
DICE Embeddings

Release 0.1.3.2

Caglar Demir

Sep 15, 2026

Contents:

1	Dicee Manual	2
2	Installation	3
2.1	Installation from Source	3
3	Download Knowledge Graphs	3
4	Knowledge Graph Embedding Models	3
5	How to Train	3
6	Creating an Embedding Vector Database	5
6.1	Learning Embeddings	5
6.2	Loading Embeddings into Qdrant Vector Database	6
6.3	Launching Webservice	6
7	Answering Complex Queries	6
8	Predicting Missing Links	8
9	Downloading Pretrained Models	8
10	How to Deploy	8
11	Docker	8
12	Coverage Report	8
13	How to cite	10
14	dicee	12
14.1	Submodules	12
14.2	Attributes	287
14.3	Classes	287
14.4	Package Contents	288
	Python Module Index	302

DICE Embeddings¹: Hardware-agnostic Framework for Large-scale Knowledge Graph Embeddings:

1 Dicee Manual

Version: dicee 0.3.2

GitHub repository: <https://github.com/dice-group/dice-embeddings>

Publisher and maintainer: Caglar Demir²

Contact: caglar.demir@upb.de

License: OSI Approved :: MIT License

Dicee is a hardware-agnostic framework for large-scale knowledge graph embeddings.

Knowledge graph embedding research has mainly focused on learning continuous representations of knowledge graphs towards the link prediction problem. Recently developed frameworks can be effectively applied in a wide range of research-related applications. Yet, using these frameworks in real-world applications becomes more challenging as the size of the knowledge graph grows

We developed the DICE Embeddings framework (dicee) to compute embeddings for large-scale knowledge graphs in a hardware-agnostic manner. To achieve this goal, we rely on

1. **Pandas**³ & Co. to use parallelism at preprocessing a large knowledge graph,
2. **PyTorch**⁴ & Co. to learn knowledge graph embeddings via multi-CPU, GPUs, TPUs or computing cluster, and
3. **Huggingface**⁵ to ease the deployment of pre-trained models.

Why Pandas⁶ & Co. ? A large knowledge graph can be read and preprocessed (e.g. removing literals) by pandas, modin, or polars in parallel. Through polars, a knowledge graph having more than 1 billion triples can be read in parallel fashion. Importantly, using these frameworks allow us to perform all necessary computations on a single CPU as well as a cluster of computers.

Why PyTorch⁷ & Co. ? PyTorch is one of the most popular machine learning frameworks available at the time of writing. PytorchLightning facilitates scaling the training procedure of PyTorch without boilerplate. In our framework, we combine **PyTorch**⁸ & **PytorchLightning**⁹. Users can choose the trainer class (e.g., DDP by Pytorch) to train large knowledge graph embedding models with billions of parameters. PytorchLightning allows us to use state-of-the-art model parallelism techniques (e.g. Fully Sharded Training, FairScale, or DeepSpeed) without extra effort. With our framework, practitioners can directly use PytorchLightning for model parallelism to train gigantic embedding models.

Why Huggingface¹⁰? Seamlessly deploy and share pre-trained embedding models through the Huggingface ecosystem.

¹ <https://github.com/dice-group/dice-embeddings>

² <https://github.com/Demirrr>

³ <https://pandas.pydata.org/>

⁴ <https://pytorch.org/>

⁵ <https://huggingface.co/>

⁶ <https://pandas.pydata.org/>

⁷ <https://pytorch.org/>

⁸ <https://pytorch.org/>

⁹ <https://www.pytorchlightning.ai/>

¹⁰ <https://huggingface.co/>

2 Installation

2.1 Installation from Source

```
git clone https://github.com/dice-group/dice-embeddings.git
conda create -n dice python=3.10.13 --no-default-packages && conda activate dice &&
↪ cd dice-embeddings &&
pip3 install -e .
```

or

```
pip install dicee
```

3 Download Knowledge Graphs

```
wget https://files.dice-research.org/datasets/dice-embeddings/KGs.zip --no-check-
↪ certificate && unzip KGs.zip
```

To test the Installation

```
python -m pytest -p no:warnings -x # Runs >114 tests leading to > 15 mins
python -m pytest -p no:warnings --lf # run only the last failed test
python -m pytest -p no:warnings --ff # to run the failures first and then the rest of
↪ the tests.
```

4 Knowledge Graph Embedding Models

1. TransE, DistMult, ComplEx, ConEx, QMult, OMult, ConvO, ConvQ, Keci
2. All 44 models available in <https://github.com/pykeen/pykeen#models>

For more, please refer to examples.

5 How to Train

To Train a KGE model (KECI) and evaluate it on the train, validation, and test sets of the UMLS benchmark dataset.

```
from dicee.executer import Execute
from dicee.config import Namespace
args = Namespace()
args.model = 'Keci'
args.scoring_technique = "KvsAll" # 1vsAll, or AllvsAll, or NegSample
args.dataset_dir = "KGs/UMLS"
args.path_to_store_single_run = "Keci_UMLS"
args.num_epochs = 100
args.embedding_dim = 32
args.batch_size = 1024
reports = Execute(args).start()
print(reports["Train"]["MRR"]) # => 0.9912
print(reports["Test"]["MRR"]) # => 0.8155
# See the Keci_UMLS folder embeddings and all other files
```

where the data is in the following form

```
$ head -3 KGs/UMLS/train.txt
acquired_abnormality    location_of      experimental_model_of_disease
anatomical_abnormality  manifestation_of physiologic_function
alga    isa        entity
```

A KGE model can also be trained from the command line

```
dicee --dataset_dir "KGs/UMLS" --model Keci --eval_model "train_val_test"
```

dicee automatically detects available GPUs and trains a model with distributed data parallels technique. Under the hood, dicee uses lightning as a default trainer.

```
# Train a model by only using the GPU-0
CUDA_VISIBLE_DEVICES=0 dicee --dataset_dir "KGs/UMLS" --model Keci --eval_model
↪ "train_val_test"
# Train a model by only using GPU-1
CUDA_VISIBLE_DEVICES=1 dicee --dataset_dir "KGs/UMLS" --model Keci --eval_model
↪ "train_val_test"
NCCL_P2P_DISABLE=1 CUDA_VISIBLE_DEVICES=0,1 python dicee/scripts/run.py --trainer PL -
↪ --dataset_dir "KGs/UMLS" --model Keci --eval_model "train_val_test"
```

Under the hood, dicee executes run.py script and uses lightning as a default trainer

```
# Two equivalent executions
# (1)
dicee --dataset_dir "KGs/UMLS" --model Keci --eval_model "train_val_test"
# Evaluate Keci on Train set: Evaluate Keci on Train set
# {'H@1': 0.9518788343558282, 'H@3': 0.9988496932515337, 'H@10': 1.0, 'MRR': 0.
↪ 9753123402351737}
# Evaluate Keci on Validation set: Evaluate Keci on Validation set
# {'H@1': 0.6932515337423313, 'H@3': 0.9041411042944786, 'H@10': 0.9754601226993865,
↪ 'MRR': 0.8072362996241839}
# Evaluate Keci on Test set: Evaluate Keci on Test set
# {'H@1': 0.6951588502269289, 'H@3': 0.9039334341906202, 'H@10': 0.9750378214826021,
↪ 'MRR': 0.8064032293278861}

# (2)
CUDA_VISIBLE_DEVICES=0,1 python dicee/scripts/run.py --trainer PL --dataset_dir "KGs/
↪ UMLS" --model Keci --eval_model "train_val_test"
# Evaluate Keci on Train set: Evaluate Keci on Train set
# {'H@1': 0.9518788343558282, 'H@3': 0.9988496932515337, 'H@10': 1.0, 'MRR': 0.
↪ 9753123402351737}
# Evaluate Keci on Train set: Evaluate Keci on Train set
# Evaluate Keci on Validation set: Evaluate Keci on Validation set
# {'H@1': 0.6932515337423313, 'H@3': 0.9041411042944786, 'H@10': 0.9754601226993865,
↪ 'MRR': 0.8072362996241839}
# Evaluate Keci on Test set: Evaluate Keci on Test set
# {'H@1': 0.6951588502269289, 'H@3': 0.9039334341906202, 'H@10': 0.9750378214826021,
↪ 'MRR': 0.8064032293278861}
```

Similarly, models can be easily trained with torchrun

```
torchrun --standalone --nnodes=1 --nproc_per_node=gpu dicee/scripts/run.py --trainer_
→torchDDP --dataset_dir "KGs/UMLS" --model Keci --eval_model "train_val_test"
# Evaluate Keci on Train set: Evaluate Keci on Train set: Evaluate Keci on Train set
# {'H@1': 0.9518788343558282, 'H@3': 0.9988496932515337, 'H@10': 1.0, 'MRR': 0.
→9753123402351737}
# Evaluate Keci on Validation set: Evaluate Keci on Validation set
# {'H@1': 0.6932515337423313, 'H@3': 0.9041411042944786, 'H@10': 0.9754601226993865,
→'MRR': 0.8072499937521418}
# Evaluate Keci on Test set: Evaluate Keci on Test set
{'H@1': 0.6951588502269289, 'H@3': 0.9039334341906202, 'H@10': 0.9750378214826021,
→'MRR': 0.8064032293278861}
```

You can also train a model in multi-node multi-gpu setting.

```
torchrun --nnodes 2 --nproc_per_node=gpu --node_rank 0 --rdzv_id 455 --rdzv_backend_
→c10d --rdzv_endpoint=nebula dicee/scripts/run.py --trainer torchDDP --dataset_dir_
→KGs/UMLS
torchrun --nnodes 2 --nproc_per_node=gpu --node_rank 1 --rdzv_id 455 --rdzv_backend_
→c10d --rdzv_endpoint=nebula dicee/scripts/run.py --trainer torchDDP --dataset_dir_
→KGs/UMLS
```

Train a KGE model by providing the path of a single file and store all parameters under newly created directory called KeciFamilyRun.

```
dicee --path_single_kg "KGs/Family/family-benchmark_rich_background.owl" --model Keci_
→--path_to_store_single_run KeciFamilyRun --backend rdflib
```

where the data is in the following form

```
$ head -3 KGs/Family/train.txt
_:1 <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://www.w3.org/2002/07/owl
→#Ontology> .
<http://www.benchmark.org/family#hasChild> <http://www.w3.org/1999/02/22-rdf-syntax-ns
→#type> <http://www.w3.org/2002/07/owl#ObjectProperty> .
<http://www.benchmark.org/family#hasParent> <http://www.w3.org/1999/02/22-rdf-syntax-
→ns#type> <http://www.w3.org/2002/07/owl#ObjectProperty> .
```

Apart from n-triples or standard link prediction dataset formats, we support ["owl", "nt", "turtle", "rdf/xml", "n3"]*. Moreover, a KGE model can be also trained by providing an endpoint of a triple store.

```
dicee --sparql_endpoint "http://localhost:3030/mutagenesis/" --model Keci
```

For more, please refer to examples.

6 Creating an Embedding Vector Database

6.1 Learning Embeddings

```
# Train an embedding model
dicee --dataset_dir KGs/Countries-S1 --path_to_store_single_run CountryEmbeddings --
→model Keci --p 0 --q 1 --embedding_dim 32 --adaptive_swa
```

6.2 Loading Embeddings into Qdrant Vector Database

```
# Ensure that Qdrant available
# docker pull qdrant/qdrant && docker run -p 6333:6333 -p 6334:6334 -v $(pwd)/
↪qdrant_storage:/qdrant/storage:z qdrant/qdrant
diceeindex --path_model "CountryEmbeddings" --collection_name "dummy" --location
↪"localhost"
```

6.3 Launching Webservice

```
diceeserve --path_model "CountryEmbeddings" --collection_name "dummy" --collection_
↪location "localhost"
```

Retrieve and Search

Get embedding of germany

```
curl -X 'GET' 'http://0.0.0.0:8000/api/get?q=germany' -H 'accept: application/json'
```

Get most similar things to europe

```
curl -X 'GET' 'http://0.0.0.0:8000/api/search?q=europe' -H 'accept: application/json'
{"result":[{"hit":"europe","score":1.0},
{"hit":"northern_europe","score":0.67126536},
{"hit":"western_europe","score":0.6010134},
{"hit":"puerto_rico","score":0.5051694},
{"hit":"southern_europe","score":0.4829831}]}
```

7 Answering Complex Queries

```
# pip install dicee
# wget https://files.dice-research.org/datasets/dice-embeddings/KGs.zip --no-check-
↪certificate & unzip KGs.zip
from dicee.executer import Execute
from dicee.config import Namespace
from dicee.knowledge_graph_embeddings import KGE
# (1) Train a KGE model
args = Namespace()
args.model = 'Keci'
args.p=0
args.q=1
args.optim = 'Adam'
args.scoring_technique = "AllvsAll"
args.path_single_kg = "KGs/Family/family-benchmark_rich_background.owl"
args.backend = "rdflib"
args.num_epochs = 200
args.batch_size = 1024
args.lr = 0.1
args.embedding_dim = 512
result = Execute(args).start()
# (2) Load the pre-trained model
```

(continues on next page)

(continued from previous page)

```
pre_trained_kge = KGE(path=result['path_experiment_folder'])
# (3) Single-hop query answering
# Query: ?E : \exist E.hasSibling(E, F9M167)
# Question: Who are the siblings of F9M167?
# Answer: [F9M157, F9F141], as (F9M167, hasSibling, F9M157) and (F9M167, hasSibling, ↵
↵F9F141)
predictions = pre_trained_kge.answer_multi_hop_query(query_type="1p",
                                                       query=('http://www.benchmark.org/
↵family#F9M167',
                                                       ('http://www.benchmark.
↵org/family#hasSibling',)),
                                                       tnorm="min", k=3)
top_entities = [topk_entity for topk_entity, query_score in predictions]
assert "http://www.benchmark.org/family#F9F141" in top_entities
assert "http://www.benchmark.org/family#F9M157" in top_entities
# (2) Two-hop query answering
# Query: ?D : \exist E.Married(D, E) \land hasSibling(E, F9M167)
# Question: To whom a sibling of F9M167 is married to?
# Answer: [F9F158, F9M142] as (F9M157 #married F9F158) and (F9F141 #married F9M142)
predictions = pre_trained_kge.answer_multi_hop_query(query_type="2p",
                                                       query=("http://www.benchmark.org/
↵family#F9M167",
                                                       ("http://www.benchmark.
↵org/family#hasSibling",
                                                       "http://www.benchmark.
↵org/family#married")),
                                                       tnorm="min", k=3)
top_entities = [topk_entity for topk_entity, query_score in predictions]
assert "http://www.benchmark.org/family#F9M142" in top_entities
assert "http://www.benchmark.org/family#F9F158" in top_entities
# (3) Three-hop query answering
# Query: ?T : \exist D.type(D,T) \land Married(D,E) \land hasSibling(E, F9M167)
# Question: What are the type of people who are married to a sibling of F9M167?
# (3) Answer: [Person, Male, Father] since F9M157 is [Brother Father Grandfather ↵
↵Male] and F9M142 is [Male Grandfather Father]
predictions = pre_trained_kge.answer_multi_hop_query(query_type="3p", query=("http://
↵www.benchmark.org/family#F9M167",
                                                       ("http://
↵www.benchmark.org/family#hasSibling",
                                                       "http://
↵www.benchmark.org/family#married",
                                                       "http://
↵www.w3.org/1999/02/22-rdf-syntax-ns#type")),
                                                       tnorm="min", k=5)
top_entities = [topk_entity for topk_entity, query_score in predictions]
print(top_entities)
assert "http://www.benchmark.org/family#Person" in top_entities
assert "http://www.benchmark.org/family#Father" in top_entities
assert "http://www.benchmark.org/family#Male" in top_entities
```

For more, please refer to examples/multi_hop_query_answering.

8 Predicting Missing Links

```
from dicee import KGE
# (1) Train a knowledge graph embedding model..
# (2) Load a pretrained model
pre_trained_kge = KGE(path='../')
# (3) Predict missing links through head entity rankings
pre_trained_kge.predict_topk(h=[".."],r=[".."],topk=10)
# (4) Predict missing links through relation rankings
pre_trained_kge.predict_topk(h=[".."],t=[".."],topk=10)
# (5) Predict missing links through tail entity rankings
pre_trained_kge.predict_topk(r=[".."],t=[".."],topk=10)
```

9 Downloading Pretrained Models

```
from dicee import KGE
# (1) Load a pretrained ConEx on DBpedia
model = KGE(url="https://files.dice-research.org/projects/DiceEmbeddings/KINSHIP-Keci-
↳dim128-epoch256-KvsAll")
```

- For more please look at dice-research.org/projects/DiceEmbeddings/¹¹

10 How to Deploy

```
from dicee import KGE
KGE(path='../').deploy(share=True, top_k=10)
```

11 Docker

To build the Docker image:

```
docker build -t dice-embeddings .
```

To test the Docker image:

```
docker run --rm -v ~/.local/share/dicee/KGs:/dicee/KGs dice-embeddings ./main.py --
↳model AConEx --embedding_dim 16
```

12 Coverage Report

The coverage report is generated using `coverage.py`¹²:

Name	Stmts	Miss	Cover	Missing
-----	-----	-----	-----	-----
dicee/___init___ .py	7	0	100%	
dicee/abstracts.py	338	115	66%	112-113, 115

(continues on next page)

¹¹ <https://files.dice-research.org/projects/DiceEmbeddings/>

¹² <https://coverage.readthedocs.io/en/7.6.0/>

(continued from previous page)

↪131, 154-155, 160, 173, 197, 240-254, 290, 303-306, 309-313, 353-364, 379-387, 402, ↪				
↪413-417, 427-428, 434-436, 442-445, 448-453, 576-596, 602-606, 610-612, 631, 658-696				
dicee/callbacks.py	248	103	58%	50-55, ↪
↪67-73, 76, 88-93, 98-103, 106-109, 116-133, 138-142, 146-147, 247, 281-285, 291-292, ↪				
↪310-316, 319, 324-325, 337-343, 349-358, 363-365, 410, 421-434, 438-473, 485-491				
dicee/config.py	97	2	98%	146-147
dicee/dataset_classes.py	430	146	66%	16, 44, ↪
↪57, 89-98, 104, 111-118, 121, 124, 127-151, 207-213, 216, 219-221, 324, 335-338, ↪				
↪354, 420-421, 439, 562-581, 583, 587-599, 606-615, 618, 622-636, 780-787, 790-794, ↪				
↪845, 866-878, 902-915, 937, 941-954, 964-967, 973, 985, 987, 989, 1012-1022				
dicee/eval_static_funcs.py	256	100	61%	104, 109, ↪
↪114, 261-356, 363-414, 442, 465-468				
dicee/evaluator.py	267	48	82%	48, 53, ↪
↪58, 77, 82-83, 86, 102, 119, 130, 134, 139, 173-184, 191-202, 310, 340-358, 452, ↪				
↪462, 480-485				
dicee/executer.py	134	16	88%	53-57, ↪
↪166-176, 235-236, 283				
dicee/knowledge_graph.py	82	10	88%	84, 94- ↪
↪95, 124, 128, 132-134, 137-138, 140				
dicee/knowledge_graph_embeddings.py	654	415	37%	25, 28- ↪
↪29, 37-50, 55-88, 91-125, 129-137, 171, 173-229, 261, 265, 276-277, 301-303, 311, ↪				
↪339-362, 493, 497-519, 523-547, 580, 656, 665, 710-716, 748, 806-1171, 1202-1263, ↪				
↪1267-1295, 1326, 1332				
dicee/models/___init___py	9	0	100%	
dicee/models/adopt.py	187	172	8%	50-86, ↪
↪99-110, 129-185, 195-242, 266-322, 346-448, 484-517				
dicee/models/base_model.py	240	35	85%	30-35, ↪
↪64, 66, 92, 99-116, 171, 204, 244, 250, 259, 262, 266, 273, 277, 279, 294, 307-308, ↪				
↪362, 365, 438, 450				
dicee/models/clifford.py	470	278	41%	10, 12, ↪
↪16, 24-25, 52-56, 79-87, 101-103, 108-109, 140-160, 184, 191, 195-256, 273-277, 289, ↪				
↪292, 297, 302, 346-361, 377-444, 464-470, 483, 486, 491, 496, 525-531, 544, 547, ↪				
↪552, 557, 567-576, 592-593, 613-685, 696-699, 724-749, 773-806, 842-846, 859, 869, ↪				
↪872, 877, 882, 887, 891, 895, 904-905, 935, 942, 947, 975-979, 1007-1016, 1026-1034, ↪				
↪1052-1054, 1072-1074, 1090-1092				
dicee/models/complex.py	162	25	85%	86-109, ↪
↪273-287				
dicee/models/dualE.py	59	10	83%	93-102, ↪
↪142-156				
dicee/models/ensemble.py	89	67	25%	7-29, 31, ↪
↪34, 37, 40, 43, 46, 49, 52-54, 56-58, 64-68, 71-90, 93-94, 97-112, 131				
dicee/models/function_space.py	262	221	16%	10-23, ↪
↪27-36, 39-48, 52-69, 76-87, 90-99, 102-111, 115-127, 135-157, 160-166, 169-186, 189- ↪				
↪195, 198-206, 209, 214-235, 244-247, 251-255, 259-268, 272-293, 302-308, 312-329, ↪				
↪333-336, 345-353, 356, 367-373, 393-407, 425-439, 444-454, 462-466, 475-479				
dicee/models/literal.py	33	1	97%	82
dicee/models/octonion.py	227	83	63%	21-44, ↪
↪320-329, 334-345, 348-370, 374-416, 426-474				
dicee/models/pykeen_models.py	55	5	91%	77-80, ↪
↪135				
dicee/models/quaternion.py	192	69	64%	7-21, 30- ↪
↪55, 68-72, 107, 185, 328-342, 345-364, 368-389, 399-426				

(continues on next page)

(continued from previous page)

dicee/models/real.py	61	12	80%	37-42, ↵
↪70-73, 91, 107-110				
dicee/models/static_funcs.py	10	0	100%	
dicee/models/transformers.py	234	189	19%	20-39, ↵
↪42, 56-71, 80-98, 101-112, 119-121, 124, 130-147, 151-176, 182-186, 189-193, 199-				
↪203, 206-208, 225-252, 261-264, 267-272, 275-300, 306-311, 315-368, 372-394, 400-410				
dicee/query_generator.py	374	346	7%	17-51, ↵
↪55, 61-64, 68-69, 77-91, 99-146, 154-187, 191-205, 211-268, 273-302, 306-442, 452-				
↪471, 479-502, 509-513, 518, 523-529				
dicee/read_preprocess_save_load_kg/___init___py	3	0	100%	
dicee/read_preprocess_save_load_kg/preprocess.py	243	40	84%	33, 39, ↵
↪76, 100-125, 131, 136-149, 175, 205, 380-381				
dicee/read_preprocess_save_load_kg/read_from_disk.py	36	11	69%	34, 38-
↪40, 47, 55, 58-72				
dicee/read_preprocess_save_load_kg/save_load_disk.py	53	21	60%	29-30, ↵
↪38, 47-68				
dicee/read_preprocess_save_load_kg/util.py	236	125	47%	159, 173-
↪175, 179-180, 198-204, 207-209, 214-216, 230, 244-247, 252-260, 265-271, 276-281, ↵				
↪286-291, 303-324, 330-386, 390-394, 398-399, 403, 407-408, 436, 441, 448-449				
dicee/sanity_checkers.py	47	19	60%	8-12, 21-
↪31, 46, 51, 58, 69-79				
dicee/static_funcs.py	483	194	60%	42, 52, ↵
↪58-63, 85, 92-96, 109-119, 129-131, 136, 143, 167, 172, 184, 190, 198, 202, 229-233,				
↪295, 303-309, 320-330, 341-361, 389, 413-414, 419-420, 437-438, 440-441, 443-444, ↵				
↪452, 470-474, 491-494, 498-503, 507-511, 515-516, 522-524, 539-553, 558-561, 566-				
↪569, 578-629, 634-646, 663-680, 683-691, 695-713, 724				
dicee/static_funcs_training.py	155	66	57%	7-10, ↵
↪222-319, 327-328				
dicee/static_preprocess_funcs.py	98	43	56%	17-25, ↵
↪50, 57, 59, 70, 83-107, 112-115, 120-123, 128-131				
dicee/trainer/___init___py	1	0	100%	
dicee/trainer/dice_trainer.py	151	18	88%	22, 30-
↪31, 33-35, 97, 104, 109-114, 152, 237, 280-283				
dicee/trainer/model_parallelism.py	99	87	12%	10-25, ↵
↪30-116, 121-132, 136, 141-197				
dicee/trainer/torch_trainer.py	77	6	92%	31, 102, ↵
↪168, 179-181				
dicee/trainer/torch_trainer_ddp.py	89	71	20%	11-14, ↵
↪43, 47-67, 78-94, 113-122, 126-136, 151-158, 168-191				

TOTAL	6948	3169	54%	

13 How to cite

Currently, we are working on our manuscript describing our framework. If you really like our work and want to cite it now, feel free to chose one :)

```
# Keci
@inproceedings{demir2023clifford,
  title={Clifford Embeddings--A Generalized Approach for Embedding in Normed Algebras}
  ↪,
```

(continues on next page)

```

    author={Demir, Caglar and Ngonga Ngomo, Axel-Cyrille},
    booktitle={Joint European Conference on Machine Learning and Knowledge Discovery in
↪Databases},
    pages={567--582},
    year={2023},
    organization={Springer}
}
# LitCQD
@inproceedings{demir2023litcqd,
    title={LitCQD: Multi-Hop Reasoning in Incomplete Knowledge Graphs with Numeric
↪Literals},
    author={Demir, Caglar and Wiebesiek, Michel and Lu, Renzhong and Ngonga Ngomo, Axel-
↪Cyrille and Heindorf, Stefan},
    booktitle={Joint European Conference on Machine Learning and Knowledge Discovery in
↪Databases},
    pages={617--633},
    year={2023},
    organization={Springer}
}
# DICE Embedding Framework
@article{demir2022hardware,
    title={Hardware-agnostic computation for large-scale knowledge graph embeddings},
    author={Demir, Caglar and Ngomo, Axel-Cyrille Ngonga},
    journal={Software Impacts},
    year={2022},
    publisher={Elsevier}
}
# KronE
@inproceedings{demir2022kronecker,
    title={Kronecker decomposition for knowledge graph embeddings},
    author={Demir, Caglar and Lienen, Julian and Ngonga Ngomo, Axel-Cyrille},
    booktitle={Proceedings of the 33rd ACM Conference on Hypertext and Social Media},
    pages={1--10},
    year={2022}
}
# QMult, OMult, ConvQ, ConvO
@InProceedings{pmlr-v157-demir21a,
    title = {Convolutional Hypercomplex Embeddings for Link Prediction},
    author = {Demir, Caglar and Moussallem, Diego and Heindorf, Stefan and Ngonga
↪Ngomo, Axel-Cyrille},
    booktitle = {Proceedings of The 13th Asian Conference on Machine Learning},
    pages = {656--671},
    year = {2021},
    editor = {Balasubramanian, Vineeth N. and Tsang, Ivor},
    volume = {157},
    series = {Proceedings of Machine Learning Research},
    month = {17--19 Nov},
    publisher = {PMLR},
    pdf = {https://proceedings.mlr.press/v157/demir21a/demir21a.pdf},
    url = {https://proceedings.mlr.press/v157/demir21a.html},
}
# ConEx

```

```
@inproceedings{demir2021convolutional,
  title={Convolutional Complex Knowledge Graph Embeddings},
  author={Caglar Demir and Axel-Cyrille Ngonga Ngomo},
  booktitle={Eighteenth Extended Semantic Web Conference - Research Track},
  year={2021},
  url={https://openreview.net/forum?id=6T45-4TFqaX}}
# Shallom
@inproceedings{demir2021shallow,
  title={A shallow neural model for relation prediction},
  author={Demir, Caglar and Moussallem, Diego and Ngomo, Axel-Cyrille Ngonga},
  booktitle={2021 IEEE 15th International Conference on Semantic Computing (ICSC)},
  pages={179--182},
  year={2021},
  organization={IEEE}
```

For any questions or wishes, please contact: caglar.demir@upb.de

14 dicee

DICE Embeddings - Knowledge Graph Embedding Library.

A library for training and using knowledge graph embedding models with support for various scoring techniques and training strategies.

Submodules:

evaluation: Model evaluation functions and Evaluator class models: KGE model implementations trainer: Training orchestration scripts: Utility scripts

14.1 Submodules

`dicee.__main__`

`dicee.abstracts`

Attributes

`logger`

Classes

<code>AbstractTrainer</code>	Abstract base class for KGE model trainers.
<code>BaseInteractiveKGE</code>	Base class for interactive, post-training use of KGE models.
<code>InteractiveQueryDecomposition</code>	Mixin that provides fuzzy-logic operators for multi-hop EPFO query answering.
<code>AbstractCallback</code>	Abstract base class for KGE training lifecycle callbacks.
<code>AbstractPPECallback</code>	Abstract base class for Post-training Parameter Ensembling (PPE) callbacks.
<code>BaseInteractiveTrainKGE</code>	Abstract/base class for training knowledge graph embedding models interactively.

Module Contents

`dicee.abstracts.logger`

class `dicee.abstracts.AbstractTrainer` (*args*, *callbacks*)

Abstract base class for KGE model trainers.

Provides the callback dispatch mechanism shared by all concrete trainer implementations (TorchTrainer, TorchD-
DPTrainer, etc.). Sub-classes call the `on_*` hooks at the appropriate points in the training loop so that any registered `AbstractCallback` can react.

Parameters

- **args** (*argparse.Namespace or similar*) – Processed configuration object. Must expose at least `random_seed` (int).
- **callbacks** (*list of AbstractCallback*) – Ordered list of callback instances to invoke at each lifecycle hook.

attributes

callbacks

is_global_zero = `True`

global_rank = `0`

local_rank = `0`

strategy = `None`

on_fit_start (**args*, ***kwargs*)

Dispatch `on_fit_start` to all registered callbacks.

Called once before the first training epoch begins.

on_fit_end (**args*, ***kwargs*)

Dispatch `on_fit_end` to all registered callbacks.

Called once after the last training epoch completes.

on_train_epoch_start (**args*, ***kwargs*)

Dispatch `on_train_epoch_start` to all registered callbacks.

Called at the beginning of every epoch.

on_train_epoch_end (**args*, ***kwargs*)

Dispatch `on_train_epoch_end` to all registered callbacks.

Called at the end of every epoch after the loss has been accumulated.

on_train_batch_end (**args*, ***kwargs*)

Dispatch `on_train_batch_end` to all registered callbacks.

Called after each mini-batch gradient update.

static save_checkpoint (*full_path: str*, *model*) → `None`

Persist model weights to disk.

Parameters

- **full_path** (*str*) – Absolute or relative file path (including filename) where the `state_dict` will be written, e.g. `'Experiments/run1/model.pt'`.

- **model** (*torch.nn.Module*) – The model whose `state_dict` is to be saved.

```
class dicee.abstracts.BaseInteractiveKGE (path: str = None, url: str = None,  

construct_ensemble: bool = False, model_name: str = None,  

apply_semantic_constraint: bool = False)
```

Base class for interactive, post-training use of KGE models.

Loads a pre-trained model from disk (or a remote URL) together with its entity/relation index mappings and exposes the prediction API used by *KGE*.

Parameters

- **path** (*str, optional*) – Path to the experiment directory produced by `Execute`. Must contain `model.pt`, `configuration.json`, `entity_to_idx.csv` and `relation_to_idx.csv`.
- **url** (*str, optional*) – Remote URL of a pre-trained model to download. Mutually exclusive with *path*.
- **construct_ensemble** (*bool, optional*) – When `True`, load all checkpoint files in *path* and average their weights to form an ensemble model. Defaults to `False`.
- **model_name** (*str, optional*) – Filename (without extension) of the checkpoint to load when multiple `.pt` files exist in *path*.
- **apply_semantic_constraint** (*bool, optional*) – Reserved for future use. Defaults to `False`.

```
construct_ensemble = False
```

```
apply_semantic_constraint = False
```

```
configs
```

```
get_eval_report () → dict
```

```
get_bpe_token_representation (str_entity_or_relation: List[str] | str) → List[List[int]] | List[int]
```

Parameters

str_entity_or_relation (*corresponds to a str or a list of strings to be tokenized via BPE and shaped.*)

Return type

A list integer(s) or a list of lists containing integer(s)

```
get_padded_bpe_triple_representation (triples: List[List[str]]) → Tuple[List, List, List]
```

Parameters

triples

```
set_model_train_mode () → None
```

Switch the underlying model to training mode.

Calls `model.train()` and re-enables gradient computation for all parameters so that subsequent calls to optimisation steps work correctly after a period of inference.

```
set_model_eval_mode () → None
```

Switch the underlying model to evaluation mode.

Calls `model.eval()` and freezes all parameters (`requires_grad = False`) so that dropout and batch-norm layers behave deterministically during inference.

property name

sample_entity (*n*: int) → List[str]

Return *n* random entity strings without replacement.

Parameters

n (*int*) – Number of entities to sample. Must be non-negative and at most `num_entities`.

Returns

Randomly selected entity string labels.

Return type

List[str]

sample_relation (*n*: int) → List[str]

Return *n* random relation strings without replacement.

Parameters

n (*int*) – Number of relations to sample. Must be non-negative and at most `num_relations`.

Returns

Randomly selected relation string labels.

Return type

List[str]

is_seen (*entity*: str = None, *relation*: str = None) → bool

Check whether an entity or relation was present in the training set.

Exactly one of *entity* or *relation* should be provided.

Parameters

- **entity** (*str*, *optional*) – Entity string label to look up.
- **relation** (*str*, *optional*) – Relation string label to look up.

Returns

True if the given string is in the respective index mapping, False otherwise.

Return type

bool

save () → None

Persist the current model weights to the experiment directory.

The checkpoint filename encodes the current timestamp so successive calls do not overwrite each other. Ensemble models are saved with an `_ensemble_` infix in the filename.

get_entity_index (*x*: str) → int

Return the integer index for a given entity string.

Parameters

x (*str*) – Entity string label (must have been seen during training).

Returns

Corresponding row index in the entity embedding matrix.

Return type

int

get_relation_index (*x: str*) → int

Return the integer index for a given relation string.

Parameters

x (*str*) – Relation string label (must have been seen during training).

Returns

Corresponding row index in the relation embedding matrix.

Return type

int

index_triple (*head_entity: List[str], relation: List[str], tail_entity: List[str]*)
→ Tuple[torch.LongTensor, torch.LongTensor, torch.LongTensor]

Convert string triple lists to integer index tensors.

Parameters

- **head_entity** (*List[str]*) – Head entity string labels.
- **relation** (*List[str]*) – Relation string labels.
- **tail_entity** (*List[str]*) – Tail entity string labels.

Returns

idx_head_entity, idx_relation, idx_tail_entity – Each has shape (n, 1) containing the integer indices for the corresponding strings.

Return type

torch.LongTensor

add_new_entity_embeddings (*entity_name: str = None, embeddings: torch.FloatTensor = None*)
→ None

Extend the entity embedding table with a new entity at inference time.

The new entity is appended to both `entity_to_idx` / `idx_to_entity` mappings and the `entity_embeddings` weight tensor so that subsequent calls to prediction methods can reference it by name.

Parameters

- **entity_name** (*str*) – String label for the new entity. If the entity already exists in the index no modification is made.
- **embeddings** (*torch.FloatTensor*) – 1-D float tensor of length `embedding_dim` containing the pre-computed embedding for the new entity.

get_entity_embeddings (*items: List[str]*) → torch.FloatTensor

Return the embedding vectors for the given entity strings.

For standard (non-BPE) models the method looks up each string in `entity_to_idx` and returns the corresponding rows of the entity embedding matrix. For BPE models subword token embeddings are fetched and flattened into a single vector per entity.

Parameters

items (*List[str]*) – Entity string labels to retrieve.

Returns

Shape (len(items), embedding_dim).

Return type

torch.FloatTensor

get_relation_embeddings (*items: List[str]*) → torch.FloatTensor

Return the embedding vectors for the given relation strings.

Parameters

items (*List[str]*) – Relation string labels to retrieve.

Returns

Shape (len(items), embedding_dim).

Return type

torch.FloatTensor

construct_input_and_output (*head_entity: List[str], relation: List[str], tail_entity: List[str], labels*)
→ Tuple[torch.LongTensor, torch.FloatTensor]

Build an indexed triple tensor and a label tensor from string inputs.

Parameters

- **head_entity** (*List[str]*) – Head entity string labels.
- **relation** (*List[str]*) – Relation string labels.
- **tail_entity** (*List[str]*) – Tail entity string labels.
- **labels** (*list or array-like*) – Binary or soft labels (one per triple) used as training targets.

Returns

- **x** (*torch.LongTensor*) – Shape (n, 3) integer-indexed triples.
- **labels** (*torch.FloatTensor*) – Shape (n,) float label tensor.

parameters ()

class dicee.abstracts.**InteractiveQueryDecomposition**

Mixin that provides fuzzy-logic operators for multi-hop EPFO query answering.

The three families of operators — T-norm, T-conorm, and negation norm — are applied element-wise over entity score tensors to compose complex queries from atomic link-prediction results (e.g. 2p, 3p, 2i, ip, up).

t_norm (*tens_1: torch.Tensor, tens_2: torch.Tensor, tnorm: str = 'min'*) → torch.Tensor

Apply a T-norm to combine two entity score distributions.

Parameters

- **tens_1** (*torch.Tensor*) – Score tensors of identical shape, values in [0, 1].
- **tens_2** (*torch.Tensor*) – Score tensors of identical shape, values in [0, 1].
- **tnorm** (*str*) – Operator to use. 'min' applies the Gödel (min) T-norm; 'prod' applies the product T-norm.

Returns

Element-wise combined scores of the same shape as the inputs.

Return type

torch.Tensor

tensor_t_norm (*subquery_scores: torch.FloatTensor, tnorm: str = 'min'*) → torch.FloatTensor

Compute T-norm over $[0,1]^{n \times d}$ where n denotes the number of hops and d denotes number of entities

t_conorm (*tens_1*: torch.Tensor, *tens_2*: torch.Tensor, *tconorm*: str = 'min') → torch.Tensor

Apply a T-conorm (S-norm) to combine two score distributions (union).

Parameters

- **tens_1** (torch.Tensor) – Score tensors of identical shape, values in [0, 1].
- **tens_2** (torch.Tensor) – Score tensors of identical shape, values in [0, 1].
- **tconorm** (str) – Operator to use. 'min' applies the Gödel (max) T-conorm; 'prod' applies the probabilistic sum T-conorm.

Returns

Element-wise combined scores of the same shape as the inputs.

Return type

torch.Tensor

negnorm (*tens_1*: torch.Tensor, *lambda_*: float, *neg_norm*: str = 'standard') → torch.Tensor

Apply a negation norm (complement) to an entity score distribution.

Parameters

- **tens_1** (torch.Tensor) – Input score tensor, values in [0, 1].
- **lambda** (float) – Shape parameter used by the Sugeno and Yager negation norms. Ignored for the standard complement.
- **neg_norm** (str) – Which negation to apply: 'standard' (1 - x), 'sugeno', or 'yager'.

Returns

Complemented score tensor of the same shape as *tens_1*.

Return type

torch.Tensor

class dicee.abstracts.**AbstractCallback**

Bases: abc.ABC, lightning.pytorch.callbacks.Callback

Abstract base class for KGE training lifecycle callbacks.

Concrete sub-classes override one or more hook methods to perform custom actions at specific points during training (e.g. weight averaging, periodic evaluation, model checkpointing). All hooks have empty default implementations so sub-classes only need to override the hooks they care about.

Callbacks are registered by passing them to the trainer's *callbacks* list. They are also compatible with PyTorch Lightning trainers because this class extends `lightning.pytorch.callbacks.Callback`.

on_init_start (*args, **kwargs)

Called when the trainer is about to be constructed.

Override to perform setup that must happen before any trainer state is initialised.

on_init_end (*args, **kwargs)

Called immediately after the trainer has been constructed.

Override to perform setup that requires a fully initialised trainer.

on_fit_start (trainer, model)

Called once before the first training epoch.

Parameters

- **trainer** (AbstractTrainer or pl.Trainer) – The active trainer instance.

- **model** (`BaseKGE`) – The model about to be trained.

`on_train_epoch_end(trainer, model)`

Called at the end of each training epoch.

Parameters

- **trainer** (`AbstractTrainer` or `pl.Trainer`) – The active trainer instance.
- **model** (`BaseKGE`) – The model being trained. `model.loss_history` contains the per-epoch average losses accumulated so far.

`on_train_batch_end(*args, **kwargs)`

Called after each mini-batch gradient update.

Override to inspect or modify the model at a finer granularity than epoch-level hooks.

`on_fit_end(*args, **kwargs)`

Called once after the final training epoch completes.

Override to perform post-training actions such as saving the final model state, computing evaluation metrics, or cleaning up resources.

class `dicee.abstracts.AbstractPPECallback(num_epochs, path, epoch_to_start, last_percent_to_consider)`

Bases: `AbstractCallback`

Abstract base class for Post-training Parameter Ensembling (PPE) callbacks.

Sub-classes implement weight-averaging strategies (SWA, EMA, SWAG, ...) by overriding `on_train_epoch_end()` and `on_fit_end()`. Common book-keeping (epoch counter, sample counter, alpha weights) is managed here.

Parameters

- **num_epochs** (`int`) – Total number of training epochs.
- **path** (`str`) – Experiment directory where averaged checkpoints will be written.
- **epoch_to_start** (`int`) – First epoch at which the averaging procedure should begin.
- **last_percent_to_consider** (`float`) – Fraction of the total training epochs (counted from the end) whose checkpoints are included in the ensemble.

`num_epochs`

`path`

`sample_counter = 0`

`epoch_count = 0`

`alphas = None`

`on_fit_start(trainer, model)`

Called once before the first training epoch.

Parameters

- **trainer** (`AbstractTrainer` or `pl.Trainer`) – The active trainer instance.
- **model** (`BaseKGE`) – The model about to be trained.

on_fit_end (*trainer, model*)

Called once after the final training epoch completes.

Override to perform post-training actions such as saving the final model state, computing evaluation metrics, or cleaning up resources.

store_ensemble (*param_ensemble*) → None

class `dicee.abstracts.BaseInteractiveTrainKGE`

Abstract/base class for training knowledge graph embedding models interactively. This class provides methods for re-training KGE models and also Literal Embedding model.

train_triples (*h: List[str], r: List[str], t: List[str], labels: List[float], iteration=2, optimizer=None*)

train_k_vs_all (*h, r, iteration=1, lr=0.001*)

Train k vs all :param head_entity: :param relation: :param iteration: :param lr: :return:

train (*kg, lr=0.1, epoch=10, batch_size=32, neg_sample_ratio=10, num_workers=1*) → None

Retrained a pretrain model on an input KG via negative sampling.

train_literals (*train_file_path: str = None, num_epochs: int = 100, lit_lr: float = 0.001, lit_normalization_type: str = 'z-norm', batch_size: int = 1024, sampling_ratio: float = None, random_seed=1, loader_backend: str = 'pandas', freeze_entity_embeddings: bool = True, gate_residual: bool = True, device: str = None, shuffle_data: bool = True*)

Trains the Literal Embeddings model using literal data.

Parameters

- **train_file_path** (*str*) – Path to the training data file.
- **num_epochs** (*int*) – Number of training epochs.
- **lit_lr** (*float*) – Learning rate for the literal model.
- **norm_type** (*str*) – Normalization type to use ('z-norm', 'min-max', or None).
- **batch_size** (*int*) – Batch size for training.
- **sampling_ratio** (*float*) – Ratio of training triples to use.
- **loader_backend** (*str*) – Backend for loading the dataset ('pandas' or 'rdflib').
- **freeze_entity_embeddings** (*bool*) – If True, freeze the entity embeddings during training.
- **gate_residual** (*bool*) – If True, use gate residual connections in the model.
- **device** (*str*) – Device to use for training ('cuda' or 'cpu'). If None, will use available GPU or CPU.
- **shuffle_data** (*bool*) – If True, shuffle the dataset before training.

`dicee.analyse_experiments`

This script should be moved to `dicee/scripts` Example: `python dicee/analyse_experiments.py --dir Experiments --features "model" "trainMRR" "testMRR"`

Attributes

logger

Classes

Experiment

Functions

get_default_arguments()
analyse(args)

Module Contents

```
dicee.analyse_experiments.logger  
  
dicee.analyse_experiments.get_default_arguments()  
  
class dicee.analyse_experiments.Experiment  
  
    model_name = []  
  
    callbacks = []  
  
    embedding_dim = []  
  
    num_params = []  
  
    num_epochs = []  
  
    batch_size = []  
  
    lr = []  
  
    byte_pair_encoding = []  
  
    aswa = []  
  
    path_dataset_folder = []  
  
    full_storage_path = []  
  
    pq = []  
  
    train_mrr = []  
  
    train_h1 = []  
  
    train_h3 = []  
  
    train_h10 = []  
  
    val_mrr = []  
  
    val_h1 = []
```

```

val_h3 = []

val_h10 = []

test_mrr = []

test_h1 = []

test_h3 = []

test_h10 = []

runtime = []

normalization = []

scoring_technique = []

save_experiment(x)

to_df()

dicee.analyse_experiments.analyse(args)

```

dicee.callbacks

Callbacks for training lifecycle events.

Provides callback classes for various training events including epoch end, model saving, weight averaging, and evaluation.

Attributes

<i>logger</i>

Classes

<i>AccumulateEpochLossCallback</i>	Callback to store epoch losses to a CSV file.
<i>PrintCallback</i>	Callback that prints training start/end times and total run-time.
<i>KGESaveCallback</i>	Callback that periodically saves model checkpoints during training.
<i>PseudoLabellingCallback</i>	Callback that augments the training set with pseudo-labelled triples.
<i>Eval</i>	Abstract base class for KGE training lifecycle callbacks.
<i>KronE</i>	Abstract base class for KGE training lifecycle callbacks.
<i>Perturb</i>	A callback for a three-Level Perturbation
<i>PeriodicEvalCallback</i>	Callback to periodically evaluate the model and optionally save checkpoints during training.
<i>LRScheduler</i>	Callback for managing learning rate scheduling and model snapshots.

Functions

<code>estimate_q(eps)</code>	estimate rate of convergence q from sequence esp
<code>compute_convergence(seq, i)</code>	

Module Contents

`dicee.callbacks.logger`

class `dicee.callbacks.AccumulateEpochLossCallback` (*path: str*)

Bases: `dicee.abstracts.AbstractCallback`

Callback to store epoch losses to a CSV file.

Parameters

path – Directory path where the loss file will be saved.

path

on_fit_end (*trainer, model*) → None

Store epoch loss history to CSV file.

Parameters

- **trainer** – The trainer instance.
- **model** – The model being trained.

class `dicee.callbacks.PrintCallback`

Bases: `dicee.abstracts.AbstractCallback`

Callback that prints training start/end times and total runtime.

start_time

on_fit_start (*trainer, pl_module*)

Called once before the first training epoch.

Parameters

- **trainer** (`AbstractTrainer` or `pl.Trainer`) – The active trainer instance.
- **model** (`BaseKGE`) – The model about to be trained.

on_fit_end (*trainer, pl_module*)

Called once after the final training epoch completes.

Override to perform post-training actions such as saving the final model state, computing evaluation metrics, or cleaning up resources.

on_train_batch_end (**args, **kwargs*)

Called after each mini-batch gradient update.

Override to inspect or modify the model at a finer granularity than epoch-level hooks.

on_train_epoch_end (**args, **kwargs*)

Called at the end of each training epoch.

Parameters

- **trainer** (`AbstractTrainer` or `pl.Trainer`) – The active trainer instance.

- **model** (`BaseKGE`) – The model being trained. `model.loss_history` contains the per-epoch average losses accumulated so far.

class `dicee.callbacks.KGESaveCallback` (*every_x_epoch: int, max_epochs: int, path: str*)

Bases: `dicee.abstracts.AbstractCallback`

Callback that periodically saves model checkpoints during training.

Parameters

- **every_x_epoch** (*int or None*) – Save a checkpoint every *every_x_epoch* epochs. When *None*, the interval defaults to `max(max_epochs // 2, 1)`.
- **max_epochs** (*int*) – Total number of training epochs (used to compute the default interval when *every_x_epoch* is *None*).
- **path** (*str*) – Directory where checkpoint files will be written.

every_x_epoch

max_epochs

epoch_counter = 0

path

on_train_batch_end (**args, **kwargs*)

Called after each mini-batch gradient update.

Override to inspect or modify the model at a finer granularity than epoch-level hooks.

on_fit_start (*trainer, pl_module*)

Called once before the first training epoch.

Parameters

- **trainer** (`AbstractTrainer` or `pl.Trainer`) – The active trainer instance.
- **model** (`BaseKGE`) – The model about to be trained.

on_train_epoch_end (**args, **kwargs*)

Called at the end of each training epoch.

Parameters

- **trainer** (`AbstractTrainer` or `pl.Trainer`) – The active trainer instance.
- **model** (`BaseKGE`) – The model being trained. `model.loss_history` contains the per-epoch average losses accumulated so far.

on_fit_end (**args, **kwargs*)

Called once after the final training epoch completes.

Override to perform post-training actions such as saving the final model state, computing evaluation metrics, or cleaning up resources.

on_epoch_end (*model, trainer, **kwargs*)

class `dicee.callbacks.PseudoLabellingCallback` (*data_module, kg, batch_size*)

Bases: `dicee.abstracts.AbstractCallback`

Callback that augments the training set with pseudo-labelled triples.

At the end of each epoch the current model scores a batch of unlabelled triples and those with a predicted probability ≥ 0.90 are appended to the training dataset (semi-supervised self-training).

Parameters

- **data_module** (*object*) – Dataset module exposing a `train_set_idx` attribute and a `train_data_loader()` method.
- **kg** (*KG*) – The knowledge graph, providing `num_entities`, `num_relations`, and `unlabelled_set`.
- **batch_size** (*int*) – Number of unlabelled triples to sample and score each epoch.

data_module

kg

num_of_epochs = 0

unlabelled_size

batch_size

create_random_data()

on_epoch_end(*trainer, model*)

`dicee.callbacks.estimate_q(eps)`

estimate rate of convergence *q* from sequence *esp*

`dicee.callbacks.compute_convergence(seq, i)`

class `dicee.callbacks.Eval` (*path, epoch_ratio: int = None*)

Bases: `dicee.abstracts.AbstractCallback`

Abstract base class for KGE training lifecycle callbacks.

Concrete sub-classes override one or more hook methods to perform custom actions at specific points during training (e.g. weight averaging, periodic evaluation, model checkpointing). All hooks have empty default implementations so sub-classes only need to override the hooks they care about.

Callbacks are registered by passing them to the trainer's *callbacks* list. They are also compatible with PyTorch Lightning trainers because this class extends `lightning.pytorch.callbacks.Callback`.

path

reports = []

epoch_ratio

epoch_counter = 0

on_fit_start(*trainer, model*)

Called once before the first training epoch.

Parameters

- **trainer** (`AbstractTrainer` or `pl.Trainer`) – The active trainer instance.
- **model** (`BaseKGE`) – The model about to be trained.

on_fit_end(*trainer, model*)

Called once after the final training epoch completes.

Override to perform post-training actions such as saving the final model state, computing evaluation metrics, or cleaning up resources.

`on_train_epoch_end(trainer, model)`

Called at the end of each training epoch.

Parameters

- **trainer** (`AbstractTrainer` or `pl.Trainer`) – The active trainer instance.
- **model** (`BaseKGE`) – The model being trained. `model.loss_history` contains the per-epoch average losses accumulated so far.

`on_train_batch_end(*args, **kwargs)`

Called after each mini-batch gradient update.

Override to inspect or modify the model at a finer granularity than epoch-level hooks.

class `dicee.callbacks.KronE`

Bases: `dicee.abstracts.AbstractCallback`

Abstract base class for KGE training lifecycle callbacks.

Concrete sub-classes override one or more hook methods to perform custom actions at specific points during training (e.g. weight averaging, periodic evaluation, model checkpointing). All hooks have empty default implementations so sub-classes only need to override the hooks they care about.

Callbacks are registered by passing them to the trainer's `callbacks` list. They are also compatible with PyTorch Lightning trainers because this class extends `lightning.pytorch.callbacks.Callback`.

f = `None`

static batch_kronecker_product (*a, b*)

Kronecker product of matrices a and b with leading batch dimensions. Batch dimensions are broadcast. The number of them must match: type a: `torch.Tensor` :type b: `torch.Tensor` :rtype: `torch.Tensor`

get_kronecker_triple_representation (*indexed_triple: torch.LongTensor*)

Get kronecker embeddings

on_fit_start (*trainer, model*)

Called once before the first training epoch.

Parameters

- **trainer** (`AbstractTrainer` or `pl.Trainer`) – The active trainer instance.
- **model** (`BaseKGE`) – The model about to be trained.

class `dicee.callbacks.Perturb` (*level: str = 'input', ratio: float = 0.0, method: str = None, scaler: float = None, frequency=None*)

Bases: `dicee.abstracts.AbstractCallback`

A callback for a three-Level Perturbation

Input Perturbation: During training an input x is perturbed by randomly replacing its element. In the context of knowledge graph embedding models, x can denote a triple, a tuple of an entity and a relation, or a tuple of two entities. A perturbation means that a component of x is randomly replaced by an entity or a relation.

Parameter Perturbation:

Output Perturbation:

level = `'input'`

ratio = `0.0`

```

method = None

scaler = None

frequency = None

on_train_batch_start (trainer, model, batch, batch_idx)
    Called when the train batch begins.

class dicee.callbacks.PeriodicEvalCallback (experiment_path: str, max_epochs: int,
    eval_every_n_epoch: int = 0, eval_at_epochs: list = None,
    save_model_every_n_epoch: bool = True, n_epochs_eval_model: str = 'val_test')
    Bases: dicee.abstracts.AbstractCallback

    Callback to periodically evaluate the model and optionally save checkpoints during training.

    Evaluates at regular intervals (every N epochs) or at explicitly specified epochs. Stores evaluation reports and model
    checkpoints.

    experiment_dir

    max_epochs

    epoch_counter = 0

    save_model_every_n_epoch = True

    reports

    n_epochs_eval_model = 'val_test'

    default_eval_model = None

    eval_epochs

    on_fit_end (trainer, model)
        Called at the end of training. Saves final evaluation report.

    on_train_epoch_end (trainer, model)
        Called at the end of each training epoch. Performs evaluation and checkpointing if scheduled.

class dicee.callbacks.LRScheduler (adaptive_lr_config: dict, total_epochs: int, experiment_dir: str,
    eta_max: float = 0.1, snapshot_dir: str = 'snapshots')
    Bases: dicee.abstracts.AbstractCallback

    Callback for managing learning rate scheduling and model snapshots.

    Supports cosine annealing (“cca”), MMCCLR (“mmcclr”), and their deferred (warmup) variants: - “deferred_cca”
    - “deferred_mmcclr”

    At the end of each learning rate cycle, the model can optionally be saved as a snapshot.

    total_epochs

    experiment_dir

    snapshot_dir

    batches_per_epoch = None

    total_steps = None

```

cycle_length = None

warmup_steps = None

lr_lambda = None

scheduler = None

step_count = 0

snapshot_loss

on_train_start (*trainer, model*)

Initialize training parameters and LR scheduler at start of training.

on_train_batch_end (*trainer, model, outputs, batch, batch_idx*)

Step the LR scheduler and save model snapshot if needed after each batch.

on_fit_end (*trainer, model*)

Called once after the final training epoch completes.

Override to perform post-training actions such as saving the final model state, computing evaluation metrics, or cleaning up resources.

dicee.config

Configuration module for DICE embeddings.

Provides the Namespace class with default configuration values for training knowledge graph embedding models.

Classes

<i>Namespace</i>	Extended Namespace with default KGE training configuration.
------------------	---

Module Contents

class `dicee.config.Namespace` (***kwargs*)

Bases: `argparse.Namespace`

Extended Namespace with default KGE training configuration.

Provides sensible defaults for all training parameters while allowing easy customization through command-line arguments or direct assignment.

dataset_dir: `str` | `None` = `None`

The path of a folder containing train.txt, and/or valid.txt and/or test.txt

save_embeddings_as_csv: `bool` = `False`

Embeddings of entities and relations are stored into CSV files to facilitate easy usage.

storage_path: `str` = `'Experiments'`

A directory named with time of execution under `storage_path` that contains related data about embeddings.

path_to_store_single_run: `str` | `None` = `None`

A single directory created that contains related data about embeddings.

```

path_single_kg = None
    Path of a file corresponding to the input knowledge graph

sparql_endpoint = None
    An endpoint of a triple store.

ultra_checkpoint: str | None = None

ultra_dim: int = 64

ultra_num_layers: int = 6

ultra_query_batch_size: int = 8

trix_checkpoint: str | None = None

trix_dim: int = 32

trix_query_batch_size: int = 8

flock_checkpoint: str | None = None

flock_dim: int = 64

flock_walk_num: int = 128

flock_walk_len: int = 128

flock_refinements: int = 6

flock_num_layers: int = 1

flock_attention_heads: int = 4

flock_test_samples: int = 1

flock_query_batch_size: int = 1

flock_seed: int | None = None

flock_prefetch_walks: bool = True

flock_compact_state: bool = True

flock_compile_sampler: bool = True

flock_pack_walks: bool = True

graph_inference_backend: str = 'auto'

graph_relation_cache_mb: int = 64

graph_projection_cache_mb: int = 64

graph_inference_compile: bool = False

grouped_negative_sampling: bool = False

strict_negative_sampling: bool = False

adversarial_temperature: float | None = None

```

model: str = 'Keci'
KGE model

optim: str = 'Adam'
Optimizer

embedding_dim: int = 64
Size of continuous vector representation of an entity/relation

num_epochs: int = 150
Number of pass over the training data

batch_size: int = 1024
Mini-batch size if it is None, an automatic batch finder technique applied

lr: float = 0.1
Learning rate

add_noise_rate: float | None = None
The ratio of added random triples into training dataset

gpus = None
Number GPUs to be used during training

callbacks
10}}

Type
Callbacks, e.g., {"PPE"}

Type
{ "last_percent_to_consider"

backend: str = 'pandas'
Backend to read, process, and index input knowledge graph. pandas, polars and rdflib available

separator: str = '\\s+'
separator for extracting head, relation and tail from a triple

trainer: str = 'torchCPUTrainer'
'torchCPUTrainer' (CPU/single GPU), 'PL' (PyTorch Lightning multi-GPU), 'torchDDP' (native DDP), 'TP' (Tensor Parallelism - implements 'Multiple Run Ensemble Learning with Low-Dimensional Knowledge Graph Embeddings')

Type
Trainer for knowledge graph embedding model. Options

scoring_technique: str = 'KvsAll'
Scoring technique for knowledge graph embedding models

neg_ratio: int = 0
Negative ratio for a true triple in NegSample training_technique

weight_decay: float = 0.0
Weight decay for all trainable params

normalization: str = 'None'
LayerNorm, BatchNorm1d, or None

init_param: `str | None = None`
 xavier_normal or None

gradient_accumulation_steps: `int = 0`
 Not tested e

num_folds_for_cv: `int = 0`
 Number of folds for CV

eval_model: `str = 'train_val_test'`
 ["None", "train", "train_val", "train_val_test", "test"]

Type
 Evaluate trained model choices

eval_tie_policy: `str = 'sort'`
 sort (legacy), optimistic, random, or pessimistic.

Type
 Prediction ties

eval_tie_seed: `int | None = None`
 Independent random tie seed; None uses random_seed.

save_model_at_every_epoch: `int | None = None`
 Not tested

label_smoothing_rate: `float = 0.0`

num_core: `int = 0`
 Number of CPUs to be used in the mini-batch loading process

random_seed: `int = 0`
 Random Seed

log_level: `str = 'INFO'`
 DEBUG, INFO, WARNING, ERROR, or CRITICAL. Dataset info, timing, and checkpoint messages are logged at INFO; set to WARNING or higher to silence them.

Type
 Logging verbosity

sample_triples_ratio: `float | None = None`
 Read some triples that are uniformly at random sampled. Ratio being between 0 and 1

read_only_few: `int | None = None`
 Read only first few triples

pykeen_model_kwargs
 Additional keyword arguments for pykeen models

pl_trainer_kwargs
 Additional keyword arguments for the PyTorch Lightning Trainer

kernel_size: `int = 3`
 Size of a square kernel in a convolution operation

num_of_output_channels: `int = 32`
 Number of slices in the generated feature map by convolution.

margin: float = 4.0
Margin used by margin-based scoring functions (e.g. TransE, TransH, RotatE), and by torch.nn.MarginRankingLoss when scoring_technique='NegSampleMargin'

p: int = 0
P parameter of Clifford Embeddings

q: int = 1
Q parameter of Clifford Embeddings

input_dropout_rate: float = 0.0
Dropout rate on embeddings of input triples

hidden_dropout_rate: float = 0.0
Dropout rate on hidden representations of input triples

feature_map_dropout_rate: float = 0.0
Dropout rate on a feature map generated by a convolution operation

byte_pair_encoding: bool = False
Byte pair encoding

Type
WIP

adaptive_swa: bool = False
Adaptive stochastic weight averaging

swa: bool = False
Stochastic weight averaging

swag: bool = False
Stochastic weight averaging - Gaussian

ema: bool = False
Exponential Moving Average

twa: bool = False
Trainable weight averaging

block_size: int | None = None
block size of LLM

continual_learning: str | None = None
Path of a pretrained model size of LLM

auto_batch_finding: bool = False
A flag for using auto batch finding

eval_every_n_epochs: int = 0
Evaluate model every n epochs. If 0, no evaluation is applied.

save_every_n_epochs: bool = False
Save model every n epochs. If True, save model at every epoch.

eval_at_epochs: list | None = None
List of epoch numbers at which to evaluate the model (e.g., 1 5 10).

n_epochs_eval_model: str = 'val_test'

Evaluating link prediction performance on data splits while performing periodic evaluation.

adaptive_lr

“cca”}

Type

Adaptive learning rate parameters, e.g., ‘{“scheduler_name”

swa_start_epoch: int | None = None

Epoch at which to start applying stochastic weight averaging.

swa_c_epochs: int = 1

Number of epochs to average over for SWA, SWAG, EMA, TWA.

loss_fn: str | None = None

label_relaxation_alpha: float = 0.1

amwa: bool = False

Adaptive Momentum Weight Averaging

amwa_start_epoch: int = 0

Epoch at which to start applying AMWA.

amwa_c_epochs: int = 1

Number of epochs between AMWA updates.

amwa_monitor: str = 'MRR'

Validation metric used by AMWA to compare BaseNet and StableNet.

amwa_maximize: bool = True

Whether larger values of amwa_monitor are better.

amwa_beta: float | None = None

Fixed beta for AMWA. If None, use adaptive beta_n.

amwa_beta_window: int = 10

Number of recent delta values used to estimate beta_n.

amwa_beta_init: float = 1.0

Initial beta used before enough delta history exists.

amwa_beta_floor: float = 1e-08

Numerical floor for beta_n.

__iter__()

dicee.dataset_classes

Dataset classes for knowledge graph embedding training.

This package groups the various `torch.utils.data.Dataset` implementations used throughout DICE Embeddings into thematic sub-modules:

- `_bpe` – Byte-pair-encoding related datasets.
- `_negative_sampling` – Negative sampling based datasets.
- `_label_based` – Multi-label / multi-class scoring datasets.

- `_literal` – Literal (numeric) embedding dataset.
- `_factory` – `construct_dataset` / `reload_dataset` helpers.

All public names are re-exported here so that existing `from dicee.dataset_classes import ...` statements continue to work unchanged.

Classes

<code>BPE_NegativeSamplingDataset</code>	Dataset for negative sampling with byte-pair encoded triples.
<code>MultiClassClassificationDataset</code>	Dataset for autoregressive multi-class classification on sub-word units.
<code>MultiLabelDataset</code>	Multi-label dataset for BPE-based KvsAll / AllvsAll training.
<code>AllvsAll</code>	Dataset for AllvsAll training (multi-label, exhaustive).
<code>FSDP1vsSampleDataset</code>	Positive-triple dataset for FSDP 1vsSample training with true-negative sampling.
<code>KvsAll</code>	Dataset for KvsAll training (multi-label).
<code>KvsSampleDataset</code>	Dataset for KvsSample training (dynamic multi-label).
<code>OnevsAllDataset</code>	Dataset for the 1-vs-All training strategy (multi-class).
<code>LiteralDataset</code>	Dataset for loading and processing literal data for Literal Embedding models.
<code>FixedNegSampleDataset</code>	Pre-computed (fixed) negative sampling dataset.
<code>OnevsSample</code>	Dataset for 1-vs-Sample training (dynamic multi-class with negatives).
<code>TriplePredictionDataset</code>	Dataset for triple prediction with on-the-fly negative sampling.

Functions

<code>construct_dataset(→ torch.utils.data.Dataset)</code>	Build the appropriate dataset for the given training configuration.
<code>reload_dataset(path, form_of_labelling, ...)</code>	Reload training data from disk and construct a Pytorch dataset.

Package Contents

class `dicee.dataset_classes.BPE_NegativeSamplingDataset` (*train_set: torch.LongTensor, ordered_shaped_bpe_entities: torch.LongTensor, neg_ratio: int*)

Bases: `dicee.dataset_classes._storage.WorkerDataset`

Dataset for negative sampling with byte-pair encoded triples.

Each sample is a BPE-encoded triple. The custom `collate_fn` constructs negatives by corrupting head or tail entities with random BPE entities.

Parameters

- **`train_set`** (*torch.LongTensor*) – Integer-encoded triples of shape $(N, 3)$.
- **`ordered_shaped_bpe_entities`** (*torch.LongTensor*) – All BPE entity representations, ordered by entity index.

- **neg_ratio** (*int*) – Number of negative samples per positive triple.

train_set

ordered_bpe_entities

num_bpe_entities

neg_ratio

num_datapoints

__len__ ()

__getitem__ (*idx*)

collate_fn (*batch_shaped_bpe_triples: List[Tuple[torch.Tensor, torch.Tensor]]*)

class dicee.dataset_classes.**MultiClassClassificationDataset** (
subword_units: numpy.ndarray, block_size: int = 8)

Bases: dicee.dataset_classes._storage.WorkerDataset

Dataset for autoregressive multi-class classification on sub-word units.

Splits a flat sequence of sub-word token ids into overlapping windows of size `block_size` for next-token prediction.

Parameters

- **subword_units** (*numpy.ndarray*) – 1-D array of sub-word token ids.
- **block_size** (*int, optional*) – Context window length (default 8).

train_data

block_size = 8

num_of_data_points

collate_fn = None

__len__ ()

__getitem__ (*idx*)

class dicee.dataset_classes.**MultiLabelDataset** (*train_set: torch.LongTensor, train_indices_target,*
target_dim: int, torch_ordered_shaped_bpe_entities: torch.LongTensor)

Bases: dicee.dataset_classes._storage.WorkerDataset

Multi-label dataset for BPE-based KvsAll / AllvsAll training.

Each sample is a BPE-encoded (head, relation) pair together with a binary multi-label target vector over all entities.

Parameters

- **train_set** (*torch.LongTensor*) – BPE-encoded input pairs of shape (N, 2, token_length).
- **train_indices_target** (*sequence of integer sequences or RaggedIndices*) – Per-sample lists of positive target entity indices.
- **target_dim** (*int*) – Dimensionality of the target vector (number of entities).

- **torch_ordered_shaped_bpe_entities** (*torch.LongTensor*) – Ordered BPE entity representations.

train_set

train_indices_target

target_dim

num_datapoints

torch_ordered_shaped_bpe_entities

collate_fn = None

__len__()

__getitem__(*idx*)

`dicee.dataset_classes.construct_dataset` (* (Keyword-only parameters separator (PEP 3102)),
train_set: numpy.ndarray | list, valid_set=None, test_set=None, ordered_bpe_entities=None,
train_target_indices=None, target_dim: int = None, entity_to_idx: dict, relation_to_idx: dict,
form_of_labelling: str, scoring_technique: str, neg_ratio: int, label_smoothing_rate: float,
byte_pair_encoding=None, block_size: int = None, seed: int = None, sort_train_set: bool = True,
grouped_negative_sampling: bool = False, strict_negative_sampling: bool = False,
adversarial_temperature: float | None = None) → `torch.utils.data.Dataset`

Build the appropriate dataset for the given training configuration.

Parameters

- **train_set** (*numpy.ndarray* or *list*) – Raw integer-indexed triples.
- **entity_to_idx** (*dict*) – Name → index mappings.
- **relation_to_idx** (*dict*) – Name → index mappings.
- **form_of_labelling** (*str*) – 'EntityPrediction' or 'RelationPrediction'.
- **scoring_technique** (*str*) – One of 'NegSample', 'NegSampleMargin', 'Fixed-NegSample', '1vsAll', '1vsSample', 'KvsAll', 'AllvsAll', 'KvsSample'.
- **neg_ratio** (*int*) – Negative sample ratio.
- **label_smoothing_rate** (*float*) – Label smoothing coefficient.

Return type

`torch.utils.data.Dataset`

`dicee.dataset_classes.reload_dataset` (*path: str, form_of_labelling, scoring_technique, neg_ratio,*
label_smoothing_rate)

Reload training data from disk and construct a Pytorch dataset.

class `dicee.dataset_classes.AllvsAll` (*train_set_idx: numpy.ndarray, entity_idx, relation_idx,*
label_smoothing_rate=0.0)

Bases: `dicee.dataset_classes._storage.WorkerDataset`

Dataset for AllvsAll training (multi-label, exhaustive).

Extends the KvsAll idea: every *possible* (entity, relation) combination is included — not just those observed in the KG. Pairs without any known tail entities receive an all-zeros label vector.

Parameters

- **train_set_idx** (*numpy.ndarray*) – (N, 3) integer-indexed triples.

- **entity_idx**s (*dict*) – Entity-name → index mapping.
- **relation_idx**s (*dict*) – Relation-name → index mapping.
- **label_smoothing_rate** (*float, optional*) – Label smoothing coefficient (default 0.0).

train_data: torch.Tensor

train_target: dicee.dataset_classes._storage.RaggedIndices

label_smoothing_rate

collate_fn = None

target_dim

__len__()

__getitem__(*idx*)

class dicee.dataset_classes.**FSDP1vsSampleDataset** (*train_set_idx: numpy.ndarray, entity_idx*s, *relation_idx*s, *form, neg_ratio=None, label_smoothing_rate: float = 0.0*)

Bases: dicee.dataset_classes._storage.WorkerDataset

Positive-triple dataset for FSDP 1vsSample training with true-negative sampling.

Each dataset item is a single positive triple (h, r, t). The collate_fn builds the full (source, target_idx, labels) batch in a DataLoader worker, sampling true negatives via index remapping so they never coincide with any known positive tail for that (h, r) pair.

Fixed batch width follows KvsSample:

max_num_of_classes = max_positives_per_pair + neg_ratio

Each sample contributes 1 positive and (max_num_of_classes - 1) negatives.

train_data

num_entities

num_relations

neg_ratio = None

label_smoothing_rate = 0.0

max_num_of_classes

num_negatives

collate_fn

__len__()

__getitem__(*idx*)

class dicee.dataset_classes.**KvsAll** (*train_set_idx: numpy.ndarray, entity_idx*s, *relation_idx*s, *form, store=None, label_smoothing_rate: float = 0.0*)

Bases: dicee.dataset_classes._storage.WorkerDataset

Dataset for KvsAll training (multi-label).

D := {(x, y)_i}_{i=1}^{N} where

- $x = (h, r)$ is a unique (entity, relation) pair observed in the KG,
- $y \in [0, 1]^{\{IE\}}$ is a multi-label vector with $y_j = 1$ iff $(h, r, e_j) \in KG$.

Parameters

- **train_set_idx** (*numpy.ndarray*) – (N, 3) integer-indexed triples.
- **entity_idxes** (*dict*) – Entity-name $\rightarrow$ index mapping.
- **relation_idxes** (*dict*) – Relation-name $\rightarrow$ index mapping.
- **form** (*str*) – 'EntityPrediction' or 'RelationPrediction'.
- **label_smoothing_rate** (*float, optional*) – Label smoothing coefficient (default 0.0).

train_data: torch.Tensor

train_target: dicee.dataset_classes._storage.RaggedIndices

label_smoothing_rate

collate_fn = None

__len__ ()

__getitem__ (*idx*)

class dicee.dataset_classes.**KvsSampleDataset** (*train_set_idx: numpy.ndarray, entity_idxes, relation_idxes, form, store=None, neg_ratio=None, label_smoothing_rate: float = 0.0*)

Bases: dicee.dataset_classes._storage.WorkerDataset

Dataset for KvsSample training (dynamic multi-label).

Like **KvsAll** but sub-samples the target vector at each access to keep mini-batch sizes manageable when the entity set is large.

Parameters

- **train_set_idx** (*numpy.ndarray*) – (N, 3) integer-indexed triples.
- **entity_idxes** (*dict*) – Entity-name $\rightarrow$ index mapping.
- **relation_idxes** (*dict*) – Relation-name $\rightarrow$ index mapping.
- **form** (*str*) – 'EntityPrediction'.
- **neg_ratio** (*int*) – Number of negative samples per positive target.
- **label_smoothing_rate** (*float, optional*) – Label smoothing coefficient (default 0.0).

train_data: torch.Tensor

train_target: dicee.dataset_classes._storage.RaggedIndices

neg_ratio = None

num_entities

label_smoothing_rate

collate_fn = None

max_num_of_classes

__len__()

__getitem__(*idx*)

class dicee.dataset_classes.**OnevsAllDataset**(*train_set_idx: numpy.ndarray, entity_idx*)

Bases: dicee.dataset_classes._storage.WorkerDataset

Dataset for the 1-vs-All training strategy (multi-class).

Each sample is a (*head, relation*) pair with a one-hot target vector whose single active position corresponds to the true tail entity.

Parameters

- **train_set_idx**(*numpy.ndarray*) – (*N, 3*) integer-indexed triples.
- **entity_idx**(*dict*) – Entity-name → index mapping (used to determine the target dimension).

train_data

target_dim

collate_fn = None

__len__()

__getitem__(*idx*)

class dicee.dataset_classes.**LiteralDataset**(*file_path: str, ent_idx: dict = None, normalization_type: str = 'z-norm', sampling_ratio: float = None, loader_backend: str = 'pandas'*)

Bases: torch.utils.data.Dataset

Dataset for loading and processing literal data for Literal Embedding models.

Handles loading, normalization, and preparation of (*entity, attribute, value*) triples. Supports z-score and min-max normalization as well as optional sub-sampling for ablation studies.

Parameters

- **file_path**(*str*) – Path to the training data file (CSV / TSV / RDF).
- **ent_idx**(*dict*) – Entity-name → index mapping.
- **normalization_type**(*str, optional*) – 'z-norm', 'min-max', or None (default 'z-norm').
- **sampling_ratio**(*float or None, optional*) – Fraction of the training set to keep (default None = use all).
- **loader_backend**(*str, optional*) – 'pandas' or 'rdflib' (default 'pandas').

train_file_path

loader_backend = 'pandas'

normalization_type = 'z-norm'

normalization_params

sampling_ratio = None

`entity_to_idx = None`

`num_entities`

`__getitem__ (index)`

`__len__ ()`

`static load_and_validate_literal_data (file_path: str = None, loader_backend: str = 'pandas')
→ pandas.DataFrame`

Load and validate a literal data file.

Parameters

- **file_path** (*str*) – Path to the data file.
- **loader_backend** (*str*) – 'pandas' or 'rdflib'.

Returns

Three-column DataFrame with columns head, attribute, value.

Return type

pandas.DataFrame

`static denormalize (preds_norm, attributes, normalization_params) → numpy.ndarray`

Reverse the normalization applied during training.

Parameters

- **preds_norm** (*numpy.ndarray*) – Normalized predictions.
- **attributes** (*list*) – Attribute names corresponding to each prediction.
- **normalization_params** (*dict*) – Parameters stored during training.

Returns

Denormalized predictions.

Return type

numpy.ndarray

`class dicee.dataset_classes.FixedNegSampleDataset (train_set: numpy.ndarray, num_entities: int,
num_relations: int, neg_sample_ratio: int = 1, label_smoothing_rate: float = 0.0, seed: int = None)`

Bases: `dicee.dataset_classes._storage.WorkerDataset`

Pre-computed (fixed) negative sampling dataset.

At construction time every positive triple is paired with one random negative (head- or tail-corrupted) using vectorized operations for efficiency. The pairs are stored so that `__getitem__` is a simple lookup.

This is useful when you want deterministic negatives across epochs (e.g., for reproducibility or debugging).

Parameters

- **train_set** (*numpy.ndarray*) – (N, 3) integer-indexed triples.
- **num_entities** (*int*) – Total number of entities.
- **num_relations** (*int*) – Total number of relations.
- **neg_sample_ratio** (*int, optional*) – Number of negative samples per positive triple (default 1).
- **label_smoothing_rate** (*float, optional*) – Label smoothing coefficient (default 0.0).


```

neg_sample_ratio = 1

num_entities

num_relations

label_smoothing_rate = 0.0

collate_fn = None

seed = None

train_triples

length

__len__() → int

__getitem__(idx: int) → Tuple[torch.Tensor, torch.Tensor]

```

class dicee.dataset_classes.**OnevsSample** (*train_set: numpy.ndarray, num_entities: int, num_relations: int, neg_sample_ratio: int = None, label_smoothing_rate: float = 0.0*)

Bases: dicee.dataset_classes._storage.WorkerDataset

Dataset for 1-vs-Sample training (dynamic multi-class with negatives).

For every positive triple (*h*, *r*, *t*) the dataset draws *neg_sample_ratio* random entities as negatives and returns a label vector that marks the true tail and the negatives.

Parameters

- **train_set** (*numpy.ndarray*) – (*N*, 3) integer-indexed triples.
- **num_entities** (*int*) – Total number of entities.
- **num_relations** (*int*) – Total number of relations.
- **neg_sample_ratio** (*int*) – Number of negative samples per positive.
- **label_smoothing_rate** (*float, optional*) – Label smoothing coefficient (default 0.0).

```

train_data

num_entities

num_relations

neg_sample_ratio = None

label_smoothing_rate

collate_fn = None

__len__()

__getitem__(idx)

```

```
class dicee.dataset_classes.TriplePredictionDataset (train_set: numpy.ndarray,  

num_entities: int, num_relations: int, neg_sample_ratio: int = 1, label_smoothing_rate: float = 0.0,  

seed: int = None, sort_train_set: bool = True)
```

Bases: `dicee.dataset_classes._storage.WorkerDataset`

Dataset for triple prediction with on-the-fly negative sampling.

Each item is a single positive triple; the custom `collate_fn` generates a batch of mixed positive and negative triples.

Parameters

- **train_set** (*numpy.ndarray*) – (N, 3) integer-indexed triples.
- **num_entities** (*int*) – Total number of entities.
- **num_relations** (*int*) – Total number of relations.
- **neg_sample_ratio** (*int, optional*) – Number of negative samples per positive triple (default 1).
- **label_smoothing_rate** (*float, optional*) – Label smoothing coefficient (default 0.0).

label_smoothing_rate

neg_sample_ratio

seed = None

length

num_entities

num_relations

__len__()

__getitem__ (*idx*)

collate_fn (*batch: List[torch.Tensor]*)

`dicee.eval_static_funcs`

Static evaluation functions for KGE models.

This module provides backward compatibility by re-exporting from the new `dicee.evaluation` module.

Deprecated since version Use: `dicee.evaluation` submodules instead. This module will be removed in a future version.

Functions

<code>evaluate_ensemble_link_prediction_performance</code>	Evaluate link prediction performance of an ensemble of KGE models.
<code>evaluate_link_prediction_performance(→ Dict[str, float])</code>	Evaluate link prediction performance with head and tail prediction.
<code>evaluate_link_prediction_performance_with_bpe</code>	Evaluate link prediction with BPE encoding (head and tail).

continues on next page

Table 12 – continued from previous page

<code>evaluate_link_prediction_performance_with_j</code>	Evaluate link prediction with BPE encoding and reciprocals.
<code>evaluate_link_prediction_performance_with_</code>	Evaluate link prediction with reciprocal relations.
<code>evaluate_lp_bpe_k_vs_all(→ Dict[str, float])</code>	Evaluate BPE link prediction with KvsAll scoring.
<code>evaluate_literal_prediction(→ Optional[pandas.DataFrame])</code>	Evaluate trained literal prediction model on a test file.

Module Contents

`dicee.eval_static_funcs.evaluate_ensemble_link_prediction_performance` (*models*: List, *triples*, *er_vocab*: Dict[Tuple, List], *weights*: List[float] | None = None, *batch_size*: int = 512, *weighted_averaging*: bool = True, *normalize_scores*: bool = True, *, *tie_policy*: str = 'sort', *tie_seed*: int = 0) → Dict[str, float]

Evaluate link prediction performance of an ensemble of KGE models.

Combines predictions from multiple models using weighted or simple averaging, with optional score normalization.

Parameters

- **models** – List of KGE models (e.g., snapshots from training).
- **triples** – Test triples as numpy array or list, shape (N, 3), with integer indices (head, relation, tail).
- **er_vocab** – Mapping (head_idx, rel_idx) -> list of tail indices for filtered evaluation.
- **weights** – Weights for model averaging. Required if `weighted_averaging` is True. Must sum to 1 for proper averaging.
- **batch_size** – Batch size for processing triples.
- **weighted_averaging** – If True, use weighted averaging of predictions. If False, use simple mean.
- **normalize_scores** – If True, normalize scores to [0, 1] range per sample before averaging.
- **tie_policy** – Exact prediction ties: sort (legacy), optimistic, random, or pessimistic. Applied after combining model scores.
- **tie_seed** – Independent random tie seed; reset on each evaluation call.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

Raises

AssertionError – If `weighted_averaging` is True but weights are not provided or have wrong length.

Example

```
>>> from dicee.evaluation import evaluate_ensemble_link_prediction_performance
>>> models = [model1, model2, model3]
>>> weights = [0.5, 0.3, 0.2]
>>> results = evaluate_ensemble_link_prediction_performance(
...     models, test_triples, er_vocab,
...     weights=weights, weighted_averaging=True
... )
>>> print(f"MRR: {results['MRR']:.4f}")
```

```
dicee.eval_static_funcs.evaluate_link_prediction_performance(model, triples,
    er_vocab: Dict[Tuple, List], re_vocab: Dict[Tuple, List], *, tie_policy: str = 'sort', tie_seed: int = 0)
    → Dict[str, float]
```

Evaluate link prediction performance with head and tail prediction.

Performs filtered evaluation where known correct answers are filtered out before computing ranks.

Parameters

- **model** – KGE model wrapper with entity/relation mappings.
- **triples** – Test triples as list of (head, relation, tail) strings.
- **er_vocab** – Mapping (entity, relation) -> list of valid tail entities.
- **re_vocab** – Mapping (relation, entity) -> list of valid head entities.
- **tie_policy** – How exact prediction ties are ranked: sort (legacy), optimistic, random, or pessimistic.
- **tie_seed** – Independent random tie seed; reset on each evaluation call.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

```
dicee.eval_static_funcs.evaluate_link_prediction_performance_with_bpe(model,
    within_entities: List[str], triples: List[Tuple[str]], er_vocab: Dict[Tuple, List],
    re_vocab: Dict[Tuple, List], *, tie_policy: str = 'sort', tie_seed: int = 0) → Dict[str, float]
```

Evaluate link prediction with BPE encoding (head and tail).

Parameters

- **model** – KGE model wrapper with BPE support.
- **within_entities** – List of entities to evaluate within.
- **triples** – Test triples as list of (head, relation, tail) tuples.
- **er_vocab** – Mapping (entity, relation) -> list of valid tail entities.
- **re_vocab** – Mapping (relation, entity) -> list of valid head entities.
- **tie_policy** – How exact prediction ties are ranked: sort (legacy), optimistic, random, or pessimistic.
- **tie_seed** – Independent random tie seed; reset on each evaluation call.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

```
dicee.eval_static_funcs.evaluate_link_prediction_performance_with_bpe_reciprocals(
    model, within_entities: List[str], triples: List[List[str]], er_vocab: Dict[Tuple, List], *,
    tie_policy: str = 'sort', tie_seed: int = 0) → Dict[str, float]
```

Evaluate link prediction with BPE encoding and reciprocals.

Parameters

- **model** – KGE model wrapper with BPE support.
- **within_entities** – List of entities to evaluate within.
- **triples** – Test triples as list of [head, relation, tail] strings.
- **er_vocab** – Mapping (entity, relation) -> list of valid tail entities.

- **tie_policy** – How exact prediction ties are ranked: sort (legacy), optimistic, random, or pessimistic.
- **tie_seed** – Independent random tie seed; reset on each evaluation call.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

```
dicee.eval_static_funcs.evaluate_link_prediction_performance_with_reciprocals(
    model, triples, er_vocab: Dict[Tuple, List], *, tie_policy: str = 'sort', tie_seed: int = 0)
    → Dict[str, float]
```

Evaluate link prediction with reciprocal relations.

Optimized for models trained with reciprocal triples where only tail prediction is needed.

Parameters

- **model** – KGE model wrapper.
- **triples** – Test triples as list of (head, relation, tail) strings.
- **er_vocab** – Mapping (entity, relation) -> list of valid tail entities.
- **tie_policy** – How exact prediction ties are ranked: sort (legacy), optimistic, random, or pessimistic.
- **tie_seed** – Independent random tie seed; reset on each evaluation call.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

```
dicee.eval_static_funcs.evaluate_lp_bpe_k_vs_all(model, triples: List[List[str]],
    er_vocab: Dict | None = None, batch_size: int | None = None,
    func_triple_to_bpe_representation: Callable | None = None,
    str_to_bpe_entity_to_idx: Dict | None = None, *, tie_policy: str = 'sort', tie_seed: int = 0)
    → Dict[str, float]
```

Evaluate BPE link prediction with KvsAll scoring.

Parameters

- **model** – The KGE model wrapper.
- **triples** – List of string triples.
- **er_vocab** – Entity-relation vocabulary for filtering.
- **batch_size** – Batch size for processing.
- **func_triple_to_bpe_representation** – Function to convert triples to BPE.
- **str_to_bpe_entity_to_idx** – Mapping from string entities to BPE indices.
- **tie_policy** – How exact prediction ties are ranked: sort (legacy), optimistic, random, or pessimistic.
- **tie_seed** – Independent random tie seed; reset on each evaluation call.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

Raises

ValueError – If batch_size is not provided.

```
dicee.eval_static_funcs.evaluate_literal_prediction(kge_model, eval_file_path: str = None,
    store_lit_preds: bool = True, eval_literals: bool = True, loader_backend: str = 'pandas',
    return_attr_error_metrics: bool = False) → pandas.DataFrame | None
```

Evaluate trained literal prediction model on a test file.

Evaluates the literal prediction capabilities of a KGE model by computing MAE and RMSE metrics for each attribute.

Parameters

- **kge_model** – Trained KGE model with literal prediction capability.
- **eval_file_path** – Path to the evaluation file containing test literals.
- **store_lit_preds** – If True, stores predictions to CSV file.
- **eval_literals** – If True, evaluates and prints error metrics.
- **loader_backend** – Backend for loading dataset ('pandas' or 'rdflib').
- **return_attr_error_metrics** – If True, returns the metrics DataFrame.

Returns

DataFrame with per-attribute MAE and RMSE if `return_attr_error_metrics` is True, otherwise None.

Raises

- **RuntimeError** – If the KGE model doesn't have a trained literal model.
- **AssertionError** – If model is invalid or test set has no valid data.

Example

```
>>> from dicee import KGE
>>> from dicee.evaluation import evaluate_literal_prediction
>>> model = KGE(path="pretrained_model")
>>> metrics = evaluate_literal_prediction(
...     model,
...     eval_file_path="test_literals.csv",
...     return_attr_error_metrics=True
... )
>>> print(metrics)
```

dicee.evaluation

Evaluation module for knowledge graph embedding models.

This module provides comprehensive evaluation capabilities for KGE models, including link prediction, literal prediction, and ensemble evaluation.

Modules:

link_prediction: Functions for evaluating link prediction performance
literal_prediction: Functions for evaluating literal/attribute prediction
ensemble: Functions for ensemble model evaluation
evaluator: Main Evaluator class for integrated evaluation
utils: Shared utility functions for evaluation
_filtering: Internal helpers for filtered ranking (not exported)

Example

```
>>> from dicee.evaluation import Evaluator
>>> from dicee.evaluation.link_prediction import evaluate_link_prediction_performance
>>> from dicee.evaluation.ensemble import evaluate_ensemble_link_prediction_
    ↪ performance
```

Submodules

`dicee.evaluation.ensemble`

Ensemble evaluation functions.

This module provides functions for evaluating ensemble models, including weighted averaging and score normalization.

Functions

<code>evaluate_ensemble_link_prediction_performance</code>	Evaluate link prediction performance of an ensemble of KGE models.
--	--

Module Contents

`dicee.evaluation.ensemble.evaluate_ensemble_link_prediction_performance` (*models*: List, *triples*, *er_vocab*: Dict[Tuple, List], *weights*: List[float] | None = None, *batch_size*: int = 512, *weighted_averaging*: bool = True, *normalize_scores*: bool = True, *, *tie_policy*: str = 'sort', *tie_seed*: int = 0) → Dict[str, float]

Evaluate link prediction performance of an ensemble of KGE models.

Combines predictions from multiple models using weighted or simple averaging, with optional score normalization.

Parameters

- **models** – List of KGE models (e.g., snapshots from training).
- **triples** – Test triples as numpy array or list, shape (N, 3), with integer indices (head, relation, tail).
- **er_vocab** – Mapping (head_idx, rel_idx) -> list of tail indices for filtered evaluation.
- **weights** – Weights for model averaging. Required if `weighted_averaging` is True. Must sum to 1 for proper averaging.
- **batch_size** – Batch size for processing triples.
- **weighted_averaging** – If True, use weighted averaging of predictions. If False, use simple mean.
- **normalize_scores** – If True, normalize scores to [0, 1] range per sample before averaging.
- **tie_policy** – Exact prediction ties: sort (legacy), optimistic, random, or pessimistic. Applied after combining model scores.
- **tie_seed** – Independent random tie seed; reset on each evaluation call.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

Raises

AssertionError – If `weighted_averaging` is `True` but weights are not provided or have wrong length.

Example

```
>>> from dicee.evaluation import evaluate_ensemble_link_prediction_performance
>>> models = [model1, model2, model3]
>>> weights = [0.5, 0.3, 0.2]
>>> results = evaluate_ensemble_link_prediction_performance(
...     models, test_triples, er_vocab,
...     weights=weights, weighted_averaging=True
... )
>>> print(f"MRR: {results['MRR']:.4f}")
```

`dicee.evaluation.evaluator`

Main Evaluator class for KGE model evaluation.

This module provides the Evaluator class which orchestrates evaluation of knowledge graph embedding models across different datasets and scoring techniques.

Attributes

logger

VALID_SCORING_TECHNIQUES

Classes

Evaluator

Evaluator class for KGE models in various downstream tasks.

Module Contents

`dicee.evaluation.evaluator.logger`

`dicee.evaluation.evaluator.VALID_SCORING_TECHNIQUES`

class `dicee.evaluation.evaluator.Evaluator` (*args*, *is_continual_training*: *bool* = *False*)

Evaluator class for KGE models in various downstream tasks.

Orchestrates link prediction evaluation with different scoring techniques including standard evaluation and byte-pair encoding based evaluation.

er_vocab

Entity-relation to tail vocabulary for filtered ranking.

re_vocab

Relation-entity (tail) to head vocabulary.

ee_vocab

Entity-entity to relation vocabulary.

num_entities

Total number of entities in the knowledge graph.

num_relations

Total number of relations in the knowledge graph.

args

Configuration arguments.

report

Dictionary storing evaluation results.

during_training

Whether evaluation is happening during training.

Example

```
>>> from dicee.evaluation import Evaluator
>>> evaluator = Evaluator(args)
>>> results = evaluator.eval(dataset, model, 'EntityPrediction')
>>> print(f"Test MRR: {results['Test']['MRR']:.4f}")
```

re_vocab: Dict | None = None

er_vocab: Dict | None = None

ee_vocab: Dict | None = None

func_triple_to_bpe_representation = None

is_continual_training = False

num_entities: int | None = None

num_relations: int | None = None

domain_constraints_per_rel = None

range_constraints_per_rel = None

args

report: Dict

during_training = False

vocab_preparation (*dataset*) → None

Prepare vocabularies from the dataset for evaluation.

Resolves any future objects and saves vocabularies to disk.

Parameters

dataset – Knowledge graph dataset with vocabulary attributes.

eval (*dataset*, *trained_model*, *form_of_labelling*: str, *during_training*: bool = False) → Dict | None

Evaluate the trained model on the dataset.

Parameters

- **dataset** – Knowledge graph dataset (KG instance).

- **trained_model** – The trained KGE model.
- **form_of_labelling** – Type of labelling ('EntityPrediction' or 'RelationPrediction').
- **during_training** – Whether evaluation is during training.

Returns

Dictionary of evaluation metrics, or None if evaluation is skipped.

eval_rank_of_head_and_tail_entity (*, *train_set*, *valid_set=None*, *test_set=None*, *trained_model*)
→ None

Evaluate with negative sampling scoring.

eval_rank_of_head_and_tail_byte_pair_encoded_entity (*, *train_set=None*, *valid_set=None*,
test_set=None, *ordered_bpe_entities*, *trained_model*) → None

Evaluate with BPE-encoded entities and negative sampling.

eval_with_byte (*, *raw_train_set*, *raw_valid_set=None*, *raw_test_set=None*, *trained_model*,
form_of_labelling) → None

Evaluate BytE model with generation.

eval_with_bpe_vs_all (*, *raw_train_set*, *raw_valid_set=None*, *raw_test_set=None*, *trained_model*,
form_of_labelling) → None

Evaluate with BPE and KvsAll scoring.

eval_with_vs_all (*, *train_set*, *valid_set=None*, *test_set=None*, *trained_model*, *form_of_labelling*)
→ None

Evaluate with KvsAll or 1vsAll scoring.

evaluate_lp_k_vs_all (*model*, *triple_idx*, *info: str | None = None*,
form_of_labelling: str | None = None) → Dict[str, float]

Filtered link prediction evaluation with KvsAll scoring.

Parameters

- **model** – The trained model to evaluate.
- **triple_idx** – Integer-indexed test triples.
- **info** – Description to print.
- **form_of_labelling** – 'EntityPrediction' or 'RelationPrediction'.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

evaluate_lp_with_byte (*model*, *triples: List[List[str]]*, *info: str | None = None*) → Dict[str, float]

Evaluate BytE model with text generation.

Parameters

- **model** – BytE model.
- **triples** – String triples.
- **info** – Description to print.

Returns

Dictionary with placeholder metrics (-1 values).

evaluate_lp_bpe_k_vs_all (*model*, *triples: List[List[str]]*, *info: str | None = None*,
form_of_labelling: str | None = None) → Dict[str, float]

Evaluate BPE model with KvsAll scoring.

Parameters

- **model** – BPE-enabled model.
- **triples** – String triples.
- **info** – Description to print.
- **form_of_labelling** – Type of labelling.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

evaluate_lp (*model, triple_idx, info: str*) → Dict[str, float]

Evaluate link prediction with negative sampling.

Parameters

- **model** – The model to evaluate.
- **triple_idx** – Integer-indexed triples.
- **info** – Description to print.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

dummy_eval (*trained_model, form_of_labelling: str*) → None

Run evaluation from saved data (for continual training).

Parameters

- **trained_model** – The trained model.
- **form_of_labelling** – Type of labelling.

eval_with_data (*dataset, trained_model, triple_idx: numpy.ndarray, form_of_labelling: str*)
→ Dict[str, float]

Evaluate a trained model on a given dataset.

Parameters

- **dataset** – Knowledge graph dataset.
- **trained_model** – The trained model.
- **triple_idx** – Integer-indexed triples to evaluate.
- **form_of_labelling** – Type of labelling.

Returns

Dictionary with evaluation metrics.

Raises

ValueError – If scoring technique is invalid.

dicee.evaluation.link_prediction

Link prediction evaluation functions.

This module provides various functions for evaluating link prediction performance of knowledge graph embedding models.

Attributes

`logger`

Functions

<code>evaluate_link_prediction_performance(→ Dict[str, float])</code>	Evaluate link prediction performance with head and tail prediction.
<code>evaluate_link_prediction_performance_with_reciprocals(→ Dict[str, float])</code>	Evaluate link prediction with reciprocal relations.
<code>evaluate_link_prediction_performance_with_reciprocals_and_bpe(→ Dict[str, float])</code>	Evaluate link prediction with BPE encoding and reciprocals.
<code>evaluate_link_prediction_performance_with_reciprocals_and_bpe_head_tail(→ Dict[str, float])</code>	Evaluate link prediction with BPE encoding (head and tail).
<code>evaluate_lp(→ Dict[str, float])</code>	Evaluate link prediction with batched processing.
<code>evaluate_bpe_lp(→ Dict[str, float])</code>	Evaluate link prediction with BPE-encoded entities.
<code>evaluate_lp_bpe_k_vs_all(→ Dict[str, float])</code>	Evaluate BPE link prediction with KvsAll scoring.

Module Contents

`dicee.evaluation.link_prediction.logger`

`dicee.evaluation.link_prediction.evaluate_link_prediction_performance(model, triples, er_vocab: Dict[Tuple, List], re_vocab: Dict[Tuple, List], *, tie_policy: str = 'sort', tie_seed: int = 0) → Dict[str, float]`

Evaluate link prediction performance with head and tail prediction.

Performs filtered evaluation where known correct answers are filtered out before computing ranks.

Parameters

- **model** – KGE model wrapper with entity/relation mappings.
- **triples** – Test triples as list of (head, relation, tail) strings.
- **er_vocab** – Mapping (entity, relation) -> list of valid tail entities.
- **re_vocab** – Mapping (relation, entity) -> list of valid head entities.
- **tie_policy** – How exact prediction ties are ranked: sort (legacy), optimistic, random, or pessimistic.
- **tie_seed** – Independent random tie seed; reset on each evaluation call.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

`dicee.evaluation.link_prediction.evaluate_link_prediction_performance_with_reciprocals(model, triples, er_vocab: Dict[Tuple, List], *, tie_policy: str = 'sort', tie_seed: int = 0) → Dict[str, float]`

Evaluate link prediction with reciprocal relations.

Optimized for models trained with reciprocal triples where only tail prediction is needed.

Parameters

- **model** – KGE model wrapper.
- **triples** – Test triples as list of (head, relation, tail) strings.

- **er_vocab** – Mapping (entity, relation) -> list of valid tail entities.
- **tie_policy** – How exact prediction ties are ranked: sort (legacy), optimistic, random, or pessimistic.
- **tie_seed** – Independent random tie seed; reset on each evaluation call.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

`dicee.evaluation.link_prediction.`

evaluate_link_prediction_performance_with_bpe_reciprocals (*model*,
within_entities: List[str], *triples*: List[List[str]], *er_vocab*: Dict[Tuple, List], *, *tie_policy*: str = 'sort',
tie_seed: int = 0) → Dict[str, float]

Evaluate link prediction with BPE encoding and reciprocals.

Parameters

- **model** – KGE model wrapper with BPE support.
- **within_entities** – List of entities to evaluate within.
- **triples** – Test triples as list of [head, relation, tail] strings.
- **er_vocab** – Mapping (entity, relation) -> list of valid tail entities.
- **tie_policy** – How exact prediction ties are ranked: sort (legacy), optimistic, random, or pessimistic.
- **tie_seed** – Independent random tie seed; reset on each evaluation call.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

`dicee.evaluation.link_prediction.evaluate_link_prediction_performance_with_bpe` (
model, *within_entities*: List[str], *triples*: List[Tuple[str]], *er_vocab*: Dict[Tuple, List],
re_vocab: Dict[Tuple, List], *, *tie_policy*: str = 'sort', *tie_seed*: int = 0) → Dict[str, float]

Evaluate link prediction with BPE encoding (head and tail).

Parameters

- **model** – KGE model wrapper with BPE support.
- **within_entities** – List of entities to evaluate within.
- **triples** – Test triples as list of (head, relation, tail) tuples.
- **er_vocab** – Mapping (entity, relation) -> list of valid tail entities.
- **re_vocab** – Mapping (relation, entity) -> list of valid head entities.
- **tie_policy** – How exact prediction ties are ranked: sort (legacy), optimistic, random, or pessimistic.
- **tie_seed** – Independent random tie seed; reset on each evaluation call.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

`dicee.evaluation.link_prediction.evaluate_lp` (*model*, *triple_idx*, *num_entities*: int,
er_vocab: Dict[Tuple, List], *re_vocab*: Dict[Tuple, List], *info*: str = 'Eval Starts',
batch_size: int = 128, *chunk_size*: int = 1000, *, *tie_policy*: str = 'sort', *tie_seed*: int = 0,
tie_generator: torch.Generator | None = None,
graph_rank_plan: dicee.evaluation._graph_inference.GraphRankPlan | None = None,
reuse_graph_queries: bool = True) → Dict[str, float]

Evaluate link prediction with batched processing.

Memory-efficient evaluation using chunked entity scoring.

Parameters

- **model** – The KGE model to evaluate.
- **triple_idx** – Integer-indexed triples as numpy array.
- **num_entities** – Total number of entities.
- **er_vocab** – Mapping (head_idx, rel_idx) -> list of tail indices.
- **re_vocab** – Mapping (rel_idx, tail_idx) -> list of head indices.
- **info** – Description to print.
- **batch_size** – Batch size for triple processing.
- **chunk_size** – Chunk size for entity scoring.
- **tie_generator** – Optional CPU generator to continue random ties across calls. When supplied, its state takes precedence over tie_seed.
- **graph_rank_plan** – Optional whole-split plan shared across progress batches.
- **reuse_graph_queries** – Reuse deterministic ULTRA/TRIX queries. Flock keeps its original query and sampling order regardless of this option.
- **tie_policy** – How exact prediction ties are ranked: sort (legacy), optimistic, random, or pessimistic.
- **tie_seed** – Independent random tie seed; reset on each evaluation call.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

```
dicee.evaluation.link_prediction.evaluate_bpe_lp(model, triple_idx: List[Tuple],  
all_bpe_shaped_entities, er_vocab: Dict[Tuple, List], re_vocab: Dict[Tuple, List],  
info: str = 'Eval Starts', *, tie_policy: str = 'sort', tie_seed: int = 0) → Dict[str, float]
```

Evaluate link prediction with BPE-encoded entities.

Parameters

- **model** – The KGE model to evaluate.
- **triple_idx** – List of BPE-encoded triple tuples.
- **all_bpe_shaped_entities** – All entities with BPE representations.
- **er_vocab** – Mapping for tail filtering.
- **re_vocab** – Mapping for head filtering.
- **info** – Description to print.
- **tie_policy** – How exact prediction ties are ranked: sort (legacy), optimistic, random, or pessimistic.
- **tie_seed** – Independent random tie seed; reset on each evaluation call.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

```
dicee.evaluation.link_prediction.evaluate_lp_bpe_k_vs_all(model, triples: List[List[str]],
    er_vocab: Dict | None = None, batch_size: int | None = None,
    func_triple_to_bpe_representation: Callable | None = None,
    str_to_bpe_entity_to_idx: Dict | None = None, *, tie_policy: str = 'sort', tie_seed: int = 0)
    → Dict[str, float]
```

Evaluate BPE link prediction with KvsAll scoring.

Parameters

- **model** – The KGE model wrapper.
- **triples** – List of string triples.
- **er_vocab** – Entity-relation vocabulary for filtering.
- **batch_size** – Batch size for processing.
- **func_triple_to_bpe_representation** – Function to convert triples to BPE.
- **str_to_bpe_entity_to_idx** – Mapping from string entities to BPE indices.
- **tie_policy** – How exact prediction ties are ranked: sort (legacy), optimistic, random, or pessimistic.
- **tie_seed** – Independent random tie seed; reset on each evaluation call.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

Raises

ValueError – If batch_size is not provided.

dicee.evaluation.literal_prediction

Literal prediction evaluation functions.

This module provides functions for evaluating literal/attribute prediction performance of knowledge graph embedding models.

Attributes

logger

Functions

<i>evaluate_literal_prediction</i> (→ Optional[pandas.DataFrame])	Evaluate trained literal prediction model on a test file.
---	---

Module Contents

dicee.evaluation.literal_prediction.**logger**

```
dicee.evaluation.literal_prediction.evaluate_literal_prediction(kge_model,
    eval_file_path: str = None, store_lit_preds: bool = True, eval_literals: bool = True,
    loader_backend: str = 'pandas', return_attr_error_metrics: bool = False)
    → pandas.DataFrame | None
```

Evaluate trained literal prediction model on a test file.

Evaluates the literal prediction capabilities of a KGE model by computing MAE and RMSE metrics for each attribute.

Parameters

- **kge_model** – Trained KGE model with literal prediction capability.
- **eval_file_path** – Path to the evaluation file containing test literals.
- **store_lit_preds** – If True, stores predictions to CSV file.
- **eval_literals** – If True, evaluates and prints error metrics.
- **loader_backend** – Backend for loading dataset ('pandas' or 'rdflib').
- **return_attr_error_metrics** – If True, returns the metrics DataFrame.

Returns

DataFrame with per-attribute MAE and RMSE if `return_attr_error_metrics` is True, otherwise None.

Raises

- **RuntimeError** – If the KGE model doesn't have a trained literal model.
- **AssertionError** – If model is invalid or test set has no valid data.

Example

```
>>> from dicee import KGE
>>> from dicee.evaluation import evaluate_literal_prediction
>>> model = KGE(path="pretrained_model")
>>> metrics = evaluate_literal_prediction(
...     model,
...     eval_file_path="test_literals.csv",
...     return_attr_error_metrics=True
... )
>>> print(metrics)
```

dicee.evaluation.utils

Utility functions for evaluation module.

This module contains shared helper functions used across different evaluation components.

Attributes

`DEFAULT_HITS_RANGE`

`ALL_HITS_RANGE`

Functions

<code>make_iterable_verbose(→ Iterable)</code>	Wrap an iterable with tqdm progress bar if verbose is True.
<code>compute_metrics_from_ranks(→ Dict[str, float])</code>	Compute standard link prediction metrics from ranks.
<code>compute_metrics_from_ranks_simple(→ Dict[str, float])</code>	Compute link prediction metrics without scaling factor.
<code>update_hits(→ None)</code>	Update hits dictionary based on rank.
<code>create_hits_dict(→ Dict[int, List[float]])</code>	Create an initialized hits dictionary.
<code>efficient_zero_grad(→ None)</code>	Efficiently zero gradients using <code>parameter.grad = None</code> .

Module Contents

`dicee.evaluation.utils.DEFAULT_HITS_RANGE: List[int] = [1, 3, 10]`

`dicee.evaluation.utils.ALL_HITS_RANGE: List[int]`

`dicee.evaluation.utils.make_iterable_verbose` (*iterable_object: Iterable, verbose: bool, desc: str = 'Default', position: int | None = None, leave: bool = True*) → Iterable

Wrap an iterable with tqdm progress bar if verbose is True.

Parameters

- **iterable_object** – The iterable to potentially wrap.
- **verbose** – Whether to show progress bar.
- **desc** – Description for the progress bar.
- **position** – Position of the progress bar.
- **leave** – Whether to leave the progress bar after completion.

Returns

The original iterable or a tqdm-wrapped version.

`dicee.evaluation.utils.compute_metrics_from_ranks` (*ranks: List[int], num_triples: int, hits_dict: Dict[int, List[float]], scale_factor: int = 1*) → Dict[str, float]

Compute standard link prediction metrics from ranks.

Parameters

- **ranks** – List of ranks for each prediction.
- **num_triples** – Total number of triples evaluated.
- **hits_dict** – Dictionary mapping hit levels to lists of hits.
- **scale_factor** – Factor to scale the denominator (e.g., 2 for head+tail).

Returns

Dictionary containing H@1, H@3, **H@10**, and MRR metrics.

`dicee.evaluation.utils.compute_metrics_from_ranks_simple` (*ranks: List[int], num_triples: int, hits_dict: Dict[int, List[float]]*) → Dict[str, float]

Compute link prediction metrics without scaling factor.

Parameters

- **ranks** – List of ranks for each prediction.
- **num_triples** – Total number of triples evaluated.
- **hits_dict** – Dictionary mapping hit levels to lists of hits.

Returns

Dictionary containing H@1, H@3, H@10, and MRR metrics.

```
dicee.evaluation.utils.update_hits (hits: Dict[int, List[float]], rank: int,  
hits_range: List[int] | None = None) → None
```

Update hits dictionary based on rank.

Parameters

- **hits** – Dictionary to update in-place.
- **rank** – The rank to check against hit levels.
- **hits_range** – List of hit levels to check (default: ALL_HITS_RANGE).

```
dicee.evaluation.utils.create_hits_dict (hits_range: List[int] | None = None)  
→ Dict[int, List[float]]
```

Create an initialized hits dictionary.

Parameters

hits_range – List of hit levels to initialize (default: ALL_HITS_RANGE).

Returns

Dictionary with empty lists for each hit level.

```
dicee.evaluation.utils.efficient_zero_grad (model) → None
```

Efficiently zero gradients using parameter.grad = None.

This is more efficient than optimizer.zero_grad() as it avoids memory operations.

See: https://pytorch.org/tutorials/recipes/recipes/tuning_guide.html

Parameters

model – PyTorch model to zero gradients for.

Classes

<i>Evaluator</i>	Evaluator class for KGE models in various downstream tasks.
------------------	---

Functions

<i>evaluate_ensemble_link_prediction_performance</i>	Evaluate link prediction performance of an ensemble of KGE models.
<i>evaluate_bpe_lp</i> (→ Dict[str, float])	Evaluate link prediction with BPE-encoded entities.
<i>evaluate_link_prediction_performance</i> (→ Dict[str, float])	Evaluate link prediction performance with head and tail prediction.
<i>evaluate_link_prediction_performance_with_j</i>	Evaluate link prediction with BPE encoding (head and tail).
<i>evaluate_link_prediction_performance_with_j</i>	Evaluate link prediction with BPE encoding and reciprocals.
<i>evaluate_link_prediction_performance_with_</i>	Evaluate link prediction with reciprocal relations.
<i>evaluate_lp</i> (→ Dict[str, float])	Evaluate link prediction with batched processing.
<i>evaluate_lp_bpe_k_vs_all</i> (→ Dict[str, float])	Evaluate BPE link prediction with KvsAll scoring.

continues on next page

Table 23 – continued from previous page

<code>evaluate_literal_prediction(→ Optional[pandas.DataFrame])</code>	Evaluate trained literal prediction model on a test file.
--	---

Package Contents

`dicee.evaluation.evaluate_ensemble_link_prediction_performance` (*models: List, triples, er_vocab: Dict[Tuple, List], weights: List[float] | None = None, batch_size: int = 512, weighted_averaging: bool = True, normalize_scores: bool = True, *, tie_policy: str = 'sort', tie_seed: int = 0*) → Dict[str, float]

Evaluate link prediction performance of an ensemble of KGE models.

Combines predictions from multiple models using weighted or simple averaging, with optional score normalization.

Parameters

- **models** – List of KGE models (e.g., snapshots from training).
- **triples** – Test triples as numpy array or list, shape (N, 3), with integer indices (head, relation, tail).
- **er_vocab** – Mapping (head_idx, rel_idx) -> list of tail indices for filtered evaluation.
- **weights** – Weights for model averaging. Required if `weighted_averaging` is `True`. Must sum to 1 for proper averaging.
- **batch_size** – Batch size for processing triples.
- **weighted_averaging** – If `True`, use weighted averaging of predictions. If `False`, use simple mean.
- **normalize_scores** – If `True`, normalize scores to [0, 1] range per sample before averaging.
- **tie_policy** – Exact prediction ties: sort (legacy), optimistic, random, or pessimistic. Applied after combining model scores.
- **tie_seed** – Independent random tie seed; reset on each evaluation call.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

Raises

AssertionError – If `weighted_averaging` is `True` but weights are not provided or have wrong length.

Example

```
>>> from dicee.evaluation import evaluate_ensemble_link_prediction_performance
>>> models = [model1, model2, model3]
>>> weights = [0.5, 0.3, 0.2]
>>> results = evaluate_ensemble_link_prediction_performance(
...     models, test_triples, er_vocab,
...     weights=weights, weighted_averaging=True
... )
>>> print(f"MRR: {results['MRR']:.4f}")
```

class `dicee.evaluation.Evaluator` (*args, is_continual_training: bool = False*)

Evaluator class for KGE models in various downstream tasks.

Orchestrates link prediction evaluation with different scoring techniques including standard evaluation and byte-pair encoding based evaluation.

er_vocab

Entity-relation to tail vocabulary for filtered ranking.

re_vocab

Relation-entity (tail) to head vocabulary.

ee_vocab

Entity-entity to relation vocabulary.

num_entities

Total number of entities in the knowledge graph.

num_relations

Total number of relations in the knowledge graph.

args

Configuration arguments.

report

Dictionary storing evaluation results.

during_training

Whether evaluation is happening during training.

Example

```
>>> from dicee.evaluation import Evaluator
>>> evaluator = Evaluator(args)
>>> results = evaluator.eval(dataset, model, 'EntityPrediction')
>>> print(f"Test MRR: {results['Test']['MRR']:.4f}")
```

re_vocab: Dict | None = None

er_vocab: Dict | None = None

ee_vocab: Dict | None = None

func_triple_to_bpe_representation = None

is_continual_training = False

num_entities: int | None = None

num_relations: int | None = None

domain_constraints_per_rel = None

range_constraints_per_rel = None

args

report: Dict

during_training = False

vocab_preparation (*dataset*) → None

Prepare vocabularies from the dataset for evaluation.

Resolves any future objects and saves vocabularies to disk.

Parameters

dataset – Knowledge graph dataset with vocabulary attributes.

eval (*dataset, trained_model, form_of_labelling: str, during_training: bool = False*) → Dict | None

Evaluate the trained model on the dataset.

Parameters

- **dataset** – Knowledge graph dataset (KG instance).
- **trained_model** – The trained KGE model.
- **form_of_labelling** – Type of labelling ('EntityPrediction' or 'RelationPrediction').
- **during_training** – Whether evaluation is during training.

Returns

Dictionary of evaluation metrics, or None if evaluation is skipped.

eval_rank_of_head_and_tail_entity (*, *train_set, valid_set=None, test_set=None, trained_model*)
→ None

Evaluate with negative sampling scoring.

eval_rank_of_head_and_tail_byte_pair_encoded_entity (*, *train_set=None, valid_set=None, test_set=None, ordered_bpe_entities, trained_model*) → None

Evaluate with BPE-encoded entities and negative sampling.

eval_with_byte (*, *raw_train_set, raw_valid_set=None, raw_test_set=None, trained_model, form_of_labelling*) → None

Evaluate Byte model with generation.

eval_with_bpe_vs_all (*, *raw_train_set, raw_valid_set=None, raw_test_set=None, trained_model, form_of_labelling*) → None

Evaluate with BPE and KvsAll scoring.

eval_with_vs_all (*, *train_set, valid_set=None, test_set=None, trained_model, form_of_labelling*)
→ None

Evaluate with KvsAll or 1vsAll scoring.

evaluate_lp_k_vs_all (*model, triple_idx, info: str | None = None, form_of_labelling: str | None = None*) → Dict[str, float]

Filtered link prediction evaluation with KvsAll scoring.

Parameters

- **model** – The trained model to evaluate.
- **triple_idx** – Integer-indexed test triples.
- **info** – Description to print.
- **form_of_labelling** – 'EntityPrediction' or 'RelationPrediction'.

Returns

Dictionary with H@1, H@3, H@10, and MRR metrics.

evaluate_lp_with_byte (*model*, *triples*: List[List[str]], *info*: str | None = None) → Dict[str, float]

Evaluate ByteE model with text generation.

Parameters

- **model** – ByteE model.
- **triples** – String triples.
- **info** – Description to print.

Returns

Dictionary with placeholder metrics (-1 values).

evaluate_lp_bpe_k_vs_all (*model*, *triples*: List[List[str]], *info*: str | None = None, *form_of_labelling*: str | None = None) → Dict[str, float]

Evaluate BPE model with KvsAll scoring.

Parameters

- **model** – BPE-enabled model.
- **triples** – String triples.
- **info** – Description to print.
- **form_of_labelling** – Type of labelling.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

evaluate_lp (*model*, *triple_idx*, *info*: str) → Dict[str, float]

Evaluate link prediction with negative sampling.

Parameters

- **model** – The model to evaluate.
- **triple_idx** – Integer-indexed triples.
- **info** – Description to print.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

dummy_eval (*trained_model*, *form_of_labelling*: str) → None

Run evaluation from saved data (for continual training).

Parameters

- **trained_model** – The trained model.
- **form_of_labelling** – Type of labelling.

eval_with_data (*dataset*, *trained_model*, *triple_idx*: numpy.ndarray, *form_of_labelling*: str) → Dict[str, float]

Evaluate a trained model on a given dataset.

Parameters

- **dataset** – Knowledge graph dataset.
- **trained_model** – The trained model.
- **triple_idx** – Integer-indexed triples to evaluate.
- **form_of_labelling** – Type of labelling.

Returns

Dictionary with evaluation metrics.

Raises

ValueError – If scoring technique is invalid.

```
dicee.evaluation.evaluate_bpe_lp(model, triple_idx: List[Tuple], all_bpe_shaped_entities,  
    er_vocab: Dict[Tuple, List], re_vocab: Dict[Tuple, List], info: str = 'Eval Starts', *,  
    tie_policy: str = 'sort', tie_seed: int = 0) → Dict[str, float]
```

Evaluate link prediction with BPE-encoded entities.

Parameters

- **model** – The KGE model to evaluate.
- **triple_idx** – List of BPE-encoded triple tuples.
- **all_bpe_shaped_entities** – All entities with BPE representations.
- **er_vocab** – Mapping for tail filtering.
- **re_vocab** – Mapping for head filtering.
- **info** – Description to print.
- **tie_policy** – How exact prediction ties are ranked: sort (legacy), optimistic, random, or pessimistic.
- **tie_seed** – Independent random tie seed; reset on each evaluation call.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

```
dicee.evaluation.evaluate_link_prediction_performance(model, triples,  
    er_vocab: Dict[Tuple, List], re_vocab: Dict[Tuple, List], *, tie_policy: str = 'sort', tie_seed: int = 0)  
→ Dict[str, float]
```

Evaluate link prediction performance with head and tail prediction.

Performs filtered evaluation where known correct answers are filtered out before computing ranks.

Parameters

- **model** – KGE model wrapper with entity/relation mappings.
- **triples** – Test triples as list of (head, relation, tail) strings.
- **er_vocab** – Mapping (entity, relation) -> list of valid tail entities.
- **re_vocab** – Mapping (relation, entity) -> list of valid head entities.
- **tie_policy** – How exact prediction ties are ranked: sort (legacy), optimistic, random, or pessimistic.
- **tie_seed** – Independent random tie seed; reset on each evaluation call.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

```
dicee.evaluation.evaluate_link_prediction_performance_with_bpe(model,  
    within_entities: List[str], triples: List[Tuple[str]], er_vocab: Dict[Tuple, List],  
    re_vocab: Dict[Tuple, List], *, tie_policy: str = 'sort', tie_seed: int = 0) → Dict[str, float]
```

Evaluate link prediction with BPE encoding (head and tail).

Parameters

- **model** – KGE model wrapper with BPE support.

- **within_entities** – List of entities to evaluate within.
- **triples** – Test triples as list of (head, relation, tail) tuples.
- **er_vocab** – Mapping (entity, relation) -> list of valid tail entities.
- **re_vocab** – Mapping (relation, entity) -> list of valid head entities.
- **tie_policy** – How exact prediction ties are ranked: sort (legacy), optimistic, random, or pessimistic.
- **tie_seed** – Independent random tie seed; reset on each evaluation call.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

```
dicee.evaluation.evaluate_link_prediction_performance_with_bpe_reciprocals(model,
    within_entities: List[str], triples: List[List[str]], er_vocab: Dict[Tuple, List], *, tie_policy: str = 'sort',
    tie_seed: int = 0) → Dict[str, float]
```

Evaluate link prediction with BPE encoding and reciprocals.

Parameters

- **model** – KGE model wrapper with BPE support.
- **within_entities** – List of entities to evaluate within.
- **triples** – Test triples as list of [head, relation, tail] strings.
- **er_vocab** – Mapping (entity, relation) -> list of valid tail entities.
- **tie_policy** – How exact prediction ties are ranked: sort (legacy), optimistic, random, or pessimistic.
- **tie_seed** – Independent random tie seed; reset on each evaluation call.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

```
dicee.evaluation.evaluate_link_prediction_performance_with_reciprocals(model, triples,
    er_vocab: Dict[Tuple, List], *, tie_policy: str = 'sort', tie_seed: int = 0) → Dict[str, float]
```

Evaluate link prediction with reciprocal relations.

Optimized for models trained with reciprocal triples where only tail prediction is needed.

Parameters

- **model** – KGE model wrapper.
- **triples** – Test triples as list of (head, relation, tail) strings.
- **er_vocab** – Mapping (entity, relation) -> list of valid tail entities.
- **tie_policy** – How exact prediction ties are ranked: sort (legacy), optimistic, random, or pessimistic.
- **tie_seed** – Independent random tie seed; reset on each evaluation call.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

```
dicee.evaluation.evaluate_lp(model, triple_idx, num_entities: int, er_vocab: Dict[Tuple, List],
    re_vocab: Dict[Tuple, List], info: str = 'Eval Starts', batch_size: int = 128, chunk_size: int = 1000,
    *, tie_policy: str = 'sort', tie_seed: int = 0, tie_generator: torch.Generator | None = None,
    graph_rank_plan: dicee.evaluation._graph_inference.GraphRankPlan | None = None,
    reuse_graph_queries: bool = True) → Dict[str, float]
```


Evaluate link prediction with batched processing.

Memory-efficient evaluation using chunked entity scoring.

Parameters

- **model** – The KGE model to evaluate.
- **triple_idx** – Integer-indexed triples as numpy array.
- **num_entities** – Total number of entities.
- **er_vocab** – Mapping (head_idx, rel_idx) -> list of tail indices.
- **re_vocab** – Mapping (rel_idx, tail_idx) -> list of head indices.
- **info** – Description to print.
- **batch_size** – Batch size for triple processing.
- **chunk_size** – Chunk size for entity scoring.
- **tie_generator** – Optional CPU generator to continue random ties across calls. When supplied, its state takes precedence over tie_seed.
- **graph_rank_plan** – Optional whole-split plan shared across progress batches.
- **reuse_graph_queries** – Reuse deterministic ULTRA/TRIX queries. Flock keeps its original query and sampling order regardless of this option.
- **tie_policy** – How exact prediction ties are ranked: sort (legacy), optimistic, random, or pessimistic.
- **tie_seed** – Independent random tie seed; reset on each evaluation call.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

```
dicee.evaluation.evaluate_lp_bpe_k_vs_all(model, triples: List[List[str]],
    er_vocab: Dict | None = None, batch_size: int | None = None,
    func_triple_to_bpe_representation: Callable | None = None,
    str_to_bpe_entity_to_idx: Dict | None = None, *, tie_policy: str = 'sort', tie_seed: int = 0)
    → Dict[str, float]
```

Evaluate BPE link prediction with KvsAll scoring.

Parameters

- **model** – The KGE model wrapper.
- **triples** – List of string triples.
- **er_vocab** – Entity-relation vocabulary for filtering.
- **batch_size** – Batch size for processing.
- **func_triple_to_bpe_representation** – Function to convert triples to BPE.
- **str_to_bpe_entity_to_idx** – Mapping from string entities to BPE indices.
- **tie_policy** – How exact prediction ties are ranked: sort (legacy), optimistic, random, or pessimistic.
- **tie_seed** – Independent random tie seed; reset on each evaluation call.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

Raises

ValueError – If `batch_size` is not provided.

`dicee.evaluation.evaluate_literal_prediction(kge_model, eval_file_path: str = None, store_lit_preds: bool = True, eval_literals: bool = True, loader_backend: str = 'pandas', return_attr_error_metrics: bool = False) → pandas.DataFrame | None`

Evaluate trained literal prediction model on a test file.

Evaluates the literal prediction capabilities of a KGE model by computing MAE and RMSE metrics for each attribute.

Parameters

- **kge_model** – Trained KGE model with literal prediction capability.
- **eval_file_path** – Path to the evaluation file containing test literals.
- **store_lit_preds** – If True, stores predictions to CSV file.
- **eval_literals** – If True, evaluates and prints error metrics.
- **loader_backend** – Backend for loading dataset ('pandas' or 'rdflib').
- **return_attr_error_metrics** – If True, returns the metrics DataFrame.

Returns

DataFrame with per-attribute MAE and RMSE if `return_attr_error_metrics` is True, otherwise None.

Raises

- **RuntimeError** – If the KGE model doesn't have a trained literal model.
- **AssertionError** – If model is invalid or test set has no valid data.

Example

```
>>> from dicee import KGE
>>> from dicee.evaluation import evaluate_literal_prediction
>>> model = KGE(path="pretrained_model")
>>> metrics = evaluate_literal_prediction(
...     model,
...     eval_file_path="test_literals.csv",
...     return_attr_error_metrics=True
... )
>>> print(metrics)
```

`dicee.evaluator`

Evaluator module for knowledge graph embedding models.

This module provides backward compatibility by re-exporting from the new `dicee.evaluation` module.

Deprecated since version Use: `dicee.evaluation.Evaluator` instead. This module will be removed in a future version.

Classes

<i>Evaluator</i>	Evaluator class for KGE models in various downstream tasks.
------------------	---

Module Contents

class `dicee.evaluator.Evaluator` (*args*, *is_continual_training*: *bool* = *False*)

Evaluator class for KGE models in various downstream tasks.

Orchestrates link prediction evaluation with different scoring techniques including standard evaluation and byte-pair encoding based evaluation.

er_vocab

Entity-relation to tail vocabulary for filtered ranking.

re_vocab

Relation-entity (tail) to head vocabulary.

ee_vocab

Entity-entity to relation vocabulary.

num_entities

Total number of entities in the knowledge graph.

num_relations

Total number of relations in the knowledge graph.

args

Configuration arguments.

report

Dictionary storing evaluation results.

during_training

Whether evaluation is happening during training.

Example

```
>>> from dicee.evaluation import Evaluator
>>> evaluator = Evaluator(args)
>>> results = evaluator.eval(dataset, model, 'EntityPrediction')
>>> print(f"Test MRR: {results['Test']['MRR']:.4f}")
```

re_vocab: Dict | None = None

er_vocab: Dict | None = None

ee_vocab: Dict | None = None

func_triple_to_bpe_representation = None

is_continual_training = False

num_entities: int | None = None

num_relations: int | None = None

domain_constraints_per_rel = None

range_constraints_per_rel = None

args

report: Dict

during_training = False

vocab_preparation (*dataset*) → None

Prepare vocabularies from the dataset for evaluation.

Resolves any future objects and saves vocabularies to disk.

Parameters

dataset – Knowledge graph dataset with vocabulary attributes.

eval (*dataset, trained_model, form_of_labelling: str, during_training: bool = False*) → Dict | None

Evaluate the trained model on the dataset.

Parameters

- **dataset** – Knowledge graph dataset (KG instance).
- **trained_model** – The trained KGE model.
- **form_of_labelling** – Type of labelling ('EntityPrediction' or 'RelationPrediction').
- **during_training** – Whether evaluation is during training.

Returns

Dictionary of evaluation metrics, or None if evaluation is skipped.

eval_rank_of_head_and_tail_entity (*, *train_set, valid_set=None, test_set=None, trained_model*)
→ None

Evaluate with negative sampling scoring.

eval_rank_of_head_and_tail_byte_pair_encoded_entity (*, *train_set=None, valid_set=None, test_set=None, ordered_bpe_entities, trained_model*) → None

Evaluate with BPE-encoded entities and negative sampling.

eval_with_byte (*, *raw_train_set, raw_valid_set=None, raw_test_set=None, trained_model, form_of_labelling*) → None

Evaluate Byte model with generation.

eval_with_bpe_vs_all (*, *raw_train_set, raw_valid_set=None, raw_test_set=None, trained_model, form_of_labelling*) → None

Evaluate with BPE and KvsAll scoring.

eval_with_vs_all (*, *train_set, valid_set=None, test_set=None, trained_model, form_of_labelling*)
→ None

Evaluate with KvsAll or 1vsAll scoring.

evaluate_lp_k_vs_all (*model, triple_idx, info: str | None = None, form_of_labelling: str | None = None*) → Dict[str, float]

Filtered link prediction evaluation with KvsAll scoring.

Parameters

- **model** – The trained model to evaluate.

- **triple_idx** – Integer-indexed test triples.
- **info** – Description to print.
- **form_of_labelling** – ‘EntityPrediction’ or ‘RelationPrediction’.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

evaluate_lp_with_byte (*model, triples: List[List[str]], info: str | None = None*) → Dict[str, float]

Evaluate Byte model with text generation.

Parameters

- **model** – Byte model.
- **triples** – String triples.
- **info** – Description to print.

Returns

Dictionary with placeholder metrics (-1 values).

evaluate_lp_bpe_k_vs_all (*model, triples: List[List[str]], info: str | None = None, form_of_labelling: str | None = None*) → Dict[str, float]

Evaluate BPE model with KvsAll scoring.

Parameters

- **model** – BPE-enabled model.
- **triples** – String triples.
- **info** – Description to print.
- **form_of_labelling** – Type of labelling.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

evaluate_lp (*model, triple_idx, info: str*) → Dict[str, float]

Evaluate link prediction with negative sampling.

Parameters

- **model** – The model to evaluate.
- **triple_idx** – Integer-indexed triples.
- **info** – Description to print.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

dummy_eval (*trained_model, form_of_labelling: str*) → None

Run evaluation from saved data (for continual training).

Parameters

- **trained_model** – The trained model.
- **form_of_labelling** – Type of labelling.

eval_with_data (*dataset, trained_model, triple_idx: numpy.ndarray, form_of_labelling: str*)
→ Dict[str, float]

Evaluate a trained model on a given dataset.

Parameters

- **dataset** – Knowledge graph dataset.
- **trained_model** – The trained model.
- **triple_idx** – Integer-indexed triples to evaluate.
- **form_of_labelling** – Type of labelling.

Returns

Dictionary with evaluation metrics.

Raises

ValueError – If scoring technique is invalid.

dicee.executer

Executor module for training, retraining and evaluating KGE models.

This module provides the Execute and ContinuousExecute classes for managing the full lifecycle of knowledge graph embedding model training.

Attributes

<i>logger</i>

Classes

<i>Execute</i>	Executor class for training, retraining and evaluating KGE models.
<i>ContinuousExecute</i>	A subclass of Execute Class for retraining

Module Contents

`dicee.executer.logger`

class `dicee.executer.Execute` (*args, continuous_training: bool = False*)

Executor class for training, retraining and evaluating KGE models.

Handles the complete workflow: 1. Loading & Preprocessing & Serializing input data 2. Training & Validation & Testing 3. Storing all necessary information

args

Processed input arguments.

distributed

Whether distributed training is enabled.

rank

Process rank in distributed training.

world_size
Total number of processes.

local_rank
Local GPU rank.

trainer
Training handler instance.

trained_model
The trained model after training completes.

knowledge_graph
The loaded knowledge graph.

report
Dictionary storing training metrics and results.

evaluator
Model evaluation handler.

distributed

rank

world_size

local_rank

args

is_continual_training = False

trainer: *dicее.trainer.DICE_Trainer* | None = None

trained_model = None

knowledge_graph: *dicее.knowledge_graph.KG* | None = None

report: Dict

evaluator: *dicее.evaluator.Evaluator* | None = None

start_time: float | None = None

is_local_rank_zero() → bool

is_global_rank_zero() → bool

cleanup()

setup_executor() → None
Set up storage directories for the experiment.

Creates or reuses experiment directories based on configuration. Saves the configuration to a JSON file.

create_and_store_kg() → None
Create knowledge graph and store as memory-mapped file.

Only executed on global rank 0 in distributed training, so exactly one process writes the shared memmap/vocab files regardless of how many nodes are involved. Skips if memmap already exists.

load_from_memmap () → None

Load knowledge graph from memory-mapped file.

save_trained_model () → None

Save a knowledge graph embedding model

- (1) Send model to eval mode and cpu.
- (2) Store the memory footprint of the model.
- (3) Save the model into disk.
- (4) Update the stats of KG again ?

Parameter

rtype

None

end (*form_of_labelling: str*) → dict

End training

- (1) Store trained model.
- (2) Report runtimes.
- (3) Eval model if required.

Parameter

rtype

A dict containing information about the training and/or evaluation

write_report () → None

Report training related information in a report.json file

start () → dict

Start training

(1) Loading the Data # (2) Create an evaluator object. # (3) Create a trainer object. # (4) Start the training

Parameter

rtype

A dict containing information about the training and/or evaluation

class `dicee.executer.ContinuousExecute` (*args*)

Bases: `Execute`

A subclass of Execute Class for retraining

- (1) Loading & Preprocessing & Serializing input data.
- (2) Training & Validation & Testing
- (3) Storing all necessary info

During the continual learning we can only modify * **num_epochs** * parameter. Trained model stored in the same folder as the seed model for the training. Trained model is noted with the current time.

continual_start() → dict

Start Continual Training

- (1) Initialize training.
- (2) Start continual training.
- (3) Save trained model.

Parameter

rtype

A dict containing information about the training and/or evaluation

dicee.knowledge_graph

Knowledge Graph module for data loading and preprocessing.

Provides the KG class for handling knowledge graph data including loading, preprocessing, and indexing operations.

Classes

<i>KG</i>	Knowledge Graph container and processor.
-----------	--

Module Contents

```
class dicee.knowledge_graph.KG (dataset_dir: str | None = None, byte_pair_encoding: bool = False,  
padding: bool = False, add_noise_rate: float | None = None, sparql_endpoint: str | None = None,  
path_single_kg: str | None = None, path_for_deserialization: str | None = None,  
add_reciprocal: bool | None = None, eval_model: str | None = None,  
read_only_few: int | None = None, sample_triples_ratio: float | None = None,  
path_for_serialization: str | None = None, entity_to_idx: Dict | None = None,  
relation_to_idx: Dict | None = None, backend: str | None = None,  
training_technique: str | None = None, separator: str | None = None)
```

Knowledge Graph container and processor.

Handles loading, preprocessing, and indexing of knowledge graph data from various sources including files, SPARQL endpoints, and serialized formats.

dataset_dir

Path to directory containing train/valid/test files.

num_entities

Total number of unique entities.

num_relations

Total number of unique relations.

train_set

Indexed training triples as numpy array.

valid_set

Indexed validation triples (optional).

test_set

Indexed test triples (optional).

```

entity_to_idx
    Mapping from entity strings to indices.
relation_to_idx
    Mapping from relation strings to indices.
dataset_dir = None
sparql_endpoint = None
path_single_kg = None
byte_pair_encoding = False
ordered_shaped_bpe_tokens = None
add_noise_rate = None
num_entities: int | None = None
num_relations: int | None = None
path_for_deserialization = None
add_reciprocal = None
eval_model = None
read_only_few = None
sample_triples_ratio = None
path_for_serialization = None
entity_to_idx = None
relation_to_idx = None
backend = 'pandas'
training_technique = None
separator = None
raw_train_set = None
raw_valid_set = None
raw_test_set = None
train_set = None
valid_set = None
test_set = None
idx_entity_to_bpe_shaped: Dict
enc
num_tokens

```

```

num_bpe_entities: int | None = None

padding = False

dummy_id

max_length_subword_tokens: int | None = None

train_set_target = None

target_dim: int | None = None

train_target_indices = None

ordered_bpe_entities = None

description_of_input = None

describe() → None
    Generate a description string of the dataset statistics.

property_entities_str: List[str]
    Get list of all entity strings.

property_relations_str: List[str]
    Get list of all relation strings.

exists(h: str, r: str, t: str) → bool
    Check if a triple exists in the training set.

Parameters
    • h – Head entity string.
    • r – Relation string.
    • t – Tail entity string.

Returns
    True if the triple exists, False otherwise.

__iter__() → Iterator[Tuple[str, str, str]]
    Iterate over training triples as string tuples.

__len__() → int
    Return number of triples in the raw training set.

func_triple_to_bpe_representation(triple: List[str])

```

dicee.knowledge_graph_embeddings

Attributes

logger

Classes

<i>KGE</i>	Knowledge Graph Embedding Class for interactive usage of pre-trained models
------------	---

Module Contents

`dicee.knowledge_graph_embeddings.logger`

class `dicee.knowledge_graph_embeddings.KGE` (*path=None, url=None, construct_ensemble=False, model_name=None*)

Bases: `dicee.abstracts.BaseInteractiveKGE`, `dicee.abstracts.InteractiveQueryDecomposition`, `dicee.abstracts.BaseInteractiveTrainKGE`

Knowledge Graph Embedding Class for interactive usage of pre-trained models

`__str__` ()

`to` (*device: str*) $\rightarrow$ None

`get_transductive_entity_embeddings` (*indices: torch.LongTensor | List[str], as_pytorch=False, as_numpy=False, as_list=True*) $\rightarrow$ `torch.FloatTensor | numpy.ndarray | List[float]`

`create_vector_database` (*collection_name: str, distance: str, location: str = 'localhost', port: int = 6333*)

`generate` (*h="", r=""*)

`eval_lp_performance` (*dataset=List[Tuple[str, str, str]], filtered=True, *, tie_policy=None, tie_seed=None*)

Evaluate head/tail ranks, inheriting saved tie settings unless overridden.

`tie_policy` accepts sort (legacy), optimistic, random, or pessimistic. `tie_seed` seeds an independent random stream for each evaluation.

`predict_missing_head_entity` (*relation: List[str] | str, tail_entity: List[str] | str, within=None, batch_size=2, topk=1, return_indices=False*) $\rightarrow$ Tuple

Given a relation and a tail entity, return top k ranked head entity.

$\operatorname{argmax}_{\{e \in E\}} f(e, r, t)$, where $r \in R$, $t \in E$.

Parameter

`relation`: Union[List[str], str]

String representation of selected relations.

`tail_entity`: Union[List[str], str]

String representation of selected entities.

`k`: int

Highest ranked k entities.

Returns: Tuple

Highest K scores and entities

predict_missing_relations (*head_entity: List[str] | str, tail_entity: List[str] | str, within=None, batch_size=2, topk=1, return_indices=False*) → Tuple

Given a head entity and a tail entity, return top k ranked relations.

$\text{argmax}_{\{r \in R\}} f(h,r,t)$, where $h, t \in E$.

Parameter

head_entity: List[str]

String representation of selected entities.

tail_entity: List[str]

String representation of selected entities.

k: int

Highest ranked k entities.

Returns: Tuple

Highest K scores and entities

predict_missing_tail_entity (*head_entity: List[str] | str, relation: List[str] | str, within: List[str] = None, batch_size=2, topk=1, return_indices=False*) → torch.FloatTensor

Given a head entity and a relation, return top k ranked entities

$\text{argmax}_{\{e \in E\}} f(h,r,e)$, where $h \in E$ and $r \in R$.

Parameter

head_entity: List[str]

String representation of selected entities.

tail_entity: List[str]

String representation of selected entities.

Returns: Tuple

scores

predict (*, *h: List[str] | str | None = None, r: List[str] | str | None = None, t: List[str] | str | None = None, within: List[str] | None = None, logits: bool = True*) → torch.FloatTensor

Predict scores for triples or missing triple elements.

Parameters

- **h** – Head entity/entities. None to predict heads.
- **r** – Relation/relations. None to predict relations.
- **t** – Tail entity/entities. None to predict tails.
- **within** – Optional list of entities to restrict predictions to.
- **logits** – If True, return raw scores. If False, return sigmoid scores (0-1).

Returns

- Single triple (h, r, t): scalar score
- Missing element: vector of all possible scores

Return type

torch.FloatTensor of scores. Shape depends on the query type

Raises

- **TypeError** – If inputs are not strings or lists of strings.
- **ValueError** – If a required argument for the query type is missing.

Examples

```
>>> # Score a specific triple
>>> model.predict(h="Mongolia", r="isLocatedIn", t="Asia", logits=False)
tensor(0.9523)
```

```
>>> # Get scores for all possible tail entities
>>> model.predict(h="Mongolia", r="isLocatedIn", t=None)
tensor([0.21, 0.95, 0.03, ...]) # One score per entity
```

predict_topk (*, h: str | List[str] | None = None, r: str | List[str] | None = None, t: str | List[str] | None = None, topk: int = 10, within: List[str] | None = None, batch_size: int = 1024) → List[Tuple[str, float]] | List[List[Tuple[str, float]]]

Predict top-k missing items in a given triple pattern.

Parameters

- **h** – Head entity/entities. None to predict heads.
- **r** – Relation/relations. None to predict relations.
- **t** – Tail entity/entities. None to predict tails.
- **topk** – Number of top predictions to return.
- **within** – Optional list of entities to restrict predictions to.
- **batch_size** – Batch size for processing multiple queries.

Returns

List[(item, score), ...] of length topk. For batch query: List of such lists, one per query.

Return type

For single query

Raises

- **TypeError** – If h, r, or t is not a str or list of str.
- **ValueError** – If the required arguments for a query type are None.

Examples

```
>>> model.predict_topk(h=["Mongolia"], r=["isLocatedIn"], topk=3)
[('Asia', 0.99), ('Europe', 0.02), ...]
```

```
>>> model.predict_topk(r=["isLocatedIn"], t=["Asia"], topk=5)
[('Mongolia', 0.85), ('China', 0.82), ...]
```

triple_score (*h: List[str] | str = None, r: List[str] | str = None, t: List[str] | str = None, logits=False*)
→ torch.FloatTensor

Predict triple score

Parameter

head_entity: List[str]

String representation of selected entities.

relation: List[str]

String representation of selected relations.

tail_entity: List[str]

String representation of selected entities.

logits: bool

If logits is True, unnormalized score returned

Returns: Tuple

pytorch tensor of triple score

return_multi_hop_query_results (*aggregated_query_for_all_entities, k: int, only_scores*)

single_hop_query_answering (*query: tuple, only_scores: bool = True, k: int = None, use_logits: bool = True*)

answer_multi_hop_query (*query_type: str | None = None, query: Tuple[str | Tuple[str, str], ...] | None = None, queries: List[Tuple[str | Tuple[str, str], ...]] | None = None, tnorm: str = 'prod', neg_norm: str = 'standard', lambda_: float = 0.0, k: int = 10, only_scores: bool = False, use_logits: bool = True*)
→ List[Tuple[str, torch.Tensor]] | List[List[Tuple[str, torch.Tensor]]]

Answer multi-hop EPFO (Existential Positive First-Order) queries.

Supports 9 query types: 1p, 2p, 3p, 2i, 3i, ip, pi, 2u, up. See docs/guides/multi_hop_queries.md for detailed query patterns.

Parameters

- **query_type** – Query pattern name. One of: - 1p: (e, (r,)) # One-hop - 2p: (e, (r1, r2)) # Two-hop - 3p: (e, (r1, r2, r3)) # Three-hop - 2i: ((e1, (r1,)), (e2, (r2,))) # Two-way intersection - 3i: ((e1, (r1,)), (e2, (r2,)), (e3, (r3,))) # Three-way intersection - ip: (((e1, (r1,)), (e2, (r2,))), (r3,)) # Intersection + projection - pi: ((e, (r1, r2)), (r3,)) # Projection + intersection (2i meets 2p) - 2u: ((e1, (r1,)), (e2, (r2,))) # Two-way union - up: ((e, (r1, r2)), (e, (r3,))) # Union + projection
- **query** – Single query tuple matching the query_type pattern.
- **queries** – Batch of queries. If provided, query must be None.
- **tnorm** – T-norm for intersection/union. Options: “prod”, “min”.
- **neg_norm** – Negation norm. Options: “standard”, “sugeno”, “yager”.

- **lambda** – Parameter for sugeno and yager negation (0.0-1.0).
- **k** – Number of top answer entities to return.
- **only_scores** – If True, return only scores tensor. If False, return (entity, score) tuples.
- **use_logits** – If True, use raw model logits. If False, use sigmoid probabilities.

Returns

List[(entity, score), ...] of top-k answers. For batch queries: List of such lists, one per query.

Return type

For single query

Raises

- **ValueError** – If query_type is not in {1p, 2p, 3p, 2i, 3i, ip, pi, 2u, up}.
- **AssertionError** – If query structure doesn't match query_type pattern.

Examples

```
>>> # 1p: Find entities located in Asia
>>> model.answer_multi_hop_query(
...     query_type="1p",
...     query=("Asia", ("isLocatedIn",)),
...     k=5
... )
[("Mongolia", 0.92), ("China", 0.89), ...]
```

```
>>> # 2p: Two-hop query (e.g., "capital of countries in Europe")
>>> model.answer_multi_hop_query(
...     query_type="2p",
...     query=("Europe", ("isLocatedIn", "hasCapital")),
...     k=3
... )
[("Paris", 0.85), ("Berlin", 0.82), ...]
```

```
>>> # 2i: Intersection query
>>> model.answer_multi_hop_query(
...     query_type="2i",
...     query=((("Asia", ("isLocatedIn",)), ("Mountains", ("hasGeography",))),
...     k=5
... )
[("Nepal", 0.78), ("Tibet", 0.65), ...]
```

➡ See also

- docs/guides/multi_hop_queries.md: Complete guide with all query patterns
- tests/test_answer_multi_hop_query.py: Usage examples

find_missing_triples (confidence: float, entities: List[str] = None, relations: List[str] = None, topk: int = 10, at_most: int = sys.maxsize) → Set

Find missing triples

Iterative over a set of entities E and a set of relation R :

forall e in E and forall r in R f(e,r,x)

Return (e,r,x)

notin G and f(e,r,x) > confidence

confidence: float

A threshold for an output of a sigmoid function given a triple.

topk: int

Highest ranked k item to select triples with f(e,r,x) > confidence .

at_most: int

Stop after finding at_most missing triples

{(e,r,x) | f(e,r,x) > confidence and (e,r,x)

notin G

predict_literals (*entity: List[str] | str = None, attribute: List[str] | str = None, denormalize_preds: bool = True*) → numpy.ndarray

Predicts literal values for given entities and attributes.

Parameters

- **entity** (*Union[List[str], str]*) – Entity or list of entities to predict literals for.
- **attribute** (*Union[List[str], str]*) – Attribute or list of attributes to predict literals for.
- **denormalize_preds** (*bool*) – If True, denormalizes the predictions.

Returns

Predictions for the given entities and attributes.

Return type

numpy ndarray

dicee.losses

Submodules

dicee.losses.custom_losses

Classes

<i>DefaultBCELoss</i>	Base class for all neural network modules.
<i>WeightedBCELoss</i>	Base class for all neural network modules.
<i>LabelSmoothingLoss</i>	Base class for all neural network modules.
<i>AdaptiveLabelSmoothingLoss</i>	Base class for all neural network modules.
<i>LabelRelaxationLoss</i>	Base class for all neural network modules.
<i>AdaptiveLabelRelaxationLoss</i>	Base class for all neural network modules.
<i>ConfidenceBasedAdaptiveLabelRelaxation-Loss</i>	Base class for all neural network modules.
<i>CombinedLSandLR</i>	Base class for all neural network modules.
<i>CombinedAdaptiveLSandAdaptiveLR</i>	Base class for all neural network modules.

continues on next page

Table 30 – continued from previous page

<i>AggregatedLSandLR</i>	Base class for all neural network modules.
<i>ACLS</i>	Base class for all neural network modules.

Module Contents

class dicee.losses.custom_losses.DefaultBCELoss

Bases: torch.nn.Module

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

forward (*pred, target*)

class dicee.losses.custom_losses.WeightedBCELoss

Bases: torch.nn.Module

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

forward (*pred, target, current_epoch*)

```
class dicee.losses.custom_losses.LabelSmoothingLoss(smoothness_ratio=0.0)
```

Bases: `torch.nn.Module`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

smoothness_ratio = 0.0

forward (*pred, target*)

```
class dicee.losses.custom_losses.AdaptiveLabelSmoothingLoss (min_smoothing_factor=0.01,  
    max_smoothing_factor=0.2, smoothing_factor_step=0.01, initial_smoothing_factor=0.1)
```

Bases: `torch.nn.Module`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn  
import torch.nn.functional as F  
  
class Model(nn.Module):  
    def __init__(self) -> None:  
        super().__init__()  
        self.conv1 = nn.Conv2d(1, 20, 5)  
        self.conv2 = nn.Conv2d(20, 20, 5)  
  
    def forward(self, x):  
        x = F.relu(self.conv1(x))  
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

min_smoothing_factor = 0.01

max_smoothing_factor = 0.2

smoothing_factor_step = 0.01

```

smoothing_factor = 0.1

prev_loss = None

eps = 1e-14

forward(logits, target, current_epoch)

```

```
class dicee.losses.custom_losses.LabelRelaxationLoss(alpha=0.0)
```

Bases: torch.nn.Module

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```

import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))

```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

```
alpha = 0.0
```

```
gz_threshold = 0.1
```

```
eps = 1e-14
```

```
forward(pred, target)
```

```
class dicee.losses.custom_losses.AdaptiveLabelRelaxationLoss(min_alpha=0.01,
max_alpha=0.2, alpha_step=0.01, initial_alpha=0.1)
```

Bases: torch.nn.Module

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

min_alpha = 0.01

max_alpha = 0.2

alpha_step = 0.01

alpha = 0.1

prev_loss = None

eps = 1e-14

gz_threshold = 0.1

forward (*pred, target*)

```
class dicee.losses.custom_losses.ConfidenceBasedAdaptiveLabelRelaxationLoss(
    alpha=0.1)
```

Bases: `torch.nn.Module`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```

import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))

```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

alpha = 0.1

gz_threshold = 0.1

eps = 1e-14

forward (*pred, target, current_epoch*)

class `dicee.losses.custom_losses.CombinedLSandLR` (*smoothness_ratio=0.0, alpha=0.0*)

Bases: `torch.nn.Module`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```

import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):

```

(continues on next page)

(continued from previous page)

```
x = F.relu(self.conv1(x))
return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

smoothness_ratio = 0.0

alpha = 0.0

forward (*pred, target, current_epoch*)

class dicee.losses.custom_losses.CombinedAdaptiveLSandAdaptiveLR

Bases: `torch.nn.Module`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

adaptive_label_smoothing

adaptive_label_relaxation

criterion = ''

forward (*pred, target, current_epoch*)

class dicee.losses.custom_losses.**AggregatedLSandLR** (*smoothness_ratio=0.1, alpha=0.1*)

Bases: torch.nn.Module

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

smoothness_ratio = 0.1

alpha = 0.1

forward (*pred, target, current_epoch*)

class dicee.losses.custom_losses.**ACLS** (*pos_lambda: float = 1.0, neg_lambda: float = 0.1, alpha: float = 0.1, margin: float = 10.0, num_classes: int = 200, ignore_index: int = -100*)

Bases: torch.nn.Module

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the submodules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

```
pos_lambda = 1.0
neg_lambda = 0.1
alpha = 0.1
margin = 10.0
num_classes = 200
ignore_index = -100
cross_entropy
property names
get_reg(inputs, targets)
forward(inputs, targets)
```

`dicee.models`

Submodules

`dicee.models.adopt`

ADOPT Optimizer Implementation.

This module implements the ADOPT (Adaptive Optimization with Precise Tracking) algorithm, an advanced optimization method for training neural networks.

ADOPT Overview:

ADOPT is an adaptive learning rate optimization algorithm that combines the benefits of momentum-based methods with per-parameter learning rate adaptation. Unlike Adam, which applies momentum to raw gradients, ADOPT normalizes gradients first and then applies momentum, leading to more stable training dynamics.

Key Features: - Gradient normalization before momentum application - Adaptive per-parameter learning rates - Optional gradient clipping that grows with training steps - Support for decoupled weight decay (AdamW-style) - Multiple execution modes: single-tensor, multi-tensor (foreach), and fused (planned)

Algorithm Comparison:

Adam: $m = \beta_1 * m + (1 - \beta_1) * g$, $\theta = \theta - \alpha * m / \sqrt{v}$ ADOPT: $m = \beta_1 * m + (1 - \beta_1) * g / \sqrt{v}$, $\theta = \theta - \alpha * m$

The key difference is that ADOPT normalizes gradients before momentum, which provides better stability and can lead to improved convergence.

Classes:

- ADOPT: Main optimizer class (extends torch.optim.Optimizer)

Functions:

- adopt: Functional API for ADOPT algorithm computation
- _single_tensor_adopt: Single-tensor implementation (TorchScript compatible)
- _multi_tensor_adopt: Multi-tensor implementation using foreach operations

Performance:

- Single-tensor: Default, compatible with torch.jit.script
- Multi-tensor (foreach): 2-3x faster on GPU through vectorization
- Fused (planned): Would provide maximum performance via specialized kernels

Example:

```
>>> import torch
>>> from dicee.models.adopt import ADOPT
>>>
>>> model = torch.nn.Linear(10, 1)
>>> optimizer = ADOPT(model.parameters(), lr=0.001, weight_decay=0.01, decouple=True)
>>>
>>> # Training loop
>>> for epoch in range(num_epochs):
...     optimizer.zero_grad()
...     output = model(input)
...     loss = criterion(output, target)
...     loss.backward()
...     optimizer.step()
```

References:

Original implementation: <https://github.com/iShohei220/adopt>

Notes:

This implementation is based on the original ADOPT implementation and adapted to work with the PyTorch optimizer interface and the dicee framework.

Classes

<code>ADOPT</code>	ADOPT Optimizer.
--------------------	------------------

Functions

<code>adopt(params, grads, exp_avgs, exp_avg_sqs, state_steps)</code>	Functional API that performs ADOPT algorithm computation.
---	---

Module Contents

```
class dicee.models.adopt.ADOPT (params: torch.optim.optimizer.ParamsT,  
    lr: float | torch.Tensor = 0.001, betas: Tuple[float, float] = (0.9, 0.9999), eps: float = 1e-06,  
    clip_lambda: Callable[[int], float] | None = lambda step: ..., weight_decay: float = 0.0,  
    decouple: bool = False, *, foreach: bool | None = None, maximize: bool = False,  
    capturable: bool = False, differentiable: bool = False, fused: bool | None = None)
```

Bases: torch.optim.optimizer.Optimizer

ADOPT Optimizer.

ADOPT is an adaptive learning rate optimization algorithm that combines momentum-based updates with adaptive per-parameter learning rates. It uses exponential moving averages of gradients and squared gradients, with gradient clipping for stability.

The algorithm performs the following key operations: 1. Normalizes gradients by the square root of the second moment estimate 2. Applies optional gradient clipping based on the training step 3. Updates parameters using momentum-smoothed normalized gradients 4. Supports decoupled weight decay (AdamW-style) or L2 regularization

Mathematical formulation:

$$m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * \text{clip}(g_t / \sqrt{v_t}) \quad v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * g_t^2 \quad \theta_t = \theta_{t-1} - \alpha * m_t$$

where:

- θ_t : parameter at step t
- g_t : gradient at step t
- m_t : first moment estimate (momentum)
- v_t : second moment estimate (variance)
- α : learning rate
- β_1, β_2 : exponential decay rates
- $\text{clip}()$: optional gradient clipping function

Reference:

Original implementation: <https://github.com/iShohei220/adopt>

Parameters

- **params** (*ParamsT*) – Iterable of parameters to optimize or dicts defining parameter groups.
- **lr** (*float or Tensor, optional*) – Learning rate. Can be a float or 1-element Tensor. Default: 1e-3
- **betas** (*Tuple[float, float], optional*) – Coefficients (β_1, β_2) for computing running averages of gradient and its square. β_1 controls momentum, β_2 controls variance. Default: (0.9, 0.9999)
- **eps** (*float, optional*) – Term added to denominator to improve numerical stability. Default: 1e-6
- **clip_lambda** (*Callable[[int], float], optional*) – Function that takes the step number and returns the gradient clipping threshold. Common choices: - lambda step: step**0.25 (default, gradually increases clipping threshold) - lambda step: 1.0 (constant clipping) - None (no clipping) Default: lambda step: step**0.25
- **weight_decay** (*float, optional*) – Weight decay coefficient (L2 penalty). Default: 0.0
- **decouple** (*bool, optional*) – If True, uses decoupled weight decay (AdamW-style), applying weight decay directly to parameters. If False, adds weight decay to gradients (L2 regularization). Default: False
- **foreach** (*bool, optional*) – If True, uses the faster foreach implementation for multi-tensor operations. Default: None (auto-select)
- **maximize** (*bool, optional*) – If True, maximizes parameters instead of minimizing. Useful for reinforcement learning. Default: False
- **capturable** (*bool, optional*) – If True, the optimizer is safe to capture in a CUDA graph. Requires learning rate as Tensor. Default: False
- **differentiable** (*bool, optional*) – If True, the optimization step can be differentiated. Useful for meta-learning. Default: False
- **fused** (*bool, optional*) – If True, uses fused kernel implementation (currently not supported). Default: None

Raises

- **ValueError** – If learning rate, epsilon, betas, or weight_decay are invalid.
- **RuntimeError** – If fused is enabled (not currently supported).
- **RuntimeError** – If lr is a Tensor with foreach=True and capturable=False.

Example

```
>>> # Basic usage
>>> optimizer = ADOPT(model.parameters(), lr=0.001)
>>> optimizer.zero_grad()
>>> loss.backward()
>>> optimizer.step()
```

```
>>> # With decoupled weight decay
>>> optimizer = ADOPT(model.parameters(), lr=0.001, weight_decay=0.01,
↳decouple=True)
```

```
>>> # Custom gradient clipping
>>> optimizer = ADOPT(model.parameters(), clip_lambda=lambda step: max(1.0,
↳step**0.5))
```

Note

- For most use cases, the default hyperparameters work well
- Consider using decouple=True for better generalization (similar to AdamW)
- The clip_lambda function helps stabilize training in early steps

clip_lambda

__setstate__ (state)

Restore optimizer state from a checkpoint.

This method handles backward compatibility when loading optimizer state from older versions. It ensures all required fields are present with default values and properly converts step counters to tensors if needed.

Key responsibilities: 1. Set default values for newly added hyperparameters 2. Convert old-style scalar step counters to tensor format 3. Place step tensors on appropriate devices based on capturable/fused modes

Parameters

state (dict) – Optimizer state dictionary (typically from torch.load()).

Note

- This enables loading checkpoints saved with older ADOPT versions
- Step counters are converted to appropriate device/dtype for compatibility
- Capturable and fused modes require step tensors on parameter devices

step (closure=None)

Perform a single optimization step.

This method executes one iteration of the ADOPT optimization algorithm across all parameter groups. It orchestrates the following workflow:

1. Optionally evaluates a closure to recompute the loss (useful for algorithms like LBFGS or when loss needs multiple evaluations)
2. For each parameter group: - Collects parameters with gradients and their associated state - Extracts hyperparameters (betas, learning rate, etc.) - Calls the functional adopt() API to perform the actual update
3. Returns the loss value if a closure was provided

The functional API (adopt()) handles three execution modes: - Single-tensor: Updates one parameter at a time (default, JIT-compatible) - Multi-tensor (foreach): Batches operations for better performance - Fused: Uses fused CUDA kernels (not yet implemented)

Gradient scaling support: This method is compatible with automatic mixed precision (AMP) training. It can access `grad_scale` and `found_inf` attributes for gradient unscaling and inf/nan detection when used with `GradScaler`.

Parameters

closure (*Callable, optional*) – A callable that reevaluates the model and returns the loss. The closure should: - Enable gradients (`torch.enable_grad()`) - Compute forward pass - Compute loss - Compute backward pass - Return the loss value Example: `lambda: (loss := model(x), loss.backward(), loss)[-1]` Default: `None`

Returns

The loss value returned by the closure, or `None` if no closure was provided.

Return type

Optional[`Tensor`]

Example

```
>>> # Standard usage
>>> loss = criterion(model(input), target)
>>> loss.backward()
>>> optimizer.step()
```

```
>>> # With closure (e.g., for line search)
>>> def closure():
...     optimizer.zero_grad()
...     output = model(input)
...     loss = criterion(output, target)
...     loss.backward()
...     return loss
>>> loss = optimizer.step(closure)
```

Note

- Call `zero_grad()` before computing gradients for the next step
- CUDA graph capture is checked for safety when `capturable=True`
- The method is thread-safe for different parameter groups

`dicee.models.adopt.adopt` (*params: List[torch.Tensor], grads: List[torch.Tensor], exp_avgs: List[torch.Tensor], exp_avg_sqs: List[torch.Tensor], state_steps: List[torch.Tensor], foreach: bool | None = None, capturable: bool = False, differentiable: bool = False, fused: bool | None = None, grad_scale: torch.Tensor | None = None, found_inf: torch.Tensor | None = None, has_complex: bool = False, *, beta1: float, beta2: float, lr: float | torch.Tensor, clip_lambda: Callable[[int], float] | None, weight_decay: float, decouple: bool, eps: float, maximize: bool*)

Functional API that performs ADOPT algorithm computation.

This is the main functional interface for the ADOPT optimization algorithm. It dispatches to one of three implementations based on the execution mode:

1. **Single-tensor mode** (default): Updates parameters one at a time - Compatible with `torch.jit.script` - More flexible but slower - Used when `foreach=False` or automatically for small models

2. **Multi-tensor (foreach) mode:** Batches operations across tensors - 2-3x faster on GPU through vectorization
- Groups tensors by device/dtype automatically - Used when foreach=True
3. **Fused mode:** Uses specialized fused kernels (not yet implemented) - Would provide maximum performance
- Currently raises RuntimeError if enabled

Algorithm overview (ADOPT):

ADOPT adapts learning rates per-parameter while using momentum on normalized gradients. The key innovation is normalizing gradients before momentum, which provides more stable training than standard Adam.

Mathematical formulation:

```
# Normalize gradient by its historical variance  $\text{normed\_g\_t} = \text{g\_t} / \sqrt{(\text{v\_t} + \epsilon)}$ 

# Optional gradient clipping for stability  $\text{normed\_g\_t} = \text{clip}(\text{normed\_g\_t}, \text{threshold}(t))$ 

# Momentum on normalized gradients (key difference from Adam)  $\text{m\_t} = \beta_1 * \text{m}_{\{t-1\}} + (1 - \beta_1) * \text{normed\_g\_t}$ 

# Parameter update  $\theta\_t = \theta_{\{t-1\}} - \alpha * \text{m\_t}$ 

# Update variance estimate  $\text{v\_t} = \beta_2 * \text{v}_{\{t-1\}} + (1 - \beta_2) * \text{g\_t}^2$ 
```

where:

- θ : parameters
- g : gradients
- m : first moment (momentum of normalized gradients)
- v : second moment (variance of raw gradients)
- α : learning rate
- β_1, β_2 : exponential decay rates
- ϵ : numerical stability constant
- $\text{clip}()$: gradient clipping function based on step

Automatic mode selection:

When foreach and fused are both None (default), the function automatically selects the best implementation based on:
- Parameter types and devices - Whether differentiable mode is enabled - Learning rate type (float vs Tensor)
- Capturable mode requirements

param params

Parameters to optimize.

type params

List[Tensor]

param grads

Gradients for each parameter.

type grads

List[Tensor]

param exp_avgs

First moment estimates (momentum).

type exp_avgs

List[Tensor]

param exp_avg_sqs
Second moment estimates (variance).

type exp_avg_sqs
List[Tensor]

param state_steps
Step counters (must be singleton tensors).

type state_steps
List[Tensor]

param foreach
Whether to use multi-tensor implementation. None: auto-select based on configuration (default).

type foreach
Optional[bool]

param capturable
If True, ensure CUDA graph capture safety.

type capturable
bool

param differentiable
If True, allow gradients through optimization step.

type differentiable
bool

param fused
If True, use fused kernels (not implemented).

type fused
Optional[bool]

param grad_scale
Gradient scaler for AMP training.

type grad_scale
Optional[Tensor]

param found_inf
Flag for inf/nan detection in AMP.

type found_inf
Optional[Tensor]

param has_complex
Whether any parameters are complex-valued.

type has_complex
bool

param beta1
Exponential decay rate for first moment (momentum). Typical range: 0.9-0.95.

type beta1
float

param beta2
Exponential decay rate for second moment (variance). Typical range: 0.999-0.9999 (higher than Adam).

type beta2
float

param lr
Learning rate. Can be a scalar Tensor for dynamic learning rate with capturable=True.

type lr
Union[float, Tensor]

param clip_lambda
Function that maps step number to gradient clipping threshold. None disables clipping.

type clip_lambda
Optional[Callable[[int], float]]

param weight_decay
Weight decay coefficient (L2 penalty).

type weight_decay
float

param decouple
If True, use decoupled weight decay (AdamW-style). Recommended for better generalization.

type decouple
bool

param eps
Small constant for numerical stability in normalization.

type eps
float

param maximize
If True, maximize objective instead of minimize.

type maximize
bool

raises RuntimeError
If torch.jit.script is used with foreach or fused.

raises RuntimeError
If state_steps contains non-tensor elements.

raises RuntimeError
If fused=True (not yet implemented).

raises RuntimeError
If lr is Tensor with foreach=True and capturable=False.

Example

```
>>> # Typically called by ADOPT optimizer, not directly
>>> adopt (
...     params=[p1, p2],
...     grads=[g1, g2],
...     exp_avgs=[m1, m2],
...     exp_avg_sqs=[v1, v2],
...     state_steps=[step1, step2],
...     beta1=0.9,
```

(continues on next page)

(continued from previous page)

```
...     beta2=0.9999,  
...     lr=0.001,  
...     clip_lambda=lambda s: s**0.25,  
...     weight_decay=0.01,  
...     decouple=True,  
...     eps=1e-6,  
...     maximize=False,  
... )
```

Note

- For distributed training, this API is compatible with torch/distributed/opt
- The foreach mode is generally preferred for GPU training
- Complex parameters are handled transparently by viewing as real
- First optimization step only initializes variance, doesn't update parameters

See also

- ADOPT class: High-level optimizer interface
- `_single_tensor_adopt`: Single-tensor implementation details
- `_multi_tensor_adopt`: Multi-tensor implementation details

dicee.models.base_model

Attributes

logger

Classes

<i>BaseKGELightning</i>	Thin PyTorch Lightning wrapper shared by all KGE models.
<i>BaseKGE</i>	Base class for all Knowledge Graph Embedding models.
<i>IdentityClass</i>	No-op normalisation / dropout placeholder.

Module Contents

`dicee.models.base_model.logger`

class `dicee.models.base_model.BaseKGELightning` (*args, **kwargs)

Bases: `lightning.LightningModule`

Thin PyTorch Lightning wrapper shared by all KGE models.

Provides the standard Lightning training loop hooks (`training_step`, `on_train_epoch_end`, `configure_optimizers`) as well as a helper for reporting model size. All concrete KGE models should extend `BaseKGE` rather than this class directly.

args: `Dict[str, Any]`

loss: `torch.nn.Module`

training_step_outputs = []

mem_of_model() → `Dict`

Size of model in MB and number of params

training_step(*batch*, *batch_idx=None*)

Execute one optimisation step for the given mini-batch.

Handles two- and three-element batches produced by the different dataset classes (`KvsAll` / `NegSample` vs. `KvsSample`).

Parameters

- **batch** (*tuple*) – (*x*, *y*) for standard scoring, or (*x*, *y_select*, *y*) for sample-based labelling.
- **batch_idx** (*int*, *optional*) – Index of the current batch (unused, kept for Lightning API compat).

Returns

Scalar loss value for this batch.

Return type

`torch.FloatTensor`

loss_function(*yhat_batch: torch.Tensor*, *y_batch: torch.Tensor*) → `torch.Tensor`

Compute the loss between model predictions and targets.

Delegates to `self.loss` which is configured in `BaseKGE.__init__` based on the scoring technique (`BCEWithLogitsLoss` for entity/relation prediction, `CrossEntropyLoss` for classification, `MarginRankingLoss` for `scoring_technique='NegSampleMargin'`).

For `NegSampleMargin`, *yhat_batch*/*y_batch* still arrive in the flat `NegSample` layout (positives first, followed by `neg_ratio` tiled blocks of negatives, row-aligned by triple). This method splits that flat batch into positive/negative score pairs before delegating to `MarginRankingLoss`.

When `adversarial_temperature` is set (grouped/strict negative sampling), *yhat_batch*/*y_batch* instead arrive as (*batch*, `1 + neg_ratio`) groups with the positive in column 0, produced by `GroupedNegativeSamplingDataset`. This delegates to `grouped_adversarial_bce()` instead of `self.loss`.

Parameters

- **yhat_batch** (`torch.FloatTensor`) – Model output scores, shape (*batch_size*, *).
- **y_batch** (`torch.FloatTensor`) – Ground-truth labels of the same shape as *yhat_batch*.

Returns

Scalar loss value.

Return type

`torch.FloatTensor`

`on_train_epoch_end(*args, **kwargs)`

Called in the training loop at the very end of the epoch.

To access all batch outputs at the end of the epoch, you can cache step outputs as an attribute of the `LightningModule` and access them in this hook:

```
class MyLightningModule(L.LightningModule):
    def __init__(self):
        super().__init__()
        self.training_step_outputs = []

    def training_step(self):
        loss = ...
        self.training_step_outputs.append(loss)
        return loss

    def on_train_epoch_end(self):
        # do something with all training_step outputs, for example:
        epoch_mean = torch.stack(self.training_step_outputs).mean()
        self.log("training_epoch_mean", epoch_mean)
        # free up the memory
        self.training_step_outputs.clear()
```

`test_epoch_end(outputs: List[Any])`

`test_dataloader()` → None

An iterable or collection of iterables specifying test samples.

For more information about multiple dataloaders, see this section.

For data processing use the following pattern:

- download in `prepare_data()`
- process and split in `setup()`

However, the above are only necessary for distributed processing.

Warning

do not assign state in `prepare_data`

- `test()`
- `prepare_data()`
- `setup()`

Note

Lightning tries to add the correct sampler for distributed and arbitrary hardware. There is no need to set it yourself.

Note

If you don't need a test dataset and a `test_step()`, you don't need to implement this method.

`val_dataloader()` → None

An iterable or collection of iterables specifying validation samples.

For more information about multiple dataloaders, see this section.

The dataloader you return will not be reloaded unless you set **:param-ref:~lightning.pytorch.trainer.trainer.Trainer.reload_dataloaders_every_n_epochs`** to a positive integer.

It's recommended that all data downloads and preparation happen in `prepare_data()`.

- `fit()`
- `validate()`
- `prepare_data()`
- `setup()`

Note

Lightning tries to add the correct sampler for distributed and arbitrary hardware There is no need to set it yourself.

Note

If you don't need a validation dataset and a `validation_step()`, you don't need to implement this method.

`predict_dataloader()` → None

An iterable or collection of iterables specifying prediction samples.

For more information about multiple dataloaders, see this section.

It's recommended that all data downloads and preparation happen in `prepare_data()`.

- `predict()`
- `prepare_data()`
- `setup()`

Note

Lightning tries to add the correct sampler for distributed and arbitrary hardware There is no need to set it yourself.

Returns

A `torch.utils.data.DataLoader` or a sequence of them specifying prediction samples.

`train_dataloader()` → None

An iterable or collection of iterables specifying training samples.

For more information about multiple dataloaders, see this section.

The dataloader you return will not be reloaded unless you set `:param-ref:~lightning.pytorch.trainer.trainer.Trainer.reload_dataloaders_every_n_epochs`` to a positive integer.

For data processing use the following pattern:

- download in `prepare_data()`
- process and split in `setup()`

However, the above are only necessary for distributed processing.

Warning

do not assign state in `prepare_data`

- `fit()`
- `prepare_data()`
- `setup()`

Note

Lightning tries to add the correct sampler for distributed and arbitrary hardware. There is no need to set it yourself.

`configure_optimizers(parameters=None)`

Instantiate and return the optimiser for training.

The optimiser type is taken from `self.optimizer_name` which is set in `BaseKGE.init_params_with_sanity_checking()` from the `--optim` argument. Supported values: 'SGD', 'Adam', 'Adopt', 'AdamW', 'NAdam', 'Adagrad', 'ASGD', 'Muon'.

Parameters

parameters (*iterable, optional*) – Model parameters to optimise. Defaults to `self.parameters()` when None.

Returns

The configured optimiser instance.

Return type

`torch.optim.Optimizer`

`class dicee.models.base_model.BaseKGE(args: dict)`

Bases: `BaseKGELightning`

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from `BaseKGELightning` and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- `forward_triples()` — score a batch of (h, r, t) triples.
- `forward_k_vs_all()` — score a (h, r) batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

```
args
embedding_dim = None
num_entities: int | None = None
num_relations: int | None = None
num_tokens = None
learning_rate = None
apply_unit_norm = None
input_dropout_rate = None
hidden_dropout_rate = None
optimizer_name = None
feature_map_dropout_rate = None
kernel_size = None
num_of_output_channels = None
weight_decay = None
selected_optimizer = None
normalizer_class = None
normalize_head_entity_embeddings
normalize_relation_embeddings
normalize_tail_entity_embeddings
hidden_normalizer
param_init
input_dp_ent_real
input_dp_rel_real
hidden_dropout
loss_history = []
byte_pair_encoding
```


max_length_subword_tokens

block_size

defer_large_embeddings

init_entity_embeddings (*embedding_dim*: int | None = None) → None

Create (or re-create) the entity embedding table.

This is the single place that honours *defer_large_embeddings*: when entity rows are sharded across FSDP ranks the table must stay None until the trainer allocates the sharded adapter. Subclasses that need an entity table of a non-default width must call this method instead of assigning `self.entity_embeddings` directly, so the deferral is never silently undone.

Parameters

embedding_dim (*Optional*[int]) – Width of the table. Defaults to *embedding_dim*.

init_relation_embeddings (*embedding_dim*: int | None = None) → None

Create (or re-create) the relation embedding table.

Relation tables are never deferred - they are small enough to be replicated on every rank.

Parameters

embedding_dim (*Optional*[int]) – Width of the table. Defaults to *embedding_dim*.

forward_byte_pair_encoded_k_vs_all (*x*: torch.LongTensor) → torch.FloatTensor

KvsAll scoring for BPE-encoded head entities and relations.

Retrieves subword-unit embeddings for the head entity and relation, reduces them to fixed-size vectors via a linear projection, then scores against all BPE entity embeddings.

Parameters

x (torch.LongTensor) – Shape (batch_size, 2, T) BPE token indices where dim 1 indexes [head, relation] and T is max_length_subword_tokens.

Returns

Shape (batch_size, num_bpe_entities) score matrix.

Return type

torch.FloatTensor

forward_byte_pair_encoded_triple (*x*: Tuple[torch.LongTensor, torch.LongTensor])
→ torch.FloatTensor

NegSample scoring for BPE-encoded (head, relation, tail) triples.

Retrieves subword-unit embeddings for all three elements and reduces them to fixed-size vectors via a linear projection before computing the triple score.

Parameters

x (torch.LongTensor) – Shape (batch_size, 3, T) BPE token indices.

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

init_params_with_sanity_checking() → None

Populate model hyper-parameters from `self.args` with safe defaults.

Reads embedding dimension, learning rate, dropout rates, normalisation strategy, optimizer name, and parameter initialisation scheme from the `args` dict. Falls back to sensible defaults for any missing key so that minimal `args` dicts (e.g. for unit tests) are still valid.

forward (*x*: torch.LongTensor | Tuple[torch.LongTensor, torch.LongTensor],
y_idx: torch.LongTensor | None = None) → torch.Tensor

Route the forward pass to the appropriate scoring method.

Inspects the shape and type of *x* to decide which low-level scorer to call:

- Tuple (*x*, *y_idx*) → *forward_k_vs_sample()*
- (batch, 3) tensor → *forward_triples()*
- (batch, 2) tensor → *forward_k_vs_all()*
- BPE triple tensor → *forward_byte_pair_encoded_triple()*
- BPE pair tensor → *forward_byte_pair_encoded_k_vs_all()*

Parameters

- **x** (torch.LongTensor or Tuple[torch.LongTensor, torch.LongTensor]) – Either a plain index tensor or a (triple_idx, target_idx) tuple for sample-based labelling.
- **y_idx** (torch.LongTensor, optional) – Target entity indices used by *forward_k_vs_sample()*. Ignored when *x* is a plain tensor.

Returns

Score tensor whose shape depends on the selected scorer.

Return type

torch.FloatTensor

forward_triples (*x*: torch.LongTensor) → torch.Tensor

Score a batch of (head, relation, tail) index triples.

Parameters

- **x** (torch.LongTensor) – Shape (batch_size, 3) integer tensor where each row is [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

forward_k_vs_all (*args, **kwargs)

Score a (head, relation) batch against every entity.

Sub-classes must override this method. The default implementation raises ValueError to make missing overrides obvious at runtime.

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_k_vs_sample (*args, **kwargs)

Score a (head, relation) batch against a sampled subset of entities.

Used by KvsSample and 1vsSample datasets. Sub-classes that support sample-based labelling must override this method.

Returns

Shape (batch_size, k) score matrix where k is the number of sampled target entities.

Return type

torch.FloatTensor

get_triple_representation (*idx_hrt*)

→ Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for a triple index batch.

Parameters

idx_hrt (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor with columns [head_idx, relation_idx, tail_idx].

Returns

head_ent_emb, rel_ent_emb, tail_ent_emb – Each has shape (batch_size, embedding_dim) after applying the configured dropout and normalisation.

Return type

torch.FloatTensor

get_head_relation_representation (*indexed_triple*)

→ Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for head entities and relations.

Parameters

indexed_triple (*torch.LongTensor*) – Shape (batch_size, 2) integer tensor with columns [head_idx, relation_idx].

Returns

head_ent_emb, rel_ent_emb – Each has shape (batch_size, embedding_dim) after applying the configured dropout and normalisation.

Return type

torch.FloatTensor

get_sentence_representation (*x: torch.LongTensor*)

→ Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve BPE subword-unit embeddings for a batch of triples.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3, T) where T is max_length_subword_tokens.

Returns

head_ent_emb, rel_emb, tail_emb – Each has shape (batch_size, T , embedding_dim).

Return type

torch.FloatTensor

get_bpe_head_and_relation_representation (*x: torch.LongTensor*)

→ Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve unit-normalised BPE embeddings for head entities and relations.

Each entity/relation is represented as a sequence of T subword tokens. Their token embeddings are L2-normalised across the sequence dimension so that the resulting matrix has unit Frobenius norm.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 2, T) where dim 1 indexes [head, relation] and T is max_length_subword_tokens.

Returns

head_ent_emb, rel_emb – Each has shape (batch_size, T, embedding_dim), L2-normalised over the (T, D) dimensions.

Return type

torch.FloatTensor

get_embeddings() → Tuple[numpy.ndarray, numpy.ndarray]

Return the entity and relation embedding matrices as numpy arrays.

Returns

- **entity_embeddings** (numpy.ndarray) – Shape (num_entities, embedding_dim).
- **relation_embeddings** (numpy.ndarray) – Shape (num_relations, embedding_dim).

class dicee.models.base_model.IdentityClass (args=None)

Bases: torch.nn.Module

No-op normalisation / dropout placeholder.

Used whenever no normalisation layer is requested (--normalization None). All inputs are returned unchanged so that the rest of the model code does not need conditional checks around normalisation calls.

args = None

__call__(x)

static forward(x)

dicee.models.clifford

Attributes

logger

Classes

<i>Keci</i>	Keci: Knowledge Graph Embedding via Clifford Algebra.
<i>KeciTransformer</i>	Keci with Transformer architecture.
<i>TransformerBlock</i>	A single transformer block with self-attention and MLP.
<i>TransformerSelfAttention</i>	Multi-head self-attention for the Keci Transformer.
<i>TransformerMLP</i>	MLP for the Keci Transformer.
<i>CKeci</i>	Without learning dimension scaling
<i>DeCaL</i>	Base class for all Knowledge Graph Embedding models.

Module Contents

dicee.models.clifford.**logger**

class dicee.models.clifford.Keci (args)

Bases: *dicee.models.base_model.BaseKGE*

Keci: Knowledge Graph Embedding via Clifford Algebra.

Embeds entities and relations as multi-vectors in the Clifford algebra $Cl_{\{p,q\}}(R^d)$ and scores triples via the Clifford product. The algebra is parameterised by two non-negative integers p and q :

- $p = 0, q = 0$ — reduces to a standard bilinear (DistMult-like) model.
- $p = 0, q = 1$ — equivalent to ComplEx.
- Larger p and q capture higher-order geometric interactions.

The embedding dimension must satisfy `embedding_dim % (p + q + 1) == 0`; the resulting quotient is stored as `self.r`.

Parameters

args (*dict*) – Configuration dictionary. Recognised keys (beyond those in `BaseKGE`): p (int, default 0) and q (int, default 0).

References

Demir et al., *Clifford Embeddings — A Generalized Approach for Embedding in Normed Algebras*, ECML 2023.

name = 'Keci'

p

q

r

requires_grad_for_interactions = True

compute_sigma_pp (*hp, rp*)

Compute $\text{sigma_pp} = \sum_{i=1}^{p-1} \sum_{k=i+1}^p (h_i r_k - h_k r_i) e_i e_k$

sigma_pp captures the interactions between along p bases For instance, let p e_1, e_2, e_3 , we compute interactions between $e_1 e_2, e_1 e_3$, and $e_2 e_3$ This can be implemented with a nested two for loops

results = [] for i in range(p - 1):

for k in range(i + 1, p):

 results.append(hp[:, :, i] * rp[:, :, k] - hp[:, :, k] * rp[:, :, i])

sigma_pp = torch.stack(results, dim=2) assert sigma_pp.shape == (b, r, int((p * (p - 1)) / 2))

Yet, this computation would be quite inefficient. Instead, we compute interactions along all p , e.g., $e_1 e_1, e_1 e_2, e_1 e_3$,

$e_2 e_1, e_2 e_2, e_2 e_3, e_3 e_1, e_3 e_2, e_3 e_3$

Then select the triangular matrix without diagonals: $e_1 e_2, e_1 e_3, e_2 e_3$.

compute_sigma_qq (*hq, rq*)

Compute $\text{sigma_qq} = \sum_{j=1}^{q-1} \sum_{k=j+1}^q (h_j r_k - h_k r_j) e_j e_k$ sigma_qq captures the interactions between along q bases For instance, let q e_1, e_2, e_3 , we compute interactions between $e_1 e_2, e_1 e_3$, and $e_2 e_3$ This can be implemented with a nested two for loops

results = [] for j in range(q - 1):

for k in range(j + 1, q):

 results.append(hq[:, :, j] * rq[:, :, k] - hq[:, :, k] * rq[:, :, j])

sigma_qq = torch.stack(results, dim=2) assert sigma_qq.shape == (b, r, int((q * (q - 1)) / 2))

Yet, this computation would be quite inefficient. Instead, we compute interactions along all p , e.g., $e_1 e_1, e_1 e_2, e_1 e_3$,

$e_2 e_1, e_2 e_2, e_2 e_3, e_3 e_1, e_3 e_2, e_3 e_3$

Then select the triangular matrix without diagonals: $e_1 e_2, e_1 e_3, e_2 e_3$.

```

compute_sigma_pq (*, hp, hq, rp, rq)
    sum_{i=1}^p sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j
    results = []
    sigma_pq = torch.zeros(b, r, p, q)
    for i in range(p):
        for j in range(q):
            sigma_pq[:, :, i, j] = hp[:, :, i] * rq[:, :, j] - hq[:, :, j] * rp[:, :, i]
    print(sigma_pq.shape)

apply_coefficients (hp, hq, rp, rq)
    Multiplying a base vector with its scalar coefficient

clifford_multiplication (h0, hp, hq, r0, rp, rq)
    Compute our CL multiplication

    h = h_0 + sum_{i=1}^p h_i e_i + sum_{j=p+1}^{p+q} h_j e_j
    r = r_0 + sum_{i=1}^p r_i e_i + sum_{j=p+1}^{p+q} r_j e_j

    e_i^2 = +1 for i ≤ p
    e_j^2 = -1 for p < j ≤ p+q
    e_i e_j = -e_j e_i for i < j

    h r = sigma_0 + sigma_p + sigma_q + sigma_{pp} + sigma_{pq} + sigma_{qq} where
    (1) sigma_0 = h_0 r_0 + sum_{i=1}^p (h_i r_i) e_i - sum_{j=p+1}^{p+q} (h_j r_j) e_j
    (2) sigma_p = sum_{i=1}^p (h_i r_i + h_i r_0) e_i
    (3) sigma_q = sum_{j=p+1}^{p+q} (h_j r_j + h_j r_0) e_j
    (4) sigma_{pp} = sum_{i=1}^{p-1} sum_{k=i+1}^p (h_i r_k - h_k r_i) e_i e_k
    (5) sigma_{qq} = sum_{j=1}^{p+q-1} sum_{k=j+1}^{p+q} (h_j r_k - h_k r_j) e_j e_k
    (6) sigma_{pq} = sum_{i=1}^p sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j

```

```

construct_cl_multivector (x: torch.FloatTensor, r: int, p: int, q: int)
    → tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

```

Split a flat embedding vector into the three Clifford components.

Given an embedding x of dimension $d = r + r \cdot p + r \cdot q$, returns the scalar part a_0 , the p -blade part a_p , and the q -blade part a_q .

Parameters

- **x** (*torch.FloatTensor*) – Shape (batch_size, d).
- **r** (*int*) – Scalar block size (embedding_dim // (p + q + 1)).
- **p** (*int*) – Number of positive-signature basis elements.
- **q** (*int*) – Number of negative-signature basis elements.

Returns

- **a0** (*torch.FloatTensor*) – Shape (batch_size, r) — scalar (grade-0) part.
- **ap** (*torch.FloatTensor*) – Shape (batch_size, r, p) — positive-blade part.
- **aq** (*torch.FloatTensor*) – Shape (batch_size, r, q) — negative-blade part.

forward_k_vs_with_explicit (*x*: torch.Tensor) → torch.FloatTensor

KvsAll scoring using an explicit loop over sigma_pp/qq/pq terms.

Functionally equivalent to `forward_k_vs_all()` but computes the higher-order interaction terms (sigma_pp, sigma_qq, sigma_pq) with explicit nested loops rather than einsum contractions. Kept for reference and correctness verification.

Parameters

x (torch.Tensor) – Shape (batch_size, 2) integer tensor [head_idx, relation_idx].

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

k_vs_all_score (bpe_head_ent_emb: torch.FloatTensor, bpe_rel_ent_emb: torch.FloatTensor, *E*: torch.FloatTensor) → torch.FloatTensor

Compute Clifford-product scores for a head/relation batch vs. all entities.

Decomposes the head-entity and relation embeddings into Clifford multi-vectors, performs the $Cl_{\{p,q\}}$ product, and inner-products the result against the entity embedding matrix *E*.

Parameters

- **bpe_head_ent_emb** (torch.FloatTensor) – Head-entity embeddings, shape (batch_size, embedding_dim).
- **bpe_rel_ent_emb** (torch.FloatTensor) – Relation embeddings, shape (batch_size, embedding_dim).
- **E** (torch.FloatTensor) – All entity embeddings, shape (num_entities, embedding_dim).

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_k_vs_all (*x*: torch.Tensor) → torch.FloatTensor

Kvsall training

- (1) Retrieve real-valued embedding vectors for heads and relations $\mathbb{R}^d$.
- (2) Construct head entity and relation embeddings according to $Cl_{\{p,q\}}(\mathbb{R}^d)$.
- (3) Perform Cl multiplication
- (4) Inner product of (3) and all entity embeddings

forward_k_vs_with_explicit and this function are identical Parameter ——— *x*: torch.LongTensor with (n,2) shape :rtype: torch.FloatTensor with (n, **IEI**) shape

construct_batch_selected_cl_multivector (*x*: torch.FloatTensor, *r*: int, *p*: int, *q*: int) → tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Split a batched, *k*-selected embedding tensor into Clifford components.

A variant of `construct_cl_multivector()` for tensors that have an extra *k* dimension (e.g. when scoring against *k* sampled targets).

Parameters

- **x** (*torch.FloatTensor*) – Shape (batch_size, k, d).
- **r** (*int*) – Scalar block size.
- **p** (*int*) – Number of positive-signature basis elements.
- **q** (*int*) – Number of negative-signature basis elements.

Returns

- **a0** (*torch.FloatTensor*) – Shape (batch_size, k, r).
- **ap** (*torch.FloatTensor*) – Shape (batch_size, k, r, p).
- **aq** (*torch.FloatTensor*) – Shape (batch_size, k, r, q).

forward_k_vs_sample (*x: torch.LongTensor, target_entity_idx: torch.LongTensor*) → *torch.FloatTensor*

Parameter

x: *torch.LongTensor* with (n,2) shape

target_entity_idx: *torch.LongTensor* with (n, k) shape k denotes the selected number of examples.

rtype

torch.FloatTensor with (n, k) shape

score (*h, r, t*)

forward_triples (*x: torch.Tensor*) → *torch.FloatTensor*

Parameter

x: *torch.LongTensor* with (n,3) shape

rtype

torch.FloatTensor with (n) shape

class `dicee.models.clifford.KeciTransformer` (*args*)

Bases: *Keci*

Keci with Transformer architecture.

Concatenates h0, hp, hq, r0, rp, rq into a single embedding vector and processes through transformer.

name = 'KeciTransformer'

use_clifford_mul

seq_len = 1

transformer

lm_head

forward_k_vs_all (*x: torch.Tensor*) → *torch.FloatTensor*

Kvsall training

Parameter

x: torch.LongTensor with (n,2) shape

rtype

torch.FloatTensor with (n, **IE**) shape

```
class dicee.models.clifford.TransformerBlock (n_embd, n_head, dropout=0.0, bias=False)
```

Bases: torch.nn.Module

A single transformer block with self-attention and MLP.

ln_1

attn

ln_2

mlp

forward (x)

```
class dicee.models.clifford.TransformerSelfAttention (n_embd, n_head, dropout=0.0,  
                                                    bias=False)
```

Bases: torch.nn.Module

Multi-head self-attention for the Keci Transformer.

c_attn

c_proj

attn_dropout

resid_dropout

n_head

n_embd

dropout = 0.0

forward (x)

```
class dicee.models.clifford.TransformerMLP (n_embd, dropout=0.0, bias=False)
```

Bases: torch.nn.Module

MLP for the Keci Transformer.

c_fc

gelu

c_proj

dropout

forward (x)

```
class dicee.models.clifford.CKeci (args)
    Bases: Keci

    Without learning dimension scaling

    name = 'CKeci'

    requires_grad_for_interactions = False
```

```
class dicee.models.clifford.DeCaL (args)
    Bases: dicee.models.base_model.BaseKGE
```

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from `BaseKGELightning` and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- `forward_triples()` — score a batch of (h, r, t) triples.
- `forward_k_vs_all()` — score a (h, r) batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

```
name = 'DeCaL'
```

```
p
```

```
q
```

```
r
```

```
re
```

```
forward_triples (x: torch.Tensor) → torch.FloatTensor
```

Parameter

x: torch.LongTensor with (n,) shape

rtype

torch.FloatTensor with (n) shape

```
cl_pqr (a: torch.tensor) → torch.tensor
```

Input: tensor(batch_size, emb_dim) → output: tensor with 1+p+q+r components with size (batch_size, emb_dim/(1+p+q+r)) each.

1) takes a tensor of size (batch_size, emb_dim), split it into 1 + p + q + r components, hence 1+p+q+r must be a divisor of the emb_dim. 2) Return a list of the 1+p+q+r components vectors, each are tensors of size (batch_size, emb_dim/(1+p+q+r))

```
compute_sigmas_single (list_h_emb, list_r_emb, list_t_emb)
```

here we compute all the sums with no others vectors interaction taken with the scalar product with t, that is,

$$s_0 = h_0 r_0 t_0 \quad s_1 = \sum_{i=1}^p h_i r_i t_0 \quad s_2 = \sum_{j=p+1}^{p+q} h_j r_j t_0 \quad s_3 = \sum_{i=1}^q (h_0 r_i t_i + h_i r_0 t_i) \quad s_4 = \sum_{i=p+1}^{p+q} (h_0 r_i t_i + h_i r_0 t_i) \quad s_5 = \sum_{i=p+q+1}^{p+q+r} (h_0 r_i t_i + h_i r_0 t_i)$$

and return:

$$\sigma_0 t = \sigma_0 \cdot t_0 = s_0 + s_1 - s_2 s_3, s_4 \text{ and } s_5$$

compute_sigmas_multivect (*list_h_emb, list_r_emb*)

Here we compute and return all the sums with vectors interaction for the same and different bases.

For same bases vectors interaction we have

$$\sigma_{pp} = \sum_{i=1}^{p-1} \sum_{i'=i+1}^p (h_i r_{i'} - h_{i'} r_i) (\text{model the interactions between } e_i \text{ and } e_{i'} \text{ for } 1 \leq i, i' \leq p) \sigma_{qq} = \sum_{j=p+1}^{p+q-1} \sum_{j'=j+1}^{p+q} (h_j r_{j'} - h_{j'} r_j)$$

For different base vector interactions, we have

$$\sigma_{pq} = \sum_{i=1}^p \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) (\text{interactions between } e_i \text{ and } e_j \text{ for } 1 \leq i \leq p \text{ and } p+1 \leq j \leq p+q) \sigma_{pr} = \sum_{i=1}^p \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i)$$

forward_k_vs_all (*x: torch.Tensor*) → torch.FloatTensor

Kvsall training

- (1) Retrieve real-valued embedding vectors for heads and relations
- (2) Construct head entity and relation embeddings according to $\text{Cl}_{\{p,q,r\}}(\mathbb{R}^d)$.
- (3) Perform Cl multiplication
- (4) Inner product of (3) and all entity embeddings

forward_k_vs_with_explicit and this functions are identical Parameter ——— x: torch.LongTensor with (n,) shape :rtype: torch.FloatTensor with (n, **IE**) shape

apply_coefficients (*h0, hp, hq, hk, r0, rp, rq, rk*)

Multiplying a base vector with its scalar coefficient

construct_cl_multivector (*x: torch.FloatTensor, re: int, p: int, q: int, r: int*)
→ tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Construct a batch of multivectors $\text{Cl}_{\{p,q,r\}}(\mathbb{R}^d)$

Parameter

x: torch.FloatTensor with (n,d) shape

returns

- **a0** (*torch.FloatTensor*)
- **ap** (*torch.FloatTensor*)
- **aq** (*torch.FloatTensor*)
- **ar** (*torch.FloatTensor*)

compute_sigma_pp (*hp, rp*)

Compute .. math:

$$\sigma_{pp} = \sum_{i=1}^{p-1} \sum_{i'=i+1}^p (x_{iy_{i'}} - x_{i'y_i})$$

$\text{sigma_}\{pp\}$ captures the interactions between along p bases For instance, let $p \in \{e_1, e_2, e_3\}$, we compute interactions between $e_1 e_2, e_1 e_3$, and $e_2 e_3$ This can be implemented with a nested two for loops

```
results = [] for i in range(p - 1):
```

```
    for k in range(i + 1, p):
```

```
        results.append(hp[:, :, i] * rp[:, :, k] - hp[:, :, k] * rp[:, :, i])
```

```
sigma_pp = torch.stack(results, dim=2) assert sigma_pp.shape == (b, r, int((p * (p - 1)) / 2))
```

Yet, this computation would be quite inefficient. Instead, we compute interactions along all p , e.g., $e_1 e_1, e_1 e_2, e_1 e_3,$

$e_2 e_1, e_2 e_2, e_2 e_3, e_3 e_1, e_3 e_2, e_3 e_3$

Then select the triangular matrix without diagonals: $e_1 e_2, e_1 e_3, e_2 e_3$.

compute_sigma_qq (hq, rq)

Compute

$$\sigma_{q,q}^* = \sum_{j=p+1}^{p+q-1} \sum_{j'=j+1}^{p+q} (x_j y_{j'} - x_{j'} y_j) Eq.16$$

$\text{sigma_}\{q\}$ captures the interactions between along q bases For instance, let $q \in \{e_1, e_2, e_3\}$, we compute interactions between $e_1 e_2, e_1 e_3$, and $e_2 e_3$ This can be implemented with a nested two for loops

```
results = [] for j in range(q - 1):
```

```
    for k in range(j + 1, q):
```

```
        results.append(hq[:, :, j] * rq[:, :, k] - hq[:, :, k] * rq[:, :, j])
```

```
sigma_qq = torch.stack(results, dim=2) assert sigma_qq.shape == (b, r, int((q * (q - 1)) / 2))
```

Yet, this computation would be quite inefficient. Instead, we compute interactions along all p , e.g., $e_1 e_1, e_1 e_2, e_1 e_3,$

$e_2 e_1, e_2 e_2, e_2 e_3, e_3 e_1, e_3 e_2, e_3 e_3$

Then select the triangular matrix without diagonals: $e_1 e_2, e_1 e_3, e_2 e_3$.

compute_sigma_rr (hk, rk)

$$\sigma_{r,r}^* = \sum_{k=p+q+1}^{p+q+r-1} \sum_{k'=k+1}^p (x_k y_{k'} - x_{k'} y_k)$$

compute_sigma_pq ($*, hp, hq, rp, rq$)

Compute

$$\sum_{i=1}^p \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j$$

```
results = [] sigma_pq = torch.zeros(b, r, p, q) for i in range(p):
```

```
    for j in range(q):
```

```
        sigma_pq[:, :, i, j] = hp[:, :, i] * rq[:, :, j] - hq[:, :, j] * rp[:, :, i]
```

```
print(sigma_pq.shape)
```

```
compute_sigma_pr(*, hp, hk, rp, rk)
```

Compute

$$\sum_{i=1}^p \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j$$

```
results = [] sigma_pq = torch.zeros(b, r, p, q) for i in range(p):
```

```
    for j in range(q):
```

```
        sigma_pq[:, :, i, j] = hp[:, :, i] * rq[:, :, j] - hq[:, :, j] * rp[:, :, i]
```

```
    print(sigma_pq.shape)
```

```
compute_sigma_qr(*, hq, hk, rq, rk)
```

$$\sum_{i=1}^p \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j$$

```
results = [] sigma_pq = torch.zeros(b, r, p, q) for i in range(p):
```

```
    for j in range(q):
```

```
        sigma_pq[:, :, i, j] = hp[:, :, i] * rq[:, :, j] - hq[:, :, j] * rp[:, :, i]
```

```
    print(sigma_pq.shape)
```

dicее.models.complex

Classes

<i>ConEx</i>	Convolutional ComplEx Knowledge Graph Embeddings
<i>AConEx</i>	Additive Convolutional ComplEx Knowledge Graph Embeddings
<i>Complex</i>	Base class for all Knowledge Graph Embedding models.
<i>RotatE</i>	RotatE: knowledge graph embedding by relational rotation in complex space.

Module Contents

```
class dicее.models.complex.ConEx(args)
```

Bases: *dicее.models.base_model.BaseKGE*

Convolutional ComplEx Knowledge Graph Embeddings

```
name = 'ConEx'
```

```
conv2d
```

```
fc_num_input
```

```
fc1
```

```
norm_fc1
```

```
bn_conv2d
```

```
feature_map_dropout
```

residual_convolution (*C_1*: *Tuple[torch.Tensor, torch.Tensor]*,
C_2: *Tuple[torch.Tensor, torch.Tensor]*) → *torch.FloatTensor*

Compute residual score of two complex-valued embeddings. :param C_1: a tuple of two pytorch tensors that corresponds complex-valued embeddings :param C_2: a tuple of two pytorch tensors that corresponds complex-valued embeddings :return:

forward_k_vs_all (*x*: *torch.Tensor*) → *torch.FloatTensor*

Score a (head, relation) batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_triples (*x*: *torch.Tensor*) → *torch.FloatTensor*

Score a batch of (head, relation, tail) index triples.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor where each row is [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

forward_k_vs_sample (*x*: *torch.Tensor*, *target_entity_idx*: *torch.Tensor*)

Score a (head, relation) batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns

Shape (batch_size, k) score matrix where *k* is the number of sampled target entities.

Return type

torch.FloatTensor

class `dicee.models.complex.AConEx` (*args*)

Bases: `dicee.models.base_model.BaseKGE`

Additive Convolutional ComplEx Knowledge Graph Embeddings

name = 'AConEx'

conv2d

fc_num_input

fc1

norm_fc1

bn_conv2d

feature_map_dropout

residual_convolution (*C_1*: *Tuple[torch.Tensor, torch.Tensor]*,
C_2: *Tuple[torch.Tensor, torch.Tensor]*) → *torch.FloatTensor*

Compute residual score of two complex-valued embeddings. :param C_1: a tuple of two pytorch tensors that corresponds complex-valued embeddings :param C_2: a tuple of two pytorch tensors that corresponds complex-valued embeddings :return:

forward_k_vs_all (*x*: *torch.Tensor*) → *torch.FloatTensor*

Score a (head, relation) batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_triples (*x*: *torch.Tensor*) → *torch.FloatTensor*

Score a batch of (head, relation, tail) index triples.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor where each row is [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

forward_k_vs_sample (*x*: *torch.Tensor*, *target_entity_idx*: *torch.Tensor*)

Score a (head, relation) batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns

Shape (batch_size, k) score matrix where *k* is the number of sampled target entities.

Return type

torch.FloatTensor

class `dicee.models.complex.Complex` (*args*)

Bases: `dicee.models.base_model.BaseKGE`

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from `BaseKGELightning` and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- `forward_triples()` — score a batch of (h, r, t) triples.
- `forward_k_vs_all()` — score a (h, r) batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

```

name = 'Complex'

static score(head_ent_emb: torch.FloatTensor, rel_ent_emb: torch.FloatTensor,
            tail_ent_emb: torch.FloatTensor)

static k_vs_all_score(emb_h: torch.FloatTensor, emb_r: torch.FloatTensor,
                    emb_E: torch.FloatTensor)

```

Parameters

- `emb_h`
- `emb_r`
- `emb_E`

forward_k_vs_all (*x*: torch.LongTensor) → torch.FloatTensor

Score a (head, relation) batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_k_vs_sample (*x*: torch.LongTensor, target_entity_idx: torch.LongTensor)

Score a (head, relation) batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns

Shape (batch_size, k) score matrix where *k* is the number of sampled target entities.

Return type

torch.FloatTensor

```
class dicee.models.complex.RotatE(args)
```

Bases: `dicee.models.base_model.BaseKGE`

RotatE: knowledge graph embedding by relational rotation in complex space.

Represents each entity as a complex vector in $\mathbb{C}^{(d/2)}$, obtained by splitting its *d*-dimensional embedding into a real half and an imaginary half. Each relation is represented by *d*/2 phase angles θ that define a unit-modulus rotation $r_i = e^{\{i\theta_i\}}$; since a phase angle needs only one real number, the relation embedding table is reinitialised at half of the entity embedding size. A true triple (*h*, *r*, *t*) should satisfy $h \circ r \approx t$ under element-wise complex multiplication, giving the score:

$$f(h, r, t) = \text{margin} - ||h \circ r - t||_2$$

Unlike TransE, RotatE can model symmetric, antisymmetric, inverse, and composition relation patterns.

References

Sun et al., *RotatE: Knowledge Graph Embedding by Relational Rotation in Complex Space*, ICLR 2019. <https://arxiv.org/abs/1902.10197>

```
name = 'RotatE'
```


margin

half_dim

relation_embeddings

score (*head_ent_emb*: torch.FloatTensor, *rel_ent_emb*: torch.FloatTensor,
tail_ent_emb: torch.FloatTensor) → torch.FloatTensor

Score a batch of triples using the RotatE margin-distance formula.

Parameters

- **head_ent_emb** (torch.FloatTensor) – Each has shape (batch_size, embedding_dim).
- **tail_ent_emb** (torch.FloatTensor) – Each has shape (batch_size, embedding_dim).
- **rel_ent_emb** (torch.FloatTensor) – Shape (batch_size, d/2) relation phase angles θ .

Returns

Shape (batch_size,) scores equal to $\text{margin} - ||h \circ r - t||_2$.

Return type

torch.FloatTensor

forward_k_vs_all (*x*: torch.Tensor) → torch.FloatTensor

KvsAll forward pass: score head/relation against all entities.

Computes $\text{margin} - ||h \circ r - e||_2$ for every entity embedding e .

Parameters

x (torch.Tensor) – Shape (batch_size, 2) integer tensor [head_idx, relation_idx].

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_k_vs_sample (*x*: torch.Tensor, *target_entity_idx*: torch.Tensor) → torch.FloatTensor

KvsSample forward pass: score head/relation against a sampled entity subset.

Computes $\text{margin} - ||h \circ r - e||_2$ for each of the k sampled entities e .

Parameters

- **x** (torch.Tensor) – Shape (batch_size, 2) integer tensor [head_idx, relation_idx].
- **target_entity_idx** (torch.Tensor) – Shape (batch_size, k) indices of the k target entities per sample.

Returns

Shape (batch_size, k) score matrix.

Return type

torch.FloatTensor

dicee.models.dualE

Classes

<i>DualE</i>	Dual Quaternion Knowledge Graph Embeddings (https://ojs.aaai.org/index.php/AAAI/article/download/16850/16657)
--------------	--

Module Contents

class dicee.models.dualE.**DualE**(args)

Bases: *dicee.models.base_model.BaseKGE*

Dual Quaternion Knowledge Graph Embeddings (<https://ojs.aaai.org/index.php/AAAI/article/download/16850/16657>)

name = 'DualE'

entity_embeddings

relation_embeddings

num_ent = None

kvsall_score(*e_1_h, e_2_h, e_3_h, e_4_h, e_5_h, e_6_h, e_7_h, e_8_h, e_1_t, e_2_t, e_3_t, e_4_t, e_5_t, e_6_t, e_7_t, e_8_t, r_1, r_2, r_3, r_4, r_5, r_6, r_7, r_8*) → torch.tensor

KvsAll scoring function

Input

x: torch.LongTensor with (n,) shape

Output

torch.FloatTensor with (n) shape

forward_triples(*idx_triple: torch.tensor*) → torch.tensor

Negative Sampling forward pass:

Input

x: torch.LongTensor with (n,) shape

Output

torch.FloatTensor with (n) shape

forward_k_vs_all(*x*)

KvsAll forward pass

Input

x: torch.LongTensor with (n,) shape

Output

torch.FloatTensor with (n) shape

$\mathbf{T}(x: \text{torch.tensor}) \rightarrow \text{torch.tensor}$

Transpose function

Input: Tensor with shape (nxm) Output: Tensor with shape (mxn)

dicее.models.ensemble

Classes

EnsembleKGE

Module Contents

```
class dicее.models.ensemble.EnsembleKGE (models: list = None, seed_model=None,
    pretrained_models: List = None)

    name

    train_mode = True

    args

    named_children()

    property example_input_array

    parameters()

    modules()

    __iter__()

    __len__()

    eval()

    to(device)

    state_dict()
        Return the state dict of the ensemble.

    load_state_dict(state_dict, strict=True)
        Load the state dict into the ensemble.

    mem_of_model()

    __call__(x_batch)

    step()

    get_embeddings()

    __str__()
```

dicee.models.flock

Pure PyTorch Flock, compatible with both official jw9730/flock checkpoints.

Architecture adapted from <https://github.com/jw9730/flock> at UPSTREAM_COMMIT. See docs/flock.md for random-walk and numerical parity details.

MIT License

Copyright (c) 2025 Jinwoo Kim

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Attributes

<code>UPSTREAM_COMMIT</code>

Classes

<code>RMSNorm</code>	Base class for all neural network modules.
<code>FeedForward</code>	Base class for all neural network modules.
<code>BidirectionalGRU</code>	Base class for all neural network modules.
<code>FlockBase</code>	Shared graph lifecycle and entity-scoring interfaces for graph foundation models.
<code>Flock</code>	Entity-prediction Flock with official <code>flock_entity.pth</code> parameters.
<code>FlockRelation</code>	Relation-prediction Flock with official <code>flock_relation.pth</code> parameters.

Functions

<code>consensus(x, logits, ids, num_ids)</code>	Stable multihead softmax pooling over all occurrences of each graph ID.
---	---

Module Contents

```
dicee.models.flock.UPSTREAM_COMMIT = 'f35103d25a78bdf4075de5c673a51de4979aa4d7'
```

```
class dicee.models.flock.RMSNorm(dim, eps=1e-05)
```

Bases: torch.nn.Module

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

eps = 1e-05

inference_backend = 'auto'

weight

forward (*x*)

```
class dicee.models.flock.FeedForward(dim)
```

Bases: torch.nn.Module

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F
```

(continues on next page)

(continued from previous page)

```
class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

w1

w2

w3

forward(*x*)

class `diccее.models.flock.BidirectionalGRU` (*dim, layers*)

Bases: `torch.nn.Module`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

gru_norm

gru

gru_out

ffn_norm

feed_forward

forward (*x*)

`dicee.models.flock.consensus` (*x*, *logits*, *ids*, *num_ids*)

Stable multihead softmax pooling over all occurrences of each graph ID.

class `dicee.models.flock.FlockBase` (*args*)

Bases: `dicee.models.graph_model.GraphKGE`

Shared graph lifecycle and entity-scoring interfaces for graph foundation models.

config_prefix = 'flock'

graph_filename = 'flock_graph.pt'

relation_prediction = False

checkpoint_hint = 'official Flock requires dim=64, walk_len=128, refinements=6, num_layers=1, attention_heads=4...'

emb_anon_node: `torch.nn.ModuleList`

emb_anon_type: `torch.nn.ModuleList`

emb_restart: `torch.nn.ModuleList`

emb_neighbor: `torch.nn.ModuleList`

emb_direction: `torch.nn.ModuleList`

emb_head_is_query: `torch.nn.ModuleList`

emb_tail_is_query: `torch.nn.ModuleList`

emb_node_is_query: `torch.nn.ModuleList`

emb_type_is_query: `torch.nn.ModuleList`

```

from_node: torch.nn.ModuleList
from_type: torch.nn.ModuleList
to_node: torch.nn.ModuleList
to_type: torch.nn.ModuleList
node_logit: torch.nn.ModuleList
type_logit: torch.nn.ModuleList
dim
walk_num
walk_len
refinements
test_samples
seed
prefetch_walks
compact_state
compile_sampler
pack_walks
node_init
type_init
net
head

sample_walks (heads, tails=None, edges=None, generator=None)
    Sample official-format records; IDs use the attached graph vocabulary.

    Returns (nodes, anonymous nodes, restarts, neighbors, relations, anonymous relations, directions), each
    shaped [T,B,S,L]. Anonymous names start at 1; no-relation markers are 2*R and L+1 respectively.

score_walks (heads, query, candidates, records)
    Score fixed walks for parity/replay; query/candidates use internal IDs.

    Entity mode: query=relations, candidates=tails. Relation mode: query=tails, candidates=relations. This
    scores the supplied records; callers constructing training records must first remove target edges.

class dicee.models.flock.Flock (args)
    Bases: FlockBase

    Entity-prediction Flock with official flock_entity.pth parameters.

    name = 'Flock'

```



```

class dicee.models.flock.FlockRelation(args)
    Bases: FlockBase, dicee.models.graph_model.RelationGraphKGE
    Relation-prediction Flock with official flock_relation.pth parameters.
    name = 'FlockRelation'
    relation_prediction = True

```

dicee.models.flock_walks

Flock's non-backtracking walk and recording protocol in PyTorch.

Semantics follow jw9730/flock at f35103d25a78bdf4075de5c673a51de4979aa4d7. The RNG is PyTorch's CPU generator, not the reference C++ per-walk MT19937. Walk records can be supplied explicitly to compare neural computations exactly.

Classes

<i>WalkGraph</i>	CPU CSR context, kept outside a model's transferable parameters.
------------------	--

Functions

<i>compiled_walk_nodes()</i>	
<i>anonymize(values[, missing_id, missing_name])</i>	Assign 1-based names in order of discovery, independently in each row.

Module Contents

```

dicee.models.flock_walks.compiled_walk_nodes()

```

```

dicee.models.flock_walks.anonymize(values, missing_id=None, missing_name=None)

```

Assign 1-based names in order of discovery, independently in each row.

```

class dicee.models.flock_walks.WalkGraph

```

CPU CSR context, kept outside a model's transferable parameters.

```

    edge_index: torch.Tensor

```

```

    edge_type: torch.Tensor

```

```

    num_nodes: int

```

```

    num_types: int

```

```

    compile_sampler: bool = True

```

```

    __post_init__()

```

```

    walk(prefix, length, remove_loops, generator=None, prefix_types=None)

```

Sample uniform neighbors, then a uniform typed edge in either direction.

```

    parse_types(walks, generator=None)

```

sample (*heads, tails, walk_num, length, refinements, generator=None*)

Return seven official-format records of shape [T,B,3N or 4N,L].

dicee.models.fsdp_models

Row-wise sharded entity embeddings for multi-GPU training.

Entity embeddings are partitioned row-wise across ranks:

- `dist.all_to_all_single` routes index requests to the owning rank
- A custom autograd Function propagates gradients back through the `all_to_all`
- `_LocalSparseAdam` updates only the rows that received a non-zero gradient

Public API:

`model.setup_fsdp_training(device, lr)` — called by trainer before loop `model.gather_entity_embeddings_on_rank_zero()` — called by trainer after loop `model.save_local_shard_checkpoint(path)` — per-rank, called periodically during loop `model.load_local_shard_checkpoint(path)` — per-rank, called by trainer on resume `create_fsdp_sharded_model_class(ModelClass)` — factory used by `static_funcs.py`

Classes

<code>RowWiseShardedEmbedding</code>	Entity embedding table sharded row-wise across ranks.
<code>FSDPShardedEntityModel</code>	Mixin that defers entity embedding allocation and wires up row-wise sharding.

Functions

<code>create_fsdp_sharded_model_class(...)</code>	Return a row-wise sharded variant of <i>model_class</i> .
---	---

Module Contents

class `dicee.models.fsdp_models.RowWiseShardedEmbedding` (*num_embeddings: int, embedding_dim: int, rank: int, world_size: int, device: torch.device, weight_dtype: torch.dtype = torch.bfloat16*)

Bases: `torch.nn.Module`

Entity embedding table sharded row-wise across ranks.

Each rank owns rows [start_row, end_row). `forward()` looks like `nn.Embedding` so every existing scoring function works unchanged.

num_embeddings

embedding_dim

rank

world_size

device

shard_size

```

start_row

end_row

local_rows

weight

forward(entity_ids: torch.LongTensor) → torch.FloatTensor

```

```
class dicee.models.fsdps_models.FSDPShardedEntityModel(args)
```

Bases: `dicee.models.base_model.BaseKGE`

Mixin that defers entity embedding allocation and wires up row-wise sharding.

Lifecycle

1. `__init__()` — entity_embeddings is None
2. `setup_fsdps_training(dev, lr)` — creates RowWiseShardedEmbedding
3. `gather_entity_embeddings_on_rank_zero()` — called by trainer after last epoch

```
setup_fsdps_training(device: torch.device, lr: float, optimizer_cls=None,
                    optimizer_kwargs: dict = None, adam_device: torch.device = None) → None
```

Create the per-rank embedding shard and its local sparse Adam.

adam_device — where exp_avg / exp_avg_sq are stored:

GPU (default) : pure GPU kernels, no PCIe, ~42.8 GiB extra GPU RAM per rank. CPU : lower GPU footprint, PCIe transfer each step.

Falls back to the embedding device (GPU) when not specified.

```
gather_entity_embeddings_on_rank_zero() → torch.nn.Embedding | None
```

Gather the full entity table on rank 0; return None on other ranks.

```
save_local_shard_checkpoint(path: str) → None
```

Save this rank's entity embedding shard + its local Adam state to *path*.

Cheap and O(local_rows) — unlike `gather_entity_embeddings_on_rank_zero()`, no collective communication is involved, so this is safe to call periodically even when the full table is too large to gather.

```
load_local_shard_checkpoint(path: str) → None
```

Load this rank's entity embedding shard + local Adam state from *path*.

The checkpoint must have been saved by a run with the same world_size: shard boundaries (start_row/local_rows) are derived from world_size and num_entities, so a mismatch means this rank does not own the same rows as when the checkpoint was written.

```
get_embeddings()
```

Return the entity and relation embedding matrices as numpy arrays.

Returns

- **entity_embeddings** (*numpy.ndarray*) — Shape (num_entities, embedding_dim).
- **relation_embeddings** (*numpy.ndarray*) — Shape (num_relations, embedding_dim).

forward_k_vs_all(*args, **kwargs)

Score a (head, relation) batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

`dicee.models.fsdps_models.create_fsdps_sharded_model_class`(
 model_class: Type[dicee.models.base_model.BaseKGE])
 → Type[dicee.models.base_model.BaseKGE]

Return a row-wise sharded variant of *model_class*.

The returned class inherits all scoring functions from *model_class* unchanged. Only *entity_embeddings* is replaced at training time.

dicee.models.function_space

Attributes

<i>logger</i>

Classes

<i>FMult</i>	Learning Knowledge Neural Graphs
<i>GFMult</i>	Learning Knowledge Neural Graphs
<i>FMult2</i>	Learning Knowledge Neural Graphs
<i>LFMult1</i>	Embedding with trigonometric functions. We represent all entities and relations in the complex number space as:
<i>LFMult</i>	Embedding with polynomial functions. We represent all entities and relations in the polynomial space as:

Module Contents

`dicee.models.function_space.logger`

class `dicee.models.function_space.FMult`(*args*)

Bases: `dicee.models.base_model.BaseKGE`

Learning Knowledge Neural Graphs

name = 'FMult'

entity_embeddings

relation_embeddings

k

num_sample = 50

gamma

roots

weights

compute_func (*weights: torch.FloatTensor*, *x*) → torch.FloatTensor

chain_func (*weights, x: torch.FloatTensor*)

forward_triples (*idx_triple: torch.Tensor*) → torch.Tensor

Score a batch of (head, relation, tail) index triples.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor where each row is [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

class dicee.models.function_space.**GFMult** (*args*)

Bases: *dicee.models.base_model.BaseKGE*

Learning Knowledge Neural Graphs

name = 'GFMult'

entity_embeddings

relation_embeddings

k

num_sample = 250

roots

weights

compute_func (*weights: torch.FloatTensor*, *x*) → torch.FloatTensor

chain_func (*weights, x: torch.FloatTensor*)

forward_triples (*idx_triple: torch.Tensor*) → torch.Tensor

Score a batch of (head, relation, tail) index triples.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor where each row is [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

```
class dicee.models.function_space.FMult2(args)
```

```
    Bases: dicee.models.base_model.BaseKGE
```

```
    Learning Knowledge Neural Graphs
```

```
    name = 'FMult2'
```

```
    n_layers = 3
```

```
    k
```

```
    n = 50
```

```
    score_func = 'compositional'
```

```
    discrete_points
```

```
    entity_embeddings
```

```
    relation_embeddings
```

```
    build_func(Vec)
```

```
    build_chain_funcs(list_Vec)
```

```
    compute_func(W, b, x) → torch.FloatTensor
```

```
    function(list_W, list_b)
```

```
    trapezoid(list_W, list_b)
```

```
    forward_triples(idx_triple: torch.Tensor) → torch.Tensor
```

```
        Score a batch of (head, relation, tail) index triples.
```

Parameters

```
    x (torch.LongTensor) – Shape (batch_size, 3) integer tensor where each row is
    [head_idx, relation_idx, tail_idx].
```

Returns

```
    Shape (batch_size,) triple scores.
```

Return type

```
    torch.FloatTensor
```

```
class dicee.models.function_space.LFMult1(args)
```

```
    Bases: dicee.models.base_model.BaseKGE
```

```
    Embedding with trigonometric functions. We represent all entities and relations in the complex number space as:
    f(x) = sum_{k=0}^{k=d-1} wk e^{kix}. and use the three differents scoring function as in the paper to evaluate
    the score
```

```
    name = 'LFMult1'
```

```
    entity_embeddings
```

```
    relation_embeddings
```

forward_triples (*idx_triple*)

Score a batch of (head, relation, tail) index triples.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor where each row is [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

tri_score (*h, r, t*)

vtp_score (*h, r, t*)

class dicee.models.function_space.**LFMult** (*args*)

Bases: *dicee.models.base_model.BaseKGE*

Embedding with polynomial functions. We represent all entities and relations in the polynomial space as: $f(x) = \sum_{i=0}^{d-1} a_k x^{i \bmod d}$ and use the three differents scoring function as in the paper to evaluate the score. We also consider combining with Neural Networks.

name = 'LFMult'

entity_embeddings

relation_embeddings

degree

m

x_values

forward_triples (*idx_triple*)

Score a batch of (head, relation, tail) index triples.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor where each row is [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

construct_multi_coeff (*x*)

poly_NN (*x, coefh, coefr, coeft*)

Constructing a 2 layers NN to represent the embeddings. $h = \text{sigma}(wh^T x + bh)$, $r = \text{sigma}(wr^T x + br)$, $t = \text{sigma}(wt^T x + bt)$

linear (*x, w, b*)

scalar_batch_NN (*a, b, c*)

element wise multiplication between a,b and c: Inputs : a, b, c ==> torch.tensor of size batch_size x m x d
Output : a tensor of size batch_size x d

tri_score (*coeff_h, coeff_r, coeff_t*)

this part implement the trilinear scoring techniques:

$$\text{score}(h,r,t) = \int_0^1 h(x)r(x)t(x) dx = \sum_{i,j,k=0}^{d-1} \text{dfrac}\{a_i*b_j*c_k\}\{1+(i+j+k)\%d\}$$

1. generate the range for i,j and k from [0 d-1]
2. perform $\text{dfrac}\{a_i*b_j*c_k\}\{1+(i+j+k)\%d\}$ in parallel for every batch
3. take the sum over each batch

vtp_score (*h, r, t*)

this part implement the vector triple product scoring techniques:

$$\text{score}(h,r,t) = \int_0^1 h(x)r(x)t(x) dx = \sum_{i,j,k=0}^{d-1} \text{dfrac}\{a_i*c_j*b_k - b_i*c_j*a_k\}\{(1+(i+j)\%d)(1+k)\}$$

1. generate the range for i,j and k from [0 d-1]
2. Compute the first and second terms of the sum
3. Multiply with then denominator and take the sum
4. take the sum over each batch

comp_func (*h, r, t*)

this part implement the function composition scoring techniques: i.e. $\text{score} = \langle \text{hor}, \text{t} \rangle$

polynomial (*coeff, x, degree*)

This function takes a matrix tensor of coefficients (*coeff*), a tensor vector of points *x* and range of integer [0,1,...d] and return a vector tensor ($\text{coeff}[0][0] + \text{coeff}[0][1]x + \dots + \text{coeff}[0][d]x^d$,

$$\text{coeff}[1][0] + \text{coeff}[1][1]x + \dots + \text{coeff}[1][d]x^d)$$

pop (*coeff, x, degree*)

This function allow us to evaluate the composition of two polynomes without for loops :) it takes a matrix tensor of coefficients (*coeff*), a matrix tensor of points *x* and range of integer [0,1,...d]

and return a tensor ($\text{coeff}[0][0] + \text{coeff}[0][1]x + \dots + \text{coeff}[0][d]x^d$,

$$\text{coeff}[1][0] + \text{coeff}[1][1]x + \dots + \text{coeff}[1][d]x^d)$$

dicee.models.graph_model

Shared graph-conditioned model support, extracted from the DICE ULTRA implementation.

Architecture and graph construction adapted from <https://github.com/DeepGraphLearning/ULTRA> at commit 427966ad8ed60420eef034063d44f3153addff90. Reference fixtures and the grouped sampling and adversarial loss implementations also follow this upstream work. Graphs are runtime context, excluded from the transferable state dictionary.

MIT License

Copyright (c) 2023 MilaGraph

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT

HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Classes

<i>GraphKGE</i>	Shared graph lifecycle and entity-scoring interfaces for graph foundation models.
<i>RelationGraphKGE</i>	Shared relation prediction API with (head, tail) query pairs.

Module Contents

class `dicee.models.graph_model.GraphKGE` (*args*)

Bases: `dicee.models.base_model.BaseKGE`

Shared graph lifecycle and entity-scoring interfaces for graph foundation models.

name = 'GraphKGE'

config_prefix = 'graph'

graph_filename = 'graph.pt'

checkpoint_hint = 'all keys and shapes must match'

deterministic_inference = False

query_batch_size

num_direct_relations = 0

clear_inference_cache()

Drop derived representations when graph, weights or device change.

set_inference_backend (*backend*)

inference_token()

train (*mode=True*)

Set the module in training mode.

This has an effect only on certain modules. See the documentation of particular modules for details of their behaviors in training/evaluation mode, i.e., whether they are affected, e.g. Dropout, BatchNorm, etc.

Parameters

mode (*bool*) – whether to set training mode (True) or evaluation mode (False). Default: True.

Returns

self

Return type

Module

init_entity_embeddings (*embedding_dim=None*)

Create (or re-create) the entity embedding table.

This is the single place that honours `defer_large_embeddings`: when entity rows are sharded across FSDP ranks the table must stay `None` until the trainer allocates the sharded adapter. Subclasses that need an entity table of a non-default width must call this method instead of assigning `self.entity_embeddings` directly, so the deferral is never silently undone.

Parameters

embedding_dim (*Optional[int]*) – Width of the table. Defaults to `embedding_dim`.

init_relation_embeddings (*embedding_dim=None*)

Create (or re-create) the relation embedding table.

Relation tables are never deferred - they are small enough to be replicated on every rank.

Parameters

embedding_dim (*Optional[int]*) – Width of the table. Defaults to `embedding_dim`.

abstractmethod get_embeddings ()

Return the entity and relation embedding matrices as numpy arrays.

Returns

- **entity_embeddings** (*numpy.ndarray*) – Shape (num_entities, embedding_dim).
- **relation_embeddings** (*numpy.ndarray*) – Shape (num_relations, embedding_dim).

set_graph (*triples, num_entities=None, num_relations=None, inverse_relations=None*)

Attach training facts; `inverse_relations` maps direct DICE IDs to inverse IDs.

Without a mapping every supplied relation is treated as a direct relation. Repeated facts and explicitly supplied inverse facts are deduplicated.

save_graph (*path*)

load_graph (*path*)

load_pretrained (*path*)

forward_grouped (*triples, head_prediction=None*)

forward_triples (*x*)

Score a batch of (head, relation, tail) index triples.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor where each row is [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

`torch.FloatTensor`

forward_k_vs_sample (*x, target_entity_idx*)

Score a (head, relation) batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns

Shape (batch_size, k) score matrix where *k* is the number of sampled target entities.

Return type

torch.FloatTensor

forward_k_vs_all(*x*)

Score a (head, relation) batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_k_vs_all_heads(*x*, *target_entity_idx=None*)

Score head candidates for DICE pairs (relation, tail).

forward(*x*, *y_idx=None*)

Route the forward pass to the appropriate scoring method.

Inspects the shape and type of *x* to decide which low-level scorer to call:

- Tuple (*x*, *y_idx*) → `forward_k_vs_sample()`
- (batch, 3) tensor → `forward_triples()`
- (batch, 2) tensor → `forward_k_vs_all()`
- BPE triple tensor → `forward_byte_pair_encoded_triple()`
- BPE pair tensor → `forward_byte_pair_encoded_k_vs_all()`

Parameters

- **x** (*torch.LongTensor* or *Tuple[torch.LongTensor, torch.LongTensor]*) – Either a plain index tensor or a (*triple_idx*, *target_idx*) tuple for sample-based labelling.
- **y_idx** (*torch.LongTensor*, optional) – Target entity indices used by `forward_k_vs_sample()`. Ignored when *x* is a plain tensor.

Returns

Score tensor whose shape depends on the selected scorer.

Return type

torch.FloatTensor

class `dicee.models.graph_model.RelationGraphKGE`(*args*)

Bases: `GraphKGE`

Shared relation prediction API with (head, tail) query pairs.

forward_grouped(*triples*, *head_prediction=None*)

forward_triples(*x*)

Score a batch of (head, relation, tail) index triples.

Parameters

- **x** (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor where each row is [*head_idx*, *relation_idx*, *tail_idx*].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

forward_k_vs_sample (*x*, *target_entity_idx*)

Score relation candidates [K] or [B,K] for (head, tail) pairs.

forward_k_vs_all (*x*)

Score a (head, relation) batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_k_vs_all_relations

forward_k_vs_all_heads (*x*, *target_entity_idx=None*)

Score head candidates for DICE pairs (relation, tail).

dicee.models.literal**Classes**

LiteralEmbeddings

A model for learning and predicting numerical literals using pre-trained KGE.

Module Contents

```
class dicee.models.literal.LiteralEmbeddings (num_of_data_properties: int, embedding_dims: int,
        entity_embeddings: torch.tensor, dropout: float = 0.3, gate_residual=True,
        freeze_entity_embeddings=True)
```

Bases: `torch.nn.Module`

A model for learning and predicting numerical literals using pre-trained KGE.

num_of_data_properties

Number of data properties (attributes).

Type

int

embedding_dims

Dimension of the embeddings.

Type

int

entity_embeddings

Pre-trained entity embeddings.

Type
torch.tensor

dropout
Dropout rate for regularization.

Type
float

gate_residual
Whether to use gated residual connections.

Type
bool

freeze_entity_embeddings
Whether to freeze the entity embeddings during training.

Type
bool

embedding_dim

num_of_data_properties

hidden_dim

gate_residual = True

freeze_entity_embeddings = True

entity_embeddings

data_property_embeddings

fc

fc_out

dropout

gated_residual_proj

layer_norm

forward (*entity_idx*, *attr_idx*)

Parameters

- **entity_idx** (*Tensor*) – Entity indices (batch).
- **attr_idx** (*Tensor*) – Attribute (Data property) indices (batch).

Returns
scalar predictions.

Return type
Tensor

property device

dicee.models.octonion

Classes

<i>OMult</i>	Base class for all Knowledge Graph Embedding models.
<i>ConvO</i>	Base class for all Knowledge Graph Embedding models.
<i>AConvO</i>	Additive Convolutional Octonion Knowledge Graph Embeddings

Functions

<i>octonion_mul</i> (*, O_1, O_2)
<i>octonion_mul_norm</i> (*, O_1, O_2)

Module Contents

`dicee.models.octonion.octonion_mul(*, O_1, O_2)`

`dicee.models.octonion.octonion_mul_norm(*, O_1, O_2)`

class `dicee.models.octonion.OMult`(args)

Bases: `dicee.models.base_model.BaseKGE`

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from `BaseKGELightning` and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- `forward_triples()` — score a batch of (h, r, t) triples.
- `forward_k_vs_all()` — score a (h, r) batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

name = 'OMult'

static `octonion_normalizer`(*emb_rel_e0, emb_rel_e1, emb_rel_e2, emb_rel_e3, emb_rel_e4, emb_rel_e5, emb_rel_e6, emb_rel_e7*)

score(*head_ent_emb: torch.FloatTensor, rel_ent_emb: torch.FloatTensor, tail_ent_emb: torch.FloatTensor*)

k_vs_all_score(*bpe_head_ent_emb, bpe_rel_ent_emb, E*)

forward_k_vs_all(*x*)

Completed. Given a head entity and a relation (h,r), we compute scores for all possible triples, i.e., `[score(h,r,x)|x in Entities] => [0.0,0.1,...,0.8]`, shape=> (1, **Entities**) Given a batch of head entities and relations => shape (size of batch, |Entities|)

```
class dicee.models.octonion.ConvO(args: dict)
```

Bases: `dicee.models.base_model.BaseKGE`

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from `BaseKGELightning` and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- `forward_triples()` — score a batch of (h, r, t) triples.
- `forward_k_vs_all()` — score a (h, r) batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

```
name = 'ConvO'
```

```
conv2d
```

```
fc_num_input
```

```
fc1
```

```
bn_conv2d
```

```
norm_fc1
```

```
feature_map_dropout
```

```
static octonion_normalizer(emb_rel_e0, emb_rel_e1, emb_rel_e2, emb_rel_e3, emb_rel_e4,  
emb_rel_e5, emb_rel_e6, emb_rel_e7)
```

```
residual_convolution(O_1, O_2)
```

```
forward_triples(x: torch.Tensor) → torch.Tensor
```

Score a batch of (head, relation, tail) index triples.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor where each row is [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

```
forward_k_vs_all(x: torch.Tensor)
```

Given a head entity and a relation (h,r), we compute scores for all entities. [score(h,r,x)|x in Entities] => [0.0,0.1,...,0.8], shape=> (1, **Entities**) Given a batch of head entities and relations => shape (size of batch,|Entities)

```
class dicee.models.octonion.AConvO(args: dict)
```

Bases: `dicee.models.base_model.BaseKGE`

Additive Convolutional Octonion Knowledge Graph Embeddings

```

name = 'AConvO'

conv2d

fc_num_input

fc1

bn_conv2d

norm_fc1

feature_map_dropout

static octonion_normalizer(emb_rel_e0, emb_rel_e1, emb_rel_e2, emb_rel_e3, emb_rel_e4,
                           emb_rel_e5, emb_rel_e6, emb_rel_e7)

residual_convolution(O_1, O_2)

forward_triples(x: torch.Tensor) → torch.Tensor
    Score a batch of (head, relation, tail) index triples.

    Parameters
        x (torch.LongTensor) – Shape (batch_size, 3) integer tensor where each row is
        [head_idx, relation_idx, tail_idx].

    Returns
        Shape (batch_size,) triple scores.

    Return type
        torch.FloatTensor

forward_k_vs_all(x: torch.Tensor)
    Given a head entity and a relation (h,r), we compute scores for all entities. [score(h,r,x)|x in Entities] =>
    [0.0,0.1,...,0.8], shape=> (1, Entities) Given a batch of head entities and relations => shape (size of batch,|
    Entities)

```

dicee.models.pykeen_models

Attributes

logger

Classes

PykeenKGE

A class for using knowledge graph embedding models implemented in Pykeen

Module Contents

`dicee.models.pykeen_models.logger`


```

class dicee.models.pykeen_models.PykeenKGE (args: dict)
    Bases: dicee.models.base_model.BaseKGE

    A class for using knowledge graph embedding models implemented in Pykeen

    Notes: Pykeen_DistMult: C Pykeen_ComplEx: Pykeen_QuatE: Pykeen_MuRE: Pykeen_CP: Pykeen_HolE: Py-
    keen_HolE: Pykeen_HolE: Pykeen_TransD: Pykeen_TransE: Pykeen_TransF: Pykeen_TransH: Pykeen_TransR:

    model_kwargs

    name

    model

    loss_history = []

    args

    entity_embeddings = None

    relation_embeddings = None

    forward_k_vs_all (x: torch.LongTensor)
        # => Explicit version by this we can apply bn and dropout

        # (1) Retrieve embeddings of heads and relations + apply Dropout & Normalization if given. h, r =
        self.get_head_relation_representation(x) # (2) Reshape (1). if self.last_dim > 0:

            h = h.reshape(len(x), self.embedding_dim, self.last_dim) r = r.reshape(len(x), self.embedding_dim,
            self.last_dim)

        # (3) Reshape all entities. if self.last_dim > 0:

            t = self.entity_embeddings.weight.reshape(self.num_entities, self.embedding_dim, self.last_dim)

        else:

            t = self.entity_embeddings.weight

        # (4) Call the score_t from interactions to generate triple scores. return self.interaction.score_t(h=h, r=r,
        all_entities=t, slice_size=1)

    forward_triples (x: torch.LongTensor) → torch.FloatTensor
        # => Explicit version by this we can apply bn and dropout

        # (1) Retrieve embeddings of heads, relations and tails and apply Dropout & Normalization if given. h, r, t =
        self.get_triple_representation(x) # (2) Reshape (1). if self.last_dim > 0:

            h = h.reshape(len(x), self.embedding_dim, self.last_dim) r = r.reshape(len(x), self.embedding_dim,
            self.last_dim) t = t.reshape(len(x), self.embedding_dim, self.last_dim)

        # (3) Compute the triple score return self.interaction.score(h=h, r=r, t=t, slice_size=None, slice_dim=0)

    abstractmethod forward_k_vs_sample (x: torch.LongTensor, target_entity_idx)
        Score a (head, relation) batch against a sampled subset of entities.

        Used by KvsSample and 1vsSample datasets. Sub-classes that support sample-based labelling must over-
        ride this method.

    Returns
        Shape (batch_size, k) score matrix where k is the number of sampled target entities.

```

Return type
torch.FloatTensor

dicее.models.quaternion

Classes

<i>QMult</i>	Base class for all Knowledge Graph Embedding models.
<i>ConvQ</i>	Convolutional Quaternion Knowledge Graph Embeddings
<i>ACovQ</i>	Additive Convolutional Quaternion Knowledge Graph Embeddings

Functions

<i>quaternion_mul_with_unit_norm</i> (*, Q_1, Q_2)
--

Module Contents

dicее.models.quaternion.**quaternion_mul_with_unit_norm**(*, Q_1, Q_2)

class dicее.models.quaternion.**QMult** (*args*)

Bases: *dicее.models.base_model.BaseKGE*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from `BaseKGELightning` and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- `forward_triples()` — score a batch of (*h*, *r*, *t*) triples.
- `forward_k_vs_all()` — score a (*h*, *r*) batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

name = 'QMult'

explicit = True

quaternion_multiplication_followed_by_inner_product (*h*, *r*, *t*)

Parameters

- **h** – shape: (**batch_dims*, dim) The head representations.
- **r** – shape: (**batch_dims*, dim) The head representations.
- **t** – shape: (**batch_dims*, dim) The tail representations.

Returns

Triple scores.

static quaternion_normalizer (*x*: torch.FloatTensor) → torch.FloatTensor

Normalize the length of relation vectors, if the forward constraint has not been applied yet.

Absolute value of a quaternion

$$|a + bi + cj + dk| = \sqrt{a^2 + b^2 + c^2 + d^2}$$

L2 norm of quaternion vector:

$$\|x\|^2 = \sum_{i=1}^d |x_i|^2 = \sum_{i=1}^d (x_i.re^2 + x_i.im_1^2 + x_i.im_2^2 + x_i.im_3^2)$$

Parameters

x – The vector.

Returns

The normalized vector.

score (*head_ent_emb*: torch.FloatTensor, *rel_ent_emb*: torch.FloatTensor, *tail_ent_emb*: torch.FloatTensor)

k_vs_all_score (*bpe_head_ent_emb*, *bpe_rel_ent_emb*, *E*)

Parameters

- **bpe_head_ent_emb**
- **bpe_rel_ent_emb**
- **E**

forward_k_vs_all (*x*)

Parameters

x

forward_k_vs_sample (*x*, *target_entity_idx*)

Completed. Given a head entity and a relation (h,r), we compute scores for all possible triples,i.e., [score(h,r,x)|x in Entities] => [0.0,0.1,...,0.8], shape=> (1, **|Entities|**) Given a batch of head entities and relations => shape (size of batch,| Entities|)

class dicee.models.quaternion.**ConvQ** (*args*)

Bases: *dicee.models.base_model.BaseKGE*

Convolutional Quaternion Knowledge Graph Embeddings

name = 'ConvQ'

entity_embeddings

relation_embeddings

conv2d

fc_num_input

fc1

bn_conv1

bn_conv2

```

feature_map_dropout

residual_convolution(Q_1, Q_2)

forward_triples(indexed_triple: torch.Tensor) → torch.Tensor
    Score a batch of (head, relation, tail) index triples.

    Parameters
        x (torch.LongTensor) – Shape (batch_size, 3) integer tensor where each row is
        [head_idx, relation_idx, tail_idx].

    Returns
        Shape (batch_size,) triple scores.

    Return type
        torch.FloatTensor

forward_k_vs_all(x: torch.Tensor)
    Given a head entity and a relation (h,r), we compute scores for all entities. [score(h,r,x)|x in Entities] =>
    [0.0,0.1,...,0.8], shape=> (1, Entities) Given a batch of head entities and relations => shape (size of batch,|
    Entities)

class dicee.models.quaternion.AConvQ(args)
    Bases: dicee.models.base_model.BaseKGE
    Additive Convolutional Quaternion Knowledge Graph Embeddings

    name = 'AConvQ'

    entity_embeddings

    relation_embeddings

    conv2d

    fc_num_input

    fc1

    bn_conv1

    bn_conv2

    feature_map_dropout

    residual_convolution(Q_1, Q_2)

    forward_triples(indexed_triple: torch.Tensor) → torch.Tensor
        Score a batch of (head, relation, tail) index triples.

        Parameters
            x (torch.LongTensor) – Shape (batch_size, 3) integer tensor where each row is
            [head_idx, relation_idx, tail_idx].

        Returns
            Shape (batch_size,) triple scores.

        Return type
            torch.FloatTensor

```

forward_k_vs_all (*x*: *torch.Tensor*)

Given a head entity and a relation (h,r), we compute scores for all entities. [score(h,r,x)|x in Entities] => [0.0,0.1,...,0.8], shape=> (1, **Entities**) Given a batch of head entities and relations => shape (size of batch,|Entities)

dicee.models.real

Classes

<i>DistMult</i>	DistMult: bilinear diagonal knowledge graph embedding.
<i>TransE</i>	TransE: translation-based knowledge graph embedding.
<i>TransH</i>	TransH: translation-based knowledge graph embedding on relation-specific hyperplanes.
<i>MuRE</i>	MuRE: multi-relational graph embedding with relation-specific diagonal scaling, translation, and entity biases.
<i>Shallom</i>	Shallom: shallow neural model for relation prediction.
<i>Pyke</i>	Pyke: Physical Embedding Model for Knowledge Graphs.
<i>CoKEConfig</i>	Configuration for the CoKE (Contextualized Knowledge Graph Embedding) model.
<i>CoKE</i>	Contextualized Knowledge Graph Embedding (CoKE) model.

Module Contents

class `dicee.models.real.DistMult` (*args*)

Bases: `dicee.models.base_model.BaseKGE`

DistMult: bilinear diagonal knowledge graph embedding.

Scores a triple (h, r, t) as the element-wise product of the head, relation, and tail embeddings summed over the embedding dimension:

$$f(h, r, t) = \sum_i h_i \cdot r_i \cdot t_i$$

Simple yet effective baseline; incapable of modelling asymmetric relations.

References

Yang et al., *Embedding Entities and Relations for Learning and Inference in Knowledge Bases*, ICLR 2015. <https://arxiv.org/abs/1412.6575>

name = 'DistMult'

k_vs_all_score (*emb_h*: *torch.FloatTensor*, *emb_r*: *torch.FloatTensor*, *emb_E*: *torch.FloatTensor*)
→ *torch.FloatTensor*

Score a head/relation batch against all entity embeddings.

Computes (h * r) @ E^T after applying hidden dropout and normalisation to the element-wise product.

Parameters

- **emb_h** (*torch.FloatTensor*) – Head entity embeddings, shape (batch_size, embedding_dim).
- **emb_r** (*torch.FloatTensor*) – Relation embeddings, shape (batch_size, embedding_dim).

- **emb_E** (*torch.FloatTensor*) – All entity embeddings, shape (num_entities, embedding_dim).

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_k_vs_all (*x: torch.LongTensor*) → *torch.FloatTensor*

KvsAll forward pass: score head/relation against all entities.

Parameters

- **x** (*torch.LongTensor*) – Shape (batch_size, 2) integer tensor [head_idx, relation_idx].

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_k_vs_sample (*x: torch.LongTensor, target_entity_idx: torch.LongTensor*) → *torch.FloatTensor*

KvsSample forward pass: score head/relation against a sampled entity subset.

Parameters

- **x** (*torch.LongTensor*) – Shape (batch_size, 2) integer tensor [head_idx, relation_idx].
- **target_entity_idx** (*torch.LongTensor*) – Shape (batch_size, k) indices of the k target entities per sample.

Returns

Shape (batch_size, k) score matrix.

Return type

torch.FloatTensor

score (*h: torch.FloatTensor, r: torch.FloatTensor, t: torch.FloatTensor*) → *torch.FloatTensor*

Score a batch of (head, relation, tail) embedding triples.

Parameters

- **h** (*torch.FloatTensor*) – Each has shape (batch_size, embedding_dim).
- **r** (*torch.FloatTensor*) – Each has shape (batch_size, embedding_dim).
- **t** (*torch.FloatTensor*) – Each has shape (batch_size, embedding_dim).

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

class dicee.models.real.**TransE** (*args*)

Bases: *dicee.models.base_model.BaseKGE*

TransE: translation-based knowledge graph embedding.

Models a relation *r* as a translation in embedding space such that $h + r \approx t$ for a true triple (*h*, *r*, *t*). The score function is defined as:

$$f(h, r, t) = \text{margin} - ||h + r - t||_2$$

TransE is effective for 1-to-1 relations but struggles with reflexive, one-to-many, and many-to-one patterns.

References

Bordes et al., *Translating Embeddings for Modeling Multi-relational Data*, NeurIPS 2013. <https://proceedings.neurips.cc/paper/2013/file/1cecc7a77928ca8133fa24680a88d2f9-Paper.pdf>

name = 'TransE'

margin

score (*head_ent_emb*: torch.FloatTensor, *rel_ent_emb*: torch.FloatTensor, *tail_ent_emb*: torch.FloatTensor) → torch.FloatTensor

Score a batch of triples using the TransE margin-distance formula.

Parameters

- **head_ent_emb** (torch.FloatTensor) – Each has shape (batch_size, embedding_dim).
- **rel_ent_emb** (torch.FloatTensor) – Each has shape (batch_size, embedding_dim).
- **tail_ent_emb** (torch.FloatTensor) – Each has shape (batch_size, embedding_dim).

Returns

Shape (batch_size,) scores equal to $\text{margin} - ||h + r - t||_2$.

Return type

torch.FloatTensor

forward_k_vs_all (*x*: torch.Tensor) → torch.FloatTensor

KvsAll forward pass: score head/relation against all entities.

Computes $\text{margin} - ||h + r - e||_2$ for every entity embedding e .

Parameters

x (torch.Tensor) – Shape (batch_size, 2) integer tensor [head_idx, relation_idx].

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

class dicee.models.real.**TransH** (*args*)

Bases: `dicee.models.base_model.BaseKGE`

TransH: translation-based knowledge graph embedding on relation-specific hyperplanes.

Addresses TransE's inability to model one-to-many, many-to-one, and many-to-many relations by letting each relation r define its own hyperplane. An entity is first projected onto that hyperplane before the translation is applied, so the same entity can be positioned differently depending on the relation. Concretely, for a triple (h, r, t) with unit-norm hyperplane normal r_w and in-hyperplane translation vector r_d :

```

h_r = h - (r_w^T h) * r_w
t_r = t - (r_w^T t) * r_w
f(h, r, t) = margin - ||h_r + r_d - t_r||_2^2

```

`r_w` reuses the inherited `relation_embeddings` table and is normalised to unit norm before every use, since only a unit-norm normal yields a true orthogonal projection onto the hyperplane. `r_d` is a second, relation-specific embedding table analogous to TransE’s relation vector.

References

Wang et al., *Knowledge Graph Embedding by Translating on Hyperplanes*, AAAI 2014. <https://aaai.org/papers/8870-knowledge-graph-embedding-by-translating-on-hyperplanes/>

name = 'TransH'

margin

relation_normal_translations

forward_triples (*x*: *torch.LongTensor*) → *torch.FloatTensor*

Score a batch of (head, relation, tail) index triples.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

forward_k_vs_all (*x*: *torch.LongTensor*) → *torch.FloatTensor*

KvsAll forward pass: score head/relation against all entities.

Every candidate tail entity must be projected onto the batch row’s relation-specific hyperplane, and the hyperplane normal differs per batch row, so unlike TransE this materialises a full (batch_size, num_entities, embedding_dim) tensor of projected tail candidates – O(B * E * D) memory, versus TransE’s O(B * E).

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 2) integer tensor [head_idx, relation_idx].

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_k_vs_sample (*x*: *torch.LongTensor*, *target_entity_idx*: *torch.LongTensor*) → *torch.FloatTensor*

KvsSample forward pass: score head/relation against a sampled entity subset.

Same hyperplane-projection logic as `forward_k_vs_all()`, but restricted to the *k* sampled tail candidates instead of every entity, so it costs O(B * k * D) memory instead of O(B * E * D).

Parameters

• **x** (*torch.LongTensor*) – Shape (batch_size, 2) integer tensor [head_idx, relation_idx].

- **target_entity_idx** (*torch.LongTensor*) – Shape (batch_size, k) indices of the k target entities per sample.

Returns

Shape (batch_size, k) score matrix.

Return type

torch.FloatTensor

class `dicee.models.real.MuRE` (*args*)

Bases: `dicee.models.base_model.BaseKGE`

MuRE: multi-relational graph embedding with relation-specific diagonal scaling, translation, and entity biases.

Scores a triple (h, r, t) as:

$$f(h, r, t) = -||R_r \odot h + t_r - t||_2 + b_h + b_t$$

R_r is a relation-specific diagonal matrix applied to the head via an element-wise (Hadamard) product; it reuses the inherited `relation_embeddings` table directly, shape (num_relations, embedding_dim). t_r is a second, relation-specific translation vector (own embedding table, same shape convention: (num_relations, embedding_dim)) analogous to TransE's relation vector. b_h and b_t are learnable scalar biases indexed per entity (head and tail respectively), not per relation.

References

Balažević et al., *Multi-relational Poincaré Graph Embeddings*, NeurIPS 2019. <https://arxiv.org/abs/1905.09791>

name = 'MuRE'

relation_translations

b_h

b_t

forward_triples (*x: torch.LongTensor*) → *torch.FloatTensor*

Score a batch of (head, relation, tail) index triples.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

forward_k_vs_all (*x: torch.LongTensor*) → *torch.FloatTensor*

KvsAll forward pass: score head/relation against all entities.

Computes $-||R_r \odot h + t_r - e||_2 + b_h + b_e$ for every entity embedding e .

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 2) integer tensor [head_idx, relation_idx].

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_k_vs_sample (*x*: torch.LongTensor, *target_entity_idx*: torch.LongTensor) → torch.FloatTensor

KvsSample forward pass: score head/relation against a sampled entity subset.

Parameters

- **x** (torch.LongTensor) – Shape (batch_size, 2) integer tensor [head_idx, relation_idx].
- **target_entity_idx** (torch.LongTensor) – Shape (batch_size, k) indices of the *k* target entities per sample.

Returns

Shape (batch_size, k) score matrix.

Return type

torch.FloatTensor

class dicee.models.real.Shallom(*args*)

Bases: `dicee.models.base_model.BaseKGE`

Shallom: shallow neural model for relation prediction.

Represents each triple as the concatenation of head and tail entity embeddings and feeds it through a two-layer MLP to predict the relation. Designed for the RelationPrediction labelling form.

References

Demir et al., *A Shallow Neural Model for Relation Prediction*, ISWC 2021. <https://arxiv.org/abs/2101.09090>

name = 'Shallom'

shallom

get_embeddings () → Tuple[numpy.ndarray, None]

Return the entity and relation embedding matrices as numpy arrays.

Returns

- **entity_embeddings** (numpy.ndarray) – Shape (num_entities, embedding_dim).
- **relation_embeddings** (numpy.ndarray) – Shape (num_relations, embedding_dim).

forward_k_vs_all (*x*) → torch.FloatTensor

Score a (head, relation) batch against every entity.

Sub-classes must override this method. The default implementation raises ValueError to make missing overrides obvious at runtime.

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_triples (*x*) → torch.FloatTensor

Score a batch of triples by looking up relation scores from forward_k_vs_all.

Parameters

- **x** (torch.LongTensor) – Shape (batch_size, 3) integer tensor [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

```
class dicee.models.real.Pyke(args)
```

Bases: `dicee.models.base_model.BaseKGE`

Pyke: Physical Embedding Model for Knowledge Graphs.

Scores a triple (h, r, t) based on the average pairwise distance between head-to-relation and relation-to-tail in embedding space:

$$f(h, r, t) = \text{margin} - (||h - r||_2 + ||r - t||_2) / 2$$

The model encodes geometric proximity between entities and the relations that connect them.

name = 'Pyke'

dist_func

margin

forward_triples (x: `torch.LongTensor`) → torch.FloatTensor

Score a batch of triples using the Pyke distance formula.

Parameters

x (`torch.LongTensor`) – Shape (batch_size, 3) integer tensor [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

```
class dicee.models.real.CoKEConfig
```

Configuration for the CoKE (Contextualized Knowledge Graph Embedding) model.

block_size

Sequence length for transformer (3 for triples: head, relation, tail)

vocab_size

Total vocabulary size (num_entities + num_relations)

n_layer

Number of transformer layers

n_head

Number of attention heads per layer

n_embd

Embedding dimension (set to match model embedding_dim)

dropout

Dropout rate applied throughout the model

bias

Whether to use bias in linear layers

```

causal
    Whether to use causal masking (False for bidirectional attention)

block_size: int = 3

vocab_size: int = None

n_layer: int = 6

n_head: int = 8

n_embd: int = None

dropout: float = 0.3

bias: bool = True

causal: bool = False

```

```

class dicee.models.real.CoKE (args, config: CoKEConfig = CoKEConfig())

```

Bases: `dicee.models.base_model.BaseKGE`

Contextualized Knowledge Graph Embedding (CoKE) model. Based on: <https://arxiv.org/pdf/1911.02168>.

CoKE uses a transformer encoder to learn contextualized representations of entities and relations. For link prediction, it predicts masked elements in (head, relation, tail) triples using bidirectional attention, similar to BERT's masked language modeling approach.

The model creates a sequence [head_emb, relation_emb, mask_emb], adds positional embeddings, and processes it through transformer layers to predict the tail entity.

```

name = 'CoKE'

```

```

config

```

```

pos_emb

```

```

mask_emb

```

```

blocks

```

```

ln_f

```

```

coke_dropout

```

```

forward_k_vs_all (x: torch.Tensor)

```

Score a (head, relation) batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns

Shape (batch_size, num_entities) score matrix.

Return type

`torch.FloatTensor`

```

score (emb_h, emb_r, emb_t)

```

forward_k_vs_sample (*x: torch.LongTensor, target_entity_idx: torch.LongTensor*)

Score a (head, relation) batch against a sampled subset of entities.

Used by KvsSample and 1vsSample datasets. Sub-classes that support sample-based labelling must override this method.

Returns

Shape (batch_size, k) score matrix where *k* is the number of sampled target entities.

Return type

torch.FloatTensor

dicee.models.sampled_loss

Reusable loss for positive-first groups of negative samples.

Functions

<code>grouped_adversarial_bce</code> (→ torch.Tensor)	Balance the positive against a weighted negative group (ULTRA convention).
---	--

Module Contents

`dicee.models.sampled_loss.grouped_adversarial_bce` (*logits: torch.Tensor, targets: torch.Tensor, temperature: float*) → torch.Tensor

Balance the positive against a weighted negative group (ULTRA convention).

Positive temperature uses detached softmax(logits / temperature); zero uses uniform negative weights. This intentionally differs from ordinary mean BCE.

dicee.models.static_funcs

Functions

<code>quaternion_mul</code> (→ Tuple[torch.Tensor, torch.Tensor, ...])	Perform quaternion multiplication
--	-----------------------------------

Module Contents

`dicee.models.static_funcs.quaternion_mul` (*, *Q_1, Q_2*)
→ Tuple[torch.Tensor, torch.Tensor, torch.Tensor, torch.Tensor]

Perform quaternion multiplication :param Q_1: :param Q_2: :return:

dicee.models.transformers

Full definition of a GPT Language Model, all of it in this single file. References: 1) the official GPT-2 TensorFlow implementation released by OpenAI: <https://github.com/openai/gpt-2/blob/master/src/model.py> 2) huggingface/transformers PyTorch implementation: https://github.com/huggingface/transformers/blob/main/src/transformers/models/gpt2/modeling_gpt2.py

Attributes

<code>logger</code>

Classes

<code>ByteE</code>	Base class for all Knowledge Graph Embedding models.
<code>LayerNorm</code>	LayerNorm but with an optional bias. PyTorch doesn't support simply bias=False
<code>SelfAttention</code>	Base class for all neural network modules.
<code>MLP</code>	Base class for all neural network modules.
<code>Block</code>	Base class for all neural network modules.
<code>GPTConfig</code>	
<code>GPT</code>	Base class for all neural network modules.

Module Contents

`dicee.models.transformers.logger`

class `dicee.models.transformers.ByteE(*args, **kwargs)`

Bases: `dicee.models.base_model.BaseKGE`

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from `BaseKGE Lightning` and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- `forward_triples()` — score a batch of (h, r, t) triples.
- `forward_k_vs_all()` — score a (h, r) batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

name = 'ByteE'

config

temperature = 0.5

topk = 2

transformer

lm_head

loss_function (*yhat_batch*, *y_batch*)

Compute the loss between model predictions and targets.

Delegates to `self.loss` which is configured in `BaseKGE.__init__` based on the scoring technique (`BCEWithLogitsLoss` for entity/relation prediction, `CrossEntropyLoss` for classification, `MarginRankingLoss` for `scoring_technique='NegSampleMargin'`).

For `NegSampleMargin`, `yhat_batch/y_batch` still arrive in the flat `NegSample` layout (positives first, followed by `neg_ratio` tiled blocks of negatives, row-aligned by triple). This method splits that flat batch into positive/negative score pairs before delegating to `MarginRankingLoss`.

When `adversarial_temperature` is set (grouped/strict negative sampling), `yhat_batch/y_batch` instead arrive as `(batch, 1 + neg_ratio)` groups with the positive in column 0, produced by `GroupedNegativeSamplingDataset`. This delegates to `grouped_adversarial_bce()` instead of `self.loss`.

Parameters

- **yhat_batch** (`torch.FloatTensor`) – Model output scores, shape `(batch_size, *)`.
- **y_batch** (`torch.FloatTensor`) – Ground-truth labels of the same shape as `yhat_batch`.

Returns

Scalar loss value.

Return type

`torch.FloatTensor`

forward (*x*: `torch.LongTensor`)

Parameters

x (*B by T tensor*)

generate (*idx*, *max_new_tokens*, *temperature=1.0*, *top_k=None*)

Take a conditioning sequence of indices `idx` (`LongTensor` of shape `(b,t)`) and complete the sequence `max_new_tokens` times, feeding the predictions back into the model each time. Most likely you'll want to make sure to be in `model.eval()` mode of operation for this.

training_step (*batch*, *batch_idx=None*)

Execute one optimisation step for the given mini-batch.

Handles two- and three-element batches produced by the different dataset classes (`KvsAll` / `NegSample` vs. `KvsSample`).

Parameters

- **batch** (*tuple*) – (`x`, `y`) for standard scoring, or (`x`, `y_select`, `y`) for sample-based labelling.
- **batch_idx** (*int*, *optional*) – Index of the current batch (unused, kept for Lightning API compat).

Returns

Scalar loss value for this batch.

Return type

`torch.FloatTensor`

class `dicee.models.transformers.LayerNorm` (*ndim*, *bias*)

Bases: `torch.nn.Module`

LayerNorm but with an optional bias. PyTorch doesn't support simply `bias=False`

weight

bias

forward (*input*)

```
class dicee.models.transformers.SelfAttention(config)
```

Bases: torch.nn.Module

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

`training (bool)` – Boolean represents whether this module is in training or evaluation mode.

`c_attn`

`c_proj`

`attn_dropout`

`resid_dropout`

`n_head`

`n_embd`

`dropout`

`causal`

`flash = True`

`forward(x)`


```
class dicee.models.transformers.MLP (config)
```

Bases: torch.nn.Module

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

c_fc

gelu

c_proj

dropout

forward (*x*)

```
class dicee.models.transformers.Block (config)
```

Bases: torch.nn.Module

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```

import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))

```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

`training (bool)` – Boolean represents whether this module is in training or evaluation mode.

`ln_1`

`attn`

`ln_2`

`mlp`

`forward(x)`

```
class dicee.models.transformers.GPTConfig
```

`block_size: int = 1024`

`vocab_size: int = 50304`

`n_layer: int = 12`

`n_head: int = 12`

`n_embd: int = 768`

`dropout: float = 0.0`

`bias: bool = False`

`causal: bool = True`

```
class dicee.models.transformers.GPT(config)
```

Bases: torch.nn.Module

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

config

transformer

lm_head

get_num_params (*non_embedding=True*)

Return the number of parameters in the model. For non-embedding count (default), the position embeddings get subtracted. The token embeddings would too, except due to the parameter sharing these params are actually used as weights in the final layer, so we include them.

forward (*idx, targets=None*)

crop_block_size (*block_size*)

classmethod from_pretrained (*model_type, override_args=None*)

configure_optimizers (*weight_decay, learning_rate, betas, device_type*)

estimate_mfu (*fwdbwd_per_iter, dt*)

estimate model flops utilization (MFU) in units of A100 bfloat16 peak FLOPS

dicee.models.trix

Pure PyTorch TRIX entity and relation prediction.

Reimplementation of Zhang et al., <https://arxiv.org/abs/2502.19512>, verified against <https://github.com/yuchengz99/TRIX> at UPSTREAM_COMMIT. The module names, update schedule, binary entity-labelled relation edges, and fused convolution direction follow the released code/checkpoints (see docs/trix.md).

Attributes

<code>UPSTREAM_COMMIT</code>
<code>INTERACTIONS</code>

Classes

<code>EntityReasoner</code>	Base class for all neural network modules.
<code>RelationReasoner</code>	Base class for all neural network modules.
<code>TRIXBase</code>	Shared graph lifecycle and entity-scoring interfaces for graph foundation models.
<code>TRIX</code>	Entity predictor compatible with official <code>entity_prediction.pth</code> .
<code>TRIXRelation</code>	Relation predictor compatible with official <code>relation_prediction.pth</code> .

Functions

<code>build_relation_graph(edge_index, edge_type, ...)</code>	Sparse (relation, relation, shared entity) edges in official role order.
---	--

Module Contents

```
dicee.models.trix.UPSTREAM_COMMIT = '7596e14eefefe89e61396205a0550172cadeddb0'
```

```
dicee.models.trix.INTERACTIONS = ('hh', 'ht', 'th', 'tt')
```

```
dicee.models.trix.build_relation_graph(edge_index, edge_type, num_entities, num_relations)
```

Sparse (relation, relation, shared entity) edges in official role order.

Repeated incidences have binary support. hh/tt exclude equal relations; ht/th include them. Distinct shared entities remain distinct edges. No dense entity-by-relation-by-relation tensor is materialized.

```
class dicee.models.trix.EntityReasoner(dim, num_layers, output_dim=1)
```

Bases: `torch.nn.Module`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```

import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))

```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

layers

mlp

features (*edges, num_entities, relations, heads, rels, states=None, tails=None, candidates=None, split=False*)

```
class dicee.models.trix.RelationReasoner(dim, num_layers, entity_feedback=False)
```

Bases: `torch.nn.Module`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```

import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):

```

(continues on next page)

```
x = F.relu(self.conv1(x))
return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

step (*index, states, boundary, graph, entities, constant_relations=None*)

```
class dicee.models.trix.TRIXBase(args)
```

Bases: *dicee.models.graph_model.GraphKGE*

Shared graph lifecycle and entity-scoring interfaces for graph foundation models.

config_prefix = 'trix'

graph_filename = 'trix_graph.pt'

checkpoint_hint = 'use the matching entity/relation checkpoint with trix_dim=32;
all keys and shapes must match'

dim

property relation_graph

```
class dicee.models.trix.TRIX(args)
```

Bases: *TRIXBase*

Entity predictor compatible with official `entity_prediction.pth`.

DICE order is (head, relation, tail). Head corruption retains the original relation seed, then uses the inverse relation for both entity reasoners. Explicit reciprocal DICE queries use that same convention.

name = 'TRIX'

deterministic_inference = True

relation_model

entity_model_1

entity_model_2

relation_cache_mb

clear_inference_cache()

Drop derived representations when graph, weights or device change.

```
class dicee.models.trix.TRIXRelation(args)
```

```
    Bases: TRIXBase, dicee.models.graph_model.RelationGraphKGE
```

```
    Relation predictor compatible with official relation_prediction.pth.
```

```
    Pairs are (head, tail); triples and grouped relation corruptions use DICE's (head, relation, tail) order. One reasoning pass scores every relation for a given pair, including external inverse IDs when present in the vocabulary.
```

```
    name = 'TRIXRelation'
```

```
    relation_model
```

```
    entity_model
```

```
    mlp
```

dicee.models.ultra

Pure PyTorch ULTRA, compatible with DeepGraphLearning/ULTRA link predictors.

Architecture and graph construction adapted from <https://github.com/DeepGraphLearning/ULTRA> at commit 427966ad8ed60420eef034063d44f3153addff90. Reference fixtures and the grouped sampling and adversarial loss implementations also follow this upstream work. Graphs are runtime context, excluded from the transferable state dictionary.

MIT License

Copyright (c) 2023 MilaGraph

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Attributes

UPSTREAM_COMMIT

Classes

<i>RelationalConv</i>	ULTRA's DistMult/sum convolution, including the boundary message.
<i>RelNBFNet</i>	Base class for all neural network modules.
<i>EntityNBFNet</i>	Base class for all neural network modules.
<i>ULTRA</i>	Pure PyTorch ULTRA with the official state dictionary and DICE triple order.

Functions

<code>build_relation_graph(edge_index, edge_type, ...)</code>	Binary support of the hh, tt, ht, th incidence products, in upstream order.
---	---

Module Contents

```
dicee.models.ultra.UPSTREAM_COMMIT = '427966ad8ed60420eef034063d44f3153adddff90'
```

```
class dicee.models.ultra.RelationalConv(dim, project_relations=False)
```

Bases: torch.nn.Module

ULTRA's DistMult/sum convolution, including the boundary message.

`inference_backend = 'auto'`

`inference_compile = False`

`layer_norm`

`linear`

`forward(states, boundary, edge_index, edge_type, relations=None, *, projected=None, residual=False, constant_relations=None)`

```
class dicee.models.ultra.RelNBFNet(dim=64, num_layers=6)
```

Bases: torch.nn.Module

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

layers

dim = 64

forward(*edge_index, edge_type, num_relations, query*)

```
class dicee.models.ultra.EntityNBFNet (dim=64, num_layers=6)
```

Bases: `torch.nn.Module`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

layers

mlp

forward(*edge_index, edge_type, num_entities, relations, heads, rels, tails, projections=None*)

`dicee.models.ultra.build_relation_graph(edge_index, edge_type, num_entities, num_relations)`

Binary support of the hh, tt, ht, th incidence products, in upstream order.

class `dicee.models.ultra.ULTRA` (*args*)

Bases: `dicee.models.graph_model.GraphKGE`

Pure PyTorch ULTRA with the official state dictionary and DICE triple order.

name = 'ULTRA'

config_prefix = 'ultra'

graph_filename = 'ultra_graph.pt'

checkpoint_hint = 'official ULTRA requires ultra_dim=64, ultra_num_layers=6; all keys and shapes must match'

deterministic_inference = True

dim

num_layers

relation_model

entity_model

relation_cache_mb

projection_cache_mb

clear_inference_cache()

Drop derived representations when graph, weights or device change.

Attributes

<code>logger</code>
<code>logger</code>
<code>logger</code>

Classes

<code>ACLS</code>	Base class for all neural network modules.
<code>AdaptiveLabelRelaxationLoss</code>	Base class for all neural network modules.
<code>AdaptiveLabelSmoothingLoss</code>	Base class for all neural network modules.
<code>AggregatedLSandLR</code>	Base class for all neural network modules.
<code>CombinedAdaptiveLSandAdaptiveLR</code>	Base class for all neural network modules.
<code>CombinedLSandLR</code>	Base class for all neural network modules.
<code>ConfidenceBasedAdaptiveLabelRelaxation-Loss</code>	Base class for all neural network modules.
<code>DefaultBCELoss</code>	Base class for all neural network modules.
<code>LabelRelaxationLoss</code>	Base class for all neural network modules.
<code>LabelSmoothingLoss</code>	Base class for all neural network modules.

continues on next page

Table 69 – continued from previous page

<i>ADOPT</i>	ADOPT Optimizer.
<i>BaseKGELightning</i>	Thin PyTorch Lightning wrapper shared by all KGE models.
<i>BaseKGE</i>	Base class for all Knowledge Graph Embedding models.
<i>IdentityClass</i>	No-op normalisation / dropout placeholder.
<i>Block</i>	Base class for all neural network modules.
<i>BaseKGE</i>	Base class for all Knowledge Graph Embedding models.
<i>DistMult</i>	DistMult: bilinear diagonal knowledge graph embedding.
<i>TransE</i>	TransE: translation-based knowledge graph embedding.
<i>TransH</i>	TransH: translation-based knowledge graph embedding on relation-specific hyperplanes.
<i>MuRE</i>	MuRE: multi-relational graph embedding with relation-specific diagonal scaling, translation, and entity biases.
<i>Shallom</i>	Shallom: shallow neural model for relation prediction.
<i>Pyke</i>	Pyke: Physical Embedding Model for Knowledge Graphs.
<i>CoKEConfig</i>	Configuration for the CoKE (Contextualized Knowledge Graph Embedding) model.
<i>CoKE</i>	Contextualized Knowledge Graph Embedding (CoKE) model.
<i>BaseKGE</i>	Base class for all Knowledge Graph Embedding models.
<i>ConEx</i>	Convolutional ComplEx Knowledge Graph Embeddings
<i>AConEx</i>	Additive Convolutional ComplEx Knowledge Graph Embeddings
<i>ComplEx</i>	Base class for all Knowledge Graph Embedding models.
<i>RotatE</i>	RotatE: knowledge graph embedding by relational rotation in complex space.
<i>BaseKGE</i>	Base class for all Knowledge Graph Embedding models.
<i>IdentityClass</i>	No-op normalisation / dropout placeholder.
<i>QMult</i>	Base class for all Knowledge Graph Embedding models.
<i>ConvQ</i>	Convolutional Quaternion Knowledge Graph Embeddings
<i>AConvQ</i>	Additive Convolutional Quaternion Knowledge Graph Embeddings
<i>BaseKGE</i>	Base class for all Knowledge Graph Embedding models.
<i>IdentityClass</i>	No-op normalisation / dropout placeholder.
<i>OMult</i>	Base class for all Knowledge Graph Embedding models.
<i>ConvO</i>	Base class for all Knowledge Graph Embedding models.
<i>AConvO</i>	Additive Convolutional Octonion Knowledge Graph Embeddings
<i>Keci</i>	Keci: Knowledge Graph Embedding via Clifford Algebra.
<i>CKeci</i>	Without learning dimension scaling
<i>DeCaL</i>	Base class for all Knowledge Graph Embedding models.
<i>KeciTransformer</i>	Keci with Transformer architecture.
<i>BaseKGE</i>	Base class for all Knowledge Graph Embedding models.
<i>PykeenKGE</i>	A class for using knowledge graph embedding models implemented in Pykeen
<i>BaseKGE</i>	Base class for all Knowledge Graph Embedding models.
<i>FMult</i>	Learning Knowledge Neural Graphs
<i>GFMult</i>	Learning Knowledge Neural Graphs
<i>FMult2</i>	Learning Knowledge Neural Graphs
<i>LFMult1</i>	Embedding with trigonometric functions. We represent all entities and relations in the complex number space as:

continues on next page

Table 69 – continued from previous page

<i>LFMult</i>	Embedding with polynomial functions. We represent all entities and relations in the polynomial space as:
<i>DualE</i>	Dual Quaternion Knowledge Graph Embeddings (https://ojs.aaai.org/index.php/AAAI/article/download/16850/16657)
<i>ULTRA</i>	Pure PyTorch ULTRA with the official state dictionary and DICE triple order.
<i>TRIX</i>	Entity predictor compatible with official <code>entity_prediction.pth</code> .
<i>TRIXRelation</i>	Relation predictor compatible with official <code>relation_prediction.pth</code> .
<i>Flock</i>	Entity-prediction Flock with official <code>flock_entity.pth</code> parameters.
<i>FlockRelation</i>	Relation-prediction Flock with official <code>flock_relation.pth</code> parameters.

Functions

```

quaternion_mul(→ Tuple[torch.Tensor, torch.Tensor, Perform quaternion multiplication
...))
quaternion_mul_with_unit_norm(*, Q_1, Q_2)
octonion_mul(*, O_1, O_2)
octonion_mul_norm(*, O_1, O_2)

```

Package Contents

class `dicee.models.ACLS` (`pos_lambda: float = 1.0`, `neg_lambda: float = 0.1`, `alpha: float = 0.1`, `margin: float = 10.0`, `num_classes: int = 200`, `ignore_index: int = -100`)

Bases: `torch.nn.Module`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```

import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))

```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

```
pos_lambda = 1.0
neg_lambda = 0.1
alpha = 0.1
margin = 10.0
num_classes = 200
ignore_index = -100
cross_entropy
property names
get_reg(inputs, targets)
forward(inputs, targets)
```

```
class dicee.models.AdaptiveLabelRelaxationLoss(min_alpha=0.01, max_alpha=0.2,
        alpha_step=0.01, initial_alpha=0.1)
```

Bases: `torch.nn.Module`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the submodules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

`training (bool)` – Boolean represents whether this module is in training or evaluation mode.

```
min_alpha = 0.01
```

```
max_alpha = 0.2
```

```
alpha_step = 0.01
```

```
alpha = 0.1
```

```
prev_loss = None
```

```
eps = 1e-14
```

```
gz_threshold = 0.1
```

```
forward(pred, target)
```

```
class dicee.models.AdaptiveLabelSmoothingLoss (min_smoothing_factor=0.01,  
                                              max_smoothing_factor=0.2, smoothing_factor_step=0.01, initial_smoothing_factor=0.1)
```

Bases: `torch.nn.Module`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn  
import torch.nn.functional as F  
  
class Model(nn.Module):  
    def __init__(self) -> None:  
        super().__init__()  
        self.conv1 = nn.Conv2d(1, 20, 5)  
        self.conv2 = nn.Conv2d(20, 20, 5)  
  
    def forward(self, x):  
        x = F.relu(self.conv1(x))  
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

min_smoothing_factor = 0.01

max_smoothing_factor = 0.2

smoothing_factor_step = 0.01

smoothing_factor = 0.1

prev_loss = None

eps = 1e-14

forward (*logits, target, current_epoch*)

class `dicee.models.AggregatedLSandLR` (*smoothness_ratio=0.1, alpha=0.1*)

Bases: `torch.nn.Module`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

```
smoothness_ratio = 0.1

alpha = 0.1

forward(pred, target, current_epoch)
```

```
class dicee.models.CombinedAdaptiveLSandAdaptiveLR
```

Bases: torch.nn.Module

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

adaptive_label_smoothing

adaptive_label_relaxation

criterion = ''

forward (*pred, target, current_epoch*)

```
class dicee.models.CombinedLSandLR(smoothness_ratio=0.0, alpha=0.0)
```

Bases: torch.nn.Module

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:


```

import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))

```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

`training` (*bool*) – Boolean represents whether this module is in training or evaluation mode.

`smoothness_ratio` = 0.0

`alpha` = 0.0

`forward` (*pred, target, current_epoch*)

```
class dicee.models.ConfidenceBasedAdaptiveLabelRelaxationLoss(alpha=0.1)
```

Bases: `torch.nn.Module`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```

import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))

```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

alpha = 0.1

gz_threshold = 0.1

eps = 1e-14

forward (*pred, target, current_epoch*)

class `dicee.models.DefaultBCELoss`

Bases: `torch.nn.Module`

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the submodules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

forward (*pred, target*)

class dicee.models.**LabelRelaxationLoss** (*alpha=0.0*)

Bases: torch.nn.Module

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

alpha = 0.0

gz_threshold = 0.1

eps = 1e-14

forward (*pred, target*)

class dicee.models.**LabelSmoothingLoss** (*smoothness_ratio=0.0*)

Bases: torch.nn.Module

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

smoothness_ratio = 0.0

forward (*pred, target*)

```
class dicee.models.ADOPT (params: torch.optim.optimizer.ParamsT, lr: float | torch.Tensor = 0.001,
    betas: Tuple[float, float] = (0.9, 0.9999), eps: float = 1e-06,
    clip_lambda: Callable[[int], float] | None = lambda step: ..., weight_decay: float = 0.0,
    decouple: bool = False, *, foreach: bool | None = None, maximize: bool = False,
    capturable: bool = False, differentiable: bool = False, fused: bool | None = None)
```

Bases: `torch.optim.optimizer.Optimizer`

ADOPT Optimizer.

ADOPT is an adaptive learning rate optimization algorithm that combines momentum-based updates with adaptive per-parameter learning rates. It uses exponential moving averages of gradients and squared gradients, with gradient clipping for stability.

The algorithm performs the following key operations: 1. Normalizes gradients by the square root of the second moment estimate 2. Applies optional gradient clipping based on the training step 3. Updates parameters using momentum-smoothed normalized gradients 4. Supports decoupled weight decay (AdamW-style) or L2 regularization

Mathematical formulation:

$$m_t = \beta_1 * m_{t-1} + (1 - \beta_1) * \text{clip}(g_t / \sqrt{v_t}) \quad v_t = \beta_2 * v_{t-1} + (1 - \beta_2) * g_t^2 \quad \theta_t = \theta_{t-1} - \alpha * m_t$$

where:

- θ_t : parameter at step t
- g_t : gradient at step t
- m_t : first moment estimate (momentum)

- v_t : second moment estimate (variance)
- α : learning rate
- β_1, β_2 : exponential decay rates
- `clip()`: optional gradient clipping function

Reference:

Original implementation: <https://github.com/iShohei220/adopt>

Parameters

- **params** (*ParamsT*) – Iterable of parameters to optimize or dicts defining parameter groups.
- **lr** (*float or Tensor, optional*) – Learning rate. Can be a float or 1-element Tensor. Default: 1e-3
- **betas** (*Tuple[float, float], optional*) – Coefficients (β_1, β_2) for computing running averages of gradient and its square. β_1 controls momentum, β_2 controls variance. Default: (0.9, 0.9999)
- **eps** (*float, optional*) – Term added to denominator to improve numerical stability. Default: 1e-6
- **clip_lambda** (*Callable[[int], float], optional*) – Function that takes the step number and returns the gradient clipping threshold. Common choices: - lambda step: $\text{step} \times 0.25$ (default, gradually increases clipping threshold) - lambda step: 1.0 (constant clipping) - None (no clipping) Default: lambda step: $\text{step} \times 0.25$
- **weight_decay** (*float, optional*) – Weight decay coefficient (L2 penalty). Default: 0.0
- **decouple** (*bool, optional*) – If True, uses decoupled weight decay (AdamW-style), applying weight decay directly to parameters. If False, adds weight decay to gradients (L2 regularization). Default: False
- **foreach** (*bool, optional*) – If True, uses the faster foreach implementation for multi-tensor operations. Default: None (auto-select)
- **maximize** (*bool, optional*) – If True, maximizes parameters instead of minimizing. Useful for reinforcement learning. Default: False
- **capturable** (*bool, optional*) – If True, the optimizer is safe to capture in a CUDA graph. Requires learning rate as Tensor. Default: False
- **differentiable** (*bool, optional*) – If True, the optimization step can be differentiated. Useful for meta-learning. Default: False
- **fused** (*bool, optional*) – If True, uses fused kernel implementation (currently not supported). Default: None

Raises

- **ValueError** – If learning rate, epsilon, betas, or weight_decay are invalid.
- **RuntimeError** – If fused is enabled (not currently supported).
- **RuntimeError** – If lr is a Tensor with foreach=True and capturable=False.

Example

```
>>> # Basic usage
>>> optimizer = ADOPT(model.parameters(), lr=0.001)
>>> optimizer.zero_grad()
>>> loss.backward()
>>> optimizer.step()
```

```
>>> # With decoupled weight decay
>>> optimizer = ADOPT(model.parameters(), lr=0.001, weight_decay=0.01,
↳decouple=True)
```

```
>>> # Custom gradient clipping
>>> optimizer = ADOPT(model.parameters(), clip_lambda=lambda step: max(1.0,
↳step**0.5))
```

Note

- For most use cases, the default hyperparameters work well
- Consider using `decouple=True` for better generalization (similar to AdamW)
- The `clip_lambda` function helps stabilize training in early steps

`clip_lambda`

`__setstate__(state)`

Restore optimizer state from a checkpoint.

This method handles backward compatibility when loading optimizer state from older versions. It ensures all required fields are present with default values and properly converts step counters to tensors if needed.

Key responsibilities: 1. Set default values for newly added hyperparameters 2. Convert old-style scalar step counters to tensor format 3. Place step tensors on appropriate devices based on capturable/fused modes

Parameters

state (*dict*) – Optimizer state dictionary (typically from `torch.load()`).

Note

- This enables loading checkpoints saved with older ADOPT versions
- Step counters are converted to appropriate device/dtype for compatibility
- Capturable and fused modes require step tensors on parameter devices

`step(closure=None)`

Perform a single optimization step.

This method executes one iteration of the ADOPT optimization algorithm across all parameter groups. It orchestrates the following workflow:

1. Optionally evaluates a closure to recompute the loss (useful for algorithms like LBFGS or when loss needs multiple evaluations)

2. For each parameter group: - Collects parameters with gradients and their associated state - Extracts hyperparameters (betas, learning rate, etc.) - Calls the functional adopt() API to perform the actual update
3. Returns the loss value if a closure was provided

The functional API (adopt()) handles three execution modes: - Single-tensor: Updates one parameter at a time (default, JIT-compatible) - Multi-tensor (foreach): Batches operations for better performance - Fused: Uses fused CUDA kernels (not yet implemented)

Gradient scaling support: This method is compatible with automatic mixed precision (AMP) training. It can access grad_scale and found_inf attributes for gradient unscaling and inf/nan detection when used with GradScaler.

Parameters

closure (*Callable, optional*) – A callable that reevaluates the model and returns the loss. The closure should: - Enable gradients (torch.enable_grad()) - Compute forward pass - Compute loss - Compute backward pass - Return the loss value Example: lambda: (loss := model(x), loss.backward(), loss)[-1] Default: None

Returns

The loss value returned by the closure, or None if no closure was provided.

Return type

Optional[Tensor]

Example

```
>>> # Standard usage
>>> loss = criterion(model(input), target)
>>> loss.backward()
>>> optimizer.step()
```

```
>>> # With closure (e.g., for line search)
>>> def closure():
...     optimizer.zero_grad()
...     output = model(input)
...     loss = criterion(output, target)
...     loss.backward()
...     return loss
>>> loss = optimizer.step(closure)
```

Note

- Call zero_grad() before computing gradients for the next step
- CUDA graph capture is checked for safety when capturable=True
- The method is thread-safe for different parameter groups

dicee.models.logger

class dicee.models.BaseKGELightning(*args, **kwargs)

Bases: lightning.LightningModule

Thin PyTorch Lightning wrapper shared by all KGE models.

Provides the standard Lightning training loop hooks (`training_step`, `on_train_epoch_end`, `configure_optimizers`) as well as a helper for reporting model size. All concrete KGE models should extend `BaseKGE` rather than this class directly.

args: `Dict[str, Any]`

loss: `torch.nn.Module`

training_step_outputs = []

mem_of_model() → `Dict`

Size of model in MB and number of params

training_step(*batch*, *batch_idx=None*)

Execute one optimisation step for the given mini-batch.

Handles two- and three-element batches produced by the different dataset classes (`KvsAll` / `NegSample` vs. `KvsSample`).

Parameters

- **batch** (*tuple*) – (*x*, *y*) for standard scoring, or (*x*, *y_select*, *y*) for sample-based labelling.
- **batch_idx** (*int*, *optional*) – Index of the current batch (unused, kept for Lightning API compat).

Returns

Scalar loss value for this batch.

Return type

`torch.FloatTensor`

loss_function(*yhat_batch: torch.Tensor*, *y_batch: torch.Tensor*) → `torch.Tensor`

Compute the loss between model predictions and targets.

Delegates to `self.loss` which is configured in `BaseKGE.__init__` based on the scoring technique (`BCEWithLogitsLoss` for entity/relation prediction, `CrossEntropyLoss` for classification, `MarginRankingLoss` for `scoring_technique='NegSampleMargin'`).

For `NegSampleMargin`, *yhat_batch*/*y_batch* still arrive in the flat `NegSample` layout (positives first, followed by `neg_ratio` tiled blocks of negatives, row-aligned by triple). This method splits that flat batch into positive/negative score pairs before delegating to `MarginRankingLoss`.

When `adversarial_temperature` is set (grouped/strict negative sampling), *yhat_batch*/*y_batch* instead arrive as (*batch*, `1 + neg_ratio`) groups with the positive in column 0, produced by `GroupedNegativeSamplingDataset`. This delegates to `grouped_adversarial_bce()` instead of `self.loss`.

Parameters

- **yhat_batch** (`torch.FloatTensor`) – Model output scores, shape (*batch_size*, *).
- **y_batch** (`torch.FloatTensor`) – Ground-truth labels of the same shape as *yhat_batch*.

Returns

Scalar loss value.

Return type

`torch.FloatTensor`

`on_train_epoch_end(*args, **kwargs)`

Called in the training loop at the very end of the epoch.

To access all batch outputs at the end of the epoch, you can cache step outputs as an attribute of the `LightningModule` and access them in this hook:

```
class MyLightningModule(L.LightningModule):
    def __init__(self):
        super().__init__()
        self.training_step_outputs = []

    def training_step(self):
        loss = ...
        self.training_step_outputs.append(loss)
        return loss

    def on_train_epoch_end(self):
        # do something with all training_step outputs, for example:
        epoch_mean = torch.stack(self.training_step_outputs).mean()
        self.log("training_epoch_mean", epoch_mean)
        # free up the memory
        self.training_step_outputs.clear()
```

`test_epoch_end(outputs: List[Any])`

`test_dataloader()` → None

An iterable or collection of iterables specifying test samples.

For more information about multiple dataloaders, see this section.

For data processing use the following pattern:

- download in `prepare_data()`
- process and split in `setup()`

However, the above are only necessary for distributed processing.

Warning

do not assign state in `prepare_data`

- `test()`
- `prepare_data()`
- `setup()`

Note

Lightning tries to add the correct sampler for distributed and arbitrary hardware. There is no need to set it yourself.

Note

If you don't need a test dataset and a `test_step()`, you don't need to implement this method.

`val_dataloader()` → None

An iterable or collection of iterables specifying validation samples.

For more information about multiple dataloaders, see this section.

The dataloader you return will not be reloaded unless you set **:param-ref:~lightning.pytorch.trainer.trainer.Trainer.reload_dataloaders_every_n_epochs** to a positive integer.

It's recommended that all data downloads and preparation happen in `prepare_data()`.

- `fit()`
- `validate()`
- `prepare_data()`
- `setup()`

Note

Lightning tries to add the correct sampler for distributed and arbitrary hardware There is no need to set it yourself.

Note

If you don't need a validation dataset and a `validation_step()`, you don't need to implement this method.

`predict_dataloader()` → None

An iterable or collection of iterables specifying prediction samples.

For more information about multiple dataloaders, see this section.

It's recommended that all data downloads and preparation happen in `prepare_data()`.

- `predict()`
- `prepare_data()`
- `setup()`

Note

Lightning tries to add the correct sampler for distributed and arbitrary hardware There is no need to set it yourself.

Returns

A `torch.utils.data.DataLoader` or a sequence of them specifying prediction samples.

`train_dataloader()` → None

An iterable or collection of iterables specifying training samples.

For more information about multiple dataloaders, see this section.

The dataloader you return will not be reloaded unless you set `:param-ref:~lightning.pytorch.trainer.trainer.Trainer.reload_dataloaders_every_n_epochs`` to a positive integer.

For data processing use the following pattern:

- download in `prepare_data()`
- process and split in `setup()`

However, the above are only necessary for distributed processing.

Warning

do not assign state in `prepare_data`

- `fit()`
- `prepare_data()`
- `setup()`

Note

Lightning tries to add the correct sampler for distributed and arbitrary hardware. There is no need to set it yourself.

`configure_optimizers(parameters=None)`

Instantiate and return the optimiser for training.

The optimiser type is taken from `self.optimizer_name` which is set in `BaseKGE.init_params_with_sanity_checking()` from the `--optim` argument. Supported values: 'SGD', 'Adam', 'Adopt', 'AdamW', 'NAdam', 'Adagrad', 'ASGD', 'Muon'.

Parameters

parameters (*iterable, optional*) – Model parameters to optimise. Defaults to `self.parameters()` when None.

Returns

The configured optimiser instance.

Return type

`torch.optim.Optimizer`

`class dicee.models.BaseKGE(args: dict)`

Bases: `BaseKGELightning`

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from `BaseKGELightning` and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- `forward_triples()` — score a batch of (h, r, t) triples.
- `forward_k_vs_all()` — score a (h, r) batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

```
args
embedding_dim = None
num_entities: int | None = None
num_relations: int | None = None
num_tokens = None
learning_rate = None
apply_unit_norm = None
input_dropout_rate = None
hidden_dropout_rate = None
optimizer_name = None
feature_map_dropout_rate = None
kernel_size = None
num_of_output_channels = None
weight_decay = None
selected_optimizer = None
normalizer_class = None
normalize_head_entity_embeddings
normalize_relation_embeddings
normalize_tail_entity_embeddings
hidden_normalizer
param_init
input_dp_ent_real
input_dp_rel_real
hidden_dropout
loss_history = []
byte_pair_encoding
```

max_length_subword_tokens

block_size

defer_large_embeddings

init_entity_embeddings (*embedding_dim*: int | None = None) → None

Create (or re-create) the entity embedding table.

This is the single place that honours *defer_large_embeddings*: when entity rows are sharded across FSDP ranks the table must stay None until the trainer allocates the sharded adapter. Subclasses that need an entity table of a non-default width must call this method instead of assigning `self.entity_embeddings` directly, so the deferral is never silently undone.

Parameters

embedding_dim (*Optional*[int]) – Width of the table. Defaults to *embedding_dim*.

init_relation_embeddings (*embedding_dim*: int | None = None) → None

Create (or re-create) the relation embedding table.

Relation tables are never deferred - they are small enough to be replicated on every rank.

Parameters

embedding_dim (*Optional*[int]) – Width of the table. Defaults to *embedding_dim*.

forward_byte_pair_encoded_k_vs_all (*x*: torch.LongTensor) → torch.FloatTensor

KvsAll scoring for BPE-encoded head entities and relations.

Retrieves subword-unit embeddings for the head entity and relation, reduces them to fixed-size vectors via a linear projection, then scores against all BPE entity embeddings.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 2, T) BPE token indices where dim 1 indexes [head, relation] and T is max_length_subword_tokens.

Returns

Shape (batch_size, num_bpe_entities) score matrix.

Return type

torch.FloatTensor

forward_byte_pair_encoded_triple (*x*: Tuple[torch.LongTensor, torch.LongTensor])
→ torch.FloatTensor

NegSample scoring for BPE-encoded (head, relation, tail) triples.

Retrieves subword-unit embeddings for all three elements and reduces them to fixed-size vectors via a linear projection before computing the triple score.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3, T) BPE token indices.

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

init_params_with_sanity_checking() → None

Populate model hyper-parameters from `self.args` with safe defaults.

Reads embedding dimension, learning rate, dropout rates, normalisation strategy, optimizer name, and parameter initialisation scheme from the `args` dict. Falls back to sensible defaults for any missing key so that minimal `args` dicts (e.g. for unit tests) are still valid.

forward (*x*: torch.LongTensor | Tuple[torch.LongTensor, torch.LongTensor],
y_idx: torch.LongTensor | None = None) → torch.Tensor

Route the forward pass to the appropriate scoring method.

Inspects the shape and type of *x* to decide which low-level scorer to call:

- Tuple (*x*, *y_idx*) → *forward_k_vs_sample()*
- (batch, 3) tensor → *forward_triples()*
- (batch, 2) tensor → *forward_k_vs_all()*
- BPE triple tensor → *forward_byte_pair_encoded_triple()*
- BPE pair tensor → *forward_byte_pair_encoded_k_vs_all()*

Parameters

- **x** (torch.LongTensor or Tuple[torch.LongTensor, torch.LongTensor]) – Either a plain index tensor or a (triple_idx, target_idx) tuple for sample-based labelling.
- **y_idx** (torch.LongTensor, optional) – Target entity indices used by *forward_k_vs_sample()*. Ignored when *x* is a plain tensor.

Returns

Score tensor whose shape depends on the selected scorer.

Return type

torch.FloatTensor

forward_triples (*x*: torch.LongTensor) → torch.Tensor

Score a batch of (head, relation, tail) index triples.

Parameters

- **x** (torch.LongTensor) – Shape (batch_size, 3) integer tensor where each row is [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

forward_k_vs_all (*args, **kwargs)

Score a (head, relation) batch against every entity.

Sub-classes must override this method. The default implementation raises ValueError to make missing overrides obvious at runtime.

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_k_vs_sample (*args, **kwargs)

Score a (head, relation) batch against a sampled subset of entities.

Used by KvsSample and 1vsSample datasets. Sub-classes that support sample-based labelling must override this method.

Returns

Shape (batch_size, k) score matrix where k is the number of sampled target entities.

Return type

torch.FloatTensor

get_triple_representation (*idx_hrt*)

→ Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for a triple index batch.

Parameters

idx_hrt (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor with columns [head_idx, relation_idx, tail_idx].

Returns

head_ent_emb, rel_ent_emb, tail_ent_emb – Each has shape (batch_size, embedding_dim) after applying the configured dropout and normalisation.

Return type

torch.FloatTensor

get_head_relation_representation (*indexed_triple*)

→ Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for head entities and relations.

Parameters

indexed_triple (*torch.LongTensor*) – Shape (batch_size, 2) integer tensor with columns [head_idx, relation_idx].

Returns

head_ent_emb, rel_ent_emb – Each has shape (batch_size, embedding_dim) after applying the configured dropout and normalisation.

Return type

torch.FloatTensor

get_sentence_representation (*x: torch.LongTensor*)

→ Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve BPE subword-unit embeddings for a batch of triples.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3, T) where T is max_length_subword_tokens.

Returns

head_ent_emb, rel_emb, tail_emb – Each has shape (batch_size, T , embedding_dim).

Return type

torch.FloatTensor

get_bpe_head_and_relation_representation (*x: torch.LongTensor*)

→ Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve unit-normalised BPE embeddings for head entities and relations.

Each entity/relation is represented as a sequence of T subword tokens. Their token embeddings are L2-normalised across the sequence dimension so that the resulting matrix has unit Frobenius norm.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 2, T) where dim 1 indexes [head, relation] and T is max_length_subword_tokens.

Returns

head_ent_emb, rel_emb – Each has shape (batch_size, T, embedding_dim), L2-normalised over the (T, D) dimensions.

Return type

torch.FloatTensor

get_embeddings() → Tuple[numpy.ndarray, numpy.ndarray]

Return the entity and relation embedding matrices as numpy arrays.

Returns

- **entity_embeddings** (numpy.ndarray) – Shape (num_entities, embedding_dim).
- **relation_embeddings** (numpy.ndarray) – Shape (num_relations, embedding_dim).

```
class dicee.models.IdentityClass(args=None)
```

Bases: torch.nn.Module

No-op normalisation / dropout placeholder.

Used whenever no normalisation layer is requested (--normalization None). All inputs are returned unchanged so that the rest of the model code does not need conditional checks around normalisation calls.

args = None

__call__(x)

static forward(x)

```
class dicee.models.Block(config)
```

Bases: torch.nn.Module

Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in a tree structure. You can assign the sub-modules as regular attributes:

```
import torch.nn as nn
import torch.nn.functional as F

class Model(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.conv1 = nn.Conv2d(1, 20, 5)
        self.conv2 = nn.Conv2d(20, 20, 5)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        return F.relu(self.conv2(x))
```

Submodules assigned in this way will be registered, and will also have their parameters converted when you call `to()`, etc.

Note

As per the example above, an `__init__()` call to the parent class must be made before assignment on the child.

Variables

training (*bool*) – Boolean represents whether this module is in training or evaluation mode.

ln_1

attn

ln_2

mlp

forward(*x*)

class dicee.models.**BaseKGE** (*args: dict*)

Bases: *BaseKGELightning*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from *BaseKGELightning* and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- *forward_triples()* — score a batch of (*h*, *r*, *t*) triples.
- *forward_k_vs_all()* — score a (*h*, *r*) batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

args

embedding_dim = None

num_entities: int | None = None

num_relations: int | None = None

num_tokens = None

learning_rate = None

apply_unit_norm = None

input_dropout_rate = None

hidden_dropout_rate = None

optimizer_name = None

feature_map_dropout_rate = None

```

kernel_size = None

num_of_output_channels = None

weight_decay = None

selected_optimizer = None

normalizer_class = None

normalize_head_entity_embeddings

normalize_relation_embeddings

normalize_tail_entity_embeddings

hidden_normalizer

param_init

input_dp_ent_real

input_dp_rel_real

hidden_dropout

loss_history = []

byte_pair_encoding

max_length_subword_tokens

block_size

defer_large_embeddings

init_entity_embeddings(embedding_dim: int | None = None) → None

```

Create (or re-create) the entity embedding table.

This is the single place that honours *defer_large_embeddings*: when entity rows are sharded across FSDP ranks the table must stay *None* until the trainer allocates the sharded adapter. Subclasses that need an entity table of a non-default width must call this method instead of assigning *self.entity_embeddings* directly, so the deferral is never silently undone.

Parameters

embedding_dim (*Optional[int]*) – Width of the table. Defaults to *embedding_dim*.

```
init_relation_embeddings(embedding_dim: int | None = None) → None
```

Create (or re-create) the relation embedding table.

Relation tables are never deferred - they are small enough to be replicated on every rank.

Parameters

embedding_dim (*Optional[int]*) – Width of the table. Defaults to *embedding_dim*.

```
forward_byte_pair_encoded_k_vs_all(x: torch.LongTensor) → torch.FloatTensor
```

KvsAll scoring for BPE-encoded head entities and relations.

Retrieves subword-unit embeddings for the head entity and relation, reduces them to fixed-size vectors via a linear projection, then scores against all BPE entity embeddings.

Parameters

$\mathbf{x}$ (*torch.LongTensor*) – Shape (batch_size, 2, T) BPE token indices where dim 1 indexes [head, relation] and T is max_length_subword_tokens.

Returns

Shape (batch_size, num_bpe_entities) score matrix.

Return type

torch.FloatTensor

forward_byte_pair_encoded_triple (*x*: *Tuple[torch.LongTensor, torch.LongTensor]*)
→ torch.FloatTensor

NegSample scoring for BPE-encoded (head, relation, tail) triples.

Retrieves subword-unit embeddings for all three elements and reduces them to fixed-size vectors via a linear projection before computing the triple score.

Parameters

$\mathbf{x}$ (*torch.LongTensor*) – Shape (batch_size, 3, T) BPE token indices.

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

init_params_with_sanity_checking() → None

Populate model hyper-parameters from *self.args* with safe defaults.

Reads embedding dimension, learning rate, dropout rates, normalisation strategy, optimizer name, and parameter initialisation scheme from the *args* dict. Falls back to sensible defaults for any missing key so that minimal *args* dicts (e.g. for unit tests) are still valid.

forward (*x*: *torch.LongTensor* | *Tuple[torch.LongTensor, torch.LongTensor]*,
y_idx: *torch.LongTensor* | *None* = *None*) → torch.Tensor

Route the forward pass to the appropriate scoring method.

Inspects the shape and type of *x* to decide which low-level scorer to call:

- *Tuple* (*x*, *y_idx*) → *forward_k_vs_sample*()
- (batch, 3) tensor → *forward_triples*()
- (batch, 2) tensor → *forward_k_vs_all*()
- BPE triple tensor → *forward_byte_pair_encoded_triple*()
- BPE pair tensor → *forward_byte_pair_encoded_k_vs_all*()

Parameters

- $\mathbf{x}$ (*torch.LongTensor* or *Tuple[torch.LongTensor, torch.LongTensor]*) – Either a plain index tensor or a (triple_idx, target_idx) tuple for sample-based labelling.
- $\mathbf{y_idx}$ (*torch.LongTensor*, optional) – Target entity indices used by *forward_k_vs_sample*(). Ignored when *x* is a plain tensor.

Returns

Score tensor whose shape depends on the selected scorer.

Return type

torch.FloatTensor

forward_triples (*x*: *torch.LongTensor*) → *torch.Tensor*

Score a batch of (head, relation, tail) index triples.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor where each row is [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

forward_k_vs_all (*args, **kwargs)

Score a (head, relation) batch against every entity.

Sub-classes must override this method. The default implementation raises *ValueError* to make missing overrides obvious at runtime.

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_k_vs_sample (*args, **kwargs)

Score a (head, relation) batch against a sampled subset of entities.

Used by *KvsSample* and *1vsSample* datasets. Sub-classes that support sample-based labelling must override this method.

Returns

Shape (batch_size, k) score matrix where *k* is the number of sampled target entities.

Return type

torch.FloatTensor

get_triple_representation (idx_hrt)

→ Tuple[*torch.FloatTensor*, *torch.FloatTensor*, *torch.FloatTensor*]

Retrieve and normalise embedding vectors for a triple index batch.

Parameters

idx_hrt (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor with columns [head_idx, relation_idx, tail_idx].

Returns

head_ent_emb, rel_ent_emb, tail_ent_emb – Each has shape (batch_size, embedding_dim) after applying the configured dropout and normalisation.

Return type

torch.FloatTensor

get_head_relation_representation (indexed_triple)

→ Tuple[*torch.FloatTensor*, *torch.FloatTensor*]

Retrieve and normalise embedding vectors for head entities and relations.

Parameters

indexed_triple (*torch.LongTensor*) – Shape (batch_size, 2) integer tensor with columns [head_idx, relation_idx].

Returns

head_ent_emb, rel_ent_emb – Each has shape (batch_size, embedding_dim) after applying the configured dropout and normalisation.

Return type

torch.FloatTensor

get_sentence_representation (*x*: torch.LongTensor)

→ Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve BPE subword-unit embeddings for a batch of triples.

Parameters

x (torch.LongTensor) – Shape (batch_size, 3, T) where T is max_length_subword_tokens.

Returns

head_ent_emb, rel_emb, tail_emb – Each has shape (batch_size, T, embedding_dim).

Return type

torch.FloatTensor

get_bpe_head_and_relation_representation (*x*: torch.LongTensor)

→ Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve unit-normalised BPE embeddings for head entities and relations.

Each entity/relation is represented as a sequence of *T* subword tokens. Their token embeddings are L2-normalised across the sequence dimension so that the resulting matrix has unit Frobenius norm.

Parameters

x (torch.LongTensor) – Shape (batch_size, 2, T) where dim 1 indexes [head, relation] and T is max_length_subword_tokens.

Returns

head_ent_emb, rel_emb – Each has shape (batch_size, T, embedding_dim), L2-normalised over the (T, D) dimensions.

Return type

torch.FloatTensor

get_embeddings () → Tuple[numpy.ndarray, numpy.ndarray]

Return the entity and relation embedding matrices as numpy arrays.

Returns

- **entity_embeddings** (numpy.ndarray) – Shape (num_entities, embedding_dim).
- **relation_embeddings** (numpy.ndarray) – Shape (num_relations, embedding_dim).

class dicee.models.**DistMult** (*args*)

Bases: *dicee.models.base_model.BaseKGE*

DistMult: bilinear diagonal knowledge graph embedding.

Scores a triple (*h*, *r*, *t*) as the element-wise product of the head, relation, and tail embeddings summed over the embedding dimension:

$$f(h, r, t) = \sum_i h_i \cdot r_i \cdot t_i$$

Simple yet effective baseline; incapable of modelling asymmetric relations.

References

Yang et al., *Embedding Entities and Relations for Learning and Inference in Knowledge Bases*, ICLR 2015. <https://arxiv.org/abs/1412.6575>

name = 'DistMult'

k_vs_all_score (*emb_h*: torch.FloatTensor, *emb_r*: torch.FloatTensor, *emb_E*: torch.FloatTensor) → torch.FloatTensor

Score a head/relation batch against all entity embeddings.

Computes $(h * r) @ E^T$ after applying hidden dropout and normalisation to the element-wise product.

Parameters

- **emb_h** (torch.FloatTensor) – Head entity embeddings, shape (batch_size, embedding_dim).
- **emb_r** (torch.FloatTensor) – Relation embeddings, shape (batch_size, embedding_dim).
- **emb_E** (torch.FloatTensor) – All entity embeddings, shape (num_entities, embedding_dim).

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_k_vs_all (*x*: torch.LongTensor) → torch.FloatTensor

KvsAll forward pass: score head/relation against all entities.

Parameters

x (torch.LongTensor) – Shape (batch_size, 2) integer tensor [head_idx, relation_idx].

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_k_vs_sample (*x*: torch.LongTensor, *target_entity_idx*: torch.LongTensor) → torch.FloatTensor

KvsSample forward pass: score head/relation against a sampled entity subset.

Parameters

- **x** (torch.LongTensor) – Shape (batch_size, 2) integer tensor [head_idx, relation_idx].
- **target_entity_idx** (torch.LongTensor) – Shape (batch_size, k) indices of the *k* target entities per sample.

Returns

Shape (batch_size, k) score matrix.

Return type

torch.FloatTensor

score (*h*: torch.FloatTensor, *r*: torch.FloatTensor, *t*: torch.FloatTensor) → torch.FloatTensor

Score a batch of (head, relation, tail) embedding triples.

Parameters

- **h** (*torch.FloatTensor*) – Each has shape (batch_size, embedding_dim).
- **r** (*torch.FloatTensor*) – Each has shape (batch_size, embedding_dim).
- **t** (*torch.FloatTensor*) – Each has shape (batch_size, embedding_dim).

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

class `dicee.models.TransE` (*args*)

Bases: `dicee.models.base_model.BaseKGE`

TransE: translation-based knowledge graph embedding.

Models a relation r as a translation in embedding space such that $h + r \approx t$ for a true triple (h, r, t) . The score function is defined as:

$$f(h, r, t) = \text{margin} - ||h + r - t||_2$$

TransE is effective for 1-to-1 relations but struggles with reflexive, one-to-many, and many-to-one patterns.

References

Bordes et al., *Translating Embeddings for Modeling Multi-relational Data*, NeurIPS 2013. <https://proceedings.neurips.cc/paper/2013/file/1cecc7a77928ca8133fa24680a88d2f9-Paper.pdf>

name = 'TransE'

margin

score (*head_ent_emb: torch.FloatTensor, rel_ent_emb: torch.FloatTensor, tail_ent_emb: torch.FloatTensor*) → *torch.FloatTensor*

Score a batch of triples using the TransE margin-distance formula.

Parameters

- **head_ent_emb** (*torch.FloatTensor*) – Each has shape (batch_size, embedding_dim).
- **rel_ent_emb** (*torch.FloatTensor*) – Each has shape (batch_size, embedding_dim).
- **tail_ent_emb** (*torch.FloatTensor*) – Each has shape (batch_size, embedding_dim).

Returns

Shape (batch_size,) scores equal to $\text{margin} - ||h + r - t||_2$.

Return type

torch.FloatTensor

forward_k_vs_all (*x: torch.Tensor*) → *torch.FloatTensor*

KvsAll forward pass: score head/relation against all entities.

Computes $\text{margin} - ||h + r - e||_2$ for every entity embedding e .

Parameters

x (*torch.Tensor*) – Shape (batch_size, 2) integer tensor [head_idx, relation_idx].

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

class dicee.models.**TransH**(args)

Bases: `dicee.models.base_model.BaseKGE`

TransH: translation-based knowledge graph embedding on relation-specific hyperplanes.

Addresses TransE's inability to model one-to-many, many-to-one, and many-to-many relations by letting each relation r define its own hyperplane. An entity is first projected onto that hyperplane before the translation is applied, so the same entity can be positioned differently depending on the relation. Concretely, for a triple (h, r, t) with unit-norm hyperplane normal r_w and in-hyperplane translation vector r_d :

```
h_r = h - (r_w^T h) * r_w
t_r = t - (r_w^T t) * r_w
f(h, r, t) = margin - ||h_r + r_d - t_r||_2^2
```

r_w reuses the inherited `relation_embeddings` table and is normalised to unit norm before every use, since only a unit-norm normal yields a true orthogonal projection onto the hyperplane. r_d is a second, relation-specific embedding table analogous to TransE's relation vector.

References

Wang et al., *Knowledge Graph Embedding by Translating on Hyperplanes*, AAAI 2014. <https://aaai.org/papers/8870-knowledge-graph-embedding-by-translating-on-hyperplanes/>

name = 'TransH'

margin

relation_normal_translations

forward_triples(x: torch.LongTensor) → torch.FloatTensor

Score a batch of (head, relation, tail) index triples.

Parameters

x (torch.LongTensor) – Shape (batch_size, 3) integer tensor [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

forward_k_vs_all(x: torch.LongTensor) → torch.FloatTensor

KvsAll forward pass: score head/relation against all entities.

Every candidate tail entity must be projected onto the batch row's relation-specific hyperplane, and the hyperplane normal differs per batch row, so unlike TransE this materialises a full (batch_size, num_entities, embedding_dim) tensor of projected tail candidates – $O(B * E * D)$ memory, versus TransE's $O(B * E)$.

Parameters

x (torch.LongTensor) – Shape (batch_size, 2) integer tensor [head_idx, relation_idx].

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_k_vs_sample (*x*: torch.LongTensor, *target_entity_idx*: torch.LongTensor) → torch.FloatTensor

KvsSample forward pass: score head/relation against a sampled entity subset.

Same hyperplane-projection logic as `forward_k_vs_all()`, but restricted to the *k* sampled tail candidates instead of every entity, so it costs $O(B * k * D)$ memory instead of $O(B * E * D)$.

Parameters

- **x** (torch.LongTensor) – Shape (batch_size, 2) integer tensor [head_idx, relation_idx].
- **target_entity_idx** (torch.LongTensor) – Shape (batch_size, k) indices of the *k* target entities per sample.

Returns

Shape (batch_size, k) score matrix.

Return type

torch.FloatTensor

class dicee.models.**MuRE** (*args*)

Bases: `dicee.models.base_model.BaseKGE`

MuRE: multi-relational graph embedding with relation-specific diagonal scaling, translation, and entity biases.

Scores a triple (*h*, *r*, *t*) as:

$$f(h, r, t) = -||R_r \odot h + t_r - t||_2 + b_h + b_t$$

R_r is a relation-specific diagonal matrix applied to the head via an element-wise (Hadamard) product; it reuses the inherited `relation_embeddings` table directly, shape (num_relations, embedding_dim). t_r is a second, relation-specific translation vector (own embedding table, same shape convention: (num_relations, embedding_dim)) analogous to TransE's relation vector. b_h and b_t are learnable scalar biases indexed per entity (head and tail respectively), not per relation.

References

Balažević et al., *Multi-relational Poincaré Graph Embeddings*, NeurIPS 2019. <https://arxiv.org/abs/1905.09791>

name = 'MuRE'

relation_translations

b_h

b_t

forward_triples (*x*: torch.LongTensor) → torch.FloatTensor

Score a batch of (head, relation, tail) index triples.

Parameters

- **x** (torch.LongTensor) – Shape (batch_size, 3) integer tensor [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

forward_k_vs_all (x : torch.LongTensor) $\rightarrow$ torch.FloatTensor

KvsAll forward pass: score head/relation against all entities.

Computes $-||R_r \odot h + t_r - e||_2 + b_h + b_e$ for every entity embedding e .**Parameters** $\mathbf{x}$ (torch.LongTensor) – Shape (batch_size, 2) integer tensor [head_idx, relation_idx].**Returns**

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_k_vs_sample (x : torch.LongTensor, target_entity_idx: torch.LongTensor) $\rightarrow$ torch.FloatTensor

KvsSample forward pass: score head/relation against a sampled entity subset.

Parameters

- $\mathbf{x}$ (torch.LongTensor) – Shape (batch_size, 2) integer tensor [head_idx, relation_idx].
- **target_entity_idx** (torch.LongTensor) – Shape (batch_size, k) indices of the k target entities per sample.

Returns

Shape (batch_size, k) score matrix.

Return type

torch.FloatTensor

class dicee.models.Shallom ($args$)Bases: `dicee.models.base_model.BaseKGE`

Shallom: shallow neural model for relation prediction.

Represents each triple as the concatenation of head and tail entity embeddings and feeds it through a two-layer MLP to predict the relation. Designed for the RelationPrediction labelling form.

ReferencesDemir et al., *A Shallow Neural Model for Relation Prediction*, ISWC 2021. <https://arxiv.org/abs/2101.09090>**name** = 'Shallom'**shallom****get_embeddings** () $\rightarrow$ Tuple[numpy.ndarray, None]

Return the entity and relation embedding matrices as numpy arrays.

Returns

- **entity_embeddings** (numpy.ndarray) – Shape (num_entities, embedding_dim).
- **relation_embeddings** (numpy.ndarray) – Shape (num_relations, embedding_dim).

forward_k_vs_all (*x*) → torch.FloatTensor

Score a (head, relation) batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_triples (*x*) → torch.FloatTensor

Score a batch of triples by looking up relation scores from `forward_k_vs_all`.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

class `dicee.models.Pyke` (*args*)

Bases: `dicee.models.base_model.BaseKGE`

Pyke: Physical Embedding Model for Knowledge Graphs.

Scores a triple (h, r, t) based on the average pairwise distance between head-to-relation and relation-to-tail in embedding space:

$$f(h, r, t) = \text{margin} - (||h - r||_2 + ||r - t||_2) / 2$$

The model encodes geometric proximity between entities and the relations that connect them.

name = 'Pyke'

dist_func

margin

forward_triples (*x*: *torch.LongTensor*) → torch.FloatTensor

Score a batch of triples using the Pyke distance formula.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

class `dicee.models.CoKEConfig`

Configuration for the CoKE (Contextualized Knowledge Graph Embedding) model.

block_size

Sequence length for transformer (3 for triples: head, relation, tail)

vocab_size
Total vocabulary size (num_entities + num_relations)

n_layer
Number of transformer layers

n_head
Number of attention heads per layer

n_embd
Embedding dimension (set to match model embedding_dim)

dropout
Dropout rate applied throughout the model

bias
Whether to use bias in linear layers

causal
Whether to use causal masking (False for bidirectional attention)

block_size: int = 3

vocab_size: int = None

n_layer: int = 6

n_head: int = 8

n_embd: int = None

dropout: float = 0.3

bias: bool = True

causal: bool = False

```
class dicee.models.CoKE(args, config: CoKEConfig = CoKEConfig())
```

Bases: *[dicee.models.base_model.BaseKGE](#)*

Contextualized Knowledge Graph Embedding (CoKE) model. Based on: <https://arxiv.org/pdf/1911.02168>.

CoKE uses a transformer encoder to learn contextualized representations of entities and relations. For link prediction, it predicts masked elements in (head, relation, tail) triples using bidirectional attention, similar to BERT's masked language modeling approach.

The model creates a sequence [head_emb, relation_emb, mask_emb], adds positional embeddings, and processes it through transformer layers to predict the tail entity.

name = 'CoKE'

config

pos_emb

mask_emb

blocks

ln_f

coke_dropout

forward_k_vs_all (*x*: *torch.Tensor*)

Score a (head, relation) batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns

Shape (batch_size, num_entities) score matrix.

Return type

`torch.FloatTensor`

score (*emb_h*, *emb_r*, *emb_t*)

forward_k_vs_sample (*x*: *torch.LongTensor*, *target_entity_idx*: *torch.LongTensor*)

Score a (head, relation) batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns

Shape (batch_size, k) score matrix where *k* is the number of sampled target entities.

Return type

`torch.FloatTensor`

class `dicee.models.BaseKGE` (*args*: *dict*)

Bases: `BaseKGELightning`

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from `BaseKGELightning` and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- `forward_triples()` — score a batch of (h, r, t) triples.
- `forward_k_vs_all()` — score a (h, r) batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

args

embedding_dim = None

num_entities: int | None = None

num_relations: int | None = None

num_tokens = None

learning_rate = None

apply_unit_norm = None

```

input_dropout_rate = None
hidden_dropout_rate = None
optimizer_name = None
feature_map_dropout_rate = None
kernel_size = None
num_of_output_channels = None
weight_decay = None
selected_optimizer = None
normalizer_class = None
normalize_head_entity_embeddings
normalize_relation_embeddings
normalize_tail_entity_embeddings
hidden_normalizer
param_init
input_dp_ent_real
input_dp_rel_real
hidden_dropout
loss_history = []
byte_pair_encoding
max_length_subword_tokens
block_size
defer_large_embeddings

```

init_entity_embeddings (*embedding_dim: int | None = None*) → None

Create (or re-create) the entity embedding table.

This is the single place that honours *defer_large_embeddings*: when entity rows are sharded across FSDP ranks the table must stay `None` until the trainer allocates the sharded adapter. Subclasses that need an entity table of a non-default width must call this method instead of assigning `self.entity_embeddings` directly, so the deferral is never silently undone.

Parameters

embedding_dim (*Optional[int]*) – Width of the table. Defaults to *embedding_dim*.

init_relation_embeddings (*embedding_dim: int | None = None*) → None

Create (or re-create) the relation embedding table.

Relation tables are never deferred - they are small enough to be replicated on every rank.

Parameters

embedding_dim (*Optional[int]*) – Width of the table. Defaults to *embedding_dim*.

forward_byte_pair_encoded_k_vs_all (*x*: *torch.LongTensor*) → *torch.FloatTensor*

KvsAll scoring for BPE-encoded head entities and relations.

Retrieves subword-unit embeddings for the head entity and relation, reduces them to fixed-size vectors via a linear projection, then scores against all BPE entity embeddings.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 2, T) BPE token indices where dim 1 indexes [head, relation] and T is max_length_subword_tokens.

Returns

Shape (batch_size, num_bpe_entities) score matrix.

Return type

torch.FloatTensor

forward_byte_pair_encoded_triple (*x*: *Tuple[torch.LongTensor, torch.LongTensor]*)

→ *torch.FloatTensor*

NegSample scoring for BPE-encoded (head, relation, tail) triples.

Retrieves subword-unit embeddings for all three elements and reduces them to fixed-size vectors via a linear projection before computing the triple score.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3, T) BPE token indices.

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

init_params_with_sanity_checking() → None

Populate model hyper-parameters from *self.args* with safe defaults.

Reads embedding dimension, learning rate, dropout rates, normalisation strategy, optimizer name, and parameter initialisation scheme from the *args* dict. Falls back to sensible defaults for any missing key so that minimal *args* dicts (e.g. for unit tests) are still valid.

forward (*x*: *torch.LongTensor* | *Tuple[torch.LongTensor, torch.LongTensor]*,

y_idx: *torch.LongTensor* | None = None) → *torch.Tensor*

Route the forward pass to the appropriate scoring method.

Inspects the shape and type of *x* to decide which low-level scorer to call:

- *Tuple* (*x*, *y_idx*) → *forward_k_vs_sample()*
- (batch, 3) tensor → *forward_triples()*
- (batch, 2) tensor → *forward_k_vs_all()*
- BPE triple tensor → *forward_byte_pair_encoded_triple()*
- BPE pair tensor → *forward_byte_pair_encoded_k_vs_all()*

Parameters

• **x** (*torch.LongTensor* or *Tuple[torch.LongTensor, torch.LongTensor]*) – Either a plain index tensor or a (triple_idx, target_idx) tuple for sample-based labelling.

• **y_idx** (*torch.LongTensor*, optional) – Target entity indices used by *forward_k_vs_sample()*. Ignored when *x* is a plain tensor.

Returns

Score tensor whose shape depends on the selected scorer.

Return type

torch.FloatTensor

forward_triples (*x*: torch.LongTensor) → torch.Tensor

Score a batch of (head, relation, tail) index triples.

Parameters

x (torch.LongTensor) – Shape (batch_size, 3) integer tensor where each row is [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

forward_k_vs_all (*args, **kwargs)

Score a (head, relation) batch against every entity.

Sub-classes must override this method. The default implementation raises ValueError to make missing overrides obvious at runtime.

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_k_vs_sample (*args, **kwargs)

Score a (head, relation) batch against a sampled subset of entities.

Used by KvsSample and 1vsSample datasets. Sub-classes that support sample-based labelling must override this method.

Returns

Shape (batch_size, k) score matrix where *k* is the number of sampled target entities.

Return type

torch.FloatTensor

get_triple_representation (idx_hrt)

→ Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for a triple index batch.

Parameters

idx_hrt (torch.LongTensor) – Shape (batch_size, 3) integer tensor with columns [head_idx, relation_idx, tail_idx].

Returns

head_ent_emb, rel_ent_emb, tail_ent_emb – Each has shape (batch_size, embedding_dim) after applying the configured dropout and normalisation.

Return type

torch.FloatTensor

get_head_relation_representation (indexed_triple)

→ Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for head entities and relations.

Parameters

indexed_triple (*torch.LongTensor*) – Shape (batch_size, 2) integer tensor with columns [head_idx, relation_idx].

Returns

head_ent_emb, rel_ent_emb – Each has shape (batch_size, embedding_dim) after applying the configured dropout and normalisation.

Return type

torch.FloatTensor

get_sentence_representation (*x: torch.LongTensor*)

→ Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve BPE subword-unit embeddings for a batch of triples.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3, T) where *T* is max_length_subword_tokens.

Returns

head_ent_emb, rel_emb, tail_emb – Each has shape (batch_size, T, embedding_dim).

Return type

torch.FloatTensor

get_bpe_head_and_relation_representation (*x: torch.LongTensor*)

→ Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve unit-normalised BPE embeddings for head entities and relations.

Each entity/relation is represented as a sequence of *T* subword tokens. Their token embeddings are L2-normalised across the sequence dimension so that the resulting matrix has unit Frobenius norm.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 2, T) where dim 1 indexes [head, relation] and *T* is max_length_subword_tokens.

Returns

head_ent_emb, rel_emb – Each has shape (batch_size, T, embedding_dim), L2-normalised over the (T, D) dimensions.

Return type

torch.FloatTensor

get_embeddings () → Tuple[numpy.ndarray, numpy.ndarray]

Return the entity and relation embedding matrices as numpy arrays.

Returns

- **entity_embeddings** (*numpy.ndarray*) – Shape (num_entities, embedding_dim).
- **relation_embeddings** (*numpy.ndarray*) – Shape (num_relations, embedding_dim).

class dicee.models.ConEx (*args*)

Bases: *dicee.models.base_model.BaseKGE*

Convolutional ComplEx Knowledge Graph Embeddings

name = 'ConEx'

conv2d

```

fc_num_input

fc1

norm_fc1

bn_conv2d

feature_map_dropout

residual_convolution (C_1: Tuple[torch.Tensor, torch.Tensor],
                      C_2: Tuple[torch.Tensor, torch.Tensor]) → torch.FloatTensor
    Compute residual score of two complex-valued embeddings. :param C_1: a tuple of two pytorch tensors
    that corresponds complex-valued embeddings :param C_2: a tuple of two pytorch tensors that corresponds
    complex-valued embeddings :return:

forward_k_vs_all (x: torch.Tensor) → torch.FloatTensor
    Score a (head, relation) batch against every entity.

    Sub-classes must override this method. The default implementation raises ValueError to make missing
    overrides obvious at runtime.

    Returns
        Shape (batch_size, num_entities) score matrix.

    Return type
        torch.FloatTensor

forward_triples (x: torch.Tensor) → torch.FloatTensor
    Score a batch of (head, relation, tail) index triples.

    Parameters
        x (torch.LongTensor) – Shape (batch_size, 3) integer tensor where each row is
        [head_idx, relation_idx, tail_idx].

    Returns
        Shape (batch_size,) triple scores.

    Return type
        torch.FloatTensor

forward_k_vs_sample (x: torch.Tensor, target_entity_idx: torch.Tensor)
    Score a (head, relation) batch against a sampled subset of entities.

    Used by KvsSample and 1vsSample datasets. Sub-classes that support sample-based labelling must over-
    ride this method.

    Returns
        Shape (batch_size, k) score matrix where k is the number of sampled target entities.

    Return type
        torch.FloatTensor

class dicee.models.AConEx (args)
    Bases: dicee.models.base_model.BaseKGE
    Additive Convolutional ComplEx Knowledge Graph Embeddings
    name = 'AConEx'
    conv2d

```

fc_num_input

fc1

norm_fc1

bn_conv2d

feature_map_dropout

residual_convolution (*C_1*: *Tuple*[*torch.Tensor*, *torch.Tensor*],
 C_2: *Tuple*[*torch.Tensor*, *torch.Tensor*]) → *torch.FloatTensor*

Compute residual score of two complex-valued embeddings. :param C_1: a tuple of two pytorch tensors that corresponds complex-valued embeddings :param C_2: a tuple of two pytorch tensors that corresponds complex-valued embeddings :return:

forward_k_vs_all (*x*: *torch.Tensor*) → *torch.FloatTensor*

Score a (head, relation) batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_triples (*x*: *torch.Tensor*) → *torch.FloatTensor*

Score a batch of (head, relation, tail) index triples.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor where each row is [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

forward_k_vs_sample (*x*: *torch.Tensor*, *target_entity_idx*: *torch.Tensor*)

Score a (head, relation) batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns

Shape (batch_size, k) score matrix where *k* is the number of sampled target entities.

Return type

torch.FloatTensor

class `dicee.models.Complex` (*args*)

Bases: `dicee.models.base_model.BaseKGE`

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from `BaseKGELightning` and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- `forward_triples()` — score a batch of (h, r, t) triples.
- `forward_k_vs_all()` — score a (h, r) batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

name = 'Complex'

static score (*head_ent_emb: torch.FloatTensor, rel_ent_emb: torch.FloatTensor, tail_ent_emb: torch.FloatTensor*)

static k_vs_all_score (*emb_h: torch.FloatTensor, emb_r: torch.FloatTensor, emb_E: torch.FloatTensor*)

Parameters

- `emb_h`
- `emb_r`
- `emb_E`

forward_k_vs_all (*x: torch.LongTensor*) → `torch.FloatTensor`

Score a $(\text{head}, \text{relation})$ batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns

Shape $(\text{batch_size}, \text{num_entities})$ score matrix.

Return type

`torch.FloatTensor`

forward_k_vs_sample (*x: torch.LongTensor, target_entity_idx: torch.LongTensor*)

Score a $(\text{head}, \text{relation})$ batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns

Shape $(\text{batch_size}, k)$ score matrix where k is the number of sampled target entities.

Return type

`torch.FloatTensor`

class `dicee.models.RotatE` (*args*)

Bases: `dicee.models.base_model.BaseKGE`

RotatE: knowledge graph embedding by relational rotation in complex space.

Represents each entity as a complex vector in $\mathbb{C}^{(d/2)}$, obtained by splitting its d -dimensional embedding into a real half and an imaginary half. Each relation is represented by $d/2$ phase angles θ that define a unit-modulus rotation $r_i = e^{i\theta_i}$; since a phase angle needs only one real number, the relation embedding table is reinitialised at half of the entity embedding size. A true triple (h, r, t) should satisfy $h \circ r \approx t$ under element-wise complex multiplication, giving the score:

$$f(h, r, t) = \text{margin} - ||h \circ r - t||_2$$

Unlike TransE, RotatE can model symmetric, antisymmetric, inverse, and composition relation patterns.

References

Sun et al., *RotatE: Knowledge Graph Embedding by Relational Rotation in Complex Space*, ICLR 2019. <https://arxiv.org/abs/1902.10197>

name = 'RotatE'

margin

half_dim

relation_embeddings

score (*head_ent_emb*: torch.FloatTensor, *rel_ent_emb*: torch.FloatTensor, *tail_ent_emb*: torch.FloatTensor) → torch.FloatTensor

Score a batch of triples using the RotatE margin-distance formula.

Parameters

- **head_ent_emb** (torch.FloatTensor) – Each has shape (batch_size, embedding_dim).
- **tail_ent_emb** (torch.FloatTensor) – Each has shape (batch_size, embedding_dim).
- **rel_ent_emb** (torch.FloatTensor) – Shape (batch_size, d/2) relation phase angles θ .

Returns

Shape (batch_size,) scores equal to $\text{margin} - ||h \circ r - t||_2$.

Return type

torch.FloatTensor

forward_k_vs_all (*x*: torch.Tensor) → torch.FloatTensor

KvsAll forward pass: score head/relation against all entities.

Computes $\text{margin} - ||h \circ r - e||_2$ for every entity embedding e .

Parameters

x (torch.Tensor) – Shape (batch_size, 2) integer tensor [head_idx, relation_idx].

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_k_vs_sample (*x*: torch.Tensor, *target_entity_idx*: torch.Tensor) → torch.FloatTensor

KvsSample forward pass: score head/relation against a sampled entity subset.

Computes $\text{margin} - ||h \circ r - e||_2$ for each of the k sampled entities e .

Parameters

- **x** (torch.Tensor) – Shape (batch_size, 2) integer tensor [head_idx, relation_idx].
- **target_entity_idx** (torch.Tensor) – Shape (batch_size, k) indices of the k target entities per sample.

Returns

Shape (batch_size, k) score matrix.

Return type

torch.FloatTensor

class dicee.models.**BaseKGE** (*args: dict*)

Bases: *BaseKGELightning*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from *BaseKGELightning* and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- *forward_triples()* — score a batch of (h, r, t) triples.
- *forward_k_vs_all()* — score a (h, r) batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

args

embedding_dim = None

num_entities: int | None = None

num_relations: int | None = None

num_tokens = None

learning_rate = None

apply_unit_norm = None

input_dropout_rate = None

hidden_dropout_rate = None

optimizer_name = None

feature_map_dropout_rate = None

kernel_size = None

num_of_output_channels = None

weight_decay = None

selected_optimizer = None

normalizer_class = None

normalize_head_entity_embeddings

normalize_relation_embeddings

`normalize_tail_entity_embeddings`

`hidden_normalizer`

`param_init`

`input_dp_ent_real`

`input_dp_rel_real`

`hidden_dropout`

`loss_history = []`

`byte_pair_encoding`

`max_length_subword_tokens`

`block_size`

`defer_large_embeddings`

`init_entity_embeddings (embedding_dim: int | None = None) → None`

Create (or re-create) the entity embedding table.

This is the single place that honours `defer_large_embeddings`: when entity rows are sharded across FSDP ranks the table must stay `None` until the trainer allocates the sharded adapter. Subclasses that need an entity table of a non-default width must call this method instead of assigning `self.entity_embeddings` directly, so the deferral is never silently undone.

Parameters

`embedding_dim (Optional[int])` – Width of the table. Defaults to `embedding_dim`.

`init_relation_embeddings (embedding_dim: int | None = None) → None`

Create (or re-create) the relation embedding table.

Relation tables are never deferred - they are small enough to be replicated on every rank.

Parameters

`embedding_dim (Optional[int])` – Width of the table. Defaults to `embedding_dim`.

`forward_byte_pair_encoded_k_vs_all (x: torch.LongTensor) → torch.FloatTensor`

KvsAll scoring for BPE-encoded head entities and relations.

Retrieves subword-unit embeddings for the head entity and relation, reduces them to fixed-size vectors via a linear projection, then scores against all BPE entity embeddings.

Parameters

`x (torch.LongTensor)` – Shape `(batch_size, 2, T)` BPE token indices where dim 1 indexes `[head, relation]` and `T` is `max_length_subword_tokens`.

Returns

Shape `(batch_size, num_bpe_entities)` score matrix.

Return type

`torch.FloatTensor`

`forward_byte_pair_encoded_triple (x: Tuple[torch.LongTensor, torch.LongTensor])`

→ `torch.FloatTensor`

NegSample scoring for BPE-encoded `(head, relation, tail)` triples.

Retrieves subword-unit embeddings for all three elements and reduces them to fixed-size vectors via a linear projection before computing the triple score.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3, T) BPE token indices.

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

init_params_with_sanity_checking() → None

Populate model hyper-parameters from `self.args` with safe defaults.

Reads embedding dimension, learning rate, dropout rates, normalisation strategy, optimizer name, and parameter initialisation scheme from the `args` dict. Falls back to sensible defaults for any missing key so that minimal `args` dicts (e.g. for unit tests) are still valid.

forward (*x*: *torch.LongTensor* | *Tuple*[*torch.LongTensor*, *torch.LongTensor*],
y_idx: *torch.LongTensor* | *None* = *None*) → *torch.Tensor*

Route the forward pass to the appropriate scoring method.

Inspects the shape and type of *x* to decide which low-level scorer to call:

- *Tuple* (*x*, *y_idx*) → *forward_k_vs_sample()*
- (batch, 3) tensor → *forward_triples()*
- (batch, 2) tensor → *forward_k_vs_all()*
- BPE triple tensor → *forward_byte_pair_encoded_triple()*
- BPE pair tensor → *forward_byte_pair_encoded_k_vs_all()*

Parameters

- **x** (*torch.LongTensor* or *Tuple*[*torch.LongTensor*, *torch.LongTensor*]) – Either a plain index tensor or a (*triple_idx*, *target_idx*) tuple for sample-based labelling.
- **y_idx** (*torch.LongTensor*, *optional*) – Target entity indices used by *forward_k_vs_sample()*. Ignored when *x* is a plain tensor.

Returns

Score tensor whose shape depends on the selected scorer.

Return type

torch.FloatTensor

forward_triples (*x*: *torch.LongTensor*) → *torch.Tensor*

Score a batch of (head, relation, tail) index triples.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor where each row is [*head_idx*, *relation_idx*, *tail_idx*].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

forward_k_vs_all(*args, **kwargs)

Score a (head, relation) batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_k_vs_sample(*args, **kwargs)

Score a (head, relation) batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns

Shape (batch_size, k) score matrix where k is the number of sampled target entities.

Return type

torch.FloatTensor

get_triple_representation(idx_hrt)

→ Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for a triple index batch.

Parameters

idx_hrt (torch.LongTensor) – Shape (batch_size, 3) integer tensor with columns [head_idx, relation_idx, tail_idx].

Returns

head_ent_emb, rel_ent_emb, tail_ent_emb – Each has shape (batch_size, embedding_dim) after applying the configured dropout and normalisation.

Return type

torch.FloatTensor

get_head_relation_representation(indexed_triple)

→ Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for head entities and relations.

Parameters

indexed_triple (torch.LongTensor) – Shape (batch_size, 2) integer tensor with columns [head_idx, relation_idx].

Returns

head_ent_emb, rel_ent_emb – Each has shape (batch_size, embedding_dim) after applying the configured dropout and normalisation.

Return type

torch.FloatTensor

get_sentence_representation(x: torch.LongTensor)

→ Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve BPE subword-unit embeddings for a batch of triples.

Parameters

x (torch.LongTensor) – Shape (batch_size, 3, T) where T is max_length_subword_tokens.

Returns

head_ent_emb, rel_emb, tail_emb – Each has shape (batch_size, T, embedding_dim).

Return type

torch.FloatTensor

get_bpe_head_and_relation_representation (*x: torch.LongTensor*)
→ Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve unit-normalised BPE embeddings for head entities and relations.

Each entity/relation is represented as a sequence of T subword tokens. Their token embeddings are L2-normalised across the sequence dimension so that the resulting matrix has unit Frobenius norm.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 2, T) where dim 1 indexes [head, relation] and T is max_length_subword_tokens.

Returns

head_ent_emb, rel_emb – Each has shape (batch_size, T, embedding_dim), L2-normalised over the (T, D) dimensions.

Return type

torch.FloatTensor

get_embeddings () → Tuple[numpy.ndarray, numpy.ndarray]

Return the entity and relation embedding matrices as numpy arrays.

Returns

- **entity_embeddings** (*numpy.ndarray*) – Shape (num_entities, embedding_dim).
- **relation_embeddings** (*numpy.ndarray*) – Shape (num_relations, embedding_dim).

class dicee.models.IdentityClass (*args=None*)

Bases: torch.nn.Module

No-op normalisation / dropout placeholder.

Used whenever no normalisation layer is requested (`--normalization None`). All inputs are returned unchanged so that the rest of the model code does not need conditional checks around normalisation calls.

args = None

__call__ (*x*)

static forward (*x*)

dicee.models.quaternion_mul (*, *Q_1, Q_2*)

→ Tuple[torch.Tensor, torch.Tensor, torch.Tensor, torch.Tensor]

Perform quaternion multiplication :param Q_1: :param Q_2: :return:

dicee.models.quaternion_mul_with_unit_norm (*, *Q_1, Q_2*)

class dicee.models.QMult (*args*)

Bases: *dicee.models.base_model.BaseKGE*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from *BaseKGELightning* and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- `forward_triples()` — score a batch of (h, r, t) triples.
- `forward_k_vs_all()` — score a (h, r) batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

`name = 'QMult'`

`explicit = True`

`quaternion_multiplication_followed_by_inner_product(h, r, t)`

Parameters

- **h** – shape: (*batch_dims, dim) The head representations.
- **r** – shape: (*batch_dims, dim) The head representations.
- **t** – shape: (*batch_dims, dim) The tail representations.

Returns

Triple scores.

static quaternion_normalizer (*x: torch.FloatTensor*) → *torch.FloatTensor*

Normalize the length of relation vectors, if the forward constraint has not been applied yet.

Absolute value of a quaternion

$$|a + bi + cj + dk| = \sqrt{a^2 + b^2 + c^2 + d^2}$$

L2 norm of quaternion vector:

$$\|x\|^2 = \sum_{i=1}^d |x_i|^2 = \sum_{i=1}^d (x_i.re^2 + x_i.im_1^2 + x_i.im_2^2 + x_i.im_3^2)$$

Parameters

x – The vector.

Returns

The normalized vector.

score (*head_ent_emb: torch.FloatTensor, rel_ent_emb: torch.FloatTensor, tail_ent_emb: torch.FloatTensor*)

k_vs_all_score (*bpe_head_ent_emb, bpe_rel_ent_emb, E*)

Parameters

- `bpe_head_ent_emb`
- `bpe_rel_ent_emb`
- `E`

forward_k_vs_all (*x*)

Parameters

x

forward_k_vs_sample (*x*, *target_entity_idx*)

Completed. Given a head entity and a relation (h,r), we compute scores for all possible triples,i.e.,
[score(h,r,x)|x in Entities] => [0.0,0.1,...,0.8], shape=> (1, **Entities**) Given a batch of head entities and
relations => shape (size of batch,| Entities|)

class *dicee.models.ConvQ* (*args*)

Bases: *dicee.models.base_model.BaseKGE*

Convolutional Quaternion Knowledge Graph Embeddings

name = 'ConvQ'

entity_embeddings

relation_embeddings

conv2d

fc_num_input

fc1

bn_conv1

bn_conv2

feature_map_dropout

residual_convolution (*Q_1*, *Q_2*)

forward_triples (*indexed_triple: torch.Tensor*) → torch.Tensor

Score a batch of (head, relation, tail) index triples.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor where each row is
[head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

forward_k_vs_all (*x: torch.Tensor*)

Given a head entity and a relation (h,r), we compute scores for all entities. [score(h,r,x)|x in Entities] =>
[0.0,0.1,...,0.8], shape=> (1, **Entities**) Given a batch of head entities and relations => shape (size of batch,|
Entities|)

class *dicee.models.AConvQ* (*args*)

Bases: *dicee.models.base_model.BaseKGE*

Additive Convolutional Quaternion Knowledge Graph Embeddings

name = 'AConvQ'

entity_embeddings

relation_embeddings

conv2d

`fc_num_input`

`fc1`

`bn_conv1`

`bn_conv2`

`feature_map_dropout`

`residual_convolution(Q_1, Q_2)`

`forward_triples(indexed_triple: torch.Tensor) → torch.Tensor`

Score a batch of (head, relation, tail) index triples.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor where each row is [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

`forward_k_vs_all(x: torch.Tensor)`

Given a head entity and a relation (h,r), we compute scores for all entities. [score(h,r,x)|x in Entities] => [0.0,0.1,...,0.8], shape=> (1, **Entities**) Given a batch of head entities and relations => shape (size of batch,|Entities|)

`class dicee.models.BaseKGE(args: dict)`

Bases: *BaseKGELightning*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from *BaseKGELightning* and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- `forward_triples()` — score a batch of (h, r, t) triples.
- `forward_k_vs_all()` — score a (h, r) batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

args

`embedding_dim = None`

`num_entities: int | None = None`

`num_relations: int | None = None`

`num_tokens = None`

`learning_rate = None`

```

apply_unit_norm = None

input_dropout_rate = None

hidden_dropout_rate = None

optimizer_name = None

feature_map_dropout_rate = None

kernel_size = None

num_of_output_channels = None

weight_decay = None

selected_optimizer = None

normalizer_class = None

normalize_head_entity_embeddings

normalize_relation_embeddings

normalize_tail_entity_embeddings

hidden_normalizer

param_init

input_dp_ent_real

input_dp_rel_real

hidden_dropout

loss_history = []

byte_pair_encoding

max_length_subword_tokens

block_size

defer_large_embeddings

init_entity_embeddings (embedding_dim: int | None = None) → None

```

Create (or re-create) the entity embedding table.

This is the single place that honours *defer_large_embeddings*: when entity rows are sharded across FSDP ranks the table must stay *None* until the trainer allocates the sharded adapter. Subclasses that need an entity table of a non-default width must call this method instead of assigning *self.entity_embeddings* directly, so the deferral is never silently undone.

Parameters

embedding_dim (*Optional[int]*) – Width of the table. Defaults to *embedding_dim*.

init_relation_embeddings (*embedding_dim*: int | None = None) → None

Create (or re-create) the relation embedding table.

Relation tables are never deferred - they are small enough to be replicated on every rank.

Parameters

embedding_dim (*Optional[int]*) – Width of the table. Defaults to *embedding_dim*.

forward_byte_pair_encoded_k_vs_all (*x*: torch.LongTensor) → torch.FloatTensor

KvsAll scoring for BPE-encoded head entities and relations.

Retrieves subword-unit embeddings for the head entity and relation, reduces them to fixed-size vectors via a linear projection, then scores against all BPE entity embeddings.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 2, T) BPE token indices where dim 1 indexes [head, relation] and T is max_length_subword_tokens.

Returns

Shape (batch_size, num_bpe_entities) score matrix.

Return type

torch.FloatTensor

forward_byte_pair_encoded_triple (*x*: Tuple[torch.LongTensor, torch.LongTensor])

→ torch.FloatTensor

NegSample scoring for BPE-encoded (head, relation, tail) triples.

Retrieves subword-unit embeddings for all three elements and reduces them to fixed-size vectors via a linear projection before computing the triple score.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3, T) BPE token indices.

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

init_params_with_sanity_checking () → None

Populate model hyper-parameters from *self.args* with safe defaults.

Reads embedding dimension, learning rate, dropout rates, normalisation strategy, optimizer name, and parameter initialisation scheme from the *args* dict. Falls back to sensible defaults for any missing key so that minimal *args* dicts (e.g. for unit tests) are still valid.

forward (*x*: torch.LongTensor | Tuple[torch.LongTensor, torch.LongTensor],

y_idx: torch.LongTensor | None = None) → torch.Tensor

Route the forward pass to the appropriate scoring method.

Inspects the shape and type of *x* to decide which low-level scorer to call:

- Tuple (*x*, *y_idx*) → *forward_k_vs_sample*()
- (batch, 3) tensor → *forward_triples*()
- (batch, 2) tensor → *forward_k_vs_all*()
- BPE triple tensor → *forward_byte_pair_encoded_triple*()
- BPE pair tensor → *forward_byte_pair_encoded_k_vs_all*()

Parameters

- **x** (*torch.LongTensor* or *Tuple[torch.LongTensor, torch.LongTensor]*) – Either a plain index tensor or a (*triple_idx*, *target_idx*) tuple for sample-based labelling.
- **y_idx** (*torch.LongTensor*, *optional*) – Target entity indices used by *forward_k_vs_sample()*. Ignored when *x* is a plain tensor.

Returns

Score tensor whose shape depends on the selected scorer.

Return type

torch.FloatTensor

forward_triples (*x: torch.LongTensor*) → *torch.Tensor*

Score a batch of (head, relation, tail) index triples.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor where each row is [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

forward_k_vs_all (*args, **kwargs)

Score a (head, relation) batch against every entity.

Sub-classes must override this method. The default implementation raises *ValueError* to make missing overrides obvious at runtime.

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_k_vs_sample (*args, **kwargs)

Score a (head, relation) batch against a sampled subset of entities.

Used by *KvsSample* and *1vsSample* datasets. Sub-classes that support sample-based labelling must override this method.

Returns

Shape (batch_size, k) score matrix where *k* is the number of sampled target entities.

Return type

torch.FloatTensor

get_triple_representation (*idx_hrt*)

→ *Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]*

Retrieve and normalise embedding vectors for a triple index batch.

Parameters

idx_hrt (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor with columns [head_idx, relation_idx, tail_idx].

Returns

head_ent_emb, **rel_ent_emb**, **tail_ent_emb** – Each has shape (batch_size, embedding_dim) after applying the configured dropout and normalisation.

Return type

torch.FloatTensor

get_head_relation_representation (*indexed_triple*)

→ Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for head entities and relations.

Parameters

indexed_triple (*torch.LongTensor*) – Shape (batch_size, 2) integer tensor with columns [head_idx, relation_idx].

Returns

head_ent_emb, rel_ent_emb – Each has shape (batch_size, embedding_dim) after applying the configured dropout and normalisation.

Return type

torch.FloatTensor

get_sentence_representation (*x: torch.LongTensor*)

→ Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve BPE subword-unit embeddings for a batch of triples.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3, T) where T is max_length_subword_tokens.

Returns

head_ent_emb, rel_emb, tail_emb – Each has shape (batch_size, T, embedding_dim).

Return type

torch.FloatTensor

get_bpe_head_and_relation_representation (*x: torch.LongTensor*)

→ Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve unit-normalised BPE embeddings for head entities and relations.

Each entity/relation is represented as a sequence of T subword tokens. Their token embeddings are L2-normalised across the sequence dimension so that the resulting matrix has unit Frobenius norm.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 2, T) where dim 1 indexes [head, relation] and T is max_length_subword_tokens.

Returns

head_ent_emb, rel_emb – Each has shape (batch_size, T, embedding_dim), L2-normalised over the (T, D) dimensions.

Return type

torch.FloatTensor

get_embeddings () → Tuple[numpy.ndarray, numpy.ndarray]

Return the entity and relation embedding matrices as numpy arrays.

Returns

- **entity_embeddings** (*numpy.ndarray*) – Shape (num_entities, embedding_dim).
- **relation_embeddings** (*numpy.ndarray*) – Shape (num_relations, embedding_dim).

```
class dicee.models.IdentityClass (args=None)
```

Bases: torch.nn.Module

No-op normalisation / dropout placeholder.

Used whenever no normalisation layer is requested (`--normalization None`). All inputs are returned unchanged so that the rest of the model code does not need conditional checks around normalisation calls.

args = None

__call__(x)

static forward(x)

```
dicee.models.octonion_mul (*, O_1, O_2)
```

```
dicee.models.octonion_mul_norm (*, O_1, O_2)
```

```
class dicee.models.OMult (args)
```

Bases: `dicee.models.base_model.BaseKGE`

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from `BaseKGELightning` and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- `forward_triples()` — score a batch of (h, r, t) triples.
- `forward_k_vs_all()` — score a (h, r) batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

name = 'OMult'

static octonion_normalizer(*emb_rel_e0, emb_rel_e1, emb_rel_e2, emb_rel_e3, emb_rel_e4, emb_rel_e5, emb_rel_e6, emb_rel_e7*)

score(*head_ent_emb: torch.FloatTensor, rel_ent_emb: torch.FloatTensor, tail_ent_emb: torch.FloatTensor*)

k_vs_all_score(*bpe_head_ent_emb, bpe_rel_ent_emb, E*)

forward_k_vs_all(x)

Completed. Given a head entity and a relation (h,r), we compute scores for all possible triples,i.e., `[score(h,r,x)|x in Entities] => [0.0,0.1,...,0.8]`, shape=> (1, **Entities**) Given a batch of head entities and relations => shape (size of batch,l Entities)

```
class dicee.models.ConvO (args: dict)
```

Bases: `dicee.models.base_model.BaseKGE`

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from `BaseKGELightning` and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- `forward_triples()` — score a batch of (h, r, t) triples.
- `forward_k_vs_all()` — score a (h, r) batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

`name = 'ConvO'`

`conv2d`

`fc_num_input`

`fc1`

`bn_conv2d`

`norm_fc1`

`feature_map_dropout`

static octonion_normalizer (*emb_rel_e0, emb_rel_e1, emb_rel_e2, emb_rel_e3, emb_rel_e4, emb_rel_e5, emb_rel_e6, emb_rel_e7*)

residual_convolution (*O_1, O_2*)

forward_triples (*x: torch.Tensor*) → `torch.Tensor`

Score a batch of (head, relation, tail) index triples.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor where each row is [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

`torch.FloatTensor`

forward_k_vs_all (*x: torch.Tensor*)

Given a head entity and a relation (h,r), we compute scores for all entities. [score(h,r,x)|x in Entities] => [0.0,0.1,...,0.8], shape=> (1, **Entities**) Given a batch of head entities and relations => shape (size of batch,|Entities|)

class `dicee.models.AConvO` (*args: dict*)

Bases: `dicee.models.base_model.BaseKGE`

Additive Convolutional Octonion Knowledge Graph Embeddings

`name = 'AConvO'`

`conv2d`

`fc_num_input`

`fc1`

`bn_conv2d`

`norm_fc1`

`feature_map_dropout`

`static octonion_normalizer(emb_rel_e0, emb_rel_e1, emb_rel_e2, emb_rel_e3, emb_rel_e4, emb_rel_e5, emb_rel_e6, emb_rel_e7)`

`residual_convolution(O_1, O_2)`

`forward_triples(x: torch.Tensor) → torch.Tensor`

Score a batch of (head, relation, tail) index triples.

Parameters

x (`torch.LongTensor`) – Shape (batch_size, 3) integer tensor where each row is [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

`torch.FloatTensor`

`forward_k_vs_all(x: torch.Tensor)`

Given a head entity and a relation (h,r), we compute scores for all entities. [score(h,r,x)|x in Entities] => [0.0,0.1,...,0.8], shape=> (1, **Entities**) Given a batch of head entities and relations => shape (size of batch,|Entities|)

`class dicee.models.Keci(args)`

Bases: `dicee.models.base_model.BaseKGE`

Keci: Knowledge Graph Embedding via Clifford Algebra.

Embeds entities and relations as multi-vectors in the Clifford algebra $Cl_{\{p,q\}}(R^d)$ and scores triples via the Clifford product. The algebra is parameterised by two non-negative integers p and q :

- $p = 0, q = 0$ — reduces to a standard bilinear (DistMult-like) model.
- $p = 0, q = 1$ — equivalent to ComplEx.
- Larger p and q capture higher-order geometric interactions.

The embedding dimension must satisfy $\text{embedding_dim} \% (p + q + 1) == 0$; the resulting quotient is stored as `self.r`.

Parameters

args (`dict`) – Configuration dictionary. Recognised keys (beyond those in `BaseKGE`): p (int, default 0) and q (int, default 0).

References

Demir et al., *Clifford Embeddings — A Generalized Approach for Embedding in Normed Algebras*, ECML 2023.

`name = 'Keci'`

`p`

`q`

`r`

`requires_grad_for_interactions = True`

compute_sigma_pp (*hp, rp*)

Compute $\sigma_{pp} = \sum_{i=1}^{p-1} \sum_{k=i+1}^p (h_{i,r_k} - h_{k,r_i}) e_i e_k$

σ_{pp} captures the interactions between along p bases For instance, let $p = e_1, e_2, e_3$, we compute interactions between $e_1 e_2, e_1 e_3$, and $e_2 e_3$ This can be implemented with a nested two for loops

results = [] for i in range(p - 1):

for k in range(i + 1, p):

 results.append(hp[:, :, i] * rp[:, :, k] - hp[:, :, k] * rp[:, :, i])

sigma_pp = torch.stack(results, dim=2) assert sigma_pp.shape == (b, r, int((p * (p - 1)) / 2))

Yet, this computation would be quite inefficient. Instead, we compute interactions along all p , e.g., $e_1 e_1, e_1 e_2, e_1 e_3$,

$e_2 e_1, e_2 e_2, e_2 e_3, e_3 e_1, e_3 e_2, e_3 e_3$

Then select the triangular matrix without diagonals: $e_1 e_2, e_1 e_3, e_2 e_3$.

compute_sigma_qq (*hq, rq*)

Compute $\sigma_{qq} = \sum_{j=1}^{p+q-1} \sum_{k=j+1}^{p+q} (h_{j,r_k} - h_{k,r_j}) e_j e_k$ σ_{qq} captures the interactions between along q bases For instance, let $q = e_1, e_2, e_3$, we compute interactions between $e_1 e_2, e_1 e_3$, and $e_2 e_3$ This can be implemented with a nested two for loops

results = [] for j in range(q - 1):

for k in range(j + 1, q):

 results.append(hq[:, :, j] * rq[:, :, k] - hq[:, :, k] * rq[:, :, j])

sigma_qq = torch.stack(results, dim=2) assert sigma_qq.shape == (b, r, int((q * (q - 1)) / 2))

Yet, this computation would be quite inefficient. Instead, we compute interactions along all p , e.g., $e_1 e_1, e_1 e_2, e_1 e_3$,

$e_2 e_1, e_2 e_2, e_2 e_3, e_3 e_1, e_3 e_2, e_3 e_3$

Then select the triangular matrix without diagonals: $e_1 e_2, e_1 e_3, e_2 e_3$.

compute_sigma_pq (*, *hp, hq, rp, rq*)

$\sum_{i=1}^p \sum_{j=p+1}^{p+q} (h_{i,r_j} - h_{j,r_i}) e_i e_j$

results = [] sigma_pq = torch.zeros(b, r, p, q) for i in range(p):

for j in range(q):

 sigma_pq[:, :, i, j] = hp[:, :, i] * rq[:, :, j] - hq[:, :, j] * rp[:, :, i]

print(sigma_pq.shape)

apply_coefficients (*hp, hq, rp, rq*)

Multiplying a base vector with its scalar coefficient

clifford_multiplication (*h0, hp, hq, r0, rp, rq*)

Compute our CL multiplication

$h = h_0 + \sum_{i=1}^p h_i e_i + \sum_{j=p+1}^{p+q} h_j e_j$ $r = r_0 + \sum_{i=1}^p r_i e_i + \sum_{j=p+1}^{p+q} r_j e_j$

$e_i^2 = +1$ for $i \leq p$ $e_j^2 = -1$ for $p < j \leq p+q$ $e_i e_j = -e_j e_i$ for i

e_j

$h r = \sigma_0 + \sigma_p + \sigma_q + \sigma_{pp} + \sigma_{qq} + \sigma_{pq}$ where

(1) $\sigma_0 = h_0 r_0 + \sum_{i=1}^p (h_0 r_i - h_i r_0) e_i - \sum_{j=p+1}^{p+q} (h_j r_j) e_j$

- (2) $\text{sigma_p} = \sum_{i=1}^p (h_{0\ r_i} + h_{i\ r_0}) e_i$
- (3) $\text{sigma_q} = \sum_{j=p+1}^{p+q} (h_{0\ r_j} + h_{j\ r_0}) e_j$
- (4) $\text{sigma_pp} = \sum_{i=1}^{p-1} \sum_{k=i+1}^p (h_{i\ r_k} - h_{k\ r_i}) e_i e_k$
- (5) $\text{sigma_qq} = \sum_{j=1}^{p+q-1} \sum_{k=j+1}^{p+q} (h_{j\ r_k} - h_{k\ r_j}) e_j e_k$
- (6) $\text{sigma_pq} = \sum_{i=1}^p \sum_{j=p+1}^{p+q} (h_{i\ r_j} - h_{j\ r_i}) e_i e_j$

construct_cl_multivector (*x*: torch.FloatTensor, *r*: int, *p*: int, *q*: int)
→ tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Split a flat embedding vector into the three Clifford components.

Given an embedding *x* of dimension $d = r + r \cdot p + r \cdot q$, returns the scalar part *a0*, the *p*-blade part *ap*, and the *q*-blade part *aq*.

Parameters

- **x** (torch.FloatTensor) – Shape (batch_size, d).
- **r** (int) – Scalar block size (embedding_dim // (p + q + 1)).
- **p** (int) – Number of positive-signature basis elements.
- **q** (int) – Number of negative-signature basis elements.

Returns

- **a0** (torch.FloatTensor) – Shape (batch_size, r) — scalar (grade-0) part.
- **ap** (torch.FloatTensor) – Shape (batch_size, r, p) — positive-blade part.
- **aq** (torch.FloatTensor) – Shape (batch_size, r, q) — negative-blade part.

forward_k_vs_with_explicit (*x*: torch.Tensor) → torch.FloatTensor

KvsAll scoring using an explicit loop over sigma_pp/qq/pq terms.

Functionally equivalent to `forward_k_vs_all()` but computes the higher-order interaction terms (sigma_pp, sigma_qq, sigma_pq) with explicit nested loops rather than einsum contractions. Kept for reference and correctness verification.

Parameters

x (torch.Tensor) – Shape (batch_size, 2) integer tensor [head_idx, relation_idx].

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

k_vs_all_score (*bpe_head_ent_emb*: torch.FloatTensor, *bpe_rel_ent_emb*: torch.FloatTensor, *E*: torch.FloatTensor) → torch.FloatTensor

Compute Clifford-product scores for a head/relation batch vs. all entities.

Decomposes the head-entity and relation embeddings into Clifford multi-vectors, performs the $Cl_{\{p,q\}}$ product, and inner-products the result against the entity embedding matrix *E*.

Parameters

- **bpe_head_ent_emb** (torch.FloatTensor) – Head-entity embeddings, shape (batch_size, embedding_dim).
- **bpe_rel_ent_emb** (torch.FloatTensor) – Relation embeddings, shape (batch_size, embedding_dim).

- **E** (*torch.FloatTensor*) – All entity embeddings, shape (num_entities, embedding_dim).

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_k_vs_all (*x: torch.Tensor*) → *torch.FloatTensor*

Kvsall training

- (1) Retrieve real-valued embedding vectors for heads and relations $\mathbb{R}^d$.
- (2) Construct head entity and relation embeddings according to $Cl_{\{p,q\}}(\mathbb{R}^d)$.
- (3) Perform Cl multiplication
- (4) Inner product of (3) and all entity embeddings

forward_k_vs_with_explicit and this functions are identical Parameter ——— *x*: *torch.LongTensor* with (n,2) shape :rtype: *torch.FloatTensor* with (n, **IEI**) shape

construct_batch_selected_cl_multivector (*x: torch.FloatTensor, r: int, p: int, q: int*)
→ tuple[*torch.FloatTensor, torch.FloatTensor, torch.FloatTensor*]

Split a batched, *k*-selected embedding tensor into Clifford components.

A variant of *construct_cl_multivector()* for tensors that have an extra *k* dimension (e.g. when scoring against *k* sampled targets).

Parameters

- **x** (*torch.FloatTensor*) – Shape (batch_size, *k*, *d*).
- **r** (*int*) – Scalar block size.
- **p** (*int*) – Number of positive-signature basis elements.
- **q** (*int*) – Number of negative-signature basis elements.

Returns

- **a0** (*torch.FloatTensor*) – Shape (batch_size, *k*, *r*).
- **ap** (*torch.FloatTensor*) – Shape (batch_size, *k*, *r*, *p*).
- **aq** (*torch.FloatTensor*) – Shape (batch_size, *k*, *r*, *q*).

forward_k_vs_sample (*x: torch.LongTensor, target_entity_idx: torch.LongTensor*) → *torch.FloatTensor*

Parameter

x: *torch.LongTensor* with (n,2) shape

target_entity_idx: *torch.LongTensor* with (n, *k*) shape *k* denotes the selected number of examples.

rtype

torch.FloatTensor with (n, *k*) shape

score (*h, r, t*)

forward_triples (*x: torch.Tensor*) → *torch.FloatTensor*

Parameter

x: torch.LongTensor with (n,3) shape

rtype

torch.FloatTensor with (n) shape

```
class dicee.models.CKeci(args)
```

Bases: *Keci*

Without learning dimension scaling

```
name = 'CKeci'
```

```
requires_grad_for_interactions = False
```

```
class dicee.models.DeCaL(args)
```

Bases: *dicee.models.base_model.BaseKGE*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from *BaseKGELightning* and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- *forward_triples()* — score a batch of (h, r, t) triples.
- *forward_k_vs_all()* — score a (h, r) batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

```
name = 'DeCaL'
```

```
p
```

```
q
```

```
r
```

```
re
```

```
forward_triples(x: torch.Tensor) → torch.FloatTensor
```

Parameter

x: torch.LongTensor with (n,) shape

rtype

torch.FloatTensor with (n) shape

```
cl_pqr(a: torch.tensor) → torch.tensor
```

Input: `tensor(batch_size, emb_dim)` → output: `tensor` with `1+p+q+r` components with size `(batch_size, emb_dim/(1+p+q+r))` each.

1) takes a tensor of size `(batch_size, emb_dim)`, split it into `1 + p + q + r` components, hence `1+p+q+r` must be a divisor of the `emb_dim`. 2) Return a list of the `1+p+q+r` components vectors, each are tensors of size `(batch_size, emb_dim/(1+p+q+r))`

compute_sigmas_single (*list_h_emb, list_r_emb, list_t_emb*)

here we compute all the sums with no others vectors interaction taken with the scalar product with t, that is,

$$s0 = h_0 r_0 t_0 s1 = \sum_{i=1}^p h_i r_i t_0 s2 = \sum_{j=p+1}^{p+q} h_j r_j t_0 s3 = \sum_{i=1}^q (h_0 r_i t_i + h_i r_0 t_i) s4 = \sum_{i=p+1}^{p+q} (h_0 r_i t_i + h_i r_0 t_i) s5 = \sum_{i=p+q+1}^{p+q+r} (h_0 r_i t_i + h_i r_0 t_i)$$

and return:

$$\sigma_0 t = \sigma_0 \cdot t_0 = s0 + s1 - s2s3, s4 \text{ and } s5$$

compute_sigmas_multivect (*list_h_emb, list_r_emb*)

Here we compute and return all the sums with vectors interaction for the same and different bases.

For same bases vectors interaction we have

$$\sigma_p p = \sum_{i=1}^{p-1} \sum_{i'=i+1}^p (h_i r_{i'} - h_{i'} r_i) (\text{model the interactions between } e_i \text{ and } e_{i'} \text{ for } 1 \leq i, i' \leq p) \sigma_q q = \sum_{j=p+1}^{p+q-1} \sum_{j'=j+1}^{p+q} (h_j r_{j'} - h_{j'} r_j)$$

For different base vector interactions, we have

$$\sigma_p q = \sum_{i=1}^p \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) (\text{interactions between } e_i \text{ and } e_j \text{ for } 1 \leq i \leq p \text{ and } p+1 \leq j \leq p+q) \sigma_p r = \sum_{i=1}^p \sum_{j=p+q+1}^{p+q+r} (h_i r_j - h_j r_i)$$

forward_k_vs_all (*x: torch.Tensor*) → torch.FloatTensor

Kvsall training

- (1) Retrieve real-valued embedding vectors for heads and relations
- (2) Construct head entity and relation embeddings according to $\text{Cl}_{\{p,q,r\}}(\mathbb{R}^d)$.
- (3) Perform Cl multiplication
- (4) Inner product of (3) and all entity embeddings

forward_k_vs_with_explicit and this functions are identical Parameter ——— x: torch.LongTensor with (n,) shape :rtype: torch.FloatTensor with (n, **IEI**) shape

apply_coefficients (*h0, hp, hq, hk, r0, rp, rq, rk*)

Multiplying a base vector with its scalar coefficient

construct_cl_multivector (*x: torch.FloatTensor, re: int, p: int, q: int, r: int*)
→ tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Construct a batch of multivectors $\text{Cl}_{\{p,q,r\}}(\mathbb{R}^d)$

Parameter

x: torch.FloatTensor with (n,d) shape

returns

- **a0** (*torch.FloatTensor*)
- **ap** (*torch.FloatTensor*)
- **aq** (*torch.FloatTensor*)
- **ar** (*torch.FloatTensor*)

compute_sigma_pp (*hp, rp*)

Compute .. math:

$$\sigma_{p,p} = \sum_{i=1}^{p-1} \sum_{i'=i+1}^p (x_{i'} y_i - x_i y_{i'})$$

σ_{pp} captures the interactions between along p bases For instance, let p e_1, e_2, e_3, we compute interactions between e_1 e_2, e_1 e_3, and e_2 e_3 This can be implemented with a nested two for loops

results = [] for i in range(p - 1):

for k in range(i + 1, p):

 results.append(hp[:, :, i] * rp[:, :, k] - hp[:, :, k] * rp[:, :, i])

sigma_pp = torch.stack(results, dim=2) assert sigma_pp.shape == (b, r, int((p * (p - 1)) / 2))

Yet, this computation would be quite inefficient. Instead, we compute interactions along all p , e.g., e1e1, e1e2, e1e3,

e2e1, e2e2, e2e3, e3e1, e3e2, e3e3

Then select the triangular matrix without diagonals: e1e2, e1e3, e2e3.

compute_sigma_qq (*hq, rq*)

Compute

$$\sigma_{q,q}^* = \sum_{j=p+1}^{p+q-1} \sum_{j'=j+1}^{p+q} (x_j y_{j'} - x_{j'} y_j) Eq.16$$

$\sigma_{q,q}$ captures the interactions between along q bases For instance, let q e_1, e_2, e_3, we compute interactions between e_1 e_2, e_1 e_3, and e_2 e_3 This can be implemented with a nested two for loops

results = [] for j in range(q - 1):

for k in range(j + 1, q):

 results.append(hq[:, :, j] * rq[:, :, k] - hq[:, :, k] * rq[:, :, j])

sigma_qq = torch.stack(results, dim=2) assert sigma_qq.shape == (b, r, int((q * (q - 1)) / 2))

Yet, this computation would be quite inefficient. Instead, we compute interactions along all p , e.g., e1e1, e1e2, e1e3,

e2e1, e2e2, e2e3, e3e1, e3e2, e3e3

Then select the triangular matrix without diagonals: e1e2, e1e3, e2e3.

compute_sigma_rr (*hk, rk*)

$$\sigma_{r,r}^* = \sum_{k=p+q+1}^{p+q+r-1} \sum_{k'=k+1}^p (x_k y_{k'} - x_{k'} y_k)$$

compute_sigma_pq (*, *hp, hq, rp, rq*)

Compute

$$\sum_{i=1}^p \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j$$

results = [] sigma_pq = torch.zeros(b, r, p, q) for i in range(p):

for j in range(q):

 sigma_pq[:, :, i, j] = hp[:, :, i] * rq[:, :, j] - hq[:, :, j] * rp[:, :, i]

```

print(sigma_pq.shape)
compute_sigma_pr(* , hp, hk, rp, rk)
    Compute

```

$$\sum_{i=1}^p \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j$$

```

results = []
sigma_pq = torch.zeros(b, r, p, q)
for i in range(p):
    for j in range(q):
        sigma_pq[:, :, i, j] = hp[:, :, i] * rq[:, :, j] - hq[:, :, j] * rp[:, :, i]
print(sigma_pq.shape)
compute_sigma_qr(* , hq, hk, rq, rk)

```

$$\sum_{i=1}^p \sum_{j=p+1}^{p+q} (h_i r_j - h_j r_i) e_i e_j$$

```

results = []
sigma_pq = torch.zeros(b, r, p, q)
for i in range(p):
    for j in range(q):
        sigma_pq[:, :, i, j] = hp[:, :, i] * rq[:, :, j] - hq[:, :, j] * rp[:, :, i]
print(sigma_pq.shape)

```

```

class dicee.models.KeciTransformer(args)

```

Bases: [Keci](#)

Keci with Transformer architecture.

Concatenates h0, hp, hq, r0, rp, rq into a single embedding vector and processes through transformer.

name = 'KeciTransformer'

use_clifford_mul

seq_len = 1

transformer

lm_head

forward_k_vs_all (*x*: *torch.Tensor*) → torch.FloatTensor

Kvsall training

Parameter

x: torch.LongTensor with (n,2) shape

rtype

torch.FloatTensor with (n, **IEI**) shape

```

class dicee.models.BaseKGE(args: dict)

```

Bases: [BaseKGELightning](#)

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from [BaseKGELightning](#) and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- `forward_triples()` — score a batch of (h, r, t) triples.
- `forward_k_vs_all()` — score a (h, r) batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

```
args
embedding_dim = None
num_entities: int | None = None
num_relations: int | None = None
num_tokens = None
learning_rate = None
apply_unit_norm = None
input_dropout_rate = None
hidden_dropout_rate = None
optimizer_name = None
feature_map_dropout_rate = None
kernel_size = None
num_of_output_channels = None
weight_decay = None
selected_optimizer = None
normalizer_class = None
normalize_head_entity_embeddings
normalize_relation_embeddings
normalize_tail_entity_embeddings
hidden_normalizer
param_init
input_dp_ent_real
input_dp_rel_real
hidden_dropout
loss_history = []
byte_pair_encoding
```

max_length_subword_tokens

block_size

defer_large_embeddings

init_entity_embeddings (*embedding_dim*: int | None = None) → None

Create (or re-create) the entity embedding table.

This is the single place that honours *defer_large_embeddings*: when entity rows are sharded across FSDP ranks the table must stay None until the trainer allocates the sharded adapter. Subclasses that need an entity table of a non-default width must call this method instead of assigning `self.entity_embeddings` directly, so the deferral is never silently undone.

Parameters

embedding_dim (*Optional*[int]) – Width of the table. Defaults to *embedding_dim*.

init_relation_embeddings (*embedding_dim*: int | None = None) → None

Create (or re-create) the relation embedding table.

Relation tables are never deferred - they are small enough to be replicated on every rank.

Parameters

embedding_dim (*Optional*[int]) – Width of the table. Defaults to *embedding_dim*.

forward_byte_pair_encoded_k_vs_all (*x*: torch.LongTensor) → torch.FloatTensor

KvsAll scoring for BPE-encoded head entities and relations.

Retrieves subword-unit embeddings for the head entity and relation, reduces them to fixed-size vectors via a linear projection, then scores against all BPE entity embeddings.

Parameters

x (torch.LongTensor) – Shape (batch_size, 2, T) BPE token indices where dim 1 indexes [head, relation] and T is max_length_subword_tokens.

Returns

Shape (batch_size, num_bpe_entities) score matrix.

Return type

torch.FloatTensor

forward_byte_pair_encoded_triple (*x*: Tuple[torch.LongTensor, torch.LongTensor])
→ torch.FloatTensor

NegSample scoring for BPE-encoded (head, relation, tail) triples.

Retrieves subword-unit embeddings for all three elements and reduces them to fixed-size vectors via a linear projection before computing the triple score.

Parameters

x (torch.LongTensor) – Shape (batch_size, 3, T) BPE token indices.

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

init_params_with_sanity_checking() → None

Populate model hyper-parameters from `self.args` with safe defaults.

Reads embedding dimension, learning rate, dropout rates, normalisation strategy, optimizer name, and parameter initialisation scheme from the `args` dict. Falls back to sensible defaults for any missing key so that minimal `args` dicts (e.g. for unit tests) are still valid.

forward (*x*: torch.LongTensor | Tuple[torch.LongTensor, torch.LongTensor],
y_idx: torch.LongTensor | None = None) → torch.Tensor

Route the forward pass to the appropriate scoring method.

Inspects the shape and type of *x* to decide which low-level scorer to call:

- Tuple (*x*, *y_idx*) → *forward_k_vs_sample()*
- (batch, 3) tensor → *forward_triples()*
- (batch, 2) tensor → *forward_k_vs_all()*
- BPE triple tensor → *forward_byte_pair_encoded_triple()*
- BPE pair tensor → *forward_byte_pair_encoded_k_vs_all()*

Parameters

- **x** (torch.LongTensor or Tuple[torch.LongTensor, torch.LongTensor]) – Either a plain index tensor or a (triple_idx, target_idx) tuple for sample-based labelling.
- **y_idx** (torch.LongTensor, optional) – Target entity indices used by *forward_k_vs_sample()*. Ignored when *x* is a plain tensor.

Returns

Score tensor whose shape depends on the selected scorer.

Return type

torch.FloatTensor

forward_triples (*x*: torch.LongTensor) → torch.Tensor

Score a batch of (head, relation, tail) index triples.

Parameters

- **x** (torch.LongTensor) – Shape (batch_size, 3) integer tensor where each row is [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

forward_k_vs_all (*args, **kwargs)

Score a (head, relation) batch against every entity.

Sub-classes must override this method. The default implementation raises ValueError to make missing overrides obvious at runtime.

Returns

Shape (batch_size, num_entities) score matrix.

Return type

torch.FloatTensor

forward_k_vs_sample (*args, **kwargs)

Score a (head, relation) batch against a sampled subset of entities.

Used by KvsSample and 1vsSample datasets. Sub-classes that support sample-based labelling must override this method.

Returns

Shape (batch_size, k) score matrix where k is the number of sampled target entities.

Return type

torch.FloatTensor

get_triple_representation (idx_hrt)

→ Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for a triple index batch.

Parameters

idx_hrt (torch.LongTensor) – Shape (batch_size, 3) integer tensor with columns [head_idx, relation_idx, tail_idx].

Returns

head_ent_emb, rel_ent_emb, tail_ent_emb – Each has shape (batch_size, embedding_dim) after applying the configured dropout and normalisation.

Return type

torch.FloatTensor

get_head_relation_representation (indexed_triple)

→ Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve and normalise embedding vectors for head entities and relations.

Parameters

indexed_triple (torch.LongTensor) – Shape (batch_size, 2) integer tensor with columns [head_idx, relation_idx].

Returns

head_ent_emb, rel_ent_emb – Each has shape (batch_size, embedding_dim) after applying the configured dropout and normalisation.

Return type

torch.FloatTensor

get_sentence_representation (x: torch.LongTensor)

→ Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve BPE subword-unit embeddings for a batch of triples.

Parameters

x (torch.LongTensor) – Shape (batch_size, 3, T) where T is max_length_subword_tokens.

Returns

head_ent_emb, rel_emb, tail_emb – Each has shape (batch_size, T, embedding_dim).

Return type

torch.FloatTensor

get_bpe_head_and_relation_representation (x: torch.LongTensor)

→ Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve unit-normalised BPE embeddings for head entities and relations.

Each entity/relation is represented as a sequence of T subword tokens. Their token embeddings are L2-normalised across the sequence dimension so that the resulting matrix has unit Frobenius norm.

Parameters

x (torch.LongTensor) – Shape (batch_size, 2, T) where dim 1 indexes [head, relation] and T is max_length_subword_tokens.

Returns

head_ent_emb, rel_emb – Each has shape (batch_size, T, embedding_dim), L2-normalised over the (T, D) dimensions.

Return type

torch.FloatTensor

get_embeddings() → Tuple[numpy.ndarray, numpy.ndarray]

Return the entity and relation embedding matrices as numpy arrays.

Returns

- **entity_embeddings** (numpy.ndarray) – Shape (num_entities, embedding_dim).
- **relation_embeddings** (numpy.ndarray) – Shape (num_relations, embedding_dim).

dicee.models.logger

class dicee.models.PykeenKGE (args: dict)

Bases: *dicee.models.base_model.BaseKGE*

A class for using knowledge graph embedding models implemented in Pykeen

Notes: Pykeen_DistMult: C Pykeen_ComplEx: Pykeen_QuatE: Pykeen_MuRE: Pykeen_CP: Pykeen_HolE: Pykeen_HolE: Pykeen_HolE: Pykeen_TransD: Pykeen_TransE: Pykeen_TransF: Pykeen_TransH: Pykeen_TransR:

model_kwargs

name

model

loss_history = []

args

entity_embeddings = None

relation_embeddings = None

forward_k_vs_all (x: torch.LongTensor)

=> Explicit version by this we can apply bn and dropout

(1) Retrieve embeddings of heads and relations + apply Dropout & Normalization if given. h, r = self.get_head_relation_representation(x) # (2) Reshape (1). if self.last_dim > 0:

h = h.reshape(len(x), self.embedding_dim, self.last_dim) r = r.reshape(len(x), self.embedding_dim, self.last_dim)

(3) Reshape all entities. if self.last_dim > 0:

t = self.entity_embeddings.weight.reshape(self.num_entities, self.embedding_dim, self.last_dim)

else:

t = self.entity_embeddings.weight

(4) Call the score_t from interactions to generate triple scores. return self.interaction.score_t(h=h, r=r, all_entities=t, slice_size=1)


```

forward_triples (x: torch.LongTensor) → torch.FloatTensor
    # => Explicit version by this we can apply bn and dropout

    # (1) Retrieve embeddings of heads, relations and tails and apply Dropout & Normalization if given. h, r, t =
    self.get_triple_representation(x) # (2) Reshape (1). if self.last_dim > 0:
        h = h.reshape(len(x), self.embedding_dim, self.last_dim) r = r.reshape(len(x), self.embedding_dim,
        self.last_dim) t = t.reshape(len(x), self.embedding_dim, self.last_dim)

    # (3) Compute the triple score return self.interaction.score(h=h, r=r, t=t, slice_size=None, slice_dim=0)

abstractmethod forward_k_vs_sample (x: torch.LongTensor, target_entity_idx)
    Score a (head, relation) batch against a sampled subset of entities.

    Used by KvsSample and 1vsSample datasets. Sub-classes that support sample-based labelling must over-
    ride this method.

```

Returns

Shape (batch_size, k) score matrix where *k* is the number of sampled target entities.

Return type

torch.FloatTensor

```

class dicee.models.BaseKGE (args: dict)

```

Bases: *BaseKGELightning*

Base class for all Knowledge Graph Embedding models.

Inherits the Lightning training loop from *BaseKGELightning* and adds the embedding tables, normalisation / dropout layers, and the routing logic that dispatches `forward()` calls to the appropriate scoring method.

Sub-classes must implement at minimum:

- `forward_triples()` — score a batch of (h, r, t) triples.
- `forward_k_vs_all()` — score a (h, r) batch against every entity.

Parameters

args (*dict*) – Flat configuration dictionary produced by `vars(argparse.Namespace)`. Required keys: `embedding_dim`, `num_entities`, `num_relations`, `learning_rate` (or `lr`), `optim`, `scoring_technique`.

args

embedding_dim = None

num_entities: int | None = None

num_relations: int | None = None

num_tokens = None

learning_rate = None

apply_unit_norm = None

input_dropout_rate = None

hidden_dropout_rate = None

optimizer_name = None

```

feature_map_dropout_rate = None

kernel_size = None

num_of_output_channels = None

weight_decay = None

selected_optimizer = None

normalizer_class = None

normalize_head_entity_embeddings

normalize_relation_embeddings

normalize_tail_entity_embeddings

hidden_normalizer

param_init

input_dp_ent_real

input_dp_rel_real

hidden_dropout

loss_history = []

byte_pair_encoding

max_length_subword_tokens

block_size

defer_large_embeddings

```

init_entity_embeddings (*embedding_dim: int | None = None*) → None

Create (or re-create) the entity embedding table.

This is the single place that honours *defer_large_embeddings*: when entity rows are sharded across FSDP ranks the table must stay *None* until the trainer allocates the sharded adapter. Subclasses that need an entity table of a non-default width must call this method instead of assigning *self.entity_embeddings* directly, so the deferral is never silently undone.

Parameters

embedding_dim (*Optional[int]*) – Width of the table. Defaults to *embedding_dim*.

init_relation_embeddings (*embedding_dim: int | None = None*) → None

Create (or re-create) the relation embedding table.

Relation tables are never deferred - they are small enough to be replicated on every rank.

Parameters

embedding_dim (*Optional[int]*) – Width of the table. Defaults to *embedding_dim*.

forward_byte_pair_encoded_k_vs_all (*x*: *torch.LongTensor*) → *torch.FloatTensor*

KvsAll scoring for BPE-encoded head entities and relations.

Retrieves subword-unit embeddings for the head entity and relation, reduces them to fixed-size vectors via a linear projection, then scores against all BPE entity embeddings.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 2, T) BPE token indices where dim 1 indexes [head, relation] and T is max_length_subword_tokens.

Returns

Shape (batch_size, num_bpe_entities) score matrix.

Return type

torch.FloatTensor

forward_byte_pair_encoded_triple (*x*: *Tuple[torch.LongTensor, torch.LongTensor]*)

→ *torch.FloatTensor*

NegSample scoring for BPE-encoded (head, relation, tail) triples.

Retrieves subword-unit embeddings for all three elements and reduces them to fixed-size vectors via a linear projection before computing the triple score.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3, T) BPE token indices.

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

init_params_with_sanity_checking() → None

Populate model hyper-parameters from *self.args* with safe defaults.

Reads embedding dimension, learning rate, dropout rates, normalisation strategy, optimizer name, and parameter initialisation scheme from the *args* dict. Falls back to sensible defaults for any missing key so that minimal *args* dicts (e.g. for unit tests) are still valid.

forward (*x*: *torch.LongTensor* | *Tuple[torch.LongTensor, torch.LongTensor]*,

y_idx: *torch.LongTensor* | None = None) → *torch.Tensor*

Route the forward pass to the appropriate scoring method.

Inspects the shape and type of *x* to decide which low-level scorer to call:

- *Tuple* (*x*, *y_idx*) → *forward_k_vs_sample()*
- (batch, 3) tensor → *forward_triples()*
- (batch, 2) tensor → *forward_k_vs_all()*
- BPE triple tensor → *forward_byte_pair_encoded_triple()*
- BPE pair tensor → *forward_byte_pair_encoded_k_vs_all()*

Parameters

• **x** (*torch.LongTensor* or *Tuple[torch.LongTensor, torch.LongTensor]*) – Either a plain index tensor or a (*triple_idx*, *target_idx*) tuple for sample-based labelling.

• **y_idx** (*torch.LongTensor*, optional) – Target entity indices used by *forward_k_vs_sample()*. Ignored when *x* is a plain tensor.

Returns

Score tensor whose shape depends on the selected scorer.

Return type

`torch.FloatTensor`

forward_triples (*x*: `torch.LongTensor`) → `torch.Tensor`

Score a batch of (head, relation, tail) index triples.

Parameters

x (`torch.LongTensor`) – Shape (batch_size, 3) integer tensor where each row is [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

`torch.FloatTensor`

forward_k_vs_all (*args, **kwargs)

Score a (head, relation) batch against every entity.

Sub-classes must override this method. The default implementation raises `ValueError` to make missing overrides obvious at runtime.

Returns

Shape (batch_size, num_entities) score matrix.

Return type

`torch.FloatTensor`

forward_k_vs_sample (*args, **kwargs)

Score a (head, relation) batch against a sampled subset of entities.

Used by `KvsSample` and `1vsSample` datasets. Sub-classes that support sample-based labelling must override this method.

Returns

Shape (batch_size, k) score matrix where *k* is the number of sampled target entities.

Return type

`torch.FloatTensor`

get_triple_representation (idx_hrt)

→ Tuple[`torch.FloatTensor`, `torch.FloatTensor`, `torch.FloatTensor`]

Retrieve and normalise embedding vectors for a triple index batch.

Parameters

idx_hrt (`torch.LongTensor`) – Shape (batch_size, 3) integer tensor with columns [head_idx, relation_idx, tail_idx].

Returns

head_ent_emb, rel_ent_emb, tail_ent_emb – Each has shape (batch_size, embedding_dim) after applying the configured dropout and normalisation.

Return type

`torch.FloatTensor`

get_head_relation_representation (indexed_triple)

→ Tuple[`torch.FloatTensor`, `torch.FloatTensor`]

Retrieve and normalise embedding vectors for head entities and relations.

Parameters

indexed_triple (*torch.LongTensor*) – Shape (batch_size, 2) integer tensor with columns [head_idx, relation_idx].

Returns

head_ent_emb, rel_ent_emb – Each has shape (batch_size, embedding_dim) after applying the configured dropout and normalisation.

Return type

torch.FloatTensor

get_sentence_representation (*x: torch.LongTensor*)

→ Tuple[torch.FloatTensor, torch.FloatTensor, torch.FloatTensor]

Retrieve BPE subword-unit embeddings for a batch of triples.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3, T) where T is max_length_subword_tokens.

Returns

head_ent_emb, rel_emb, tail_emb – Each has shape (batch_size, T, embedding_dim).

Return type

torch.FloatTensor

get_bpe_head_and_relation_representation (*x: torch.LongTensor*)

→ Tuple[torch.FloatTensor, torch.FloatTensor]

Retrieve unit-normalised BPE embeddings for head entities and relations.

Each entity/relation is represented as a sequence of T subword tokens. Their token embeddings are L2-normalised across the sequence dimension so that the resulting matrix has unit Frobenius norm.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 2, T) where dim 1 indexes [head, relation] and T is max_length_subword_tokens.

Returns

head_ent_emb, rel_emb – Each has shape (batch_size, T, embedding_dim), L2-normalised over the (T, D) dimensions.

Return type

torch.FloatTensor

get_embeddings () → Tuple[numpy.ndarray, numpy.ndarray]

Return the entity and relation embedding matrices as numpy arrays.

Returns

- **entity_embeddings** (*numpy.ndarray*) – Shape (num_entities, embedding_dim).
- **relation_embeddings** (*numpy.ndarray*) – Shape (num_relations, embedding_dim).

dicee.models.logger

class dicee.models.FMult (*args*)

Bases: *dicee.models.base_model.BaseKGE*

Learning Knowledge Neural Graphs

name = 'FMult'

```

entity_embeddings

relation_embeddings

k

num_sample = 50

gamma

roots

weights

compute_func(weights: torch.FloatTensor, x) → torch.FloatTensor

chain_func(weights, x: torch.FloatTensor)

forward_triples(idx_triple: torch.Tensor) → torch.Tensor
    Score a batch of (head, relation, tail) index triples.

    Parameters
        x (torch.LongTensor) – Shape (batch_size, 3) integer tensor where each row is
        [head_idx, relation_idx, tail_idx].

    Returns
        Shape (batch_size,) triple scores.

    Return type
        torch.FloatTensor

```

class dicee.models.GFMult(args)

Bases: *dicee.models.base_model.BaseKGE*

Learning Knowledge Neural Graphs

```

name = 'GFMult'

entity_embeddings

relation_embeddings

k

num_sample = 250

roots

weights

compute_func(weights: torch.FloatTensor, x) → torch.FloatTensor

chain_func(weights, x: torch.FloatTensor)

forward_triples(idx_triple: torch.Tensor) → torch.Tensor
    Score a batch of (head, relation, tail) index triples.

    Parameters
        x (torch.LongTensor) – Shape (batch_size, 3) integer tensor where each row is
        [head_idx, relation_idx, tail_idx].

```

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

```
class dicee.models.FMult2(args)
```

Bases: *[dicee.models.base_model.BaseKGE](#)*

Learning Knowledge Neural Graphs

name = 'FMult2'

n_layers = 3

k

n = 50

score_func = 'compositional'

discrete_points

entity_embeddings

relation_embeddings

build_func(Vec)

build_chain_funcs(list_Vec)

compute_func(W, b, x) → torch.FloatTensor

function(list_W, list_b)

trapezoid(list_W, list_b)

forward_triples(idx_triple: torch.Tensor) → torch.Tensor

Score a batch of (head, relation, tail) index triples.

Parameters

x (*torch.LongTensor*) – Shape (batch_size, 3) integer tensor where each row is [head_idx, relation_idx, tail_idx].

Returns

Shape (batch_size,) triple scores.

Return type

torch.FloatTensor

```
class dicee.models.LFMult1(args)
```

Bases: *[dicee.models.base_model.BaseKGE](#)*

Embedding with trigonometric functions. We represent all entities and relations in the complex number space as: $f(x) = \sum_{k=0}^{k=d-1} w_k e^{kix}$. and use the three differents scoring function as in the paper to evaluate the score

name = 'LFMult1'

entity_embeddings

```

relation_embeddings

forward_triples (idx_triple)
    Score a batch of (head, relation, tail) index triples.

    Parameters
        x (torch.LongTensor) – Shape (batch_size, 3) integer tensor where each row is
        [head_idx, relation_idx, tail_idx].

    Returns
        Shape (batch_size,) triple scores.

    Return type
        torch.FloatTensor

tri_score (h, r, t)

vtp_score (h, r, t)

class dicee.models.LFMult (args)
    Bases: dicee.models.base_model.BaseKGE

    Embedding with polynomial functions. We represent all entities and relations in the polynomial space as:  $f(x) = \sum_{i=0}^{d-1} a_k x^{i \bmod d}$  and use the three differents scoring function as in the paper to evaluate the score.
    We also consider combining with Neural Networks.

    name = 'LFMult'

    entity_embeddings

    relation_embeddings

    degree

    m

    x_values

    forward_triples (idx_triple)
        Score a batch of (head, relation, tail) index triples.

        Parameters
            x (torch.LongTensor) – Shape (batch_size, 3) integer tensor where each row is
            [head_idx, relation_idx, tail_idx].

        Returns
            Shape (batch_size,) triple scores.

        Return type
            torch.FloatTensor

    construct_multi_coeff (x)

    poly_NN (x, coefh, coefr, coeft)
        Constructing a 2 layers NN to represent the embeddings.  $h = \text{sigma}(wh^T x + bh)$ ,  $r = \text{sigma}(wr^T x + br)$ ,
         $t = \text{sigma}(wt^T x + bt)$ 

    linear (x, w, b)

```


scalar_batch_NN (*a, b, c*)
 element wise multiplication between a,b and c: Inputs : a, b, c ==> torch.tensor of size batch_size x m x d
 Output : a tensor of size batch_size x d

tri_score (*coeff_h, coeff_r, coeff_t*)
 this part implement the trilinear scoring techniques:

$$\text{score}(h,r,t) = \int_0^1 h(x)r(x)t(x) dx = \sum_{i,j,k=0}^{d-1} \frac{a_i b_j c_k}{1+(i+j+k)d}$$

1. generate the range for i,j and k from [0 d-1]
2. perform $\frac{a_i b_j c_k}{1+(i+j+k)d}$ in parallel for every batch
3. take the sum over each batch

vtp_score (*h, r, t*)
 this part implement the vector triple product scoring techniques:

$$\text{score}(h,r,t) = \int_0^1 h(x)r(x)t(x) dx = \sum_{i,j,k=0}^{d-1} \frac{a_i c_j b_k - b_i c_j a_k}{(1+(i+j)d)(1+k)}$$

1. generate the range for i,j and k from [0 d-1]
2. Compute the first and second terms of the sum
3. Multiply with then denominator and take the sum
4. take the sum over each batch

comp_func (*h, r, t*)
 this part implement the function composition scoring techniques: i.e. score = <hor, t>

polynomial (*coeff, x, degree*)
 This function takes a matrix tensor of coefficients (coeff), a tensor vector of points x and range of integer [0,1,...d] and return a vector tensor (coeff[0][0] + coeff[0][1]x +...+ coeff[0][d]x^d,

$$\text{coeff}[1][0] + \text{coeff}[1][1]x + \dots + \text{coeff}[1][d]x^d)$$

pop (*coeff, x, degree*)
 This function allow us to evaluate the composition of two polynomes without for loops :) it takes a matrix tensor of coefficients (coeff), a matrix tensor of points x and range of integer [0,1,...d]
 and return a tensor (coeff[0][0] + coeff[0][1]x +...+ coeff[0][d]x^d,

$$\text{coeff}[1][0] + \text{coeff}[1][1]x + \dots + \text{coeff}[1][d]x^d)$$

class dicee.models.DualE (*args*)
 Bases: *dicee.models.base_model.BaseKGE*
 Dual Quaternion Knowledge Graph Embeddings (<https://ojs.aaai.org/index.php/AAAI/article/download/16850/16657>)
name = 'DualE'
entity_embeddings
relation_embeddings
num_ent = None
kvsall_score (*e_1_h, e_2_h, e_3_h, e_4_h, e_5_h, e_6_h, e_7_h, e_8_h, e_1_t, e_2_t, e_3_t, e_4_t, e_5_t, e_6_t, e_7_t, e_8_t, r_1, r_2, r_3, r_4, r_5, r_6, r_7, r_8*) → torch.tensor
 KvsAll scoring function

Input

x: torch.LongTensor with (n,) shape

Output

torch.FloatTensor with (n) shape

forward_triples (*idx_triple: torch.tensor*) → torch.tensor

Negative Sampling forward pass:

Input

x: torch.LongTensor with (n,) shape

Output

torch.FloatTensor with (n) shape

forward_k_vs_all (*x*)

KvsAll forward pass

Input

x: torch.LongTensor with (n,) shape

Output

torch.FloatTensor with (n) shape

T (*x: torch.tensor*) → torch.tensor

Transpose function

Input: Tensor with shape (nxm) Output: Tensor with shape (mxn)

```
class dicee.models.ULTRA (args)
```

Bases: *[dicee.models.graph_model.GraphKGE](#)*

Pure PyTorch ULTRA with the official state dictionary and DICE triple order.

```
name = 'ULTRA'
```

```
config_prefix = 'ultra'
```

```
graph_filename = 'ultra_graph.pt'
```

```
checkpoint_hint = 'official ULTRA requires ultra_dim=64, ultra_num_layers=6; all  
keys and shapes must match'
```

```
deterministic_inference = True
```

```
dim
```

```
num_layers
```

```
relation_model
```

```
entity_model
```

```

    relation_cache_mb

    projection_cache_mb

    clear_inference_cache()
        Drop derived representations when graph, weights or device change.
class dicee.models.TRIX(args)
    Bases: TRIXPath
    Entity predictor compatible with official entity_prediction.pth.
    DICE order is (head, relation, tail). Head corruption retains the original relation seed, then uses the inverse relation
    for both entity reasoners. Explicit reciprocal DICE queries use that same convention.

    name = 'TRIX'

    deterministic_inference = True

    relation_model

    entity_model_1

    entity_model_2

    relation_cache_mb

    clear_inference_cache()
        Drop derived representations when graph, weights or device change.
class dicee.models.TRIXRelation(args)
    Bases: TRIXPath, dicee.models.graph_model.RelationGraphKGE
    Relation predictor compatible with official relation_prediction.pth.
    Pairs are (head, tail); triples and grouped relation corruptions use DICE's (head, relation, tail) order. One reasoning
    pass scores every relation for a given pair, including external inverse IDs when present in the vocabulary.

    name = 'TRIXRelation'

    relation_model

    entity_model

    mlp

class dicee.models.Flock(args)
    Bases: FlockBase
    Entity-prediction Flock with official flock_entity.pth parameters.

    name = 'Flock'

class dicee.models.FlockRelation(args)
    Bases: FlockBase, dicee.models.graph_model.RelationGraphKGE
    Relation-prediction Flock with official flock_relation.pth parameters.

    name = 'FlockRelation'

    relation_prediction = True

```

dicee.query_generator

Attributes

logger

Classes

QueryGenerator

Module Contents

`dicee.query_generator.logger`

class `dicee.query_generator.QueryGenerator` (*train_path: str, val_path: str, test_path: str, ent2id: Dict = None, rel2id: Dict = None, seed: int = 1, gen_valid: bool = False, gen_test: bool = True*)

train_path

val_path

test_path

gen_valid = **False**

gen_test = **True**

seed = **1**

max_ans_num = **1000000.0**

mode

ent2id = **None**

rel2id: Dict = **None**

ent_in: Dict

ent_out: Dict

query_name_to_struct

list2tuple (*list_data*)

tuple2list (*x: List | Tuple*) $\rightarrow$ *List | Tuple*

Convert a nested tuple to a nested list.

set_global_seed (*seed: int*)

Set seed

construct_graph (*paths: List[str]*) $\rightarrow$ *Tuple[Dict, Dict]*

Construct graph from triples Returns dicts with incoming and outgoing edges

```

fill_query (query_structure: List[str | List], ent_in: Dict, ent_out: Dict, answer: int) → bool
    Private method for fill_query logic.

achieve_answer (query: List[str | List], ent_in: Dict, ent_out: Dict) → set
    Private method for achieve_answer logic. @TODO: Document the code

write_links (ent_out, small_ent_out)

ground_queries (query_structure: List[str | List], ent_in: Dict, ent_out: Dict, small_ent_in: Dict,
    small_ent_out: Dict, gen_num: int, query_name: str)
    Generating queries and achieving answers

unmap (query_type, queries, tp_answers, fp_answers, fn_answers)

unmap_query (query_structure, query, id2ent, id2rel)

generate_queries (query_struct: List, gen_num: int, query_type: str)
    Passing incoming and outgoing edges to ground queries depending on mode [train valid or text] and getting
    queries and answers in return @ TODO: create a class for each single query struct

save_queries (query_type: str, gen_num: int, save_path: str)

abstractmethod load_queries (path)

get_queries (query_type: str, gen_num: int)

static save_queries_and_answers (path: str, data: List[Tuple[str, Tuple[collections.defaultdict]]])
    → None
    Save Queries into Disk

static load_queries_and_answers (path: str) → List[Tuple[str, Tuple[collections.defaultdict]]]
    Load Queries from Disk to Memory

```

dicee.read_preprocess_save_load_kg

Submodules

dicee.read_preprocess_save_load_kg.preprocess

Attributes

logger

Classes

PreprocessKG

Preprocess the data in memory

Module Contents

`dicee.read_preprocess_save_load_kg.preprocess.logger`

class `dicee.read_preprocess_save_load_kg.preprocess.PreprocessKG (kg)`
 Preprocess the data in memory

kg

start () → None

Preprocess train, valid and test datasets stored in knowledge graph instance

Parameter

rtype

None

preprocess_with_byte_pair_encoding ()

preprocess_with_byte_pair_encoding_with_padding () → None

Preprocess with byte pair encoding and add padding

preprocess_with_pandas () → None

Preprocess with pandas: add reciprocal triples, construct vocabulary, and index datasets

preprocess_with_polars () → None

Preprocess with polars: add reciprocal triples and create indexed datasets

sequential_vocabulary_construction () → None

(1) Read input data into memory

(2) Remove triples with a condition

(3) **Serialize vocabularies in a pandas dataframe where**

=> the index is integer and => a single column is string (e.g. URI)

dicee.read_preprocess_save_load_kg.read_from_disk

Attributes

logger

Classes

ReadFromDisk

Read the data from disk into memory

Module Contents

dicee.read_preprocess_save_load_kg.read_from_disk.**logger**

class dicee.read_preprocess_save_load_kg.read_from_disk.**ReadFromDisk** (kg)

Read the data from disk into memory

kg

start () → None

Read a knowledge graph from disk into memory

Data will be available at the train_set, test_set, valid_set attributes.

Parameter

None

rtype

None

add_noisy_triples_into_training()

dicee.read_preprocess_save_load_kg.save_load_disk

Attributes

<i>logger</i>

Classes

<i>LoadSaveToDisk</i>

Module Contents

dicee.read_preprocess_save_load_kg.save_load_disk.**logger**

class dicee.read_preprocess_save_load_kg.save_load_disk.**LoadSaveToDisk** (*kg*)

kg

save()

load()

dicee.read_preprocess_save_load_kg.util

Attributes

<i>logger</i>

Functions

<i>polars_dataframe_indexer</i> (→ polars.DataFrame)	Replaces 'subject', 'relation', and 'object' columns in the input Polars DataFrame with their corresponding index values
<i>pandas_dataframe_indexer</i> (→ pandas.DataFrame)	Replaces 'subject', 'relation', and 'object' columns in the input Pandas DataFrame with their corresponding index values
<i>apply_reciprocal_or_noise</i> (add_reciprocal, eval_model)	Add reciprocal triples if conditions are met
<i>timeit</i> (func)	
<i>read_with_polars</i> (→ polars.DataFrame)	Load and Preprocess via Polars

continues on next page

Table 80 – continued from previous page

<code>read_with_pandas(data_path[, read_only_few, ...])</code>	Load and Preprocess via Pandas
<code>read_from_disk(→ Tuple[polars.DataFrame, pandas.DataFrame])</code>	
<code>count_triples(→ int)</code>	Returns the total number of triples in the triple store.
<code>fetch_worker(endpoint, offsets, chunk_size, ...)</code>	Worker process: fetch assigned chunks and save to disk with per-worker tqdm.
<code>read_from_triple_store_with_polars(endpoint[, ...])</code>	Main function to read all triples in parallel, save as Parquet, and load into Polars dataframe.
<code>read_from_triple_store_with_pandas([endpoint]</code>	Read triples from triple store into pandas dataframe
<code>get_filter_vocab(data[, directory])</code>	Build all evaluation filters in one pass without copying the input splits.
<code>get_er_vocab(data[, file_path])</code>	
<code>get_re_vocab(data[, file_path])</code>	
<code>get_ee_vocab(data[, file_path])</code>	
<code>create_constraints(triples[, file_path])</code>	
<code>load_with_pandas(→ None)</code>	Deserialize data
<code>save_numpy_ndarray(*, data, file_path)</code>	
<code>load_numpy_ndarray(*, file_path)</code>	
<code>save_pickle(*, data[, file_path])</code>	
<code>load_pickle(*[, file_path])</code>	
<code>create_recipriocal_triples(x)</code>	Add inverse triples into dask dataframe
<code>dataset_sanity_checking(→ None)</code>	

Module Contents

`dicee.read_preprocess_save_load_kg.util.logger`

`dicee.read_preprocess_save_load_kg.util.polars_dataframe_indexer(`
 `df_polars: polars.DataFrame, idx_entity: polars.DataFrame, idx_relation: polars.DataFrame)`
 `→ polars.DataFrame`

Replaces ‘subject’, ‘relation’, and ‘object’ columns in the input Polars DataFrame with their corresponding index values from the entity and relation index DataFrames.

This function processes the DataFrame in three main steps: 1. Replace the ‘relation’ values with the corresponding index from `idx_relation`. 2. Replace the ‘subject’ values with the corresponding index from `idx_entity`. 3. Replace the ‘object’ values with the corresponding index from `idx_entity`.

Parameters:

df_polars

[polars.DataFrame] The input Polars DataFrame containing columns: ‘subject’, ‘relation’, and ‘object’.

idx_entity

[polars.DataFrame] A Polars DataFrame that contains the mapping between entity names and their corresponding indices. Must have columns: ‘entity’ and ‘index’.

idx_relation

[polars.DataFrame] A Polars DataFrame that contains the mapping between relation names and their corresponding indices. Must have columns: ‘relation’ and ‘index’.

Returns:

polars.DataFrame

A DataFrame with the 'subject', 'relation', and 'object' columns replaced by their corresponding indices.

Example Usage:

```
>>> df_polars = pl.DataFrame({
    "subject": ["Alice", "Bob", "Charlie"],
    "relation": ["knows", "works_with", "lives_in"],
    "object": ["Dave", "Eve", "Frank"]
})
>>> idx_entity = pl.DataFrame({
    "entity": ["Alice", "Bob", "Charlie", "Dave", "Eve", "Frank"],
    "index": [0, 1, 2, 3, 4, 5]
})
>>> idx_relation = pl.DataFrame({
    "relation": ["knows", "works_with", "lives_in"],
    "index": [0, 1, 2]
})
>>> polars_dataframe_indexer(df_polars, idx_entity, idx_relation)
```

Steps:

1. Join the input DataFrame *df_polars* on the 'relation' column with *idx_relation* to replace the relations with their indices.
2. Join on 'subject' to replace it with the corresponding entity index using a left join on *idx_entity*.
3. Join on 'object' to replace it with the corresponding entity index using a left join on *idx_entity*.
4. Select only the 'subject', 'relation', and 'object' columns to return the final result.

```
dicee.read_preprocess_save_load_kg.util.pandas_dataframe_indexer(
    df_pandas: pandas.DataFrame, idx_entity: pandas.DataFrame, idx_relation: pandas.DataFrame)
    → pandas.DataFrame
```

Replaces 'subject', 'relation', and 'object' columns in the input Pandas DataFrame with their corresponding index values from the entity and relation index DataFrames.

Parameters:

df_pandas

[pd.DataFrame] The input Pandas DataFrame containing columns: 'subject', 'relation', and 'object'.

idx_entity

[pd.DataFrame] A Pandas DataFrame that contains the mapping between entity names and their corresponding indices. Must have columns: 'entity' and 'index'.

idx_relation

[pd.DataFrame] A Pandas DataFrame that contains the mapping between relation names and their corresponding indices. Must have columns: 'relation' and 'index'.

Returns:

pd.DataFrame

A DataFrame with the 'subject', 'relation', and 'object' columns replaced by their corresponding indices.

```
dicee.read_preprocess_save_load_kg.util.apply_reciprocal_or_noise (add_reciprocal: bool,  
    eval_model: str, df: object = None, info: str = None)
```

Add reciprocal triples if conditions are met

```
dicee.read_preprocess_save_load_kg.util.timeit (func)
```

```
dicee.read_preprocess_save_load_kg.util.read_with_polars (data_path,  
    read_only_few: int = None, sample_triples_ratio: float = None, separator: str = None)  
    → polars.DataFrame
```

Load and Preprocess via Polars

```
dicee.read_preprocess_save_load_kg.util.read_with_pandas (data_path,  
    read_only_few: int = None, sample_triples_ratio: float = None, separator: str = None)
```

Load and Preprocess via Pandas

```
dicee.read_preprocess_save_load_kg.util.read_from_disk (data_path: str,  
    read_only_few: int = None, sample_triples_ratio: float = None, backend: str = None,  
    separator: str = None) → Tuple[polars.DataFrame, pandas.DataFrame]
```

```
dicee.read_preprocess_save_load_kg.util.count_triples (endpoint: str) → int
```

Returns the total number of triples in the triple store.

```
dicee.read_preprocess_save_load_kg.util.fetch_worker (endpoint: str, offsets: list[int],  
    chunk_size: int, output_dir: str, worker_id: int)
```

Worker process: fetch assigned chunks and save to disk with per-worker tqdm.

```
dicee.read_preprocess_save_load_kg.util.read_from_triple_store_with_polars (  
    endpoint: str, chunk_size: int = 500000, output_dir: str = 'triples_parquet')
```

Main function to read all triples in parallel, save as Parquet, and load into Polars dataframe.

```
dicee.read_preprocess_save_load_kg.util.read_from_triple_store_with_pandas (  
    endpoint: str = None)
```

Read triples from triple store into pandas dataframe

```
dicee.read_preprocess_save_load_kg.util.get_filter_vocabs (data, directory=None)
```

Build all evaluation filters in one pass without copying the input splits.

Keep the existing defaultdict/list format and pickle filenames so saved checkpoints and evaluation APIs remain compatible.

```
dicee.read_preprocess_save_load_kg.util.get_er_vocab (data, file_path: str = None)
```

```
dicee.read_preprocess_save_load_kg.util.get_re_vocab (data, file_path: str = None)
```

```
dicee.read_preprocess_save_load_kg.util.get_ee_vocab (data, file_path: str = None)
```

```
dicee.read_preprocess_save_load_kg.util.create_constraints (triples, file_path: str = None)
```

(1) Extract domains and ranges of relations

(2) Store a mapping from relations to entities that are outside of the domain and range. Create constrained entities based on the range of relations :param triples: :return: Tuple[dict, dict]

`dicee.read_preprocess_save_load_kg.util.load_with_pandas(self) → None`

Deserialize data

`dicee.read_preprocess_save_load_kg.util.save_numpy_ndarray(*, data: numpy.ndarray, file_path: str)`

`dicee.read_preprocess_save_load_kg.util.load_numpy_ndarray(*, file_path: str)`

`dicee.read_preprocess_save_load_kg.util.save_pickle(*, data: object, file_path=str)`

`dicee.read_preprocess_save_load_kg.util.load_pickle(*, file_path=str)`

`dicee.read_preprocess_save_load_kg.util.create_recipriocal_triples(x)`

Add inverse triples into dask dataframe :param x: :return:

`dicee.read_preprocess_save_load_kg.util.dataset_sanity_checking(train_set: numpy.ndarray, num_entities: int, num_relations: int) → None`

Parameters

- **train_set**
- **num_entities**
- **num_relations**

Returns

Classes

<i>PreprocessKG</i>	Preprocess the data in memory
<i>LoadSaveToDisk</i>	
<i>ReadFromDisk</i>	Read the data from disk into memory

Package Contents

class `dicee.read_preprocess_save_load_kg.PreprocessKG(kg)`

Preprocess the data in memory

kg

start() → None

Preprocess train, valid and test datasets stored in knowledge graph instance

Parameter

rtype

None

preprocess_with_byte_pair_encoding()

preprocess_with_byte_pair_encoding_with_padding() → None

Preprocess with byte pair encoding and add padding

preprocess_with_pandas() → None

Preprocess with pandas: add reciprocal triples, construct vocabulary, and index datasets

preprocess_with_polars () → None

Preprocess with polars: add reciprocal triples and create indexed datasets

sequential_vocabulary_construction () → None

(1) Read input data into memory

(2) Remove triples with a condition

(3) **Serialize vocabularies in a pandas dataframe where**

=> the index is integer and => a single column is string (e.g. URI)

class dicee.read_preprocess_save_load_kg.**LoadSaveToDisk** (kg)

kg

save ()

load ()

class dicee.read_preprocess_save_load_kg.**ReadFromDisk** (kg)

Read the data from disk into memory

kg

start () → None

Read a knowledge graph from disk into memory

Data will be available at the train_set, test_set, valid_set attributes.

Parameter

None

rtype

None

add_noisy_triples_into_training ()

dicee.sanity_checkers

Attributes

logger

Functions

is_sparql_endpoint_alive([sparql_endpoint])

validate_knowledge_graph(args)

Validating the source of knowledge graph

sanity_checking_with_arguments(args)

sanity_check_callback_args(args)

Perform sanity checks on callback-related arguments.

Module Contents

`dicee.sanity_checkers.logger`

`dicee.sanity_checkers.is_sparql_endpoint_alive` (*sparql_endpoint: str = None*)

`dicee.sanity_checkers.validate_knowledge_graph` (*args*)

Validating the source of knowledge graph

`dicee.sanity_checkers.sanity_checking_with_arguments` (*args*)

`dicee.sanity_checkers.sanity_check_callback_args` (*args*)

Perform sanity checks on callback-related arguments.

`dicee.scripts`

Submodules

`dicee.scripts.index_serve`

```
$ docker pull qdrant/qdrant && docker run -p 6333:6333 -p 6334:6334 -v $(pwd)/qdrant_storage:/qdrant/storage:z
qdrant/qdrant $ dicee_vector_db -index -serve -path CountryEmbeddings -collection "countries_vdb"
```

Attributes

`logger`

`app`

`neural_searcher`

Classes

`NeuralSearcher`

`StringListRequest`

Functions

`get_default_arguments()`

`index(args)`

`root()`

`search_embeddings(q)`

`retrieve_embeddings(q)`

`search_embeddings_batch(request)`

`serve(args)`

`main()`

Module Contents

`dicee.scripts.index_serve.logger`

`dicee.scripts.index_serve.get_default_arguments()`

```

dicee.scripts.index_serve.index(args)

dicee.scripts.index_serve.app

dicee.scripts.index_serve.neural_searcher = None

class dicee.scripts.index_serve.NeuralSearcher(args)
    collection_name

    entity_to_idx = None

    qdrant_client

    topk = 5

    retrieve_embedding(entity: str = None, entities: List[str] = None) → List

    search(entity: str)

async dicee.scripts.index_serve.root()

async dicee.scripts.index_serve.search_embeddings(q: str)

async dicee.scripts.index_serve.retrieve_embeddings(q: str)

class dicee.scripts.index_serve.StringListRequest
    Bases: pydantic.BaseModel

    queries: List[str]

    reducer: str | None = None

async dicee.scripts.index_serve.search_embeddings_batch(request: StringListRequest)

dicee.scripts.index_serve.serve(args)

dicee.scripts.index_serve.main()

```

dicee.scripts.run

Functions

<code>get_default_arguments([description])</code>	Extends lightning Trainer's arguments with ours
<code>main()</code>	

Module Contents

```

dicee.scripts.run.get_default_arguments(description=None)
    Extends lightning Trainer's arguments with ours

dicee.scripts.run.main()

```

dicee.static_funcs

Static utility functions for DICE embeddings.

This module provides utility functions for model initialization, data loading, serialization, and various helper operations.

Attributes

`logger`
`MODEL_REGISTRY`

Functions

<code>create_recipriocal_triples(→ pandas.DataFrame)</code>	Add inverse triples to a DataFrame.
<code>get_er_vocab(→ Dict[Tuple[int, int], List[int]])</code>	Build entity-relation to tail vocabulary.
<code>get_re_vocab(→ Dict[Tuple[int, int], List[int]])</code>	Build relation-entity (tail) to head vocabulary.
<code>get_ee_vocab(→ Dict[Tuple[int, int], List[int]])</code>	Build entity-entity to relation vocabulary.
<code>timeit(→ Callable)</code>	Decorator to measure and print execution time and memory usage.
<code>setup_distributed_training(→ Dict[str, Union[bool, int]])</code>	Resolve distributed-training state and initialize custom DDP when needed.
<code>save_pickle(→ None)</code>	Save data to a pickle file.
<code>load_pickle(→ object)</code>	Load data from a pickle file.
<code>load_term_mapping(→ Union[dict, polars.DataFrame])</code>	Load term-to-index mapping from pickle or CSV file.
<code>select_model(args[, is_continual_training, storage_path])</code>	
<code>load_model(→ Tuple[object, Tuple[dict, dict]])</code>	Load weights and initialize pytorch module from namespace arguments
<code>load_model_ensemble(...)</code>	Construct Ensemble Of weights and initialize pytorch module from namespace arguments
<code>save_numpy_ndarray(*, data, file_path)</code>	
<code>numpy_data_type_changer(→ numpy.ndarray)</code>	Detect most efficient data type for a given triples
<code>save_checkpoint_model(→ None)</code>	Store Pytorch model into disk
<code>store(→ None)</code>	
<code>add_noisy_triples(→ pandas.DataFrame)</code>	Add randomly constructed triples
<code>read_or_load_kg(args, cls)</code>	
<code>intialize_model(...)</code>	Initialize a knowledge graph embedding model.
<code>load_json(→ Dict)</code>	Load JSON file into a dictionary.
<code>save_embeddings(→ None)</code>	Save embeddings to a CSV file.
<code>vocab_to_parquet(vocab_to_idx, name, ...)</code>	
<code>create_experiment_folder(→ str)</code>	Create a timestamped experiment folder.
<code>continual_training_setup_executor(→ None)</code>	
<code>exponential_function(→ torch.FloatTensor)</code>	
<code>load_numpy(→ numpy.ndarray)</code>	
<code>evaluate(entity_to_idx, scores, easy_answers, hard_answers)</code>	# @TODO: CD: Renamed this function
<code>download_file(url[, destination_folder])</code>	
<code>download_files_from_url(→ None)</code>	
<code>download_pretrained_model(→ str)</code>	
<code>write_csv_from_model_parallel(path)</code>	Create
<code>from_pretrained_model_write_embeddings_into(→ None)</code>	

Module Contents

`dicee.static_funcs.logger`

`dicee.static_funcs.MODEL_REGISTRY: Dict[str, Tuple[Type, str]]`

`dicee.static_funcs.create_recipriocal_triples(df: pandas.DataFrame) → pandas.DataFrame`

Add inverse triples to a DataFrame.

For each triple (s, p, o), creates an inverse triple (o, p_inverse, s).

Parameters

df – DataFrame with ‘subject’, ‘relation’, and ‘object’ columns.

Returns

DataFrame with original and inverse triples concatenated.

`dicee.static_funcs.get_er_vocab(data: numpy.ndarray, file_path: str | None = None)`

→ Dict[Tuple[int, int], List[int]]

Build entity-relation to tail vocabulary.

Parameters

- **data** – Array of triples with shape (n, 3) where columns are (head, relation, tail).
- **file_path** – Optional path to save the vocabulary as pickle.

Returns

Dictionary mapping (head, relation) pairs to list of tail entities.

`dicee.static_funcs.get_re_vocab(data: numpy.ndarray, file_path: str | None = None)`

→ Dict[Tuple[int, int], List[int]]

Build relation-entity (tail) to head vocabulary.

Parameters

- **data** – Array of triples with shape (n, 3) where columns are (head, relation, tail).
- **file_path** – Optional path to save the vocabulary as pickle.

Returns

Dictionary mapping (relation, tail) pairs to list of head entities.

`dicee.static_funcs.get_ee_vocab(data: numpy.ndarray, file_path: str | None = None)`

→ Dict[Tuple[int, int], List[int]]

Build entity-entity to relation vocabulary.

Parameters

- **data** – Array of triples with shape (n, 3) where columns are (head, relation, tail).
- **file_path** – Optional path to save the vocabulary as pickle.

Returns

Dictionary mapping (head, tail) pairs to list of relations.

`dicee.static_funcs.timeit(func: Callable) → Callable`

Decorator to measure and print execution time and memory usage.

Parameters

func – Function to be timed.

Returns

Wrapped function that prints timing information.

`dicee.static_funcs.setup_distributed_training(args) → Dict[str, bool | int]`

Resolve distributed-training state and initialize custom DDP when needed.

`dicee.static_funcs.save_pickle(*, data: object | None = None, file_path: str) → None`

Save data to a pickle file.

Note: Consider using more portable formats (JSON, Parquet) for new code.

Parameters

- **data** – Object to serialize. If None, nothing is saved.
- **file_path** – Path where the pickle file will be saved.

`dicee.static_funcs.load_pickle(file_path: str) → object`

Load data from a pickle file.

Note: Consider using more portable formats (JSON, Parquet) for new code.

Parameters

file_path – Path to the pickle file.

Returns

Deserialized object from the pickle file.

`dicee.static_funcs.load_term_mapping(file_path: str) → dict | polars.DataFrame`

Load term-to-index mapping from pickle or CSV file.

Attempts to load from pickle first, falls back to CSV if not found.

Parameters

file_path – Base path without extension.

Returns

Dictionary or Polars DataFrame containing the mapping.

`dicee.static_funcs.select_model(args: dict, is_continual_training: bool = None, storage_path: str = None)`

`dicee.static_funcs.load_model(path_of_experiment_folder: str, model_name='model.pt', verbose=0) → Tuple[object, Tuple[dict, dict]]`

Load weights and initialize pytorch module from namespace arguments

`dicee.static_funcs.load_model_ensemble(path_of_experiment_folder: str) → Tuple[dicee.models.base_model.BaseKGE, Tuple[pandas.DataFrame, pandas.DataFrame]]`

Construct Ensemble Of weights and initialize pytorch module from namespace arguments

- (1) Detect models under given path
- (2) Accumulate parameters of detected models
- (3) Normalize parameters
- (4) Insert (3) into model.

`dicee.static_funcs.save_numpy_ndarray(*, data: numpy.ndarray, file_path: str)`

`dicee.static_funcs.numpy_data_type_changer(train_set: numpy.ndarray, num: int) → numpy.ndarray`

Detect most efficient data type for a given triples :param train_set: :param num: :return:

`dicee.static_funcs.save_checkpoint_model(model, path: str) → None`

Store Pytorch model into disk

`dicee.static_funcs.store(trained_model, model_name: str = 'model', full_storage_path: str = None, save_embeddings_as_csv=False) → None`

`dicee.static_funcs.add_noisy_triples(train_set: pandas.DataFrame, add_noise_rate: float) → pandas.DataFrame`

Add randomly constructed triples :param train_set: :param add_noise_rate: :return:

`dicee.static_funcs.read_or_load_kg(args, cls)`

`dicee.static_funcs.initialize_model(args: Dict, verbose: int = 0, for_inference: bool = False) → Tuple[dicee.models.base_model.BaseKGE, str]`

Initialize a knowledge graph embedding model.

Parameters

- **args** – Dictionary containing model configuration including ‘model’ key.
- **verbose** – Verbosity level. If > 0, prints initialization message.
- **for_inference** – If True, never build the FSDP row-wise-sharded shell, even if `args[“trainer”] == “torchFSDP”`. A completed run’s `model.pt` (written by `TorchFSDP-Trainer._materialize_model()`) already has a plain, full `entity_embeddings.weight` — loading it into the sharded shell (which defers `entity_embeddings` until `setup_fsdg_training()`) fails with “Unexpected key(s): `entity_embeddings.weight`”. Callers loading a finished model for inference (`load_model`, `load_model_ensemble`) must set this; callers building a fresh model to train/resume with (`select_model`) must not.

Returns

Tuple of (initialized model, form of labelling string).

Raises

ValueError – If the model name is not recognized.

`dicee.static_funcs.load_json(path: str) → Dict`

Load JSON file into a dictionary.

Parameters

path – Path to the JSON file.

Returns

Dictionary containing the JSON data.

Raises

- **FileNotFoundError** – If the file does not exist.
- **json.JSONDecodeError** – If the file contains invalid JSON.

`dicee.static_funcs.save_embeddings(embeddings: numpy.ndarray, indexes: List, path: str) → None`

Save embeddings to a CSV file.

Parameters

- **embeddings** – NumPy array of embeddings with shape (n_items, embedding_dim).
- **indexes** – List of index labels (entity/relation names).
- **path** – Output file path.

`dicee.static_funcs.vocab_to_parquet(vocab_to_idx, name, path_for_serialization, print_into)`

`dicee.static_funcs.create_experiment_folder(folder_name: str = 'Experiments') → str`

Create a timestamped experiment folder.

Parameters

folder_name – Base directory name for experiments.

Returns

Full path to the created experiment folder.

`dicee.static_funcs.continual_training_setup_executor(executor) → None`

`dicee.static_funcs.exponential_function(x: numpy.ndarray, lam: float, ascending_order=True)`
→ torch.FloatTensor

`dicee.static_funcs.load_numpy(path) → numpy.ndarray`

`dicee.static_funcs.evaluate(entity_to_idx, scores, easy_answers, hard_answers)`

@TODO: CD: Renamed this function Evaluate multi hop query answering on different query types

`dicee.static_funcs.download_file(url, destination_folder='.')`

`dicee.static_funcs.download_files_from_url(base_url: str, destination_folder='.') → None`

Parameters

- **base_url** (e.g. <https://files.dice-research.org/projects/DiceEmbeddings/KINSHIP-Keci-dim128-epoch256-KvsAll>)
- **destination_folder** (e.g. `"KINSHIP-Keci-dim128-epoch256-KvsAll"`)

`dicee.static_funcs.download_pretrained_model(url: str) → str`

`dicee.static_funcs.write_csv_from_model_parallel(path: str)`

Create

`dicee.static_funcs.from_pretrained_model_write_embeddings_into_csv(path: str) → None`

dicee.static_funcs_training

Training-related static functions.

This module provides backward compatibility by re-exporting evaluation functions from the new `dicee.evaluation` module, along with training utilities.

Deprecated since version Evaluation: functions have moved to `dicee.evaluation`. Use that module for new code. This module will continue to export training utilities.

Functions

<code>evaluate_bpe_lp(→ Dict[str, float])</code>	Evaluate link prediction with BPE-encoded entities.
<code>evaluate_lp(→ Dict[str, float])</code>	Evaluate link prediction with batched processing.
<code>efficient_zero_grad(→ None)</code>	Efficiently zero gradients using <code>parameter.grad = None</code> .
<code>make_iterable_verbose(→ Iterable)</code>	Wrap an iterable with <code>tqdm</code> progress bar if <code>verbose</code> is <code>True</code> .

Module Contents

```
dicee.static_funcs_training.evaluate_bpe_lp(model, triple_idx: List[Tuple],  
      all_bpe_shaped_entities, er_vocab: Dict[Tuple, List], re_vocab: Dict[Tuple, List],  
      info: str = 'Eval Starts', *, tie_policy: str = 'sort', tie_seed: int = 0) → Dict[str, float]
```

Evaluate link prediction with BPE-encoded entities.

Parameters

- **model** – The KGE model to evaluate.
- **triple_idx** – List of BPE-encoded triple tuples.
- **all_bpe_shaped_entities** – All entities with BPE representations.
- **er_vocab** – Mapping for tail filtering.
- **re_vocab** – Mapping for head filtering.
- **info** – Description to print.
- **tie_policy** – How exact prediction ties are ranked: sort (legacy), optimistic, random, or pessimistic.
- **tie_seed** – Independent random tie seed; reset on each evaluation call.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

```
dicee.static_funcs_training.evaluate_lp(model, triple_idx, num_entities: int,  
      er_vocab: Dict[Tuple, List], re_vocab: Dict[Tuple, List], info: str = 'Eval Starts',  
      batch_size: int = 128, chunk_size: int = 1000, *, tie_policy: str = 'sort', tie_seed: int = 0,  
      tie_generator: torch.Generator | None = None,  
      graph_rank_plan: dicee.evaluation._graph_inference.GraphRankPlan | None = None,  
      reuse_graph_queries: bool = True) → Dict[str, float]
```

Evaluate link prediction with batched processing.

Memory-efficient evaluation using chunked entity scoring.

Parameters

- **model** – The KGE model to evaluate.
- **triple_idx** – Integer-indexed triples as numpy array.
- **num_entities** – Total number of entities.
- **er_vocab** – Mapping (head_idx, rel_idx) -> list of tail indices.
- **re_vocab** – Mapping (rel_idx, tail_idx) -> list of head indices.
- **info** – Description to print.
- **batch_size** – Batch size for triple processing.
- **chunk_size** – Chunk size for entity scoring.
- **tie_generator** – Optional CPU generator to continue random ties across calls. When supplied, its state takes precedence over tie_seed.
- **graph_rank_plan** – Optional whole-split plan shared across progress batches.
- **reuse_graph_queries** – Reuse deterministic ULTRA/TRIX queries. Flock keeps its original query and sampling order regardless of this option.

- **tie_policy** – How exact prediction ties are ranked: sort (legacy), optimistic, random, or pessimistic.
- **tie_seed** – Independent random tie seed; reset on each evaluation call.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

`dicee.static_funcs_training.efficient_zero_grad(model) → None`

Efficiently zero gradients using `parameter.grad = None`.

This is more efficient than `optimizer.zero_grad()` as it avoids memory operations.

See: https://pytorch.org/tutorials/recipes/recipes/tuning_guide.html

Parameters

model – PyTorch model to zero gradients for.

`dicee.static_funcs_training.make_iterable_verbose(iterable_object: Iterable, verbose: bool, desc: str = 'Default', position: int | None = None, leave: bool = True) → Iterable`

Wrap an iterable with tqdm progress bar if verbose is True.

Parameters

- **iterable_object** – The iterable to potentially wrap.
- **verbose** – Whether to show progress bar.
- **desc** – Description for the progress bar.
- **position** – Position of the progress bar.
- **leave** – Whether to leave the progress bar after completion.

Returns

The original iterable or a tqdm-wrapped version.

dicee.static_preprocess_funcs

Attributes

<code>logger</code>
<code>enable_log</code>

Functions

<code>timeit(func)</code>	
<code>preprocesses_input_args(args)</code>	Sanity Checking in input arguments
<code>create_constraints(→ Tuple[dict, dict, dict, dict])</code>	
<code>get_er_vocab(data)</code>	
<code>get_re_vocab(data)</code>	
<code>get_ee_vocab(data)</code>	
<code>mapping_from_first_two_cols_to_third(train_se</code>	

Module Contents

`dicee.static_preprocess_funcs.logger`

`dicee.static_preprocess_funcs.enable_log = False`

`dicee.static_preprocess_funcs.timeit (func)`

`dicee.static_preprocess_funcs.preprocesses_input_args (args)`

Sanity Checking in input arguments

`dicee.static_preprocess_funcs.create_constraints (triples: numpy.ndarray)`
→ Tuple[dict, dict, dict, dict]

(1) Extract domains and ranges of relations

(2) Store a mapping from relations to entities that are outside of the domain and range. Create constraints entities based on the range of relations :param triples: :return:

`dicee.static_preprocess_funcs.get_er_vocab (data)`

`dicee.static_preprocess_funcs.get_re_vocab (data)`

`dicee.static_preprocess_funcs.get_ee_vocab (data)`

`dicee.static_preprocess_funcs.mapping_from_first_two_cols_to_third (train_set_idx)`

dicee.trainer

Submodules

dicee.trainer.auto_batch_finder

Attributes

logger

Functions

<code>find_good_batch_size(→ Tuple[int, Optional[float]])</code>	Find a batch size that uses GPU memory efficiently.
--	---

Module Contents

`dicee.trainer.auto_batch_finder.logger`

`dicee.trainer.auto_batch_finder.find_good_batch_size (`
`train_loader: torch.utils.data.DataLoader, training_step_fn: Callable, device)`
→ Tuple[int, float | None]

Find a batch size that uses GPU memory efficiently.

Progressively increases batch size (with tunable delta) until GPU memory exceeds 90% or a CUDA OOM error is raised, then backs off to the last safe batch size. Only supported on CUDA devices; returns the initial batch size unchanged on CPU.

Parameters

- **train_loader** – DataLoader wrapping the training dataset.
- **training_step_fn** – Callable[[batch], float] — runs one forward+backward pass and returns the scalar loss. Each trainer passes its own implementation so this function stays trainer-agnostic.
- **device** – torch.device (or anything accepted by torch.device()) used to monitor GPU memory.

Returns

(batch_size, runtime) where runtime is the wall-clock seconds of the last successful batch, or None when batch finding was skipped.

dicee.trainer.dice_trainer

DICE Trainer module for knowledge graph embedding training.

Provides the DICE_Trainer class which supports multiple training backends including PyTorch Lightning, DDP, and custom CPU/GPU trainers.

Attributes

<i>logger</i>

Classes

<i>DICE_Trainer</i>	DICE_Trainer implement
---------------------	------------------------

Functions

<i>load_term_mapping</i> (→ polars.DataFrame)	Load term-to-index mapping from CSV file.
<i>initialize_trainer</i> (...)	Initialize the appropriate trainer based on configuration.
<i>get_callbacks</i> (→ List)	Create list of callbacks based on configuration.

Module Contents

`dicee.trainer.dice_trainer.logger`

`dicee.trainer.dice_trainer.load_term_mapping (file_path: str) → polars.DataFrame`
Load term-to-index mapping from CSV file.

Parameters

file_path – Base path without extension.

Returns

Polars DataFrame containing the mapping.

`dicee.trainer.dice_trainer.initialize_trainer (args, callbacks: List)`
→ `dicee.trainer.torch_trainer.TorchTrainer` | `dicee.trainer.model_parallelism.TensorParallel` | `dicee.trainer.torch_trainer_ddp`
Initialize the appropriate trainer based on configuration.

Parameters

- **args** – Configuration arguments containing trainer type.
- **callbacks** – List of training callbacks.

Returns

Initialized trainer instance.

Raises

AssertionError – If trainer is None after initialization.

`dicee.trainer.dice_trainer.get_callbacks(args)` → List

Create list of callbacks based on configuration.

Parameters

args – Configuration arguments.

Returns

List of callback instances.

class `dicee.trainer.dice_trainer.DICE_Trainer` (*args, is_continual_training: bool, storage_path, evaluator=None*)

DICE_Trainer implement

- 1- Pytorch Lightning trainer (<https://pytorch-lightning.readthedocs.io/en/stable/common/trainer.html>)
- 2- Multi-GPU Trainer(<https://pytorch.org/docs/stable/generated/torch.nn.parallel.DistributedDataParallel.html>)
- 3- CPU Trainer

`args`

`is_continual_training:bool`

`storage_path:str`

`evaluator:`

`report:dict`

report

args

trainer = None

is_continual_training

storage_path

evaluator = None

form_of_labelling = None

continual_start (*knowledge_graph*)

- (1) Initialize training.
- (2) Load model
- (3) Load trainer (3) Fit model

Parameter

returns

- *model*
- **form_of_labelling** (*str*)

initialize_trainer (*callbacks: List*)

→ `lightning.Trainer` | `dicее.trainer.model_parallelism.TensorParallel` | `dicее.trainer.torch_trainer.TorchTrainer` | `dicее.t`

Initialize Trainer from input arguments

initialize_or_load_model ()

init_dataloader (*dataset: torch.utils.data.Dataset*) → `torch.utils.data.DataLoader`

init_dataset () → `torch.utils.data.Dataset`

prepare_graph_model (*model, knowledge_graph*)

Attach only training facts, preserving the indexed external vocabulary.

prepare_ultra

start (*knowledge_graph: dicее.knowledge_graph.KG* | *numpy.memmap*)

→ `Tuple[dicее.models.base_model.BaseKGE, str]`

Start the training

- (1) Initialize Trainer
- (2) Initialize or load a pretrained KGE model

in DDP setup, we need to load the memory map of already read/index KG.

k_fold_cross_validation (*dataset*) → `Tuple[dicее.models.base_model.BaseKGE, str]`

Perform K-fold Cross-Validation

1. Obtain K train and test splits.
2. **For each split,**
 - 2.1 initialize trainer and model
 - 2.2. Train model with configuration provided in args.
 - 2.3. Compute the mean reciprocal rank (MRR) score of the model on the test respective split.
3. Report the mean and average MRR .

Parameters

- **self**
- **dataset**

Returns

model

dicее.trainer.model_parallelism

Tensor Parallelism trainer for ensemble knowledge graph embeddings.

This module implements the tensor parallelism training strategy described in:

“Multiple Run Ensemble Learning with Low-Dimensional Knowledge Graph Embeddings”

<https://arxiv.org/abs/2104.05003>

The `TensorParallel` trainer creates an ensemble of models, each trained on a separate GPU, allowing efficient utilization of multi-GPU systems through model parallelism.

Attributes

`logger`

Classes

`TensorParallel`

Abstract base class for KGE model trainers.

Functions

`extract_input_outputs(z[, device])`
`forward_backward_update_loss(→ float)`

Module Contents

`dicee.trainer.model_parallelism.logger`

`dicee.trainer.model_parallelism.extract_input_outputs` (*z*: list, *device*=None)

`dicee.trainer.model_parallelism.forward_backward_update_loss` (*z*: Tuple, *ensemble_model*)
→ float

class `dicee.trainer.model_parallelism.TensorParallel` (*args*, *callbacks*)

Bases: `dicee.abstracts.AbstractTrainer`

Abstract base class for KGE model trainers.

Provides the callback dispatch mechanism shared by all concrete trainer implementations (`TorchTrainer`, `TorchD-
DPTrainer`, etc.). Sub-classes call the `on_*` hooks at the appropriate points in the training loop so that any registered `AbstractCallback` can react.

Parameters

- **args** (*argparse.Namespace* or similar) – Processed configuration object. Must expose at least `random_seed` (int).
- **callbacks** (list of `AbstractCallback`) – Ordered list of callback instances to invoke at each lifecycle hook.

fit (**args*, ***kwargs*)

Train model

`dicee.trainer.torch_trainer`

Attributes

`logger`

Classes

<i>TorchTrainer</i>	TorchTrainer for using single GPU or multi CPUs on a single node
---------------------	--

Module Contents

`dicee.trainer.torch_trainer.logger`

class `dicee.trainer.torch_trainer.TorchTrainer` (*args*, *callbacks*)

Bases: `dicee.abstracts.AbstractTrainer`

TorchTrainer for using single GPU or multi CPUs on a single node

Arguments

callbacks: list of Abstract callback instances

loss_function = None

optimizer = None

model = None

train_dataloaders = None

training_step = None

process

fit (**args*, *train_dataloaders*, ***kwargs*) → None

Training starts

Arguments

kwargs: Tuple

empty dictionary

Return type

batch loss (float)

forward_backward_update (*x_batch: torch.Tensor*, *y_batch: torch.Tensor*, ***, *loss_divisor: int = 1*, *update_weights: bool = True*) → float

Compute forward, loss, backward, and parameter update

Arguments

Return type

batch loss (float)

extract_input_outputs_set_device (*batch: list*) → Tuple

Construct inputs and outputs from a batch of inputs with outputs From a batch of inputs and put

Arguments

Return type
(tuple) mini-batch on select device

dicee.trainer.torch_trainer_ddp

Attributes

<code>logger</code>

Classes

<code>TorchDDPTrainer</code>	Abstract base class for KGE model trainers.
<code>NodeTrainer</code>	

Functions

<code>make_iterable_verbose</code> (→ Iterable)

Module Contents

`dicee.trainer.torch_trainer_ddp.logger`

`dicee.trainer.torch_trainer_ddp.make_iterable_verbose` (*iterable_object*, *verbose*,
desc='Default', *position*=None, *leave*=True) → Iterable

class `dicee.trainer.torch_trainer_ddp.TorchDDPTrainer` (*args*, *callbacks*)

Bases: `dicee.abstracts.AbstractTrainer`

Abstract base class for KGE model trainers.

Provides the callback dispatch mechanism shared by all concrete trainer implementations (TorchTrainer, TorchDDPTrainer, etc.). Sub-classes call the `on_*` hooks at the appropriate points in the training loop so that any registered `AbstractCallback` can react.

Parameters

- **args** (*argparse.Namespace* or *similar*) – Processed configuration object. Must expose at least `random_seed` (int).
- **callbacks** (*list of AbstractCallback*) – Ordered list of callback instances to invoke at each lifecycle hook.

fit (**args*, ***kwargs*)

class `dicee.trainer.torch_trainer_ddp.NodeTrainer` (*trainer*, *model*: `torch.nn.Module`,
train_dataset_loader: `torch.utils.data.DataLoader`, *callbacks*, *num_epochs*: int)

trainer

local_rank

global_rank

```

optimizer

train_dataset_loader

loss_func

callbacks

model

num_epochs

loss_history = []

ctx

scaler

extract_input_outputs(z: list)

train()

```

`dicee.trainer.torch_trainer_fsdp`

Multi-GPU trainer: row-wise sharded entity embeddings + FSDP dense parameters.

Entity embeddings are sharded row-wise across ranks via `dist.all_to_all_single`. Each rank owns a contiguous slice of entity rows; forward and backward use `all_to_all` to route index requests and gradient returns to the owning rank. A per-rank `_LocalSparseAdam` updates only the rows that received a non-zero gradient each batch.

Dense parameters (relation embeddings, Clifford coefficients, ...) are wrapped with FSDP and updated by the standard dense optimizer.

Attributes

<code>logger</code>
<code>OptimizedModule</code>

Classes

<code>TorchFSDPTrainer</code>	Single-node multi-GPU trainer: row-wise sharded entity embeddings + FSDP.
-------------------------------	---

Module Contents

```

dicee.trainer.torch_trainer_fsdp.logger

dicee.trainer.torch_trainer_fsdp.OptimizedModule = None

class dicee.trainer.torch_trainer_fsdp.TorchFSDPTrainer(args, callbacks)
    Bases: dicee.abstracts.AbstractTrainer

    Single-node multi-GPU trainer: row-wise sharded entity embeddings + FSDP.

```

Entity embeddings

Sharded row-wise across GPUs. `all_to_all` routes each index request to its owning rank; embeddings and gradients travel back the same way. `_LocalSparseAdam` updates only the rows accessed in each batch — $O(\text{active_rows})$ not $O(\text{shard_size})$. Each entity receives gradient from every rank's batches.

Dense parameters (relation embeddings, Clifford coefficients, ...)

Wrapped with FSDP; updated by the standard dense optimizer `step()`.

Usage

Launched with `torchrun`:

```
torchrun -nproc_per_node=4 -u dicee --trainer torchFSDP ...
```

`local_rank`

`global_rank`

`is_global_zero`

`device`

`model = None`

`raw_model = None`

`optimizer = None`

`loss_func = None`

`train_dataset_loader = None`

`ptdtype`

`ctx`

`scaler`

`sharding_strategy`

`gradient_clip_val`

`use_compile`

`num_workers`

`prefetch_factor`

`entity_optim_device`

`checkpoint_every_n_epochs`

`fit(*args, **kwargs)`

`extract_input_outputs(z)`

Classes

DICE_Trainer

DICE_Trainer implement

Package Contents

```
class dicee.trainer.DICE_Trainer (args, is_continual_training: bool, storage_path, evaluator=None)
```

DICE_Trainer implement

- 1- Pytorch Lightning trainer (<https://pytorch-lightning.readthedocs.io/en/stable/common/trainer.html>)
- 2- Multi-GPU Trainer(<https://pytorch.org/docs/stable/generated/torch.nn.parallel.DistributedDataParallel.html>)
- 3- CPU Trainer

args

is_continual_training:bool

storage_path:str

evaluator:

report:dict

report

args

trainer = None

is_continual_training

storage_path

evaluator = None

form_of_labelling = None

continual_start (*knowledge_graph*)

- (1) Initialize training.
- (2) Load model
- (3) Load trainer (3) Fit model

Parameter

returns

- *model*
- **form_of_labelling** (*str*)

initialize_trainer (*callbacks: List*)

→ *lightning.Trainer* | *dicee.trainer.model_parallelism.TensorParallel* | *dicee.trainer.torch_trainer.TorchTrainer* | *dicee.t*

Initialize Trainer from input arguments

initialize_or_load_model ()

init_dataloader (*dataset: torch.utils.data.Dataset*) → torch.utils.data.DataLoader

init_dataset () → torch.utils.data.Dataset

prepare_graph_model (*model, knowledge_graph*)

Attach only training facts, preserving the indexed external vocabulary.

prepare_ultra

start (*knowledge_graph: dicee.knowledge_graph.KG | numpy.memmap*)

→ Tuple[*dicee.models.base_model.BaseKGE*, str]

Start the training

(1) Initialize Trainer

(2) Initialize or load a pretrained KGE model

in DDP setup, we need to load the memory map of already read/index KG.

k_fold_cross_validation (*dataset*) → Tuple[*dicee.models.base_model.BaseKGE*, str]

Perform K-fold Cross-Validation

1. Obtain K train and test splits.

2. **For each split,**

2.1 initialize trainer and model 2.2. Train model with configuration provided in args. 2.3. Compute the mean reciprocal rank (MRR) score of the model on the test respective split.

3. Report the mean and average MRR .

Parameters

- **self**
- **dataset**

Returns

model

dicee.weight_averaging

Classes

<i>AMWA</i>	Adaptive Momentum Weight Averaging (AMWA).
<i>ASWA</i>	Adaptive stochastic weight averaging
<i>SWA</i>	Stochastic Weight Averaging callback.
<i>SWAG</i>	Stochastic Weight Averaging - Gaussian (SWAG).
<i>EMA</i>	Exponential Moving Average (EMA) callback.
<i>TWA</i>	Train with Weight Averaging (TWA) using subspace projection + averaging.

Module Contents

class `dicee.weight_averaging.AMWA` (*amwa_start_epoch: int = 0, amwa_c_epochs: int = 1, path: str | None = None, monitor: str = 'MRR', maximize: bool = True, beta: float | None = None, beta_window: int = 10, beta_init: float = 1.0, beta_floor: float = 1e-08*)

Bases: `dicee.abstracts.AbstractCallback`

Adaptive Momentum Weight Averaging (AMWA).

Paper: <https://www.sciencedirect.com/science/article/pii/S0031320325009586>

BaseNet is the ordinary model optimized during training, while StableNet is updated once per epoch via weight averaging,

$$w_stable \leftarrow m * w_stable + (1 - m) * w_base$$

where the adaptive momentum is computed from the validation-performance difference between StableNet and BaseNet,

$$m(\Delta) = 1 / (1 + \exp(-\max(0, \Delta) / \beta_n))$$

with Δ defined so that $\Delta > 0$ means StableNet performs better than BaseNet. This yields the exact behavior described in the paper: StableNet stays close to its current solution when it already validates better, and it takes a larger step toward BaseNet when BaseNet validates better. The paper sets β_n to the standard deviation of Δ over the past 10 epochs; this implementation follows that default and also allows a fixed β for the ablation setting.

param amwa_start_epoch

Epoch at which AMWA starts tracking StableNet.

type amwa_start_epoch

int

param amwa_c_epochs

Update interval in epochs.

type amwa_c_epochs

int

param path

Directory used to persist `amwa.pt` and `amwa_history.json`.

type path

str, optional

param monitor

Validation metric to monitor, e.g. "MRR".

type monitor

str

param maximize

Whether larger values of `monitor` are better.

type maximize

bool

param beta

If provided, use a fixed β instead of the adaptive β_n from the paper. This matches the paper's fixed-beta ablation.

type beta

float, optional

param beta_window

Number of past Δ values used to estimate β_n .

type beta_window

int

param beta_init

Beta used before enough Δ history exists.

```

    type beta_init
        float

    param beta_floor
        Numerical floor used when the empirical standard deviation is zero.

    type beta_floor
        float

    amwa_start_epoch = 0

    amwa_c_epochs = 1

    path = None

    monitor = 'MRR'

    maximize = True

    beta = None

    beta_window

    beta_init

    beta_floor

    current_epoch = -1

    initial_eval_setting: str | None = None

    stable_state_dict: Dict[str, torch.Tensor] | None = None

    delta_history: List[float] = []

    momentum_history: List[float] = []

    beta_history: List[float] = []

    base_history: List[float] = []

    stable_history: List[float] = []

    history: List[Dict[str, Any]] = []

    on_train_epoch_start(trainer, model)
        Called when the train epoch begins.

    on_train_epoch_end(trainer, model)
        Called at the end of each training epoch.

```

Parameters

- **trainer** (`AbstractTrainer` or `pl.Trainer`) – The active trainer instance.
- **model** (`BaseKGE`) – The model being trained. `model.loss_history` contains the per-epoch average losses accumulated so far.

on_fit_end (*trainer, model*)

Called once after the final training epoch completes.

Override to perform post-training actions such as saving the final model state, computing evaluation metrics, or cleaning up resources.

```
class dicee.weight_averaging.ASWA(num_epochs, path)
```

Bases: `dicee.abstracts.AbstractCallback`

Adaptive stochastic weight averaging ASWE keeps track of the validation performance and updates the ensemble model accordingly.

path

num_epochs

initial_eval_setting = None

epoch_count = 0

alphas = []

val_aswa = -1

on_fit_end(trainer, model)

Called once after the final training epoch completes.

Override to perform post-training actions such as saving the final model state, computing evaluation metrics, or cleaning up resources.

static compute_mrr(trainer, model) → float

get_aswa_state_dict(model)

decide(running_model_state_dict, ensemble_state_dict, val_running_model, mrr_updated_ensemble_model)

Perform Hard Update, software or rejection

Parameters

- **running_model_state_dict**
- **ensemble_state_dict**
- **val_running_model**
- **mrr_updated_ensemble_model**

on_train_epoch_end(trainer, model)

Called at the end of each training epoch.

Parameters

- **trainer** (`AbstractTrainer` or `pl.Trainer`) – The active trainer instance.
- **model** (`BaseKGE`) – The model being trained. `model.loss_history` contains the per-epoch average losses accumulated so far.

```
class dicee.weight_averaging.SWA(swa_start_epoch, swa_c_epochs: int = 1, lr_init: float = 0.1, swa_lr: float = 0.05, max_epochs: int = None)
```

Bases: `dicee.abstracts.AbstractCallback`

Stochastic Weight Averaging callback.

Initialize SWA callback.

swa_start_epoch: int

The epoch at which to start SWA.

swa_c_epochs: int
The number of epochs to use for SWA.

lr_init: float
The initial learning rate.

swa_lr: float
The learning rate to use during SWA.

max_epochs: int
The maximum number of epochs. `args.num_epochs`

swa_start_epoch

swa_c_epochs = 1

swa_lr = 0.05

lr_init = 0.1

max_epochs = None

swa_model = None

swa_n = 0

current_epoch = -1

static moving_average (*swa_model, running_model, alpha*)
Update SWA model with moving average of current model. Math: # SWA update: $\theta_{\text{swa}} \leftarrow (1 - \alpha) * \theta_{\text{swa}} + \alpha * \theta$ # $\alpha = 1 / (n + 1)$, where n = number of models already averaged # α is tracked via `self.swa_n` in code

on_train_epoch_start (*trainer, model*)
Update learning rate according to SWA schedule.

on_train_epoch_end (*trainer, model*)
Apply SWA averaging if conditions are met.

on_fit_end (*trainer, model*)
Replace main model with SWA model at the end of training.

class `dicee.weight_averaging.SWAG` (*swa_start_epoch, swa_c_epochs: int = 1, lr_init: float = 0.1, swa_lr: float = 0.05, max_epochs: int = None, max_num_models: int = 20, var_clamp: float = 1e-30*)

Bases: `dicee.abstracts.AbstractCallback`

Stochastic Weight Averaging - Gaussian (SWAG). Parameters

swa_start_epoch
[int] Epoch at which to start collecting weights.

swa_c_epochs
[int] Interval of epochs between updates.

lr_init
[float] Initial LR.

swa_lr
[float] LR in SWA / GSWA phase.

```

    max_epochs
        [int] Total number of epochs.

    max_num_models
        [int] Number of models to keep for low-rank covariance approx.

    var_clamp
        [float] Clamp low variance for stability.

swa_start_epoch

swa_c_epochs = 1

swa_lr = 0.05

lr_init = 0.1

max_epochs = None

max_num_models = 20

var_clamp = 1e-30

mean = None

sq_mean = None

deviations = []

gswa_n = 0

current_epoch = -1

get_mean_and_var()
    Return mean + variance (diagonal part).

sample (base_model, scale=0.5)
    Sample new model from SWAG posterior distribution.

    Math: # From SWAG, posterior is approximated as:  $\theta \sim N(\text{mean}, \Sigma)$  # where  $\Sigma \approx \text{diag}(\text{var}) + (1/(K-1)) * D D^T$  # - mean = running average of weights # - var = elementwise variance (sq_mean - mean2) # - D = [dev_1, dev_2, ..., dev_K], deviations from mean (low-rank approx) # - K = number of collected models

    # Sampling step: # 1.  $\theta_{\text{diag}} = \text{mean} + \text{scale} * \text{std} \odot \varepsilon$ , where  $\varepsilon \sim N(0, I)$  # 2.  $\theta_{\text{lowrank}} = \theta_{\text{diag}} + (D z) / \sqrt{K-1}$ , where  $z \sim N(0, I_K)$  # Final sample =  $\theta_{\text{lowrank}}$ 

on_train_epoch_start (trainer, model)
    Update LR schedule (same as SWA).

on_train_epoch_end (trainer, model)
    Collect Gaussian stats at the end of epochs after swa_start.

on_fit_end (trainer, model)
    Set model weights to the collected SWAG mean at the end of training.

class dicee.weight_averaging.EMA(ema_start_epoch: int, decay: float = 0.999,
    max_epochs: int = None, ema_c_epochs: int = 1)

Bases: dicee.abstracts.AbstractCallback

Exponential Moving Average (EMA) callback.

Parameters

```

- **ema_start_epoch** (*int*) – Epoch to start EMA.
- **decay** (*float*) – EMA decay rate (typical: 0.99 - 0.9999) Math: $\theta_{\text{ema}} \leftarrow \text{decay} * \theta_{\text{ema}} + (1 - \text{decay}) * \theta$
- **max_epochs** (*int*) – Maximum number of epochs.

ema_start_epoch

decay = 0.999

max_epochs = None

ema_c_epochs = 1

ema_model = None

current_epoch = -1

static ema_update (*ema_model, running_model, decay: float*)

Update EMA model with exponential moving average of current model. Math: # EMA update: $\theta_{\text{ema}} \leftarrow (1 - \alpha) * \theta_{\text{ema}} + \alpha * \theta$ # $\alpha = 1 - \text{decay}$, where decay is the EMA smoothing factor (typical 0.99 - 0.999) # α controls how much of the current model θ contributes to the EMA # decay is fixed in code -> can be extended to scheduled

on_train_epoch_start (*trainer, model*)

Track current epoch.

on_train_epoch_end (*trainer, model*)

Update EMA if past start epoch.

on_fit_end (*trainer, model*)

Replace main model with EMA model at the end of training.

class dicee.weight_averaging.**TWA** (*twa_start_epoch: int, lr_init: float, num_samples: int = 5, reg_lambda: float = 0.0, max_epochs: int = None, twa_c_epochs: int = 1*)

Bases: *dicee.abstracts.AbstractCallback*

Train with Weight Averaging (TWA) using subspace projection + averaging.

Parameters

twa_start_epoch

[int] Epoch to start TWA.

lr_init

[float] Learning rate used for β updates.

num_samples

[int] Number of sampled weight snapshots to build projection subspace.

reg_lambda

[float] Regularization coefficient for β updates.

max_epochs

[int] Total number of training epochs.

twa_c_epochs

[int] Interval of epochs between TWA updates.

twa_start_epoch

```

num_samples = 5

reg_lambda = 0.0

max_epochs = None

lr_init

twa_c_epochs = 1

current_epoch = -1

weight_samples = []

twa_model = None

base_weights = None

P = None

beta = None

sample_weights(model)
    Collect sampled weights from the current model and maintain rolling buffer.

build_projection(weight_samples, k=None)
    Build projection subspace from collected weight samples. :param weight_samples: list of flat weight tensors [(D,), ...] :param k: number of basis vectors to keep. Defaults to min(N, D).

    Returns
        (D,) base weight vector (average) P: (D, k) projection matrix with top-k basis directions

    Return type
        mean_w

on_train_epoch_start(trainer, model)
    Track epoch.

on_train_epoch_end(trainer, model)
    Main TWA logic: build subspace and update in  $\beta$  space.

    # Math: # TWA weight update: #  $w_{twa} = \text{mean\_w} + P * \beta$  #  $\text{mean\_w} = (1/n) * \sum_i w_i$  (SWA baseline)
    #  $\beta \leftarrow (1 - \eta * \lambda) * \beta - \eta * P^T * g$  #  $g$  = gradient of training loss w.r.t. full model weights
    #  $\eta$  = learning rate,  $\lambda$  = ridge regularization #  $P$  = orthonormal basis spanning sampled checkpoints  $\{w_i\}$ 

on_fit_end(trainer, model)
    Replace with TWA model at the end.

```

14.2 Attributes

`__version__`

14.3 Classes

<i>Execute</i>	Executor class for training, retraining and evaluating KGE models.
<i>KGE</i>	Knowledge Graph Embedding Class for interactive usage of pre-trained models
<i>QueryGenerator</i>	
<i>DICE_Trainer</i>	DICE_Trainer implement
<i>Evaluator</i>	Evaluator class for KGE models in various downstream tasks.

14.4 Package Contents

class `dicee.Execute` (*args*, *continuous_training*: *bool = False*)

Executor class for training, retraining and evaluating KGE models.

Handles the complete workflow: 1. Loading & Preprocessing & Serializing input data 2. Training & Validation & Testing 3. Storing all necessary information

args

Processed input arguments.

distributed

Whether distributed training is enabled.

rank

Process rank in distributed training.

world_size

Total number of processes.

local_rank

Local GPU rank.

trainer

Training handler instance.

trained_model

The trained model after training completes.

knowledge_graph

The loaded knowledge graph.

report

Dictionary storing training metrics and results.

evaluator

Model evaluation handler.

distributed

rank

world_size

local_rank

args


```

is_continual_training = False

trainer: dicce.trainer.DICE_Trainer | None = None

trained_model = None

knowledge_graph: dicce.knowledge_graph.KG | None = None

report: Dict

evaluator: dicce.evaluator.Evaluator | None = None

start_time: float | None = None

is_local_rank_zero() → bool

is_global_rank_zero() → bool

cleanup()

setup_executor() → None
    Set up storage directories for the experiment.

    Creates or reuses experiment directories based on configuration. Saves the configuration to a JSON file.

create_and_store_kg() → None
    Create knowledge graph and store as memory-mapped file.

    Only executed on global rank 0 in distributed training, so exactly one process writes the shared memmap/vocab
    files regardless of how many nodes are involved. Skips if memmap already exists.

load_from_memmap() → None
    Load knowledge graph from memory-mapped file.

save_trained_model() → None
    Save a knowledge graph embedding model

    (1) Send model to eval mode and cpu.
    (2) Store the memory footprint of the model.
    (3) Save the model into disk.
    (4) Update the stats of KG again ?

```

Parameter

rtype
None

```

end(form_of_labelling: str) → dict
    End training

    (1) Store trained model.
    (2) Report runtimes.
    (3) Eval model if required.

```

Parameter

rtype

A dict containing information about the training and/or evaluation

write_report () → None

Report training related information in a report.json file

start () → dict

Start training

(1) Loading the Data # (2) Create an evaluator object. # (3) Create a trainer object. # (4) Start the training

Parameter

rtype

A dict containing information about the training and/or evaluation

class dicee.KGE (*path=None, url=None, construct_ensemble=False, model_name=None*)

Bases: *dicee.abstracts.BaseInteractiveKGE, dicee.abstracts.InteractiveQueryDecomposition, dicee.abstracts.BaseInteractiveTrainKGE*

Knowledge Graph Embedding Class for interactive usage of pre-trained models

__str__ ()

to (*device: str*) → None

get_transductive_entity_embeddings (*indices: torch.LongTensor | List[str], as_pytorch=False, as_numpy=False, as_list=True*) → *torch.FloatTensor | numpy.ndarray | List[float]*

create_vector_database (*collection_name: str, distance: str, location: str = 'localhost', port: int = 6333*)

generate (*h="", r=""*)

eval_lp_performance (*dataset=List[Tuple[str, str, str]], filtered=True, *, tie_policy=None, tie_seed=None*)

Evaluate head/tail ranks, inheriting saved tie settings unless overridden.

tie_policy accepts sort (legacy), optimistic, random, or pessimistic. *tie_seed* seeds an independent random stream for each evaluation.

predict_missing_head_entity (*relation: List[str] | str, tail_entity: List[str] | str, within=None, batch_size=2, topk=1, return_indices=False*) → *Tuple*

Given a relation and a tail entity, return top k ranked head entity.

$\text{argmax}_{\{e \in E\}} f(e, r, t)$, where $r \in R, t \in E$.

Parameter

relation: Union[List[str], str]

String representation of selected relations.

tail_entity: Union[List[str], str]

String representation of selected entities.

k: int

Highest ranked k entities.

Returns: Tuple

Highest K scores and entities

predict_missing_relations (*head_entity: List[str] | str, tail_entity: List[str] | str, within=None, batch_size=2, topk=1, return_indices=False*) → Tuple

Given a head entity and a tail entity, return top k ranked relations.

$\text{argmax}_{\{r \in R\}} f(h,r,t)$, where $h, t \in E$.

Parameter

head_entity: List[str]

String representation of selected entities.

tail_entity: List[str]

String representation of selected entities.

k: int

Highest ranked k entities.

Returns: Tuple

Highest K scores and entities

predict_missing_tail_entity (*head_entity: List[str] | str, relation: List[str] | str, within: List[str] = None, batch_size=2, topk=1, return_indices=False*) → torch.FloatTensor

Given a head entity and a relation, return top k ranked entities

$\text{argmax}_{\{e \in E\}} f(h,r,e)$, where $h \in E$ and $r \in R$.

Parameter

head_entity: List[str]

String representation of selected entities.

tail_entity: List[str]

String representation of selected entities.

Returns: Tuple

scores

predict (*, *h: List[str] | str | None = None, r: List[str] | str | None = None, t: List[str] | str | None = None, within: List[str] | None = None, logits: bool = True*) → torch.FloatTensor

Predict scores for triples or missing triple elements.

Parameters

- **h** – Head entity/entities. None to predict heads.
- **r** – Relation/relations. None to predict relations.
- **t** – Tail entity/entities. None to predict tails.
- **within** – Optional list of entities to restrict predictions to.
- **logits** – If True, return raw scores. If False, return sigmoid scores (0-1).

Returns

- Single triple (h, r, t): scalar score
- Missing element: vector of all possible scores

Return type

torch.FloatTensor of scores. Shape depends on the query type

Raises

- **TypeError** – If inputs are not strings or lists of strings.
- **ValueError** – If a required argument for the query type is missing.

Examples

```
>>> # Score a specific triple
>>> model.predict(h="Mongolia", r="isLocatedIn", t="Asia", logits=False)
tensor(0.9523)
```

```
>>> # Get scores for all possible tail entities
>>> model.predict(h="Mongolia", r="isLocatedIn", t=None)
tensor([0.21, 0.95, 0.03, ...]) # One score per entity
```

predict_topk (*, h: str | List[str] | None = None, r: str | List[str] | None = None, t: str | List[str] | None = None, topk: int = 10, within: List[str] | None = None, batch_size: int = 1024) → List[Tuple[str, float]] | List[List[Tuple[str, float]]]

Predict top-k missing items in a given triple pattern.

Parameters

- **h** – Head entity/entities. None to predict heads.
- **r** – Relation/relations. None to predict relations.
- **t** – Tail entity/entities. None to predict tails.
- **topk** – Number of top predictions to return.
- **within** – Optional list of entities to restrict predictions to.
- **batch_size** – Batch size for processing multiple queries.

Returns

List[(item, score), ...] of length topk. For batch query: List of such lists, one per query.

Return type

For single query

Raises

- **TypeError** – If h, r, or t is not a str or list of str.
- **ValueError** – If the required arguments for a query type are None.

Examples

```
>>> model.predict_topk(h=["Mongolia"], r=["isLocatedIn"], topk=3)
[('Asia', 0.99), ('Europe', 0.02), ...]
```

```
>>> model.predict_topk(r=["isLocatedIn"], t=["Asia"], topk=5)
[('Mongolia', 0.85), ('China', 0.82), ...]
```

triple_score (*h: List[str] | str = None, r: List[str] | str = None, t: List[str] | str = None, logits=False*)
→ torch.FloatTensor
Predict triple score

Parameter

head_entity: List[str]
String representation of selected entities.

relation: List[str]
String representation of selected relations.

tail_entity: List[str]
String representation of selected entities.

logits: bool
If logits is True, unnormalized score returned

Returns: Tuple

pytorch tensor of triple score

return_multi_hop_query_results (*aggregated_query_for_all_entities, k: int, only_scores*)

single_hop_query_answering (*query: tuple, only_scores: bool = True, k: int = None, use_logits: bool = True*)

answer_multi_hop_query (*query_type: str | None = None, query: Tuple[str | Tuple[str, str], ...] | None = None, queries: List[Tuple[str | Tuple[str, str], ...]] | None = None, tnorm: str = 'prod', neg_norm: str = 'standard', lambda_: float = 0.0, k: int = 10, only_scores: bool = False, use_logits: bool = True*)
→ List[Tuple[str, torch.Tensor]] | List[List[Tuple[str, torch.Tensor]]]

Answer multi-hop EPFO (Existential Positive First-Order) queries.

Supports 9 query types: 1p, 2p, 3p, 2i, 3i, ip, pi, 2u, up. See docs/guides/multi_hop_queries.md for detailed query patterns.

Parameters

- **query_type** – Query pattern name. One of: - 1p: (e, (r,)) # One-hop - 2p: (e, (r1, r2)) # Two-hop - 3p: (e, (r1, r2, r3)) # Three-hop - 2i: ((e1, (r1,)), (e2, (r2,))) # Two-way intersection - 3i: ((e1, (r1,)), (e2, (r2,)), (e3, (r3,))) # Three-way intersection - ip: (((e1, (r1,)), (e2, (r2,))), (r3,)) # Intersection + projection - pi: ((e, (r1, r2)), (r3,)) # Projection + intersection (2i meets 2p) - 2u: ((e1, (r1,)), (e2, (r2,))) # Two-way union - up: ((e, (r1, r2)), (e, (r3,))) # Union + projection
- **query** – Single query tuple matching the query_type pattern.
- **queries** – Batch of queries. If provided, query must be None.
- **tnorm** – T-norm for intersection/union. Options: “prod”, “min”.
- **neg_norm** – Negation norm. Options: “standard”, “sugeno”, “yager”.
- **lambda** – Parameter for sugeno and yager negation (0.0-1.0).
- **k** – Number of top answer entities to return.
- **only_scores** – If True, return only scores tensor. If False, return (entity, score) tuples.

- **use_logits** – If True, use raw model logits. If False, use sigmoid probabilities.

Returns

List[(entity, score), ...] of top-k answers. For batch queries: List of such lists, one per query.

Return type

For single query

Raises

- **ValueError** – If query_type is not in {1p, 2p, 3p, 2i, 3i, ip, pi, 2u, up}.
- **AssertionError** – If query structure doesn't match query_type pattern.

Examples

```
>>> # 1p: Find entities located in Asia
>>> model.answer_multi_hop_query(
...     query_type="1p",
...     query=("Asia", ("isLocatedIn",)),
...     k=5
... )
[("Mongolia", 0.92), ("China", 0.89), ...]
```

```
>>> # 2p: Two-hop query (e.g., "capital of countries in Europe")
>>> model.answer_multi_hop_query(
...     query_type="2p",
...     query=("Europe", ("isLocatedIn", "hasCapital")),
...     k=3
... )
[("Paris", 0.85), ("Berlin", 0.82), ...]
```

```
>>> # 2i: Intersection query
>>> model.answer_multi_hop_query(
...     query_type="2i",
...     query=((("Asia", ("isLocatedIn",)), ("Mountains", ("hasGeography",))),
...     k=5
... )
[("Nepal", 0.78), ("Tibet", 0.65), ...]
```

➡ See also

- docs/guides/multi_hop_queries.md: Complete guide with all query patterns
- tests/test_answer_multi_hop_query.py: Usage examples

find_missing_triples (confidence: float, entities: List[str] = None, relations: List[str] = None, topk: int = 10, at_most: int = sys.maxsize) → Set

Find missing triples

Iterative over a set of entities E and a set of relation R :

orall e in E and orall r in R f(e,r,x)

Return (e,r,x)

otin G and $f(e,r,x) > \text{confidence}$

confidence: float

A threshold for an output of a sigmoid function given a triple.

topk: int

Highest ranked k item to select triples with $f(e,r,x) > \text{confidence}$.

at_most: int

Stop after finding at_most missing triples

$\{(e,r,x) \mid f(e,r,x) > \text{confidence} \text{ and } (e,r,x)$

otin G

predict_literals (*entity: List[str] | str = None, attribute: List[str] | str = None, denormalize_preds: bool = True*) $\rightarrow$ numpy.ndarray

Predicts literal values for given entities and attributes.

Parameters

- **entity** (*Union[List[str], str]*) – Entity or list of entities to predict literals for.
- **attribute** (*Union[List[str], str]*) – Attribute or list of attributes to predict literals for.
- **denormalize_preds** (*bool*) – If True, denormalizes the predictions.

Returns

Predictions for the given entities and attributes.

Return type

numpy ndarray

class dicee.**QueryGenerator** (*train_path: str, val_path: str, test_path: str, ent2id: Dict = None, rel2id: Dict = None, seed: int = 1, gen_valid: bool = False, gen_test: bool = True*)

train_path

val_path

test_path

gen_valid = False

gen_test = True

seed = 1

max_ans_num = 1000000.0

mode

ent2id = None

rel2id: Dict = None

ent_in: Dict

ent_out: Dict

```

query_name_to_struct

list2tuple (list_data)

tuple2list (x: List | Tuple) → List | Tuple
    Convert a nested tuple to a nested list.

set_global_seed (seed: int)
    Set seed

construct_graph (paths: List[str]) → Tuple[Dict, Dict]
    Construct graph from triples Returns dicts with incoming and outgoing edges

fill_query (query_structure: List[str | List], ent_in: Dict, ent_out: Dict, answer: int) → bool
    Private method for fill_query logic.

achieve_answer (query: List[str | List], ent_in: Dict, ent_out: Dict) → set
    Private method for achieve_answer logic. @TODO: Document the code

write_links (ent_out, small_ent_out)

ground_queries (query_structure: List[str | List], ent_in: Dict, ent_out: Dict, small_ent_in: Dict,
    small_ent_out: Dict, gen_num: int, query_name: str)
    Generating queries and achieving answers

unmap (query_type, queries, tp_answers, fp_answers, fn_answers)

unmap_query (query_structure, query, id2ent, id2rel)

generate_queries (query_struct: List, gen_num: int, query_type: str)
    Passing incoming and outgoing edges to ground queries depending on mode [train valid or text] and getting
    queries and answers in return @ TODO: create a class for each single query struct

save_queries (query_type: str, gen_num: int, save_path: str)

abstractmethod load_queries (path)

get_queries (query_type: str, gen_num: int)

static save_queries_and_answers (path: str, data: List[Tuple[str, Tuple[collections.defaultdict]]])
    → None
    Save Queries into Disk

static load_queries_and_answers (path: str) → List[Tuple[str, Tuple[collections.defaultdict]]]
    Load Queries from Disk to Memory

class dicee.DICE_Trainer (args, is_continual_training: bool, storage_path, evaluator=None)

DICE_Trainer implement
    1- Pytorch Lightning trainer (https://pytorch-lightning.readthedocs.io/en/stable/common/trainer.html)
    2- Multi-GPU Trainer(https://pytorch.org/docs/stable/generated/torch.nn.parallel.DistributedDataParallel.html)
    3- CPU Trainer

    args
    is_continual_training:bool
    storage_path:str
    evaluator:
    report:dict

```


report

args

trainer = None

is_continual_training

storage_path

evaluator = None

form_of_labelling = None

continual_start (*knowledge_graph*)

- (1) Initialize training.
- (2) Load model
- (3) Load trainer (3) Fit model

Parameter

returns

- *model*
- **form_of_labelling** (*str*)

initialize_trainer (*callbacks: List*)

→ `lightning.Trainer` | `dicke.trainer.model_parallelism.TensorParallel` | `dicke.trainer.torch_trainer.TorchTrainer` | `dicke.t`

Initialize Trainer from input arguments

initialize_or_load_model ()

init_dataloader (*dataset: torch.utils.data.Dataset*) → `torch.utils.data.DataLoader`

init_dataset () → `torch.utils.data.Dataset`

prepare_graph_model (*model, knowledge_graph*)

Attach only training facts, preserving the indexed external vocabulary.

prepare_ultra

start (*knowledge_graph: dicke.knowledge_graph.KG* | `numpy.memmap`)

→ `Tuple[dicke.models.base_model.BaseKGE, str]`

Start the training

- (1) Initialize Trainer
- (2) Initialize or load a pretrained KGE model

in DDP setup, we need to load the memory map of already read/index KG.

k_fold_cross_validation (*dataset*) → `Tuple[dicke.models.base_model.BaseKGE, str]`

Perform K-fold Cross-Validation

1. Obtain K train and test splits.
2. **For each split,**
 - 2.1 initialize trainer and model
 - 2.2. Train model with configuration provided in args.
 - 2.3. Compute the mean reciprocal rank (MRR) score of the model on the test respective split.

3. Report the mean and average MRR .

Parameters

- **self**
- **dataset**

Returns

model

```
class dicee.Evaluator(args, is_continual_training: bool = False)
```

Evaluator class for KGE models in various downstream tasks.

Orchestrates link prediction evaluation with different scoring techniques including standard evaluation and byte-pair encoding based evaluation.

er_vocab

Entity-relation to tail vocabulary for filtered ranking.

re_vocab

Relation-entity (tail) to head vocabulary.

ee_vocab

Entity-entity to relation vocabulary.

num_entities

Total number of entities in the knowledge graph.

num_relations

Total number of relations in the knowledge graph.

args

Configuration arguments.

report

Dictionary storing evaluation results.

during_training

Whether evaluation is happening during training.

Example

```
>>> from dicee.evaluation import Evaluator
>>> evaluator = Evaluator(args)
>>> results = evaluator.eval(dataset, model, 'EntityPrediction')
>>> print(f"Test MRR: {results['Test']['MRR']:.4f}")
```

re_vocab: Dict | None = None

er_vocab: Dict | None = None

ee_vocab: Dict | None = None

func_triple_to_bpe_representation = None

is_continual_training = False

```

num_entities: int | None = None

num_relations: int | None = None

domain_constraints_per_rel = None

range_constraints_per_rel = None

args

report: Dict

during_training = False

```

vocab_preparation (*dataset*) → None

Prepare vocabularies from the dataset for evaluation.

Resolves any future objects and saves vocabularies to disk.

Parameters

dataset – Knowledge graph dataset with vocabulary attributes.

eval (*dataset, trained_model, form_of_labelling: str, during_training: bool = False*) → Dict | None

Evaluate the trained model on the dataset.

Parameters

- **dataset** – Knowledge graph dataset (KG instance).
- **trained_model** – The trained KGE model.
- **form_of_labelling** – Type of labelling ('EntityPrediction' or 'RelationPrediction').
- **during_training** – Whether evaluation is during training.

Returns

Dictionary of evaluation metrics, or None if evaluation is skipped.

eval_rank_of_head_and_tail_entity (*, *train_set, valid_set=None, test_set=None, trained_model*) → None

Evaluate with negative sampling scoring.

eval_rank_of_head_and_tail_byte_pair_encoded_entity (*, *train_set=None, valid_set=None, test_set=None, ordered_bpe_entities, trained_model*) → None

Evaluate with BPE-encoded entities and negative sampling.

eval_with_byte (*, *raw_train_set, raw_valid_set=None, raw_test_set=None, trained_model, form_of_labelling*) → None

Evaluate Byte model with generation.

eval_with_bpe_vs_all (*, *raw_train_set, raw_valid_set=None, raw_test_set=None, trained_model, form_of_labelling*) → None

Evaluate with BPE and KvsAll scoring.

eval_with_vs_all (*, *train_set, valid_set=None, test_set=None, trained_model, form_of_labelling*) → None

Evaluate with KvsAll or 1vsAll scoring.

evaluate_lp_k_vs_all (*model, triple_idx, info: str | None = None, form_of_labelling: str | None = None*) → Dict[str, float]

Filtered link prediction evaluation with KvsAll scoring.

Parameters

- **model** – The trained model to evaluate.
- **triple_idx** – Integer-indexed test triples.
- **info** – Description to print.
- **form_of_labelling** – ‘EntityPrediction’ or ‘RelationPrediction’.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

evaluate_lp_with_byte (*model, triples: List[List[str]], info: str | None = None*) → Dict[str, float]

Evaluate Byte model with text generation.

Parameters

- **model** – Byte model.
- **triples** – String triples.
- **info** – Description to print.

Returns

Dictionary with placeholder metrics (-1 values).

evaluate_lp_bpe_k_vs_all (*model, triples: List[List[str]], info: str | None = None, form_of_labelling: str | None = None*) → Dict[str, float]

Evaluate BPE model with KvsAll scoring.

Parameters

- **model** – BPE-enabled model.
- **triples** – String triples.
- **info** – Description to print.
- **form_of_labelling** – Type of labelling.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

evaluate_lp (*model, triple_idx, info: str*) → Dict[str, float]

Evaluate link prediction with negative sampling.

Parameters

- **model** – The model to evaluate.
- **triple_idx** – Integer-indexed triples.
- **info** – Description to print.

Returns

Dictionary with H@1, H@3, **H@10**, and MRR metrics.

dummy_eval (*trained_model, form_of_labelling: str*) → None

Run evaluation from saved data (for continual training).

Parameters

- **trained_model** – The trained model.
- **form_of_labelling** – Type of labelling.

eval_with_data (*dataset, trained_model, triple_idx: numpy.ndarray, form_of_labelling: str*)
→ Dict[str, float]

Evaluate a trained model on a given dataset.

Parameters

- **dataset** – Knowledge graph dataset.
- **trained_model** – The trained model.
- **triple_idx** – Integer-indexed triples to evaluate.
- **form_of_labelling** – Type of labelling.

Returns

Dictionary with evaluation metrics.

Raises

ValueError – If scoring technique is invalid.

`dicee.__version__ = '0.3.3'`

Python Module Index

d

- `dicee`, 12
- `dicee.__main__`, 12
- `dicee.abstracts`, 12
- `dicee.analyse_experiments`, 20
- `dicee.callbacks`, 22
- `dicee.config`, 28
- `dicee.dataset_classes`, 33
- `dicee.eval_static_funcs`, 42
- `dicee.evaluation`, 46
 - `dicee.evaluation.ensemble`, 47
 - `dicee.evaluation.evaluator`, 48
 - `dicee.evaluation.link_prediction`, 51
 - `dicee.evaluation.literal_prediction`, 55
 - `dicee.evaluation.utils`, 56
- `dicee.evaluator`, 66
- `dicee.executer`, 70
- `dicee.knowledge_graph`, 73
- `dicee.knowledge_graph_embeddings`, 75
- `dicee.losses`, 81
 - `dicee.losses.custom_losses`, 81
- `dicee.models`, 90
 - `dicee.models.adopt`, 90
 - `dicee.models.base_model`, 99
 - `dicee.models.clifford`, 108
 - `dicee.models.complex`, 117
 - `dicee.models.dualE`, 122
 - `dicee.models.ensemble`, 123
 - `dicee.models.flock`, 124
 - `dicee.models.flock_walks`, 129
 - `dicee.models.fsdp_models`, 130
 - `dicee.models.function_space`, 132
 - `dicee.models.graph_model`, 136
 - `dicee.models.literal`, 140
 - `dicee.models.octonion`, 142
 - `dicee.models.pykeen_models`, 144
 - `dicee.models.quaternion`, 146
 - `dicee.models.real`, 149
 - `dicee.models.sampled_loss`, 157
 - `dicee.models.static_funcs`, 157
 - `dicee.models.transformers`, 157
 - `dicee.models.trix`, 164
 - `dicee.models.ultra`, 167
- `dicee.query_generator`, 252
- `dicee.read_preprocess_save_load_kg`, 253
 - `dicee.read_preprocess_save_load_kg.preprocess`, 253
 - `dicee.read_preprocess_save_load_kg.read_from_disk`, 254
 - `dicee.read_preprocess_save_load_kg.save_load_disk`, 255
 - `dicee.read_preprocess_save_load_kg.util`, 255
- `dicee.sanity_checkers`, 260
- `dicee.scripts`, 261
 - `dicee.scripts.index_serve`, 261
 - `dicee.scripts.run`, 262
- `dicee.static_funcs`, 262
 - `dicee.static_funcs_training`, 267
 - `dicee.static_preprocess_funcs`, 269
- `dicee.trainer`, 270
 - `dicee.trainer.auto_batch_finder`, 270
 - `dicee.trainer.dice_trainer`, 271
 - `dicee.trainer.model_parallelism`, 273
 - `dicee.trainer.torch_trainer`, 274
 - `dicee.trainer.torch_trainer_ddp`, 276
 - `dicee.trainer.torch_trainer_fsdp`, 277
 - `dicee.weight_averaging`, 280

Index

Non-alphabetical

`__call__` () (*dicee.models.base_model.IdentityClass method*), 108
`__call__` () (*dicee.models.ensemble.EnsembleKGE method*), 123
`__call__` () (*dicee.models.IdentityClass method*), 192, 218, 226
`__getitem__` () (*dicee.dataset_classes.AllvsAll method*), 37
`__getitem__` () (*dicee.dataset_classes.BPE_NegativeSamplingDataset method*), 35
`__getitem__` () (*dicee.dataset_classes.FixedNegSampleDataset method*), 41
`__getitem__` () (*dicee.dataset_classes.FSDP1vsSampleDataset method*), 37
`__getitem__` () (*dicee.dataset_classes.KvsAll method*), 38
`__getitem__` () (*dicee.dataset_classes.KvsSampleDataset method*), 39
`__getitem__` () (*dicee.dataset_classes.LiteralDataset method*), 40
`__getitem__` () (*dicee.dataset_classes.MultiClassClassificationDataset method*), 35
`__getitem__` () (*dicee.dataset_classes.MultiLabelDataset method*), 36
`__getitem__` () (*dicee.dataset_classes.OnevsAllDataset method*), 39
`__getitem__` () (*dicee.dataset_classes.OnevsSample method*), 41
`__getitem__` () (*dicee.dataset_classes.TriplePredictionDataset method*), 42
`__iter__` () (*dicee.config.Namespace method*), 33
`__iter__` () (*dicee.knowledge_graph.KG method*), 75
`__iter__` () (*dicee.models.ensemble.EnsembleKGE method*), 123
`__len__` () (*dicee.dataset_classes.AllvsAll method*), 37
`__len__` () (*dicee.dataset_classes.BPE_NegativeSamplingDataset method*), 35
`__len__` () (*dicee.dataset_classes.FixedNegSampleDataset method*), 41
`__len__` () (*dicee.dataset_classes.FSDP1vsSampleDataset method*), 37
`__len__` () (*dicee.dataset_classes.KvsAll method*), 38
`__len__` () (*dicee.dataset_classes.KvsSampleDataset method*), 39
`__len__` () (*dicee.dataset_classes.LiteralDataset method*), 40
`__len__` () (*dicee.dataset_classes.MultiClassClassificationDataset method*), 35
`__len__` () (*dicee.dataset_classes.MultiLabelDataset method*), 36
`__len__` () (*dicee.dataset_classes.OnevsAllDataset method*), 39
`__len__` () (*dicee.dataset_classes.OnevsSample method*), 41
`__len__` () (*dicee.dataset_classes.TriplePredictionDataset method*), 42
`__len__` () (*dicee.knowledge_graph.KG method*), 75
`__len__` () (*dicee.models.ensemble.EnsembleKGE method*), 123
`__post_init__` () (*dicee.models.flock_walks.WalkGraph method*), 129
`__setstate__` () (*dicee.models.ADOPT method*), 182
`__setstate__` () (*dicee.models.adopt.ADOPT method*), 94
`__str__` () (*dicee.KGE method*), 290
`__str__` () (*dicee.knowledge_graph_embeddings.KGE method*), 76
`__str__` () (*dicee.models.ensemble.EnsembleKGE method*), 123
`__version__` (*in module dicee*), 301

A

`AbstractCallback` (*class in dicee.abstracts*), 18
`AbstractPPECallback` (*class in dicee.abstracts*), 19
`AbstractTrainer` (*class in dicee.abstracts*), 13
`AccumulateEpochLossCallback` (*class in dicee.callbacks*), 23
`achieve_answer` () (*dicee.query_generator.QueryGenerator method*), 253
`achieve_answer` () (*dicee.QueryGenerator method*), 296
`ACLS` (*class in dicee.losses.custom_losses*), 89
`ACLS` (*class in dicee.models*), 172
`AConEx` (*class in dicee.models*), 210
`AConEx` (*class in dicee.models.complex*), 118
`AConvO` (*class in dicee.models*), 227
`AConvO` (*class in dicee.models.octonion*), 143
`AConvQ` (*class in dicee.models*), 220
`AConvQ` (*class in dicee.models.quaternion*), 148
`adaptive_label_relaxation` (*dicee.losses.custom_losses.CombinedAdaptiveLSandAdaptiveLR attribute*), 89
`adaptive_label_relaxation` (*dicee.models.CombinedAdaptiveLSandAdaptiveLR attribute*), 176
`adaptive_label_smoothing` (*dicee.losses.custom_losses.CombinedAdaptiveLSandAdaptiveLR attribute*), 89
`adaptive_label_smoothing` (*dicee.models.CombinedAdaptiveLSandAdaptiveLR attribute*), 176
`adaptive_lr` (*dicee.config.Namespace attribute*), 33
`adaptive_swa` (*dicee.config.Namespace attribute*), 32
`AdaptiveLabelRelaxationLoss` (*class in dicee.losses.custom_losses*), 85
`AdaptiveLabelRelaxationLoss` (*class in dicee.models*), 173
`AdaptiveLabelSmoothingLoss` (*class in dicee.losses.custom_losses*), 84

AdaptiveLabelSmoothingLoss (class in *dicее.models*), 174
 add_new_entity_embeddings() (*dicее.abstracts.BaseInteractiveKGE* method), 16
 add_noise_rate (*dicее.config.Namespace* attribute), 30
 add_noise_rate (*dicее.knowledge_graph.KG* attribute), 74
 add_noisy_triples() (in module *dicее.static_funcs*), 266
 add_noisy_triples_into_training() (*dicее.read_preprocess_save_load_kg.read_from_disk.ReadFromDisk* method), 255
 add_noisy_triples_into_training() (*dicее.read_preprocess_save_load_kg.ReadFromDisk* method), 260
 add_reciprocal (*dicее.knowledge_graph.KG* attribute), 74
 ADOPT (class in *dicее.models*), 180
 ADOPT (class in *dicее.models.adopt*), 92
 adopt() (in module *dicее.models.adopt*), 95
 adversarial_temperature (*dicее.config.Namespace* attribute), 29
 AggregatedLSandLR (class in *dicее.losses.custom_losses*), 89
 AggregatedLSandLR (class in *dicее.models*), 175
 ALL_HITS_RANGE (in module *dicее.evaluation.utils*), 57
 AllvsAll (class in *dicее.dataset_classes*), 36
 alpha (*dicее.losses.custom_losses.ACLS* attribute), 90
 alpha (*dicее.losses.custom_losses.AdaptiveLabelRelaxationLoss* attribute), 86
 alpha (*dicее.losses.custom_losses.AggregatedLSandLR* attribute), 89
 alpha (*dicее.losses.custom_losses.CombinedLSandLR* attribute), 88
 alpha (*dicее.losses.custom_losses.ConfidenceBasedAdaptiveLabelRelaxationLoss* attribute), 87
 alpha (*dicее.losses.custom_losses.LabelRelaxationLoss* attribute), 85
 alpha (*dicее.models.ACLS* attribute), 173
 alpha (*dicее.models.AdaptiveLabelRelaxationLoss* attribute), 174
 alpha (*dicее.models.AggregatedLSandLR* attribute), 176
 alpha (*dicее.models.CombinedLSandLR* attribute), 177
 alpha (*dicее.models.ConfidenceBasedAdaptiveLabelRelaxationLoss* attribute), 178
 alpha (*dicее.models.LabelRelaxationLoss* attribute), 179
 alpha_step (*dicее.losses.custom_losses.AdaptiveLabelRelaxationLoss* attribute), 86
 alpha_step (*dicее.models.AdaptiveLabelRelaxationLoss* attribute), 174
 alphas (*dicее.abstracts.AbstractPPECallback* attribute), 19
 alphas (*dicее.weight_averaging.ASWA* attribute), 283
 AMWA (class in *dicее.weight_averaging*), 280
 amwa (*dicее.config.Namespace* attribute), 33
 amwa_beta (*dicее.config.Namespace* attribute), 33
 amwa_beta_floor (*dicее.config.Namespace* attribute), 33
 amwa_beta_init (*dicее.config.Namespace* attribute), 33
 amwa_beta_window (*dicее.config.Namespace* attribute), 33
 amwa_c_epochs (*dicее.config.Namespace* attribute), 33
 amwa_c_epochs (*dicее.weight_averaging.AMWA* attribute), 282
 amwa_maximize (*dicее.config.Namespace* attribute), 33
 amwa_monitor (*dicее.config.Namespace* attribute), 33
 amwa_start_epoch (*dicее.config.Namespace* attribute), 33
 amwa_start_epoch (*dicее.weight_averaging.AMWA* attribute), 282
 analyse() (in module *dicее.analyse_experiments*), 22
 anonymize() (in module *dicее.models.flock_walks*), 129
 answer_multi_hop_query() (*dicее.KGE* method), 293
 answer_multi_hop_query() (*dicее.knowledge_graph_embeddings.KGE* method), 79
 app (in module *dicее.scripts.index_serve*), 262
 apply_coefficients() (*dicее.models.clifford.DeCaL* method), 115
 apply_coefficients() (*dicее.models.clifford.Keci* method), 110
 apply_coefficients() (*dicее.models.DeCaL* method), 233
 apply_coefficients() (*dicее.models.Keci* method), 229
 apply_reciprocal_or_noise() (in module *dicее.read_preprocess_save_load_kg.util*), 258
 apply_semantic_constraint (*dicее.abstracts.BaseInteractiveKGE* attribute), 14
 apply_unit_norm (*dicее.models.base_model.BaseKGE* attribute), 104
 apply_unit_norm (*dicее.models.BaseKGE* attribute), 188, 193, 205, 214, 221, 236, 241
 args (*dicее.DICE_Trainer* attribute), 297
 args (*dicее.evaluation.Evaluator* attribute), 60
 args (*dicее.evaluation.evaluator.Evaluator* attribute), 49
 args (*dicее.Evaluator* attribute), 298, 299
 args (*dicее.evaluator.Evaluator* attribute), 67, 68
 args (*dicее.Execute* attribute), 288
 args (*dicее.executer.Execute* attribute), 70, 71
 args (*dicее.models.base_model.BaseKGE* attribute), 104
 args (*dicее.models.base_model.BaseKGELightning* attribute), 100
 args (*dicее.models.base_model.IdentityClass* attribute), 108
 args (*dicее.models.BaseKGE* attribute), 188, 193, 205, 214, 221, 236, 241

- args (*diccee.models.BaseKGELighting* attribute), 184
- args (*diccee.models.ensemble.EnsembleKGE* attribute), 123
- args (*diccee.models.IdentityClass* attribute), 192, 218, 226
- args (*diccee.models.pykeen_models.PykeenKGE* attribute), 145
- args (*diccee.models.PykeenKGE* attribute), 240
- args (*diccee.trainer.DICE_Trainer* attribute), 279
- args (*diccee.trainer.dice_trainer.DICE_Trainer* attribute), 272
- ASWA (class in *diccee.weight_averaging*), 282
- aswa (*diccee.analyse_experiments.Experiment* attribute), 21
- attn (*diccee.models.Block* attribute), 193
- attn (*diccee.models.clifford.TransformerBlock* attribute), 113
- attn (*diccee.models.transformers.Block* attribute), 162
- attn_dropout (*diccee.models.clifford.TransformerSelfAttention* attribute), 113
- attn_dropout (*diccee.models.transformers.SelfAttention* attribute), 160
- attributes (*diccee.abstracts.AbstractTrainer* attribute), 13
- auto_batch_finding (*diccee.config.Namespace* attribute), 32

B

- b_h (*diccee.models.MuRE* attribute), 201
- b_h (*diccee.models.real.MuRE* attribute), 153
- b_t (*diccee.models.MuRE* attribute), 201
- b_t (*diccee.models.real.MuRE* attribute), 153
- backend (*diccee.config.Namespace* attribute), 30
- backend (*diccee.knowledge_graph.KG* attribute), 74
- base_history (*diccee.weight_averaging.AMWA* attribute), 282
- base_weights (*diccee.weight_averaging.TWA* attribute), 287
- BaseInteractiveKGE (class in *diccee.abstracts*), 14
- BaseInteractiveTrainKGE (class in *diccee.abstracts*), 20
- BaseKGE (class in *diccee.models*), 187, 193, 205, 214, 221, 235, 241
- BaseKGE (class in *diccee.models.base_model*), 103
- BaseKGELighting (class in *diccee.models.base_model*), 183
- BaseKGELighting (class in *diccee.models.base_model*), 99
- batch_kronecker_product () (*diccee.callbacks.KronE* static method), 26
- batch_size (*diccee.analyse_experiments.Experiment* attribute), 21
- batch_size (*diccee.callbacks.PseudoLabellingCallback* attribute), 25
- batch_size (*diccee.config.Namespace* attribute), 30
- batches_per_epoch (*diccee.callbacks.LRScheduler* attribute), 27
- beta (*diccee.weight_averaging.AMWA* attribute), 282
- beta (*diccee.weight_averaging.TWA* attribute), 287
- beta_floor (*diccee.weight_averaging.AMWA* attribute), 282
- beta_history (*diccee.weight_averaging.AMWA* attribute), 282
- beta_init (*diccee.weight_averaging.AMWA* attribute), 282
- beta_window (*diccee.weight_averaging.AMWA* attribute), 282
- bias (*diccee.models.CoKEConfig* attribute), 204
- bias (*diccee.models.real.CoKEConfig* attribute), 155, 156
- bias (*diccee.models.transformers.GPTConfig* attribute), 162
- bias (*diccee.models.transformers.LayerNorm* attribute), 159
- BidirectionalGRU (class in *diccee.models.flock*), 126
- Block (class in *diccee.models*), 192
- Block (class in *diccee.models.transformers*), 161
- block_size (*diccee.config.Namespace* attribute), 32
- block_size (*diccee.dataset_classes.MultiClassClassificationDataset* attribute), 35
- block_size (*diccee.models.base_model.BaseKGE* attribute), 105
- block_size (*diccee.models.BaseKGE* attribute), 189, 194, 206, 215, 222, 237, 242
- block_size (*diccee.models.CoKEConfig* attribute), 203, 204
- block_size (*diccee.models.real.CoKEConfig* attribute), 155, 156
- block_size (*diccee.models.transformers.GPTConfig* attribute), 162
- blocks (*diccee.models.CoKE* attribute), 204
- blocks (*diccee.models.real.CoKE* attribute), 156
- bn_conv1 (*diccee.models.AConvQ* attribute), 221
- bn_conv1 (*diccee.models.ConvQ* attribute), 220
- bn_conv1 (*diccee.models.quaternion.AConvQ* attribute), 148
- bn_conv1 (*diccee.models.quaternion.ConvQ* attribute), 147
- bn_conv2 (*diccee.models.AConvQ* attribute), 221
- bn_conv2 (*diccee.models.ConvQ* attribute), 220
- bn_conv2 (*diccee.models.quaternion.AConvQ* attribute), 148
- bn_conv2 (*diccee.models.quaternion.ConvQ* attribute), 147

bn_conv2d (*dicee.models.AConEx* attribute), 211
 bn_conv2d (*dicee.models.AConvO* attribute), 227
 bn_conv2d (*dicee.models.complex.AConEx* attribute), 118
 bn_conv2d (*dicee.models.complex.ConEx* attribute), 117
 bn_conv2d (*dicee.models.ConEx* attribute), 210
 bn_conv2d (*dicee.models.ConvO* attribute), 227
 bn_conv2d (*dicee.models.octonion.AConvO* attribute), 144
 bn_conv2d (*dicee.models.octonion.ConvO* attribute), 143
 BPE_NegativeSamplingDataset (*class in dicee.dataset_classes*), 34
 build_chain_funcs () (*dicee.models.FMult2* method), 247
 build_chain_funcs () (*dicee.models.function_space.FMult2* method), 134
 build_func () (*dicee.models.FMult2* method), 247
 build_func () (*dicee.models.function_space.FMult2* method), 134
 build_projection () (*dicee.weight_averaging.TWA* method), 287
 build_relation_graph () (*in module dicee.models.trix*), 164
 build_relation_graph () (*in module dicee.models.ultra*), 169
 Byte (*class in dicee.models.transformers*), 158
 byte_pair_encoding (*dicee.analyse_experiments.Experiment* attribute), 21
 byte_pair_encoding (*dicee.config.Namespace* attribute), 32
 byte_pair_encoding (*dicee.knowledge_graph.KG* attribute), 74
 byte_pair_encoding (*dicee.models.base_model.BaseKGE* attribute), 104
 byte_pair_encoding (*dicee.models.BaseKGE* attribute), 188, 194, 206, 215, 222, 236, 242

C

c_attn (*dicee.models.clifford.TransformerSelfAttention* attribute), 113
 c_attn (*dicee.models.transformers.SelfAttention* attribute), 160
 c_fc (*dicee.models.clifford.TransformerMLP* attribute), 113
 c_fc (*dicee.models.transformers.MLP* attribute), 161
 c_proj (*dicee.models.clifford.TransformerMLP* attribute), 113
 c_proj (*dicee.models.clifford.TransformerSelfAttention* attribute), 113
 c_proj (*dicee.models.transformers.MLP* attribute), 161
 c_proj (*dicee.models.transformers.SelfAttention* attribute), 160
 callbacks (*dicee.abstracts.AbstractTrainer* attribute), 13
 callbacks (*dicee.analyse_experiments.Experiment* attribute), 21
 callbacks (*dicee.config.Namespace* attribute), 30
 callbacks (*dicee.trainer.torch_trainer_ddp.NodeTrainer* attribute), 277
 causal (*dicee.models.CoKEConfig* attribute), 204
 causal (*dicee.models.real.CoKEConfig* attribute), 155, 156
 causal (*dicee.models.transformers.GPTConfig* attribute), 162
 causal (*dicee.models.transformers.SelfAttention* attribute), 160
 chain_func () (*dicee.models.FMult* method), 246
 chain_func () (*dicee.models.function_space.FMult* method), 133
 chain_func () (*dicee.models.function_space.GFMult* method), 133
 chain_func () (*dicee.models.GFMult* method), 246
 checkpoint_every_n_epochs (*dicee.trainer.torch_trainer_fsdp.TorchFSDPTrainer* attribute), 278
 checkpoint_hint (*dicee.models.flock.FlockBase* attribute), 127
 checkpoint_hint (*dicee.models.graph_model.GraphKGE* attribute), 137
 checkpoint_hint (*dicee.models.trix.TRIXBase* attribute), 166
 checkpoint_hint (*dicee.models.ULTRA* attribute), 250
 checkpoint_hint (*dicee.models.ultra.ULTRA* attribute), 170
 CKeci (*class in dicee.models*), 232
 CKeci (*class in dicee.models.clifford*), 113
 cl_pqr () (*dicee.models.clifford.DeCaL* method), 114
 cl_pqr () (*dicee.models.DeCaL* method), 232
 cleanup () (*dicee.Execute* method), 289
 cleanup () (*dicee.executer.Execute* method), 71
 clear_inference_cache () (*dicee.models.graph_model.GraphKGE* method), 137
 clear_inference_cache () (*dicee.models.TRIX* method), 251
 clear_inference_cache () (*dicee.models.trix.TRIX* method), 166
 clear_inference_cache () (*dicee.models.ULTRA* method), 251
 clear_inference_cache () (*dicee.models.ultra.ULTRA* method), 170
 clifford_multiplication () (*dicee.models.clifford.Keci* method), 110
 clifford_multiplication () (*dicee.models.Keci* method), 229
 clip_lambda (*dicee.models.ADOPT* attribute), 182
 clip_lambda (*dicee.models.adopt.ADOPT* attribute), 94
 CoKE (*class in dicee.models*), 204
 CoKE (*class in dicee.models.real*), 156

coke_dropout (*dicee.models.CoKE attribute*), 204
 coke_dropout (*dicee.models.real.CoKE attribute*), 156
 CoKEConfig (*class in dicee.models*), 203
 CoKEConfig (*class in dicee.models.real*), 155
 collate_fn (*dicee.dataset_classes.AllvsAll attribute*), 37
 collate_fn (*dicee.dataset_classes.FixedNegSampleDataset attribute*), 41
 collate_fn (*dicee.dataset_classes.FSDPIvsSampleDataset attribute*), 37
 collate_fn (*dicee.dataset_classes.KvsAll attribute*), 38
 collate_fn (*dicee.dataset_classes.KvsSampleDataset attribute*), 38
 collate_fn (*dicee.dataset_classes.MultiClassClassificationDataset attribute*), 35
 collate_fn (*dicee.dataset_classes.MultiLabelDataset attribute*), 36
 collate_fn (*dicee.dataset_classes.OnevsAllDataset attribute*), 39
 collate_fn (*dicee.dataset_classes.OnevsSample attribute*), 41
 collate_fn () (*dicee.dataset_classes.BPE_NegativeSamplingDataset method*), 35
 collate_fn () (*dicee.dataset_classes.TriplePredictionDataset method*), 42
 collection_name (*dicee.scripts.index_serve.NeuralSearcher attribute*), 262
 CombinedAdaptiveLSandAdaptiveLR (*class in dicee.losses.custom_losses*), 88
 CombinedAdaptiveLSandAdaptiveLR (*class in dicee.models*), 176
 CombinedLSandLR (*class in dicee.losses.custom_losses*), 87
 CombinedLSandLR (*class in dicee.models*), 176
 comp_func () (*dicee.models.function_space.LFMult method*), 136
 comp_func () (*dicee.models.LFMult method*), 249
 compact_state (*dicee.models.flock.FlockBase attribute*), 128
 compile_sampler (*dicee.models.flock_walks.WalkGraph attribute*), 129
 compile_sampler (*dicee.models.flock.FlockBase attribute*), 128
 compiled_walk_nodes () (*in module dicee.models.flock_walks*), 129
 ComplEx (*class in dicee.models*), 211
 ComplEx (*class in dicee.models.complex*), 119
 compute_convergence () (*in module dicee.callbacks*), 25
 compute_func () (*dicee.models.FMult method*), 246
 compute_func () (*dicee.models.FMult2 method*), 247
 compute_func () (*dicee.models.function_space.FMult method*), 133
 compute_func () (*dicee.models.function_space.FMult2 method*), 134
 compute_func () (*dicee.models.function_space.GFMult method*), 133
 compute_func () (*dicee.models.GFMult method*), 246
 compute_metrics_from_ranks () (*in module dicee.evaluation.utils*), 57
 compute_metrics_from_ranks_simple () (*in module dicee.evaluation.utils*), 57
 compute_mrr () (*dicee.weight_averaging.ASWA static method*), 283
 compute_sigma_pp () (*dicee.models.clifford.DeCaL method*), 115
 compute_sigma_pp () (*dicee.models.clifford.Keci method*), 109
 compute_sigma_pp () (*dicee.models.DeCaL method*), 233
 compute_sigma_pp () (*dicee.models.Keci method*), 228
 compute_sigma_pq () (*dicee.models.clifford.DeCaL method*), 116
 compute_sigma_pq () (*dicee.models.clifford.Keci method*), 110
 compute_sigma_pq () (*dicee.models.DeCaL method*), 234
 compute_sigma_pq () (*dicee.models.Keci method*), 229
 compute_sigma_pr () (*dicee.models.clifford.DeCaL method*), 116
 compute_sigma_pr () (*dicee.models.DeCaL method*), 235
 compute_sigma_qq () (*dicee.models.clifford.DeCaL method*), 116
 compute_sigma_qq () (*dicee.models.clifford.Keci method*), 109
 compute_sigma_qq () (*dicee.models.DeCaL method*), 234
 compute_sigma_qq () (*dicee.models.Keci method*), 229
 compute_sigma_qr () (*dicee.models.clifford.DeCaL method*), 117
 compute_sigma_qr () (*dicee.models.DeCaL method*), 235
 compute_sigma_rr () (*dicee.models.clifford.DeCaL method*), 116
 compute_sigma_rr () (*dicee.models.DeCaL method*), 234
 compute_sigmas_multivect () (*dicee.models.clifford.DeCaL method*), 115
 compute_sigmas_multivect () (*dicee.models.DeCaL method*), 233
 compute_sigmas_single () (*dicee.models.clifford.DeCaL method*), 114
 compute_sigmas_single () (*dicee.models.DeCaL method*), 232
 ConEx (*class in dicee.models*), 209
 ConEx (*class in dicee.models.complex*), 117
 ConfidenceBasedAdaptiveLabelRelaxationLoss (*class in dicee.losses.custom_losses*), 86
 ConfidenceBasedAdaptiveLabelRelaxationLoss (*class in dicee.models*), 177
 config (*dicee.models.CoKE attribute*), 204
 config (*dicee.models.real.CoKE attribute*), 156
 config (*dicee.models.transformers.BytE attribute*), 158
 config (*dicee.models.transformers.GPT attribute*), 163

config_prefix (*dicee.models.flock.FlockBase attribute*), 127
 config_prefix (*dicee.models.graph_model.GraphKGE attribute*), 137
 config_prefix (*dicee.models.trix.TRIXBase attribute*), 166
 config_prefix (*dicee.models.ULTRA attribute*), 250
 config_prefix (*dicee.models.ultra.ULTRA attribute*), 170
 configs (*dicee.abstracts.BaseInteractiveKGE attribute*), 14
 configure_optimizers() (*dicee.models.base_model.BaseKGE Lightning method*), 103
 configure_optimizers() (*dicee.models.BaseKGE Lightning method*), 187
 configure_optimizers() (*dicee.models.transformers.GPT method*), 163
 consensus() (*in module dicee.models.flock*), 127
 construct_batch_selected_cl_multivector() (*dicee.models.clifford.Keci method*), 111
 construct_batch_selected_cl_multivector() (*dicee.models.Keci method*), 231
 construct_cl_multivector() (*dicee.models.clifford.DeCaL method*), 115
 construct_cl_multivector() (*dicee.models.clifford.Keci method*), 110
 construct_cl_multivector() (*dicee.models.DeCaL method*), 233
 construct_cl_multivector() (*dicee.models.Keci method*), 230
 construct_dataset() (*in module dicee.dataset_classes*), 36
 construct_ensemble (*dicee.abstracts.BaseInteractiveKGE attribute*), 14
 construct_graph() (*dicee.query_generator.QueryGenerator method*), 252
 construct_graph() (*dicee.QueryGenerator method*), 296
 construct_input_and_output() (*dicee.abstracts.BaseInteractiveKGE method*), 17
 construct_multi_coeff() (*dicee.models.function_space.LFMMult method*), 135
 construct_multi_coeff() (*dicee.models.LFMMult method*), 248
 continual_learning (*dicee.config.Namespace attribute*), 32
 continual_start() (*dicee.DICE_Trainer method*), 297
 continual_start() (*dicee.executer.ContinuousExecute method*), 72
 continual_start() (*dicee.trainer.DICE_Trainer method*), 279
 continual_start() (*dicee.trainer.dice_trainer.DICE_Trainer method*), 272
 continual_training_setup_executor() (*in module dicee.static_funcs*), 267
 ContinuousExecute (*class in dicee.executer*), 72
 conv2d (*dicee.models.AConEx attribute*), 210
 conv2d (*dicee.models.AConvO attribute*), 227
 conv2d (*dicee.models.AConvQ attribute*), 220
 conv2d (*dicee.models.complex.AConEx attribute*), 118
 conv2d (*dicee.models.complex.ConEx attribute*), 117
 conv2d (*dicee.models.ConEx attribute*), 209
 conv2d (*dicee.models.ConvO attribute*), 227
 conv2d (*dicee.models.ConvQ attribute*), 220
 conv2d (*dicee.models.octonion.AConvO attribute*), 144
 conv2d (*dicee.models.octonion.ConvO attribute*), 143
 conv2d (*dicee.models.quaternion.AConvQ attribute*), 148
 conv2d (*dicee.models.quaternion.ConvQ attribute*), 147
 ConvO (*class in dicee.models*), 226
 ConvO (*class in dicee.models.octonion*), 142
 ConvQ (*class in dicee.models*), 220
 ConvQ (*class in dicee.models.quaternion*), 147
 count_triples() (*in module dicee.read_preprocess_save_load_kg.util*), 258
 create_and_store_kg() (*dicee.Execute method*), 289
 create_and_store_kg() (*dicee.executer.Execute method*), 71
 create_constraints() (*in module dicee.read_preprocess_save_load_kg.util*), 258
 create_constraints() (*in module dicee.static_preprocess_funcs*), 270
 create_experiment_folder() (*in module dicee.static_funcs*), 266
 create_fsdp_sharded_model_class() (*in module dicee.models.fsdp_models*), 132
 create_hits_dict() (*in module dicee.evaluation.utils*), 58
 create_random_data() (*dicee.callbacks.PseudoLabellingCallback method*), 25
 create_recipriocal_triples() (*in module dicee.read_preprocess_save_load_kg.util*), 259
 create_recipriocal_triples() (*in module dicee.static_funcs*), 264
 create_vector_database() (*dicee.KGE method*), 290
 create_vector_database() (*dicee.knowledge_graph_embeddings.KGE method*), 76
 criterion (*dicee.losses.custom_losses.CombinedAdaptiveLSandAdaptiveLR attribute*), 89
 criterion (*dicee.models.CombinedAdaptiveLSandAdaptiveLR attribute*), 176
 crop_block_size() (*dicee.models.transformers.GPT method*), 163
 cross_entropy (*dicee.losses.custom_losses.ACLS attribute*), 90
 cross_entropy (*dicee.models.ACLS attribute*), 173
 ctx (*dicee.trainer.torch_trainer_ddp.NodeTrainer attribute*), 277
 ctx (*dicee.trainer.torch_trainer_fsdp.TorchFSDPTrainer attribute*), 278
 current_epoch (*dicee.weight_averaging.AMWA attribute*), 282
 current_epoch (*dicee.weight_averaging.EMA attribute*), 286

`current_epoch` (*dicee.weight_averaging.SWA attribute*), 284
`current_epoch` (*dicee.weight_averaging.SWAG attribute*), 285
`current_epoch` (*dicee.weight_averaging.TWA attribute*), 287
`cycle_length` (*dicee.callbacks.LRScheduler attribute*), 27

D

`data_module` (*dicee.callbacks.PseudoLabellingCallback attribute*), 25
`data_property_embeddings` (*dicee.models.literal.LiteralEmbeddings attribute*), 141
`dataset_dir` (*dicee.config.Namespace attribute*), 28
`dataset_dir` (*dicee.knowledge_graph.KG attribute*), 73, 74
`dataset_sanity_checking()` (in module *dicee.read_preprocess_save_load_kg.util*), 259
`DeCaL` (class in *dicee.models*), 232
`DeCaL` (class in *dicee.models.clifford*), 114
`decay` (*dicee.weight_averaging.EMA attribute*), 286
`decide()` (*dicee.weight_averaging.ASWA method*), 283
`default_eval_model` (*dicee.callbacks.PeriodicEvalCallback attribute*), 27
`DEFAULT_HITS_RANGE` (in module *dicee.evaluation.utils*), 57
`DefaultBCELoss` (class in *dicee.losses.custom_losses*), 82
`DefaultBCELoss` (class in *dicee.models*), 178
`defer_large_embeddings` (*dicee.models.base_model.BaseKGE attribute*), 105
`defer_large_embeddings` (*dicee.models.BaseKGE attribute*), 189, 194, 206, 215, 222, 237, 242
`degree` (*dicee.models.function_space.LFMulti attribute*), 135
`degree` (*dicee.models.LFMulti attribute*), 248
`delta_history` (*dicee.weight_averaging.AMWA attribute*), 282
`denormalize()` (*dicee.dataset_classes.LiteralDataset static method*), 40
`describe()` (*dicee.knowledge_graph.KG method*), 75
`description_of_input` (*dicee.knowledge_graph.KG attribute*), 75
`deterministic_inference` (*dicee.models.graph_model.GraphKGE attribute*), 137
`deterministic_inference` (*dicee.models.TRIX attribute*), 251
`deterministic_inference` (*dicee.models.trix.TRIX attribute*), 166
`deterministic_inference` (*dicee.models.ULTRA attribute*), 250
`deterministic_inference` (*dicee.models.ultra.ULTRA attribute*), 170
`deviations` (*dicee.weight_averaging.SWAG attribute*), 285
`device` (*dicee.models.fsdp_models.RowWiseShardedEmbedding attribute*), 130
`device` (*dicee.models.literal.LiteralEmbeddings property*), 141
`device` (*dicee.trainer.torch_trainer_fsdp.TorchFSDPTrainer attribute*), 278
`DICE_Trainer` (class in *dicee*), 296
`DICE_Trainer` (class in *dicee.trainer*), 279
`DICE_Trainer` (class in *dicee.trainer.dice_trainer*), 272
`dicee`
 module, 12
`dicee.__main__`
 module, 12
`dicee.abstracts`
 module, 12
`dicee.analyse_experiments`
 module, 20
`dicee.callbacks`
 module, 22
`dicee.config`
 module, 28
`dicee.dataset_classes`
 module, 33
`dicee.eval_static_funcs`
 module, 42
`dicee.evaluation`
 module, 46
`dicee.evaluation.ensemble`
 module, 47
`dicee.evaluation.evaluator`
 module, 48
`dicee.evaluation.link_prediction`
 module, 51
`dicee.evaluation.literal_prediction`
 module, 55
`dicee.evaluation.utils`
 module, 56

- dicee.evaluator
 - module, 66
- dicee.executer
 - module, 70
- dicee.knowledge_graph
 - module, 73
- dicee.knowledge_graph_embeddings
 - module, 75
- dicee.losses
 - module, 81
- dicee.losses.custom_losses
 - module, 81
- dicee.models
 - module, 90
- dicee.models.adopt
 - module, 90
- dicee.models.base_model
 - module, 99
- dicee.models.clifford
 - module, 108
- dicee.models.complex
 - module, 117
- dicee.models.dualE
 - module, 122
- dicee.models.ensemble
 - module, 123
- dicee.models.flock
 - module, 124
- dicee.models.flock_walks
 - module, 129
- dicee.models.fsdP_models
 - module, 130
- dicee.models.function_space
 - module, 132
- dicee.models.graph_model
 - module, 136
- dicee.models.literal
 - module, 140
- dicee.models.octonion
 - module, 142
- dicee.models.pykeen_models
 - module, 144
- dicee.models.quaternion
 - module, 146
- dicee.models.real
 - module, 149
- dicee.models.sampled_loss
 - module, 157
- dicee.models.static_funcs
 - module, 157
- dicee.models.transformers
 - module, 157
- dicee.models.trix
 - module, 164
- dicee.models.ultra
 - module, 167
- dicee.query_generator
 - module, 252
- dicee.read_preprocess_save_load_kg
 - module, 253
- dicee.read_preprocess_save_load_kg.preprocess
 - module, 253
- dicee.read_preprocess_save_load_kg.read_from_disk
 - module, 254
- dicee.read_preprocess_save_load_kg.save_load_disk
 - module, 255
- dicee.read_preprocess_save_load_kg.util
 - module, 255

- dicee.sanity_checkers
 - module, 260
- dicee.scripts
 - module, 261
- dicee.scripts.index_serve
 - module, 261
- dicee.scripts.run
 - module, 262
- dicee.static_funcs
 - module, 262
- dicee.static_funcs_training
 - module, 267
- dicee.static_preprocess_funcs
 - module, 269
- dicee.trainer
 - module, 270
- dicee.trainer.auto_batch_finder
 - module, 270
- dicee.trainer.dice_trainer
 - module, 271
- dicee.trainer.model_parallelism
 - module, 273
- dicee.trainer.torch_trainer
 - module, 274
- dicee.trainer.torch_trainer_ddp
 - module, 276
- dicee.trainer.torch_trainer_fsdp
 - module, 277
- dicee.weight_averaging
 - module, 280
- dim (*dicee.models.flock.FlockBase* attribute), 128
- dim (*dicee.models.trix.TRIXBase* attribute), 166
- dim (*dicee.models.ULTRA* attribute), 250
- dim (*dicee.models.ultra.RelNBFNet* attribute), 169
- dim (*dicee.models.ultra.ULTRA* attribute), 170
- discrete_points (*dicee.models.FMult2* attribute), 247
- discrete_points (*dicee.models.function_space.FMult2* attribute), 134
- dist_func (*dicee.models.Pyke* attribute), 203
- dist_func (*dicee.models.real.Pyke* attribute), 155
- DistMult (*class in dicee.models*), 197
- DistMult (*class in dicee.models.real*), 149
- distributed (*dicee.Execute* attribute), 288
- distributed (*dicee.executor.Execute* attribute), 70, 71
- domain_constraints_per_rel (*dicee.evaluation.Evaluator* attribute), 60
- domain_constraints_per_rel (*dicee.evaluation.evaluator.Evaluator* attribute), 49
- domain_constraints_per_rel (*dicee.Evaluator* attribute), 299
- domain_constraints_per_rel (*dicee.evaluator.Evaluator* attribute), 68
- download_file() (*in module dicee.static_funcs*), 267
- download_files_from_url() (*in module dicee.static_funcs*), 267
- download_pretrained_model() (*in module dicee.static_funcs*), 267
- dropout (*dicee.models.clifford.TransformerMLP* attribute), 113
- dropout (*dicee.models.clifford.TransformerSelfAttention* attribute), 113
- dropout (*dicee.models.CoKEConfig* attribute), 204
- dropout (*dicee.models.literal.LiteralEmbeddings* attribute), 141
- dropout (*dicee.models.real.CoKEConfig* attribute), 155, 156
- dropout (*dicee.models.transformers.GPTConfig* attribute), 162
- dropout (*dicee.models.transformers.MLP* attribute), 161
- dropout (*dicee.models.transformers.SelfAttention* attribute), 160
- DualE (*class in dicee.models*), 249
- DualE (*class in dicee.models.dualE*), 122
- dummy_eval() (*dicee.evaluation.Evaluator* method), 62
- dummy_eval() (*dicee.evaluation.evaluator.Evaluator* method), 51
- dummy_eval() (*dicee.Evaluator* method), 300
- dummy_eval() (*dicee.evaluator.Evaluator* method), 69
- dummy_id (*dicee.knowledge_graph.KG* attribute), 75
- during_training (*dicee.evaluation.Evaluator* attribute), 60
- during_training (*dicee.evaluation.evaluator.Evaluator* attribute), 49
- during_training (*dicee.Evaluator* attribute), 298, 299

during_training (*dicее.evaluator.Evaluator* attribute), 67, 68

E

edge_index (*dicее.models.flock_walks.WalkGraph* attribute), 129
edge_type (*dicее.models.flock_walks.WalkGraph* attribute), 129
ee_vocab (*dicее.evaluation.Evaluator* attribute), 60
ee_vocab (*dicее.evaluation.evaluator.Evaluator* attribute), 48, 49
ee_vocab (*dicее.Evaluator* attribute), 298
ee_vocab (*dicее.evaluator.Evaluator* attribute), 67
efficient_zero_grad() (in module *dicее.evaluation.utils*), 58
efficient_zero_grad() (in module *dicее.static_funcs_training*), 269
EMA (class in *dicее.weight_averaging*), 285
ema (*dicее.config.Namespace* attribute), 32
ema_c_epochs (*dicее.weight_averaging.EMA* attribute), 286
ema_model (*dicее.weight_averaging.EMA* attribute), 286
ema_start_epoch (*dicее.weight_averaging.EMA* attribute), 286
ema_update() (*dicее.weight_averaging.EMA* static method), 286
emb_anon_node (*dicее.models.flock.FlockBase* attribute), 127
emb_anon_type (*dicее.models.flock.FlockBase* attribute), 127
emb_direction (*dicее.models.flock.FlockBase* attribute), 127
emb_head_is_query (*dicее.models.flock.FlockBase* attribute), 127
emb_neighbor (*dicее.models.flock.FlockBase* attribute), 127
emb_node_is_query (*dicее.models.flock.FlockBase* attribute), 127
emb_restart (*dicее.models.flock.FlockBase* attribute), 127
emb_tail_is_query (*dicее.models.flock.FlockBase* attribute), 127
emb_type_is_query (*dicее.models.flock.FlockBase* attribute), 127
embedding_dim (*dicее.analyse_experiments.Experiment* attribute), 21
embedding_dim (*dicее.config.Namespace* attribute), 30
embedding_dim (*dicее.models.base_model.BaseKGE* attribute), 104
embedding_dim (*dicее.models.BaseKGE* attribute), 188, 193, 205, 214, 221, 236, 241
embedding_dim (*dicее.models.fsdп_models.RowWiseShardedEmbedding* attribute), 130
embedding_dim (*dicее.models.literal.LiteralEmbeddings* attribute), 141
embedding_dims (*dicее.models.literal.LiteralEmbeddings* attribute), 140
enable_log (in module *dicее.static_preprocess_funcs*), 270
enc (*dicее.knowledge_graph.KG* attribute), 74
end() (*dicее.Execute* method), 289
end() (*dicее.executer.Execute* method), 72
end_row (*dicее.models.fsdп_models.RowWiseShardedEmbedding* attribute), 131
EnsembleKGE (class in *dicее.models.ensemble*), 123
ent2id (*dicее.query_generator.QueryGenerator* attribute), 252
ent2id (*dicее.QueryGenerator* attribute), 295
ent_in (*dicее.query_generator.QueryGenerator* attribute), 252
ent_in (*dicее.QueryGenerator* attribute), 295
ent_out (*dicее.query_generator.QueryGenerator* attribute), 252
ent_out (*dicее.QueryGenerator* attribute), 295
entities_str (*dicее.knowledge_graph.KG* property), 75
entity_embeddings (*dicее.models.AConvQ* attribute), 220
entity_embeddings (*dicее.models.ConvQ* attribute), 220
entity_embeddings (*dicее.models.DualE* attribute), 249
entity_embeddings (*dicее.models.dualE.DualE* attribute), 122
entity_embeddings (*dicее.models.FMult* attribute), 245
entity_embeddings (*dicее.models.FMult2* attribute), 247
entity_embeddings (*dicее.models.function_space.FMult* attribute), 132
entity_embeddings (*dicее.models.function_space.FMult2* attribute), 134
entity_embeddings (*dicее.models.function_space.GFMult* attribute), 133
entity_embeddings (*dicее.models.function_space.LFMult* attribute), 135
entity_embeddings (*dicее.models.function_space.LFMult1* attribute), 134
entity_embeddings (*dicее.models.GFMult* attribute), 246
entity_embeddings (*dicее.models.LFMult* attribute), 248
entity_embeddings (*dicее.models.LFMult1* attribute), 247
entity_embeddings (*dicее.models.literal.LiteralEmbeddings* attribute), 140, 141
entity_embeddings (*dicее.models.pykeen_models.PykeenKGE* attribute), 145
entity_embeddings (*dicее.models.PykeenKGE* attribute), 240
entity_embeddings (*dicее.models.quaternion.AConvQ* attribute), 148
entity_embeddings (*dicее.models.quaternion.ConvQ* attribute), 147
entity_model (*dicее.models.TRIXRelation* attribute), 251
entity_model (*dicее.models.trix.TRIXRelation* attribute), 167

entity_model (*dicee.models.ULTRA attribute*), 250
 entity_model (*dicee.models.ultra.ULTRA attribute*), 170
 entity_model_1 (*dicee.models.TRIX attribute*), 251
 entity_model_1 (*dicee.models.trix.TRIX attribute*), 166
 entity_model_2 (*dicee.models.TRIX attribute*), 251
 entity_model_2 (*dicee.models.trix.TRIX attribute*), 166
 entity_optim_device (*dicee.trainer.torch_trainer_fsdp.TorchFSDPTrainer attribute*), 278
 entity_to_idx (*dicee.dataset_classes.LiteralDataset attribute*), 39
 entity_to_idx (*dicee.knowledge_graph.KG attribute*), 73, 74
 entity_to_idx (*dicee.scripts.index_serve.NeuralSearcher attribute*), 262
 EntityNBFNet (*class in dicee.models.ultra*), 169
 EntityReasoner (*class in dicee.models.trix*), 164
 epoch_count (*dicee.abstracts.AbstractPPECallback attribute*), 19
 epoch_count (*dicee.weight_averaging.ASWA attribute*), 283
 epoch_counter (*dicee.callbacks.Eval attribute*), 25
 epoch_counter (*dicee.callbacks.KGESaveCallback attribute*), 24
 epoch_counter (*dicee.callbacks.PeriodicEvalCallback attribute*), 27
 epoch_ratio (*dicee.callbacks.Eval attribute*), 25
 eps (*dicee.losses.custom_losses.AdaptiveLabelRelaxationLoss attribute*), 86
 eps (*dicee.losses.custom_losses.AdaptiveLabelSmoothingLoss attribute*), 85
 eps (*dicee.losses.custom_losses.ConfidenceBasedAdaptiveLabelRelaxationLoss attribute*), 87
 eps (*dicee.losses.custom_losses.LabelRelaxationLoss attribute*), 85
 eps (*dicee.models.AdaptiveLabelRelaxationLoss attribute*), 174
 eps (*dicee.models.AdaptiveLabelSmoothingLoss attribute*), 175
 eps (*dicee.models.ConfidenceBasedAdaptiveLabelRelaxationLoss attribute*), 178
 eps (*dicee.models.flock.RMSNORM attribute*), 125
 eps (*dicee.models.LabelRelaxationLoss attribute*), 179
 er_vocab (*dicee.evaluation.Evaluator attribute*), 60
 er_vocab (*dicee.evaluation.evaluator.Evaluator attribute*), 48, 49
 er_vocab (*dicee.Evaluator attribute*), 298
 er_vocab (*dicee.evaluator.Evaluator attribute*), 67
 estimate_mfu () (*dicee.models.transformers.GPT method*), 163
 estimate_q () (*in module dicee.callbacks*), 25
 Eval (*class in dicee.callbacks*), 25
 eval () (*dicee.evaluation.Evaluator method*), 61
 eval () (*dicee.evaluation.evaluator.Evaluator method*), 49
 eval () (*dicee.Evaluator method*), 299
 eval () (*dicee.evaluator.Evaluator method*), 68
 eval () (*dicee.models.ensemble.EnsembleKGE method*), 123
 eval_at_epochs (*dicee.config.Namespace attribute*), 32
 eval_epochs (*dicee.callbacks.PeriodicEvalCallback attribute*), 27
 eval_every_n_epochs (*dicee.config.Namespace attribute*), 32
 eval_lp_performance () (*dicee.KGE method*), 290
 eval_lp_performance () (*dicee.knowledge_graph_embeddings.KGE method*), 76
 eval_model (*dicee.config.Namespace attribute*), 31
 eval_model (*dicee.knowledge_graph.KG attribute*), 74
 eval_rank_of_head_and_tail_byte_pair_encoded_entity () (*dicee.evaluation.Evaluator method*), 61
 eval_rank_of_head_and_tail_byte_pair_encoded_entity () (*dicee.evaluation.evaluator.Evaluator method*), 50
 eval_rank_of_head_and_tail_byte_pair_encoded_entity () (*dicee.Evaluator method*), 299
 eval_rank_of_head_and_tail_byte_pair_encoded_entity () (*dicee.evaluator.Evaluator method*), 68
 eval_rank_of_head_and_tail_entity () (*dicee.evaluation.Evaluator method*), 61
 eval_rank_of_head_and_tail_entity () (*dicee.evaluation.evaluator.Evaluator method*), 50
 eval_rank_of_head_and_tail_entity () (*dicee.Evaluator method*), 299
 eval_rank_of_head_and_tail_entity () (*dicee.evaluator.Evaluator method*), 68
 eval_tie_policy (*dicee.config.Namespace attribute*), 31
 eval_tie_seed (*dicee.config.Namespace attribute*), 31
 eval_with_bpe_vs_all () (*dicee.evaluation.Evaluator method*), 61
 eval_with_bpe_vs_all () (*dicee.evaluation.evaluator.Evaluator method*), 50
 eval_with_bpe_vs_all () (*dicee.Evaluator method*), 299
 eval_with_bpe_vs_all () (*dicee.evaluator.Evaluator method*), 68
 eval_with_byte () (*dicee.evaluation.Evaluator method*), 61
 eval_with_byte () (*dicee.evaluation.evaluator.Evaluator method*), 50
 eval_with_byte () (*dicee.Evaluator method*), 299
 eval_with_byte () (*dicee.evaluator.Evaluator method*), 68
 eval_with_data () (*dicee.evaluation.Evaluator method*), 62
 eval_with_data () (*dicee.evaluation.evaluator.Evaluator method*), 51
 eval_with_data () (*dicee.Evaluator method*), 300
 eval_with_data () (*dicee.evaluator.Evaluator method*), 69

eval_with_vs_all() (*dicee.evaluation.Evaluator method*), 61
 eval_with_vs_all() (*dicee.evaluation.evaluator.Evaluator method*), 50
 eval_with_vs_all() (*dicee.Evaluator method*), 299
 eval_with_vs_all() (*dicee.evaluator.Evaluator method*), 68
 evaluate() (*in module dicee.static_funcs*), 267
 evaluate_bpe_lp() (*in module dicee.evaluation*), 63
 evaluate_bpe_lp() (*in module dicee.evaluation.link_prediction*), 54
 evaluate_bpe_lp() (*in module dicee.static_funcs_training*), 268
 evaluate_ensemble_link_prediction_performance() (*in module dicee.eval_static_funcs*), 43
 evaluate_ensemble_link_prediction_performance() (*in module dicee.evaluation*), 59
 evaluate_ensemble_link_prediction_performance() (*in module dicee.evaluation.ensemble*), 47
 evaluate_link_prediction_performance() (*in module dicee.eval_static_funcs*), 43
 evaluate_link_prediction_performance() (*in module dicee.evaluation*), 63
 evaluate_link_prediction_performance() (*in module dicee.evaluation.link_prediction*), 52
 evaluate_link_prediction_performance_with_bpe() (*in module dicee.eval_static_funcs*), 44
 evaluate_link_prediction_performance_with_bpe() (*in module dicee.evaluation*), 63
 evaluate_link_prediction_performance_with_bpe() (*in module dicee.evaluation.link_prediction*), 53
 evaluate_link_prediction_performance_with_bpe_reciprocals() (*in module dicee.eval_static_funcs*), 44
 evaluate_link_prediction_performance_with_bpe_reciprocals() (*in module dicee.evaluation*), 64
 evaluate_link_prediction_performance_with_bpe_reciprocals() (*in module dicee.evaluation.link_prediction*), 53
 evaluate_link_prediction_performance_with_reciprocals() (*in module dicee.eval_static_funcs*), 45
 evaluate_link_prediction_performance_with_reciprocals() (*in module dicee.evaluation*), 64
 evaluate_link_prediction_performance_with_reciprocals() (*in module dicee.evaluation.link_prediction*), 52
 evaluate_literal_prediction() (*in module dicee.eval_static_funcs*), 45
 evaluate_literal_prediction() (*in module dicee.evaluation*), 66
 evaluate_literal_prediction() (*in module dicee.evaluation.literal_prediction*), 55
 evaluate_lp() (*dicee.evaluation.Evaluator method*), 62
 evaluate_lp() (*dicee.evaluation.evaluator.Evaluator method*), 51
 evaluate_lp() (*dicee.Evaluator method*), 300
 evaluate_lp() (*dicee.evaluator.Evaluator method*), 69
 evaluate_lp() (*in module dicee.evaluation*), 64
 evaluate_lp() (*in module dicee.evaluation.link_prediction*), 53
 evaluate_lp() (*in module dicee.static_funcs_training*), 268
 evaluate_lp_bpe_k_vs_all() (*dicee.evaluation.Evaluator method*), 62
 evaluate_lp_bpe_k_vs_all() (*dicee.evaluation.evaluator.Evaluator method*), 50
 evaluate_lp_bpe_k_vs_all() (*dicee.Evaluator method*), 300
 evaluate_lp_bpe_k_vs_all() (*dicee.evaluator.Evaluator method*), 69
 evaluate_lp_bpe_k_vs_all() (*in module dicee.eval_static_funcs*), 45
 evaluate_lp_bpe_k_vs_all() (*in module dicee.evaluation*), 65
 evaluate_lp_bpe_k_vs_all() (*in module dicee.evaluation.link_prediction*), 54
 evaluate_lp_k_vs_all() (*dicee.evaluation.Evaluator method*), 61
 evaluate_lp_k_vs_all() (*dicee.evaluation.evaluator.Evaluator method*), 50
 evaluate_lp_k_vs_all() (*dicee.Evaluator method*), 299
 evaluate_lp_k_vs_all() (*dicee.evaluator.Evaluator method*), 68
 evaluate_lp_with_byte() (*dicee.evaluation.Evaluator method*), 61
 evaluate_lp_with_byte() (*dicee.evaluation.evaluator.Evaluator method*), 50
 evaluate_lp_with_byte() (*dicee.Evaluator method*), 300
 evaluate_lp_with_byte() (*dicee.evaluator.Evaluator method*), 69
 Evaluator (*class in dicee*), 298
 Evaluator (*class in dicee.evaluation*), 59
 Evaluator (*class in dicee.evaluation.evaluator*), 48
 evaluator (*class in dicee.evaluator*), 67
 evaluator (*dicee.DICE_Trainer attribute*), 297
 evaluator (*dicee.Execute attribute*), 288, 289
 evaluator (*dicee.executer.Execute attribute*), 71
 evaluator (*dicee.trainer.DICE_Trainer attribute*), 279
 evaluator (*dicee.trainer.dice_trainer.DICE_Trainer attribute*), 272
 every_x_epoch (*dicee.callbacks.KGESaveCallback attribute*), 24
 example_input_array (*dicee.models.ensemble.EnsembleKGE property*), 123
 Execute (*class in dicee*), 288
 Execute (*class in dicee.executer*), 70
 exists() (*dicee.knowledge_graph.KG method*), 75
 Experiment (*class in dicee.analyse_experiments*), 21
 experiment_dir (*dicee.callbacks.LRScheduler attribute*), 27
 experiment_dir (*dicee.callbacks.PeriodicEvalCallback attribute*), 27
 explicit (*dicee.models.QMult attribute*), 219
 explicit (*dicee.models.quaternion.QMult attribute*), 146
 exponential_function() (*in module dicee.static_funcs*), 267

extract_input_outputs() (*dicee.trainer.torch_trainer_ddp.NodeTrainer method*), 277
 extract_input_outputs() (*dicee.trainer.torch_trainer_fsdp.TorchFSDPTrainer method*), 278
 extract_input_outputs() (*in module dicee.trainer.model_parallelism*), 274
 extract_input_outputs_set_device() (*dicee.trainer.torch_trainer.TorchTrainer method*), 275

F

f (*dicee.callbacks.KronE attribute*), 26
 fc (*dicee.models.literal.LiteralEmbeddings attribute*), 141
 fc1 (*dicee.models.AConEx attribute*), 211
 fc1 (*dicee.models.AConvO attribute*), 227
 fc1 (*dicee.models.AConvQ attribute*), 221
 fc1 (*dicee.models.complex.AConEx attribute*), 118
 fc1 (*dicee.models.complex.ConEx attribute*), 117
 fc1 (*dicee.models.ConEx attribute*), 210
 fc1 (*dicee.models.ConvO attribute*), 227
 fc1 (*dicee.models.ConvQ attribute*), 220
 fc1 (*dicee.models.octonion.AConvO attribute*), 144
 fc1 (*dicee.models.octonion.ConvO attribute*), 143
 fc1 (*dicee.models.quaternion.AConvQ attribute*), 148
 fc1 (*dicee.models.quaternion.ConvQ attribute*), 147
 fc_num_input (*dicee.models.AConEx attribute*), 210
 fc_num_input (*dicee.models.AConvO attribute*), 227
 fc_num_input (*dicee.models.AConvQ attribute*), 220
 fc_num_input (*dicee.models.complex.AConEx attribute*), 118
 fc_num_input (*dicee.models.complex.ConEx attribute*), 117
 fc_num_input (*dicee.models.ConEx attribute*), 209
 fc_num_input (*dicee.models.ConvO attribute*), 227
 fc_num_input (*dicee.models.ConvQ attribute*), 220
 fc_num_input (*dicee.models.octonion.AConvO attribute*), 144
 fc_num_input (*dicee.models.octonion.ConvO attribute*), 143
 fc_num_input (*dicee.models.quaternion.AConvQ attribute*), 148
 fc_num_input (*dicee.models.quaternion.ConvQ attribute*), 147
 fc_out (*dicee.models.literal.LiteralEmbeddings attribute*), 141
 feature_map_dropout (*dicee.models.AConEx attribute*), 211
 feature_map_dropout (*dicee.models.AConvO attribute*), 228
 feature_map_dropout (*dicee.models.AConvQ attribute*), 221
 feature_map_dropout (*dicee.models.complex.AConEx attribute*), 118
 feature_map_dropout (*dicee.models.complex.ConEx attribute*), 117
 feature_map_dropout (*dicee.models.ConEx attribute*), 210
 feature_map_dropout (*dicee.models.ConvO attribute*), 227
 feature_map_dropout (*dicee.models.ConvQ attribute*), 220
 feature_map_dropout (*dicee.models.octonion.AConvO attribute*), 144
 feature_map_dropout (*dicee.models.octonion.ConvO attribute*), 143
 feature_map_dropout (*dicee.models.quaternion.AConvQ attribute*), 148
 feature_map_dropout (*dicee.models.quaternion.ConvQ attribute*), 147
 feature_map_dropout_rate (*dicee.config.Namespace attribute*), 32
 feature_map_dropout_rate (*dicee.models.base_model.BaseKGE attribute*), 104
 feature_map_dropout_rate (*dicee.models.BaseKGE attribute*), 188, 193, 206, 214, 222, 236, 241
 features() (*dicee.models.trix.EntityReasoner method*), 165
 feed_forward (*dicee.models.flock.BidirectionalGRU attribute*), 127
 FeedForward (*class in dicee.models.flock*), 125
 fetch_worker() (*in module dicee.read_preprocess_save_load_kg.util*), 258
 ffn_norm (*dicee.models.flock.BidirectionalGRU attribute*), 127
 fill_query() (*dicee.query_generator.QueryGenerator method*), 252
 fill_query() (*dicee.QueryGenerator method*), 296
 find_good_batch_size() (*in module dicee.trainer.auto_batch_finder*), 270
 find_missing_triples() (*dicee.KGE method*), 294
 find_missing_triples() (*dicee.knowledge_graph_embeddings.KGE method*), 80
 fit() (*dicee.trainer.model_parallelism.TensorParallel method*), 274
 fit() (*dicee.trainer.torch_trainer_ddp.TorchDDPTrainer method*), 276
 fit() (*dicee.trainer.torch_trainer_fsdp.TorchFSDPTrainer method*), 278
 fit() (*dicee.trainer.torch_trainer.TorchTrainer method*), 275
 FixedNegSampleDataset (*class in dicee.dataset_classes*), 40
 flash (*dicee.models.transformers.SelfAttention attribute*), 160
 Flock (*class in dicee.models*), 251
 Flock (*class in dicee.models.flock*), 128
 flock_attention_heads (*dicee.config.Namespace attribute*), 29

flock_checkpoint (*dicee.config.Namespace attribute*), 29
 flock_compact_state (*dicee.config.Namespace attribute*), 29
 flock_compile_sampler (*dicee.config.Namespace attribute*), 29
 flock_dim (*dicee.config.Namespace attribute*), 29
 flock_num_layers (*dicee.config.Namespace attribute*), 29
 flock_pack_walks (*dicee.config.Namespace attribute*), 29
 flock_prefetch_walks (*dicee.config.Namespace attribute*), 29
 flock_query_batch_size (*dicee.config.Namespace attribute*), 29
 flock_refinements (*dicee.config.Namespace attribute*), 29
 flock_seed (*dicee.config.Namespace attribute*), 29
 flock_test_samples (*dicee.config.Namespace attribute*), 29
 flock_walk_len (*dicee.config.Namespace attribute*), 29
 flock_walk_num (*dicee.config.Namespace attribute*), 29
 FlockBase (*class in dicee.models.flock*), 127
 FlockRelation (*class in dicee.models*), 251
 FlockRelation (*class in dicee.models.flock*), 128
 FMult (*class in dicee.models*), 245
 FMult (*class in dicee.models.function_space*), 132
 FMult2 (*class in dicee.models*), 247
 FMult2 (*class in dicee.models.function_space*), 133
 form_of_labelling (*dicee.DICE_Trainer attribute*), 297
 form_of_labelling (*dicee.trainer.DICE_Trainer attribute*), 279
 form_of_labelling (*dicee.trainer.dice_trainer.DICE_Trainer attribute*), 272
 forward () (*dicee.losses.custom_losses.ACLS method*), 90
 forward () (*dicee.losses.custom_losses.AdaptiveLabelRelaxationLoss method*), 86
 forward () (*dicee.losses.custom_losses.AdaptiveLabelSmoothingLoss method*), 85
 forward () (*dicee.losses.custom_losses.AggregatedLSandLR method*), 89
 forward () (*dicee.losses.custom_losses.CombinedAdaptiveLSandAdaptiveLR method*), 89
 forward () (*dicee.losses.custom_losses.CombinedLSandLR method*), 88
 forward () (*dicee.losses.custom_losses.ConfidenceBasedAdaptiveLabelRelaxationLoss method*), 87
 forward () (*dicee.losses.custom_losses.DefaultBCELoss method*), 82
 forward () (*dicee.losses.custom_losses.LabelRelaxationLoss method*), 85
 forward () (*dicee.losses.custom_losses.LabelSmoothingLoss method*), 84
 forward () (*dicee.losses.custom_losses.WeightedBCELoss method*), 83
 forward () (*dicee.models.ACLS method*), 173
 forward () (*dicee.models.AdaptiveLabelRelaxationLoss method*), 174
 forward () (*dicee.models.AdaptiveLabelSmoothingLoss method*), 175
 forward () (*dicee.models.AggregatedLSandLR method*), 176
 forward () (*dicee.models.base_model.BaseKGE method*), 106
 forward () (*dicee.models.base_model.IdentityClass static method*), 108
 forward () (*dicee.models.BaseKGE method*), 190, 195, 207, 216, 223, 238, 243
 forward () (*dicee.models.Block method*), 193
 forward () (*dicee.models.clifford.TransformerBlock method*), 113
 forward () (*dicee.models.clifford.TransformerMLP method*), 113
 forward () (*dicee.models.clifford.TransformerSelfAttention method*), 113
 forward () (*dicee.models.CombinedAdaptiveLSandAdaptiveLR method*), 176
 forward () (*dicee.models.CombinedLSandLR method*), 177
 forward () (*dicee.models.ConfidenceBasedAdaptiveLabelRelaxationLoss method*), 178
 forward () (*dicee.models.DefaultBCELoss method*), 178
 forward () (*dicee.models.flock.BidirectionalGRU method*), 127
 forward () (*dicee.models.flock.FeedForward method*), 126
 forward () (*dicee.models.flock.RMSNORM method*), 125
 forward () (*dicee.models.fsdp_models.RowWiseShardedEmbedding method*), 131
 forward () (*dicee.models.graph_model.GraphKGE method*), 139
 forward () (*dicee.models.IdentityClass static method*), 192, 218, 226
 forward () (*dicee.models.LabelRelaxationLoss method*), 179
 forward () (*dicee.models.LabelSmoothingLoss method*), 180
 forward () (*dicee.models.literal.LiteralEmbeddings method*), 141
 forward () (*dicee.models.transformers.Block method*), 162
 forward () (*dicee.models.transformers.BytE method*), 159
 forward () (*dicee.models.transformers.GPT method*), 163
 forward () (*dicee.models.transformers.LayerNorm method*), 159
 forward () (*dicee.models.transformers.MLP method*), 161
 forward () (*dicee.models.transformers.SelfAttention method*), 160
 forward () (*dicee.models.ultra.EntityNBFNet method*), 169
 forward () (*dicee.models.ultra.RelationalConv method*), 168
 forward () (*dicee.models.ultra.RelNBFNet method*), 169
 forward_backward_update () (*dicee.trainer.torch_trainer.TorchTrainer method*), 275

`forward_backward_update_loss()` (in module `dicee.trainer.model_parallelism`), 274
`forward_byte_pair_encoded_k_vs_all()` (`dicee.models.base_model.BaseKGE` method), 105
`forward_byte_pair_encoded_k_vs_all()` (`dicee.models.BaseKGE` method), 189, 194, 206, 215, 223, 237, 242
`forward_byte_pair_encoded_triple()` (`dicee.models.base_model.BaseKGE` method), 105
`forward_byte_pair_encoded_triple()` (`dicee.models.BaseKGE` method), 189, 195, 207, 215, 223, 237, 243
`forward_grouped()` (`dicee.models.graph_model.GraphKGE` method), 138
`forward_grouped()` (`dicee.models.graph_model.RelationGraphKGE` method), 139
`forward_k_vs_all()` (`dicee.models.AConEx` method), 211
`forward_k_vs_all()` (`dicee.models.AConvO` method), 228
`forward_k_vs_all()` (`dicee.models.AConvQ` method), 221
`forward_k_vs_all()` (`dicee.models.base_model.BaseKGE` method), 106
`forward_k_vs_all()` (`dicee.models.BaseKGE` method), 190, 196, 208, 216, 224, 238, 244
`forward_k_vs_all()` (`dicee.models.clifford.DeCaL` method), 115
`forward_k_vs_all()` (`dicee.models.clifford.Keci` method), 111
`forward_k_vs_all()` (`dicee.models.clifford.KeciTransformer` method), 112
`forward_k_vs_all()` (`dicee.models.CoKE` method), 205
`forward_k_vs_all()` (`dicee.models.ComplEx` method), 212
`forward_k_vs_all()` (`dicee.models.complex.AConEx` method), 119
`forward_k_vs_all()` (`dicee.models.complex.ComplEx` method), 120
`forward_k_vs_all()` (`dicee.models.complex.ConEx` method), 118
`forward_k_vs_all()` (`dicee.models.complex.RotatE` method), 121
`forward_k_vs_all()` (`dicee.models.ConEx` method), 210
`forward_k_vs_all()` (`dicee.models.ConvO` method), 227
`forward_k_vs_all()` (`dicee.models.ConvQ` method), 220
`forward_k_vs_all()` (`dicee.models.DeCaL` method), 233
`forward_k_vs_all()` (`dicee.models.DistMult` method), 198
`forward_k_vs_all()` (`dicee.models.DualE` method), 250
`forward_k_vs_all()` (`dicee.models.dualE.DualE` method), 122
`forward_k_vs_all()` (`dicee.models.fsdp_models.FSDPShardedEntityModel` method), 131
`forward_k_vs_all()` (`dicee.models.graph_model.GraphKGE` method), 139
`forward_k_vs_all()` (`dicee.models.graph_model.RelationGraphKGE` method), 140
`forward_k_vs_all()` (`dicee.models.Keci` method), 231
`forward_k_vs_all()` (`dicee.models.KeciTransformer` method), 235
`forward_k_vs_all()` (`dicee.models.MuRE` method), 202
`forward_k_vs_all()` (`dicee.models.octonion.AConvO` method), 144
`forward_k_vs_all()` (`dicee.models.octonion.ConvO` method), 143
`forward_k_vs_all()` (`dicee.models.octonion.OMult` method), 142
`forward_k_vs_all()` (`dicee.models.OMult` method), 226
`forward_k_vs_all()` (`dicee.models.pykeen_models.PykeenKGE` method), 145
`forward_k_vs_all()` (`dicee.models.PykeenKGE` method), 240
`forward_k_vs_all()` (`dicee.models.QMult` method), 219
`forward_k_vs_all()` (`dicee.models.quaternion.AConvQ` method), 148
`forward_k_vs_all()` (`dicee.models.quaternion.ConvQ` method), 148
`forward_k_vs_all()` (`dicee.models.quaternion.QMult` method), 147
`forward_k_vs_all()` (`dicee.models.real.CoKE` method), 156
`forward_k_vs_all()` (`dicee.models.real.DistMult` method), 150
`forward_k_vs_all()` (`dicee.models.real.MuRE` method), 153
`forward_k_vs_all()` (`dicee.models.real.Shallom` method), 154
`forward_k_vs_all()` (`dicee.models.real.TransE` method), 151
`forward_k_vs_all()` (`dicee.models.real.TransH` method), 152
`forward_k_vs_all()` (`dicee.models.RotatE` method), 213
`forward_k_vs_all()` (`dicee.models.Shallom` method), 202
`forward_k_vs_all()` (`dicee.models.TransE` method), 199
`forward_k_vs_all()` (`dicee.models.TransH` method), 200
`forward_k_vs_all_heads()` (`dicee.models.graph_model.GraphKGE` method), 139
`forward_k_vs_all_heads()` (`dicee.models.graph_model.RelationGraphKGE` method), 140
`forward_k_vs_all_relations` (`dicee.models.graph_model.RelationGraphKGE` attribute), 140
`forward_k_vs_sample()` (`dicee.models.AConEx` method), 211
`forward_k_vs_sample()` (`dicee.models.base_model.BaseKGE` method), 106
`forward_k_vs_sample()` (`dicee.models.BaseKGE` method), 190, 196, 208, 217, 224, 238, 244
`forward_k_vs_sample()` (`dicee.models.clifford.Keci` method), 112
`forward_k_vs_sample()` (`dicee.models.CoKE` method), 205
`forward_k_vs_sample()` (`dicee.models.ComplEx` method), 212
`forward_k_vs_sample()` (`dicee.models.complex.AConEx` method), 119
`forward_k_vs_sample()` (`dicee.models.complex.ComplEx` method), 120
`forward_k_vs_sample()` (`dicee.models.complex.ConEx` method), 118
`forward_k_vs_sample()` (`dicee.models.complex.RotatE` method), 121
`forward_k_vs_sample()` (`dicee.models.ConEx` method), 210

`forward_k_vs_sample()` (*dicee.models.DistMult method*), 198
`forward_k_vs_sample()` (*dicee.models.graph_model.GraphKGE method*), 138
`forward_k_vs_sample()` (*dicee.models.graph_model.RelationGraphKGE method*), 140
`forward_k_vs_sample()` (*dicee.models.Keci method*), 231
`forward_k_vs_sample()` (*dicee.models.MuRE method*), 202
`forward_k_vs_sample()` (*dicee.models.pykeen_models.PykeenKGE method*), 145
`forward_k_vs_sample()` (*dicee.models.PykeenKGE method*), 241
`forward_k_vs_sample()` (*dicee.models.QMult method*), 219
`forward_k_vs_sample()` (*dicee.models.quaternion.QMult method*), 147
`forward_k_vs_sample()` (*dicee.models.real.CoKE method*), 156
`forward_k_vs_sample()` (*dicee.models.real.DistMult method*), 150
`forward_k_vs_sample()` (*dicee.models.real.MuRE method*), 154
`forward_k_vs_sample()` (*dicee.models.real.TransH method*), 152
`forward_k_vs_sample()` (*dicee.models.RotatE method*), 213
`forward_k_vs_sample()` (*dicee.models.TransH method*), 201
`forward_k_vs_with_explicit()` (*dicee.models.clifford.Keci method*), 110
`forward_k_vs_with_explicit()` (*dicee.models.Keci method*), 230
`forward_triples()` (*dicee.models.AConEx method*), 211
`forward_triples()` (*dicee.models.AConvO method*), 228
`forward_triples()` (*dicee.models.AConvQ method*), 221
`forward_triples()` (*dicee.models.base_model.BaseKGE method*), 106
`forward_triples()` (*dicee.models.BaseKGE method*), 190, 195, 208, 216, 224, 238, 244
`forward_triples()` (*dicee.models.clifford.DeCaL method*), 114
`forward_triples()` (*dicee.models.clifford.Keci method*), 112
`forward_triples()` (*dicee.models.complex.AConEx method*), 119
`forward_triples()` (*dicee.models.complex.ConEx method*), 118
`forward_triples()` (*dicee.models.ConEx method*), 210
`forward_triples()` (*dicee.models.ConvO method*), 227
`forward_triples()` (*dicee.models.ConvQ method*), 220
`forward_triples()` (*dicee.models.DeCaL method*), 232
`forward_triples()` (*dicee.models.DualE method*), 250
`forward_triples()` (*dicee.models.dualE.DualE method*), 122
`forward_triples()` (*dicee.models.FMult method*), 246
`forward_triples()` (*dicee.models.FMult2 method*), 247
`forward_triples()` (*dicee.models.function_space.FMult method*), 133
`forward_triples()` (*dicee.models.function_space.FMult2 method*), 134
`forward_triples()` (*dicee.models.function_space.GFMult method*), 133
`forward_triples()` (*dicee.models.function_space.LFMult method*), 135
`forward_triples()` (*dicee.models.function_space.LFMult1 method*), 134
`forward_triples()` (*dicee.models.GFMult method*), 246
`forward_triples()` (*dicee.models.graph_model.GraphKGE method*), 138
`forward_triples()` (*dicee.models.graph_model.RelationGraphKGE method*), 139
`forward_triples()` (*dicee.models.Keci method*), 231
`forward_triples()` (*dicee.models.LFMult method*), 248
`forward_triples()` (*dicee.models.LFMult1 method*), 248
`forward_triples()` (*dicee.models.MuRE method*), 201
`forward_triples()` (*dicee.models.octonion.AConvO method*), 144
`forward_triples()` (*dicee.models.octonion.ConvO method*), 143
`forward_triples()` (*dicee.models.Pyke method*), 203
`forward_triples()` (*dicee.models.pykeen_models.PykeenKGE method*), 145
`forward_triples()` (*dicee.models.PykeenKGE method*), 240
`forward_triples()` (*dicee.models.quaternion.AConvQ method*), 148
`forward_triples()` (*dicee.models.quaternion.ConvQ method*), 148
`forward_triples()` (*dicee.models.real.MuRE method*), 153
`forward_triples()` (*dicee.models.real.Pyke method*), 155
`forward_triples()` (*dicee.models.real.Shallom method*), 154
`forward_triples()` (*dicee.models.real.TransH method*), 152
`forward_triples()` (*dicee.models.Shallom method*), 203
`forward_triples()` (*dicee.models.TransH method*), 200
`freeze_entity_embeddings` (*dicee.models.literal.LiteralEmbeddings attribute*), 141
`frequency` (*dicee.callbacks.Perturb attribute*), 27
`from_node` (*dicee.models.flock.FlockBase attribute*), 127
`from_pretrained()` (*dicee.models.transformers.GPT class method*), 163
`from_pretrained_model_write_embeddings_into_csv()` (*in module dicee.static_funcs*), 267
`from_type` (*dicee.models.flock.FlockBase attribute*), 128
`FSDP1vsSampleDataset` (*class in dicee.dataset_classes*), 37
`FSDPShardedEntityModel` (*class in dicee.models.fsdp_models*), 131
`full_storage_path` (*dicee.analyse_experiments.Experiment attribute*), 21

[func_triple_to_bpe_representation \(dicee.evaluation.Evaluator attribute\)](#), 60
[func_triple_to_bpe_representation \(dicee.evaluation.evaluator.Evaluator attribute\)](#), 49
[func_triple_to_bpe_representation \(dicee.Evaluator attribute\)](#), 298
[func_triple_to_bpe_representation \(dicee.evaluator.Evaluator attribute\)](#), 67
[func_triple_to_bpe_representation \(\) \(dicee.knowledge_graph.KG method\)](#), 75
[function \(\) \(dicee.models.FMult2 method\)](#), 247
[function \(\) \(dicee.models.function_space.FMult2 method\)](#), 134

G

[gamma \(dicee.models.FMult attribute\)](#), 246
[gamma \(dicee.models.function_space.FMult attribute\)](#), 132
[gate_residual \(dicee.models.literal.LiteralEmbeddings attribute\)](#), 141
[gated_residual_proj \(dicee.models.literal.LiteralEmbeddings attribute\)](#), 141
[gather_entity_embeddings_on_rank_zero \(\) \(dicee.models.fsdp_models.FSDPShardedEntityModel method\)](#), 131
[gelu \(dicee.models.clifford.TransformerMLP attribute\)](#), 113
[gelu \(dicee.models.transformers.MLP attribute\)](#), 161
[gen_test \(dicee.query_generator.QueryGenerator attribute\)](#), 252
[gen_test \(dicee.QueryGenerator attribute\)](#), 295
[gen_valid \(dicee.query_generator.QueryGenerator attribute\)](#), 252
[gen_valid \(dicee.QueryGenerator attribute\)](#), 295
[generate \(\) \(dicee.KGE method\)](#), 290
[generate \(\) \(dicee.knowledge_graph_embeddings.KGE method\)](#), 76
[generate \(\) \(dicee.models.transformers.BytE method\)](#), 159
[generate_queries \(\) \(dicee.query_generator.QueryGenerator method\)](#), 253
[generate_queries \(\) \(dicee.QueryGenerator method\)](#), 296
[get_aswa_state_dict \(\) \(dicee.weight_averaging.ASWA method\)](#), 283
[get_bpe_head_and_relation_representation \(\) \(dicee.models.base_model.BaseKGE method\)](#), 107
[get_bpe_head_and_relation_representation \(\) \(dicee.models.BaseKGE method\)](#), 191, 197, 209, 218, 225, 239, 245
[get_bpe_token_representation \(\) \(dicee.abstracts.BaseInteractiveKGE method\)](#), 14
[get_callbacks \(\) \(in module dicee.trainer.dice_trainer\)](#), 272
[get_default_arguments \(\) \(in module dicee.analyse_experiments\)](#), 21
[get_default_arguments \(\) \(in module dicee.scripts.index_serve\)](#), 261
[get_default_arguments \(\) \(in module dicee.scripts.run\)](#), 262
[get_ee_vocab \(\) \(in module dicee.read_preprocess_save_load_kg.util\)](#), 258
[get_ee_vocab \(\) \(in module dicee.static_funcs\)](#), 264
[get_ee_vocab \(\) \(in module dicee.static_preprocess_funcs\)](#), 270
[get_embeddings \(\) \(dicee.models.base_model.BaseKGE method\)](#), 108
[get_embeddings \(\) \(dicee.models.BaseKGE method\)](#), 192, 197, 209, 218, 225, 240, 245
[get_embeddings \(\) \(dicee.models.ensemble.EnsembleKGE method\)](#), 123
[get_embeddings \(\) \(dicee.models.fsdp_models.FSDPShardedEntityModel method\)](#), 131
[get_embeddings \(\) \(dicee.models.graph_model.GraphKGE method\)](#), 138
[get_embeddings \(\) \(dicee.models.real.Shallom method\)](#), 154
[get_embeddings \(\) \(dicee.models.Shallom method\)](#), 202
[get_entity_embeddings \(\) \(dicee.abstracts.BaseInteractiveKGE method\)](#), 16
[get_entity_index \(\) \(dicee.abstracts.BaseInteractiveKGE method\)](#), 15
[get_er_vocab \(\) \(in module dicee.read_preprocess_save_load_kg.util\)](#), 258
[get_er_vocab \(\) \(in module dicee.static_funcs\)](#), 264
[get_er_vocab \(\) \(in module dicee.static_preprocess_funcs\)](#), 270
[get_eval_report \(\) \(dicee.abstracts.BaseInteractiveKGE method\)](#), 14
[get_filter_vocab \(\) \(in module dicee.read_preprocess_save_load_kg.util\)](#), 258
[get_head_relation_representation \(\) \(dicee.models.base_model.BaseKGE method\)](#), 107
[get_head_relation_representation \(\) \(dicee.models.BaseKGE method\)](#), 191, 196, 208, 217, 225, 239, 244
[get_kronecker_triple_representation \(\) \(dicee.callbacks.KronE method\)](#), 26
[get_mean_and_var \(\) \(dicee.weight_averaging.SWAG method\)](#), 285
[get_num_params \(\) \(dicee.models.transformers.GPT method\)](#), 163
[get_padded_bpe_triple_representation \(\) \(dicee.abstracts.BaseInteractiveKGE method\)](#), 14
[get_queries \(\) \(dicee.query_generator.QueryGenerator method\)](#), 253
[get_queries \(\) \(dicee.QueryGenerator method\)](#), 296
[get_re_vocab \(\) \(in module dicee.read_preprocess_save_load_kg.util\)](#), 258
[get_re_vocab \(\) \(in module dicee.static_funcs\)](#), 264
[get_re_vocab \(\) \(in module dicee.static_preprocess_funcs\)](#), 270
[get_reg \(\) \(dicee.losses.custom_losses.ACLS method\)](#), 90
[get_reg \(\) \(dicee.models.ACLS method\)](#), 173
[get_relation_embeddings \(\) \(dicee.abstracts.BaseInteractiveKGE method\)](#), 16
[get_relation_index \(\) \(dicee.abstracts.BaseInteractiveKGE method\)](#), 15
[get_sentence_representation \(\) \(dicee.models.base_model.BaseKGE method\)](#), 107
[get_sentence_representation \(\) \(dicee.models.BaseKGE method\)](#), 191, 197, 209, 217, 225, 239, 245

get_transductive_entity_embeddings() (*dicee.KGE method*), 290
 get_transductive_entity_embeddings() (*dicee.knowledge_graph_embeddings.KGE method*), 76
 get_triple_representation() (*dicee.models.base_model.BaseKGE method*), 107
 get_triple_representation() (*dicee.models.BaseKGE method*), 191, 196, 208, 217, 224, 239, 244
 GFMult (*class in dicee.models*), 246
 GFMult (*class in dicee.models.function_space*), 133
 global_rank (*dicee.abstracts.AbstractTrainer attribute*), 13
 global_rank (*dicee.trainer.torch_trainer_ddp.NodeTrainer attribute*), 276
 global_rank (*dicee.trainer.torch_trainer_fsdp.TorchFSDPTrainer attribute*), 278
 GPT (*class in dicee.models.transformers*), 162
 GPTConfig (*class in dicee.models.transformers*), 162
 gpus (*dicee.config.Namespace attribute*), 30
 gradient_accumulation_steps (*dicee.config.Namespace attribute*), 31
 gradient_clip_val (*dicee.trainer.torch_trainer_fsdp.TorchFSDPTrainer attribute*), 278
 graph_filename (*dicee.models.flock.FlockBase attribute*), 127
 graph_filename (*dicee.models.graph_model.GraphKGE attribute*), 137
 graph_filename (*dicee.models.trix.TRIXBase attribute*), 166
 graph_filename (*dicee.models.ULTRA attribute*), 250
 graph_filename (*dicee.models.ultra.ULTRA attribute*), 170
 graph_inference_backend (*dicee.config.Namespace attribute*), 29
 graph_inference_compile (*dicee.config.Namespace attribute*), 29
 graph_projection_cache_mb (*dicee.config.Namespace attribute*), 29
 graph_relation_cache_mb (*dicee.config.Namespace attribute*), 29
 GraphKGE (*class in dicee.models.graph_model*), 137
 ground_queries() (*dicee.query_generator.QueryGenerator method*), 253
 ground_queries() (*dicee.QueryGenerator method*), 296
 grouped_adversarial_bce() (*in module dicee.models.sampled_loss*), 157
 grouped_negative_sampling (*dicee.config.Namespace attribute*), 29
 gru (*dicee.models.flock.BidirectionalGRU attribute*), 127
 gru_norm (*dicee.models.flock.BidirectionalGRU attribute*), 127
 gru_out (*dicee.models.flock.BidirectionalGRU attribute*), 127
 gswa_n (*dicee.weight_averaging.SWAG attribute*), 285
 gz_threshold (*dicee.losses.custom_losses.AdaptiveLabelRelaxationLoss attribute*), 86
 gz_threshold (*dicee.losses.custom_losses.ConfidenceBasedAdaptiveLabelRelaxationLoss attribute*), 87
 gz_threshold (*dicee.losses.custom_losses.LabelRelaxationLoss attribute*), 85
 gz_threshold (*dicee.models.AdaptiveLabelRelaxationLoss attribute*), 174
 gz_threshold (*dicee.models.ConfidenceBasedAdaptiveLabelRelaxationLoss attribute*), 178
 gz_threshold (*dicee.models.LabelRelaxationLoss attribute*), 179

H

half_dim (*dicee.models.complex.RotatE attribute*), 121
 half_dim (*dicee.models.RotatE attribute*), 213
 head (*dicee.models.flock.FlockBase attribute*), 128
 hidden_dim (*dicee.models.literal.LiteralEmbeddings attribute*), 141
 hidden_dropout (*dicee.models.base_model.BaseKGE attribute*), 104
 hidden_dropout (*dicee.models.BaseKGE attribute*), 188, 194, 206, 215, 222, 236, 242
 hidden_dropout_rate (*dicee.config.Namespace attribute*), 32
 hidden_dropout_rate (*dicee.models.base_model.BaseKGE attribute*), 104
 hidden_dropout_rate (*dicee.models.BaseKGE attribute*), 188, 193, 206, 214, 222, 236, 241
 hidden_normalizer (*dicee.models.base_model.BaseKGE attribute*), 104
 hidden_normalizer (*dicee.models.BaseKGE attribute*), 188, 194, 206, 215, 222, 236, 242
 history (*dicee.weight_averaging.AMWA attribute*), 282

I

IdentityClass (*class in dicee.models*), 192, 218, 225
 IdentityClass (*class in dicee.models.base_model*), 108
 idx_entity_to_bpe_shaped (*dicee.knowledge_graph.KG attribute*), 74
 ignore_index (*dicee.losses.custom_losses.ACLS attribute*), 90
 ignore_index (*dicee.models.ACLS attribute*), 173
 index() (*in module dicee.scripts.index_serve*), 261
 index_triple() (*dicee.abstracts.BaseInteractiveKGE method*), 16
 inference_backend (*dicee.models.flock.RMSNORM attribute*), 125
 inference_backend (*dicee.models.ultra.RelationalConv attribute*), 168
 inference_compile (*dicee.models.ultra.RelationalConv attribute*), 168
 inference_token() (*dicee.models.graph_model.GraphKGE method*), 137
 init_dataloader() (*dicee.DICE_Trainer method*), 297
 init_dataloader() (*dicee.trainer.DICE_Trainer method*), 279

`init_dataloader()` (*dicee.trainer.dice_trainer.DICE_Trainer method*), 273
`init_dataset()` (*dicee.DICE_Trainer method*), 297
`init_dataset()` (*dicee.trainer.DICE_Trainer method*), 280
`init_dataset()` (*dicee.trainer.dice_trainer.DICE_Trainer method*), 273
`init_entity_embeddings()` (*dicee.models.base_model.BaseKGE method*), 105
`init_entity_embeddings()` (*dicee.models.BaseKGE method*), 189, 194, 206, 215, 222, 237, 242
`init_entity_embeddings()` (*dicee.models.graph_model.GraphKGE method*), 137
`init_param` (*dicee.config.Namespace attribute*), 30
`init_params_with_sanity_checking()` (*dicee.models.base_model.BaseKGE method*), 105
`init_params_with_sanity_checking()` (*dicee.models.BaseKGE method*), 189, 195, 207, 216, 223, 237, 243
`init_relation_embeddings()` (*dicee.models.base_model.BaseKGE method*), 105
`init_relation_embeddings()` (*dicee.models.BaseKGE method*), 189, 194, 206, 215, 222, 237, 242
`init_relation_embeddings()` (*dicee.models.graph_model.GraphKGE method*), 138
`initial_eval_setting` (*dicee.weight_averaging.AMWA attribute*), 282
`initial_eval_setting` (*dicee.weight_averaging.ASWA attribute*), 283
`initialize_or_load_model()` (*dicee.DICE_Trainer method*), 297
`initialize_or_load_model()` (*dicee.trainer.DICE_Trainer method*), 273
`initialize_trainer()` (*dicee.DICE_Trainer method*), 297
`initialize_trainer()` (*dicee.trainer.DICE_Trainer method*), 279
`initialize_trainer()` (*dicee.trainer.dice_trainer.DICE_Trainer method*), 273
`initialize_trainer()` (*in module dicee.trainer.dice_trainer*), 271
`input_dp_ent_real` (*dicee.models.base_model.BaseKGE attribute*), 104
`input_dp_ent_real` (*dicee.models.BaseKGE attribute*), 188, 194, 206, 215, 222, 236, 242
`input_dp_rel_real` (*dicee.models.base_model.BaseKGE attribute*), 104
`input_dp_rel_real` (*dicee.models.BaseKGE attribute*), 188, 194, 206, 215, 222, 236, 242
`input_dropout_rate` (*dicee.config.Namespace attribute*), 32
`input_dropout_rate` (*dicee.models.base_model.BaseKGE attribute*), 104
`input_dropout_rate` (*dicee.models.BaseKGE attribute*), 188, 193, 205, 214, 222, 236, 241
`INTERACTIONS` (*in module dicee.models.trix*), 164
`InteractiveQueryDecomposition` (*class in dicee.abstracts*), 17
`intialize_model()` (*in module dicee.static_funcs*), 266
`is_continual_training` (*dicee.DICE_Trainer attribute*), 297
`is_continual_training` (*dicee.evaluation.Evaluator attribute*), 60
`is_continual_training` (*dicee.evaluation.evaluator.Evaluator attribute*), 49
`is_continual_training` (*dicee.Evaluator attribute*), 298
`is_continual_training` (*dicee.evaluator.Evaluator attribute*), 67
`is_continual_training` (*dicee.Execute attribute*), 288
`is_continual_training` (*dicee.executer.Execute attribute*), 71
`is_continual_training` (*dicee.trainer.DICE_Trainer attribute*), 279
`is_continual_training` (*dicee.trainer.dice_trainer.DICE_Trainer attribute*), 272
`is_global_rank_zero()` (*dicee.Execute method*), 289
`is_global_rank_zero()` (*dicee.executer.Execute method*), 71
`is_global_zero` (*dicee.abstracts.AbstractTrainer attribute*), 13
`is_global_zero` (*dicee.trainer.torch_trainer_fsdp.TorchFSDPTrainer attribute*), 278
`is_local_rank_zero()` (*dicee.Execute method*), 289
`is_local_rank_zero()` (*dicee.executer.Execute method*), 71
`is_seen()` (*dicee.abstracts.BaseInteractiveKGE method*), 15
`is_sparql_endpoint_alive()` (*in module dicee.sanity_checkers*), 261

K

`k` (*dicee.models.FMult attribute*), 246
`k` (*dicee.models.FMult2 attribute*), 247
`k` (*dicee.models.function_space.FMult attribute*), 132
`k` (*dicee.models.function_space.FMult2 attribute*), 134
`k` (*dicee.models.function_space.GFMult attribute*), 133
`k` (*dicee.models.GFMult attribute*), 246
`k_fold_cross_validation()` (*dicee.DICE_Trainer method*), 297
`k_fold_cross_validation()` (*dicee.trainer.DICE_Trainer method*), 280
`k_fold_cross_validation()` (*dicee.trainer.dice_trainer.DICE_Trainer method*), 273
`k_vs_all_score()` (*dicee.models.clifford.Keci method*), 111
`k_vs_all_score()` (*dicee.models.ComplEx static method*), 212
`k_vs_all_score()` (*dicee.models.complex.ComplEx static method*), 120
`k_vs_all_score()` (*dicee.models.DistMult method*), 198
`k_vs_all_score()` (*dicee.models.Keci method*), 230
`k_vs_all_score()` (*dicee.models.octonion.OMult method*), 142
`k_vs_all_score()` (*dicee.models.OMult method*), 226

`k_vs_all_score()` (*dicee.models.QMult method*), 219
`k_vs_all_score()` (*dicee.models.quaternion.QMult method*), 147
`k_vs_all_score()` (*dicee.models.real.DistMult method*), 149
`Keci` (*class in dicee.models*), 228
`Keci` (*class in dicee.models.clifford*), 108
`KeciTransformer` (*class in dicee.models*), 235
`KeciTransformer` (*class in dicee.models.clifford*), 112
`kernel_size` (*dicee.config.Namespace attribute*), 31
`kernel_size` (*dicee.models.base_model.BaseKGE attribute*), 104
`kernel_size` (*dicee.models.BaseKGE attribute*), 188, 193, 206, 214, 222, 236, 242
`KG` (*class in dicee.knowledge_graph*), 73
`kg` (*dicee.callbacks.PseudoLabellingCallback attribute*), 25
`kg` (*dicee.read_preprocess_save_load_kg.LoadSaveToDisk attribute*), 260
`kg` (*dicee.read_preprocess_save_load_kg.PreprocessKG attribute*), 259
`kg` (*dicee.read_preprocess_save_load_kg.preprocess.PreprocessKG attribute*), 253
`kg` (*dicee.read_preprocess_save_load_kg.read_from_disk.ReadFromDisk attribute*), 254
`kg` (*dicee.read_preprocess_save_load_kg.ReadFromDisk attribute*), 260
`kg` (*dicee.read_preprocess_save_load_kg.save_load_disk.LoadSaveToDisk attribute*), 255
`KGE` (*class in dicee*), 290
`KGE` (*class in dicee.knowledge_graph_embeddings*), 76
`KGSaveCallback` (*class in dicee.callbacks*), 24
`knowledge_graph` (*dicee.Execute attribute*), 288, 289
`knowledge_graph` (*dicee.executer.Execute attribute*), 71
`KronE` (*class in dicee.callbacks*), 26
`KvsAll` (*class in dicee.dataset_classes*), 37
`kvsall_score()` (*dicee.models.DualE method*), 249
`kvsall_score()` (*dicee.models.dualE.DualE method*), 122
`KvsSampleDataset` (*class in dicee.dataset_classes*), 38

L

`label_relaxation_alpha` (*dicee.config.Namespace attribute*), 33
`label_smoothing_rate` (*dicee.config.Namespace attribute*), 31
`label_smoothing_rate` (*dicee.dataset_classes.AllvsAll attribute*), 37
`label_smoothing_rate` (*dicee.dataset_classes.FixedNegSampleDataset attribute*), 41
`label_smoothing_rate` (*dicee.dataset_classes.FSDP1vsSampleDataset attribute*), 37
`label_smoothing_rate` (*dicee.dataset_classes.KvsAll attribute*), 38
`label_smoothing_rate` (*dicee.dataset_classes.KvsSampleDataset attribute*), 38
`label_smoothing_rate` (*dicee.dataset_classes.OnevsSample attribute*), 41
`label_smoothing_rate` (*dicee.dataset_classes.TriplePredictionDataset attribute*), 42
`LabelRelaxationLoss` (*class in dicee.losses.custom_losses*), 85
`LabelRelaxationLoss` (*class in dicee.models*), 179
`LabelSmoothingLoss` (*class in dicee.losses.custom_losses*), 83
`LabelSmoothingLoss` (*class in dicee.models*), 179
`layer_norm` (*dicee.models.literal.LiteralEmbeddings attribute*), 141
`layer_norm` (*dicee.models.ultra.RelationalConv attribute*), 168
`LayerNorm` (*class in dicee.models.transformers*), 159
`layers` (*dicee.models.trix.EntityReasoner attribute*), 165
`layers` (*dicee.models.ultra.EntityNBFNet attribute*), 169
`layers` (*dicee.models.ultra.RelNBFNet attribute*), 169
`learning_rate` (*dicee.models.base_model.BaseKGE attribute*), 104
`learning_rate` (*dicee.models.BaseKGE attribute*), 188, 193, 205, 214, 221, 236, 241
`length` (*dicee.dataset_classes.FixedNegSampleDataset attribute*), 41
`length` (*dicee.dataset_classes.TriplePredictionDataset attribute*), 42
`level` (*dicee.callbacks.Perturb attribute*), 26
`LFMult` (*class in dicee.models*), 248
`LFMult` (*class in dicee.models.function_space*), 135
`LFMult1` (*class in dicee.models*), 247
`LFMult1` (*class in dicee.models.function_space*), 134
`linear` (*dicee.models.ultra.RelationalConv attribute*), 168
`linear()` (*dicee.models.function_space.LFMult method*), 135
`linear()` (*dicee.models.LFMult method*), 248
`list2tuple()` (*dicee.query_generator.QueryGenerator method*), 252
`list2tuple()` (*dicee.QueryGenerator method*), 296
`LiteralDataset` (*class in dicee.dataset_classes*), 39
`LiteralEmbeddings` (*class in dicee.models.literal*), 140
`lm_head` (*dicee.models.clifford.KeciTransformer attribute*), 112
`lm_head` (*dicee.models.KeciTransformer attribute*), 235

`lm_head` (*dicee.models.transformers.BytE* attribute), 158
`lm_head` (*dicee.models.transformers.GPT* attribute), 163
`ln_1` (*dicee.models.Block* attribute), 193
`ln_1` (*dicee.models.clifford.TransformerBlock* attribute), 113
`ln_1` (*dicee.models.transformers.Block* attribute), 162
`ln_2` (*dicee.models.Block* attribute), 193
`ln_2` (*dicee.models.clifford.TransformerBlock* attribute), 113
`ln_2` (*dicee.models.transformers.Block* attribute), 162
`ln_f` (*dicee.models.CoKE* attribute), 204
`ln_f` (*dicee.models.real.CoKE* attribute), 156
`load()` (*dicee.read_preprocess_save_load_kg.LoadSaveToDisk* method), 260
`load()` (*dicee.read_preprocess_save_load_kg.save_load_disk.LoadSaveToDisk* method), 255
`load_and_validate_literal_data()` (*dicee.dataset_classes.LiteralDataset* static method), 40
`load_from_memmap()` (*dicee.Execute* method), 289
`load_from_memmap()` (*dicee.executer.Execute* method), 71
`load_graph()` (*dicee.models.graph_model.GraphKGE* method), 138
`load_json()` (in module *dicee.static_funcs*), 266
`load_local_shard_checkpoint()` (*dicee.models.fsdp_models.FSDPShardedEntityModel* method), 131
`load_model()` (in module *dicee.static_funcs*), 265
`load_model_ensemble()` (in module *dicee.static_funcs*), 265
`load_numpy()` (in module *dicee.static_funcs*), 267
`load_numpy_ndarray()` (in module *dicee.read_preprocess_save_load_kg.util*), 259
`load_pickle()` (in module *dicee.read_preprocess_save_load_kg.util*), 259
`load_pickle()` (in module *dicee.static_funcs*), 265
`load_pretrained()` (*dicee.models.graph_model.GraphKGE* method), 138
`load_queries()` (*dicee.query_generator.QueryGenerator* method), 253
`load_queries()` (*dicee.QueryGenerator* method), 296
`load_queries_and_answers()` (*dicee.query_generator.QueryGenerator* static method), 253
`load_queries_and_answers()` (*dicee.QueryGenerator* static method), 296
`load_state_dict()` (*dicee.models.ensemble.EnsembleKGE* method), 123
`load_term_mapping()` (in module *dicee.static_funcs*), 265
`load_term_mapping()` (in module *dicee.trainer.dice_trainer*), 271
`load_with_pandas()` (in module *dicee.read_preprocess_save_load_kg.util*), 258
`loader_backend` (*dicee.dataset_classes.LiteralDataset* attribute), 39
`LoadSaveToDisk` (class in *dicee.read_preprocess_save_load_kg*), 260
`LoadSaveToDisk` (class in *dicee.read_preprocess_save_load_kg.save_load_disk*), 255
`local_rank` (*dicee.abstracts.AbstractTrainer* attribute), 13
`local_rank` (*dicee.Execute* attribute), 288
`local_rank` (*dicee.executer.Execute* attribute), 71
`local_rank` (*dicee.trainer.torch_trainer_ddp.NodeTrainer* attribute), 276
`local_rank` (*dicee.trainer.torch_trainer_fsdp.TorchFSDPTrainer* attribute), 278
`local_rows` (*dicee.models.fsdp_models.RowWiseShardedEmbedding* attribute), 131
`log_level` (*dicee.config.Namespace* attribute), 31
`logger` (in module *dicee.abstracts*), 13
`logger` (in module *dicee.analyse_experiments*), 21
`logger` (in module *dicee.callbacks*), 23
`logger` (in module *dicee.evaluation.evaluator*), 48
`logger` (in module *dicee.evaluation.link_prediction*), 52
`logger` (in module *dicee.evaluation.literal_prediction*), 55
`logger` (in module *dicee.executer*), 70
`logger` (in module *dicee.knowledge_graph_embeddings*), 76
`logger` (in module *dicee.models*), 183, 240, 245
`logger` (in module *dicee.models.base_model*), 99
`logger` (in module *dicee.models.clifford*), 108
`logger` (in module *dicee.models.function_space*), 132
`logger` (in module *dicee.models.pykeen_models*), 144
`logger` (in module *dicee.models.transformers*), 158
`logger` (in module *dicee.query_generator*), 252
`logger` (in module *dicee.read_preprocess_save_load_kg.preprocess*), 253
`logger` (in module *dicee.read_preprocess_save_load_kg.read_from_disk*), 254
`logger` (in module *dicee.read_preprocess_save_load_kg.save_load_disk*), 255
`logger` (in module *dicee.read_preprocess_save_load_kg.util*), 256
`logger` (in module *dicee.sanity_checkers*), 261
`logger` (in module *dicee.scripts.index_serve*), 261
`logger` (in module *dicee.static_funcs*), 264
`logger` (in module *dicee.static_preprocess_funcs*), 270
`logger` (in module *dicee.trainer.auto_batch_finder*), 270
`logger` (in module *dicee.trainer.dice_trainer*), 271

- logger (in module *dicее.trainer.model_parallelism*), 274
- logger (in module *dicее.trainer.torch_trainer*), 275
- logger (in module *dicее.trainer.torch_trainer_ddp*), 276
- logger (in module *dicее.trainer.torch_trainer_fsdp*), 277
- loss (*dicее.models.base_model.BaseKGELightning* attribute), 100
- loss (*dicее.models.BaseKGELightning* attribute), 184
- loss_fn (*dicее.config.Namespace* attribute), 33
- loss_func (*dicее.trainer.torch_trainer_ddp.NodeTrainer* attribute), 277
- loss_func (*dicее.trainer.torch_trainer_fsdp.TorchFSDPTrainer* attribute), 278
- loss_function (*dicее.trainer.torch_trainer.TorchTrainer* attribute), 275
- loss_function() (*dicее.models.base_model.BaseKGELightning* method), 100
- loss_function() (*dicее.models.BaseKGELightning* method), 184
- loss_function() (*dicее.models.transformers.BytE* method), 158
- loss_history (*dicее.models.base_model.BaseKGE* attribute), 104
- loss_history (*dicее.models.BaseKGE* attribute), 188, 194, 206, 215, 222, 236, 242
- loss_history (*dicее.models.pykeen_models.PykeenKGE* attribute), 145
- loss_history (*dicее.models.PykeenKGE* attribute), 240
- loss_history (*dicее.trainer.torch_trainer_ddp.NodeTrainer* attribute), 277
- lr (*dicее.analyse_experiments.Experiment* attribute), 21
- lr (*dicее.config.Namespace* attribute), 30
- lr_init (*dicее.weight_averaging.SWA* attribute), 284
- lr_init (*dicее.weight_averaging.SWAG* attribute), 285
- lr_init (*dicее.weight_averaging.TWA* attribute), 287
- lr_lambda (*dicее.callbacks.LRScheduler* attribute), 28
- LRScheduler (class in *dicее.callbacks*), 27

M

- m (*dicее.models.function_space.LFMult* attribute), 135
- m (*dicее.models.LFMult* attribute), 248
- main() (in module *dicее.scripts.index_serve*), 262
- main() (in module *dicее.scripts.run*), 262
- make_iterable_verbose() (in module *dicее.evaluation.utils*), 57
- make_iterable_verbose() (in module *dicее.static_funcs_training*), 269
- make_iterable_verbose() (in module *dicее.trainer.torch_trainer_ddp*), 276
- mapping_from_first_two_cols_to_third() (in module *dicее.static_preprocess_funcs*), 270
- margin (*dicее.config.Namespace* attribute), 31
- margin (*dicее.losses.custom_losses.ACLS* attribute), 90
- margin (*dicее.models.ACLS* attribute), 173
- margin (*dicее.models.complex.RotatE* attribute), 120
- margin (*dicее.models.Pyke* attribute), 203
- margin (*dicее.models.real.Pyke* attribute), 155
- margin (*dicее.models.real.TransE* attribute), 151
- margin (*dicее.models.real.TransH* attribute), 152
- margin (*dicее.models.RotatE* attribute), 213
- margin (*dicее.models.TransE* attribute), 199
- margin (*dicее.models.TransH* attribute), 200
- mask_emb (*dicее.models.CoKE* attribute), 204
- mask_emb (*dicее.models.real.CoKE* attribute), 156
- max_alpha (*dicее.losses.custom_losses.AdaptiveLabelRelaxationLoss* attribute), 86
- max_alpha (*dicее.models.AdaptiveLabelRelaxationLoss* attribute), 174
- max_ans_num (*dicее.query_generator.QueryGenerator* attribute), 252
- max_ans_num (*dicее.QueryGenerator* attribute), 295
- max_epochs (*dicее.callbacks.KGESaveCallback* attribute), 24
- max_epochs (*dicее.callbacks.PeriodicEvalCallback* attribute), 27
- max_epochs (*dicее.weight_averaging.EMA* attribute), 286
- max_epochs (*dicее.weight_averaging.SWA* attribute), 284
- max_epochs (*dicее.weight_averaging.SWAG* attribute), 285
- max_epochs (*dicее.weight_averaging.TWA* attribute), 287
- max_length_subword_tokens (*dicее.knowledge_graph.KG* attribute), 75
- max_length_subword_tokens (*dicее.models.base_model.BaseKGE* attribute), 104
- max_length_subword_tokens (*dicее.models.BaseKGE* attribute), 188, 194, 206, 215, 222, 236, 242
- max_num_models (*dicее.weight_averaging.SWAG* attribute), 285
- max_num_of_classes (*dicее.dataset_classes.FSDP1vsSampleDataset* attribute), 37
- max_num_of_classes (*dicее.dataset_classes.KvsSampleDataset* attribute), 38
- max_smoothing_factor (*dicее.losses.custom_losses.AdaptiveLabelSmoothingLoss* attribute), 84
- max_smoothing_factor (*dicее.models.AdaptiveLabelSmoothingLoss* attribute), 175
- maximize (*dicее.weight_averaging.AMWA* attribute), 282

- `mean` (*dicee.weight_averaging.SWAG attribute*), 285
- `mem_of_model()` (*dicee.models.base_model.BaseKGELightning method*), 100
- `mem_of_model()` (*dicee.models.BaseKGELightning method*), 184
- `mem_of_model()` (*dicee.models.ensemble.EnsembleKGE method*), 123
- `method` (*dicee.callbacks.Perturb attribute*), 26
- `min_alpha` (*dicee.losses.custom_losses.AdaptiveLabelRelaxationLoss attribute*), 86
- `min_alpha` (*dicee.models.AdaptiveLabelRelaxationLoss attribute*), 174
- `min_smoothing_factor` (*dicee.losses.custom_losses.AdaptiveLabelSmoothingLoss attribute*), 84
- `min_smoothing_factor` (*dicee.models.AdaptiveLabelSmoothingLoss attribute*), 175
- `MLP` (*class in dicee.models.transformers*), 160
- `mlp` (*dicee.models.Block attribute*), 193
- `mlp` (*dicee.models.clifford.TransformerBlock attribute*), 113
- `mlp` (*dicee.models.transformers.Block attribute*), 162
- `mlp` (*dicee.models.trix.EntityReasoner attribute*), 165
- `mlp` (*dicee.models.TRIXRelation attribute*), 251
- `mlp` (*dicee.models.trix.TRIXRelation attribute*), 167
- `mlp` (*dicee.models.ultra.EntityNBFNet attribute*), 169
- `mode` (*dicee.query_generator.QueryGenerator attribute*), 252
- `mode` (*dicee.QueryGenerator attribute*), 295
- `model` (*dicee.config.Namespace attribute*), 29
- `model` (*dicee.models.pykeen_models.PykeenKGE attribute*), 145
- `model` (*dicee.models.PykeenKGE attribute*), 240
- `model` (*dicee.trainer.torch_trainer_ddp.NodeTrainer attribute*), 277
- `model` (*dicee.trainer.torch_trainer_fsdp.TorchFSDPTrainer attribute*), 278
- `model` (*dicee.trainer.torch_trainer.TorchTrainer attribute*), 275
- `model_kwargs` (*dicee.models.pykeen_models.PykeenKGE attribute*), 145
- `model_kwargs` (*dicee.models.PykeenKGE attribute*), 240
- `model_name` (*dicee.analyse_experiments.Experiment attribute*), 21
- `MODEL_REGISTRY` (*in module dicee.static_funcs*), 264
- `module`
 - `dicee`, 12
 - `dicee.__main__`, 12
 - `dicee.abstracts`, 12
 - `dicee.analyse_experiments`, 20
 - `dicee.callbacks`, 22
 - `dicee.config`, 28
 - `dicee.dataset_classes`, 33
 - `dicee.eval_static_funcs`, 42
 - `dicee.evaluation`, 46
 - `dicee.evaluation.ensemble`, 47
 - `dicee.evaluation.evaluator`, 48
 - `dicee.evaluation.link_prediction`, 51
 - `dicee.evaluation.literal_prediction`, 55
 - `dicee.evaluation.utils`, 56
 - `dicee.evaluator`, 66
 - `dicee.executer`, 70
 - `dicee.knowledge_graph`, 73
 - `dicee.knowledge_graph_embeddings`, 75
 - `dicee.losses`, 81
 - `dicee.losses.custom_losses`, 81
 - `dicee.models`, 90
 - `dicee.models.adopt`, 90
 - `dicee.models.base_model`, 99
 - `dicee.models.clifford`, 108
 - `dicee.models.complex`, 117
 - `dicee.models.dualE`, 122
 - `dicee.models.ensemble`, 123
 - `dicee.models.flock`, 124
 - `dicee.models.flock_walks`, 129
 - `dicee.models.fsdp_models`, 130
 - `dicee.models.function_space`, 132
 - `dicee.models.graph_model`, 136
 - `dicee.models.literal`, 140
 - `dicee.models.octonion`, 142
 - `dicee.models.pykeen_models`, 144
 - `dicee.models.quaternion`, 146
 - `dicee.models.real`, 149
 - `dicee.models.sampled_loss`, 157

- `dicee.models.static_funcs`, 157
- `dicee.models.transformers`, 157
- `dicee.models.trix`, 164
- `dicee.models.ultra`, 167
- `dicee.query_generator`, 252
- `dicee.read_preprocess_save_load_kg`, 253
- `dicee.read_preprocess_save_load_kg.preprocess`, 253
- `dicee.read_preprocess_save_load_kg.read_from_disk`, 254
- `dicee.read_preprocess_save_load_kg.save_load_disk`, 255
- `dicee.read_preprocess_save_load_kg.util`, 255
- `dicee.sanity_checkers`, 260
- `dicee.scripts`, 261
- `dicee.scripts.index_serve`, 261
- `dicee.scripts.run`, 262
- `dicee.static_funcs`, 262
- `dicee.static_funcs_training`, 267
- `dicee.static_preprocess_funcs`, 269
- `dicee.trainer`, 270
- `dicee.trainer.auto_batch_finder`, 270
- `dicee.trainer.dice_trainer`, 271
- `dicee.trainer.model_parallelism`, 273
- `dicee.trainer.torch_trainer`, 274
- `dicee.trainer.torch_trainer_ddp`, 276
- `dicee.trainer.torch_trainer_fsdp`, 277
- `dicee.weight_averaging`, 280
- `modules()` (*dicee.models.ensemble.EnsembleKGE method*), 123
- `momentum_history` (*dicee.weight_averaging.AMWA attribute*), 282
- `monitor` (*dicee.weight_averaging.AMWA attribute*), 282
- `moving_average()` (*dicee.weight_averaging.SWA static method*), 284
- `MultiClassClassificationDataset` (*class in dicee.dataset_classes*), 35
- `MultiLabelDataset` (*class in dicee.dataset_classes*), 35
- `MuRE` (*class in dicee.models*), 201
- `MuRE` (*class in dicee.models.real*), 153

N

- `n` (*dicee.models.FMult2 attribute*), 247
- `n` (*dicee.models.function_space.FMult2 attribute*), 134
- `n_embd` (*dicee.models.clifford.TransformerSelfAttention attribute*), 113
- `n_embd` (*dicee.models.CoKEConfig attribute*), 204
- `n_embd` (*dicee.models.real.CoKEConfig attribute*), 155, 156
- `n_embd` (*dicee.models.transformers.GPTConfig attribute*), 162
- `n_embd` (*dicee.models.transformers.SelfAttention attribute*), 160
- `n_epochs_eval_model` (*dicee.callbacks.PeriodicEvalCallback attribute*), 27
- `n_epochs_eval_model` (*dicee.config.Namespace attribute*), 32
- `n_head` (*dicee.models.clifford.TransformerSelfAttention attribute*), 113
- `n_head` (*dicee.models.CoKEConfig attribute*), 204
- `n_head` (*dicee.models.real.CoKEConfig attribute*), 155, 156
- `n_head` (*dicee.models.transformers.GPTConfig attribute*), 162
- `n_head` (*dicee.models.transformers.SelfAttention attribute*), 160
- `n_layer` (*dicee.models.CoKEConfig attribute*), 204
- `n_layer` (*dicee.models.real.CoKEConfig attribute*), 155, 156
- `n_layer` (*dicee.models.transformers.GPTConfig attribute*), 162
- `n_layers` (*dicee.models.FMult2 attribute*), 247
- `n_layers` (*dicee.models.function_space.FMult2 attribute*), 134
- `name` (*dicee.abstracts.BaseInteractiveKGE property*), 14
- `name` (*dicee.models.AConEx attribute*), 210
- `name` (*dicee.models.AConvO attribute*), 227
- `name` (*dicee.models.AConvQ attribute*), 220
- `name` (*dicee.models.CKeci attribute*), 232
- `name` (*dicee.models.clifford.CKeci attribute*), 114
- `name` (*dicee.models.clifford.DeCaL attribute*), 114
- `name` (*dicee.models.clifford.Keci attribute*), 109
- `name` (*dicee.models.clifford.KeciTransformer attribute*), 112
- `name` (*dicee.models.CoKE attribute*), 204
- `name` (*dicee.models.ComplEx attribute*), 212
- `name` (*dicee.models.complex.AConEx attribute*), 118
- `name` (*dicee.models.complex.ComplEx attribute*), 119

name (*dicee.models.complex.ConEx* attribute), 117
 name (*dicee.models.complex.RotatE* attribute), 120
 name (*dicee.models.ConEx* attribute), 209
 name (*dicee.models.ConvO* attribute), 227
 name (*dicee.models.ConvQ* attribute), 220
 name (*dicee.models.DeCaL* attribute), 232
 name (*dicee.models.DistMult* attribute), 198
 name (*dicee.models.DualE* attribute), 249
 name (*dicee.models.dualE.DualE* attribute), 122
 name (*dicee.models.ensemble.EnsembleKGE* attribute), 123
 name (*dicee.models.Flock* attribute), 251
 name (*dicee.models.flock.Flock* attribute), 128
 name (*dicee.models.flock.FlockRelation* attribute), 129
 name (*dicee.models.FlockRelation* attribute), 251
 name (*dicee.models.FMult* attribute), 245
 name (*dicee.models.FMult2* attribute), 247
 name (*dicee.models.function_space.FMult* attribute), 132
 name (*dicee.models.function_space.FMult2* attribute), 134
 name (*dicee.models.function_space.GFMult* attribute), 133
 name (*dicee.models.function_space.LFMult* attribute), 135
 name (*dicee.models.function_space.LFMult1* attribute), 134
 name (*dicee.models.GFMult* attribute), 246
 name (*dicee.models.graph_model.GraphKGE* attribute), 137
 name (*dicee.models.Keci* attribute), 228
 name (*dicee.models.KeciTransformer* attribute), 235
 name (*dicee.models.LFMult* attribute), 248
 name (*dicee.models.LFMult1* attribute), 247
 name (*dicee.models.MuRE* attribute), 201
 name (*dicee.models.octonion.AConvO* attribute), 143
 name (*dicee.models.octonion.ConvO* attribute), 143
 name (*dicee.models.octonion.OMult* attribute), 142
 name (*dicee.models.OMult* attribute), 226
 name (*dicee.models.Pyke* attribute), 203
 name (*dicee.models.pykeen_models.PykeenKGE* attribute), 145
 name (*dicee.models.PykeenKGE* attribute), 240
 name (*dicee.models.QMult* attribute), 219
 name (*dicee.models.quaternion.AConvQ* attribute), 148
 name (*dicee.models.quaternion.ConvQ* attribute), 147
 name (*dicee.models.quaternion.QMult* attribute), 146
 name (*dicee.models.real.CoKE* attribute), 156
 name (*dicee.models.real.DistMult* attribute), 149
 name (*dicee.models.real.MuRE* attribute), 153
 name (*dicee.models.real.Pyke* attribute), 155
 name (*dicee.models.real.Shallom* attribute), 154
 name (*dicee.models.real.TransE* attribute), 151
 name (*dicee.models.real.TransH* attribute), 152
 name (*dicee.models.RotatE* attribute), 213
 name (*dicee.models.Shallom* attribute), 202
 name (*dicee.models.TransE* attribute), 199
 name (*dicee.models.transformers.BytE* attribute), 158
 name (*dicee.models.TransH* attribute), 200
 name (*dicee.models.TRIX* attribute), 251
 name (*dicee.models.TRIXRelation* attribute), 251
 name (*dicee.models.trix.TRIX* attribute), 166
 name (*dicee.models.trix.TRIXRelation* attribute), 167
 name (*dicee.models.ULTRA* attribute), 250
 name (*dicee.models.ultra.ULTRA* attribute), 170
 named_children() (*dicee.models.ensemble.EnsembleKGE* method), 123
 names (*dicee.losses.custom_losses.ACLS* property), 90
 names (*dicee.models.ACLS* property), 173
 Namespace (class in *dicee.config*), 28
 neg_lambda (*dicee.losses.custom_losses.ACLS* attribute), 90
 neg_lambda (*dicee.models.ACLS* attribute), 173
 neg_ratio (*dicee.config.Namespace* attribute), 30
 neg_ratio (*dicee.dataset_classes.BPE_NegativeSamplingDataset* attribute), 35
 neg_ratio (*dicee.dataset_classes.FSDP1vsSampleDataset* attribute), 37
 neg_ratio (*dicee.dataset_classes.KvsSampleDataset* attribute), 38
 neg_sample_ratio (*dicee.dataset_classes.FixedNegSampleDataset* attribute), 40

neg_sample_ratio (*dicee.dataset_classes.OnevsSample attribute*), 41
 neg_sample_ratio (*dicee.dataset_classes.TriplePredictionDataset attribute*), 42
 negnorm() (*dicee.abstracts.InteractiveQueryDecomposition method*), 18
 net (*dicee.models.flock.FlockBase attribute*), 128
 neural_searcher (*in module dicee.scripts.index_serve*), 262
 NeuralSearcher (*class in dicee.scripts.index_serve*), 262
 node_init (*dicee.models.flock.FlockBase attribute*), 128
 node_logit (*dicee.models.flock.FlockBase attribute*), 128
 NodeTrainer (*class in dicee.trainer.torch_trainer_ddp*), 276
 norm_fc1 (*dicee.models.AConEx attribute*), 211
 norm_fc1 (*dicee.models.AConvO attribute*), 227
 norm_fc1 (*dicee.models.complex.AConEx attribute*), 118
 norm_fc1 (*dicee.models.complex.ConEx attribute*), 117
 norm_fc1 (*dicee.models.ConEx attribute*), 210
 norm_fc1 (*dicee.models.ConvO attribute*), 227
 norm_fc1 (*dicee.models.octonion.AConvO attribute*), 144
 norm_fc1 (*dicee.models.octonion.ConvO attribute*), 143
 normalization (*dicee.analyse_experiments.Experiment attribute*), 22
 normalization (*dicee.config.Namespace attribute*), 30
 normalization_params (*dicee.dataset_classes.LiteralDataset attribute*), 39
 normalization_type (*dicee.dataset_classes.LiteralDataset attribute*), 39
 normalize_head_entity_embeddings (*dicee.models.base_model.BaseKGE attribute*), 104
 normalize_head_entity_embeddings (*dicee.models.BaseKGE attribute*), 188, 194, 206, 214, 222, 236, 242
 normalize_relation_embeddings (*dicee.models.base_model.BaseKGE attribute*), 104
 normalize_relation_embeddings (*dicee.models.BaseKGE attribute*), 188, 194, 206, 214, 222, 236, 242
 normalize_tail_entity_embeddings (*dicee.models.base_model.BaseKGE attribute*), 104
 normalize_tail_entity_embeddings (*dicee.models.BaseKGE attribute*), 188, 194, 206, 214, 222, 236, 242
 normalizer_class (*dicee.models.base_model.BaseKGE attribute*), 104
 normalizer_class (*dicee.models.BaseKGE attribute*), 188, 194, 206, 214, 222, 236, 242
 num_bpe_entities (*dicee.dataset_classes.BPE_NegativeSamplingDataset attribute*), 35
 num_bpe_entities (*dicee.knowledge_graph.KG attribute*), 74
 num_classes (*dicee.losses.custom_losses.ACLS attribute*), 90
 num_classes (*dicee.models.ACLS attribute*), 173
 num_core (*dicee.config.Namespace attribute*), 31
 num_datapoints (*dicee.dataset_classes.BPE_NegativeSamplingDataset attribute*), 35
 num_datapoints (*dicee.dataset_classes.MultiLabelDataset attribute*), 36
 num_direct_relations (*dicee.models.graph_model.GraphKGE attribute*), 137
 num_embeddings (*dicee.models.fsdp_models.RowWiseShardedEmbedding attribute*), 130
 num_ent (*dicee.models.DualE attribute*), 249
 num_ent (*dicee.models.dualE.DualE attribute*), 122
 num_entities (*dicee.dataset_classes.FixedNegSampleDataset attribute*), 41
 num_entities (*dicee.dataset_classes.FSDP1vsSampleDataset attribute*), 37
 num_entities (*dicee.dataset_classes.KvsSampleDataset attribute*), 38
 num_entities (*dicee.dataset_classes.LiteralDataset attribute*), 40
 num_entities (*dicee.dataset_classes.OnevsSample attribute*), 41
 num_entities (*dicee.dataset_classes.TriplePredictionDataset attribute*), 42
 num_entities (*dicee.evaluation.Evaluator attribute*), 60
 num_entities (*dicee.evaluation.evaluator.Evaluator attribute*), 48, 49
 num_entities (*dicee.Evaluator attribute*), 298
 num_entities (*dicee.evaluator.Evaluator attribute*), 67
 num_entities (*dicee.knowledge_graph.KG attribute*), 73, 74
 num_entities (*dicee.models.base_model.BaseKGE attribute*), 104
 num_entities (*dicee.models.BaseKGE attribute*), 188, 193, 205, 214, 221, 236, 241
 num_epochs (*dicee.abstracts.AbstractPPECallback attribute*), 19
 num_epochs (*dicee.analyse_experiments.Experiment attribute*), 21
 num_epochs (*dicee.config.Namespace attribute*), 30
 num_epochs (*dicee.trainer.torch_trainer_ddp.NodeTrainer attribute*), 277
 num_epochs (*dicee.weight_averaging.ASWA attribute*), 283
 num_folds_for_cv (*dicee.config.Namespace attribute*), 31
 num_layers (*dicee.models.ULTRA attribute*), 250
 num_layers (*dicee.models.ultra.ULTRA attribute*), 170
 num_negatives (*dicee.dataset_classes.FSDP1vsSampleDataset attribute*), 37
 num_nodes (*dicee.models.flock_walks.WalkGraph attribute*), 129
 num_of_data_points (*dicee.dataset_classes.MultiClassClassificationDataset attribute*), 35
 num_of_data_properties (*dicee.models.literal.LiteralEmbeddings attribute*), 140, 141
 num_of_epochs (*dicee.callbacks.PseudoLabellingCallback attribute*), 25
 num_of_output_channels (*dicee.config.Namespace attribute*), 31
 num_of_output_channels (*dicee.models.base_model.BaseKGE attribute*), 104

num_of_output_channels (*dicee.models.BaseKGE attribute*), 188, 194, 206, 214, 222, 236, 242
 num_params (*dicee.analyse_experiments.Experiment attribute*), 21
 num_relations (*dicee.dataset_classes.FixedNegSampleDataset attribute*), 41
 num_relations (*dicee.dataset_classes.FSDP1vsSampleDataset attribute*), 37
 num_relations (*dicee.dataset_classes.OnevsSample attribute*), 41
 num_relations (*dicee.dataset_classes.TriplePredictionDataset attribute*), 42
 num_relations (*dicee.evaluation.Evaluator attribute*), 60
 num_relations (*dicee.evaluation.evaluator.Evaluator attribute*), 49
 num_relations (*dicee.Evaluator attribute*), 298, 299
 num_relations (*dicee.evaluator.Evaluator attribute*), 67
 num_relations (*dicee.knowledge_graph.KG attribute*), 73, 74
 num_relations (*dicee.models.base_model.BaseKGE attribute*), 104
 num_relations (*dicee.models.BaseKGE attribute*), 188, 193, 205, 214, 221, 236, 241
 num_sample (*dicee.models.FMult attribute*), 246
 num_sample (*dicee.models.function_space.FMult attribute*), 132
 num_sample (*dicee.models.function_space.GFMult attribute*), 133
 num_sample (*dicee.models.GFMult attribute*), 246
 num_samples (*dicee.weight_averaging.TWA attribute*), 286
 num_tokens (*dicee.knowledge_graph.KG attribute*), 74
 num_tokens (*dicee.models.base_model.BaseKGE attribute*), 104
 num_tokens (*dicee.models.BaseKGE attribute*), 188, 193, 205, 214, 221, 236, 241
 num_types (*dicee.models.flock_walks.WalkGraph attribute*), 129
 num_workers (*dicee.trainer.torch_trainer_fsdp.TorchFSDPTrainer attribute*), 278
 numpy_data_type_changer() (in module *dicee.static_funcs*), 265

O

octonion_mul() (in module *dicee.models*), 226
 octonion_mul() (in module *dicee.models.octonion*), 142
 octonion_mul_norm() (in module *dicee.models*), 226
 octonion_mul_norm() (in module *dicee.models.octonion*), 142
 octonion_normalizer() (*dicee.models.AConvO static method*), 228
 octonion_normalizer() (*dicee.models.ConvO static method*), 227
 octonion_normalizer() (*dicee.models.octonion.AConvO static method*), 144
 octonion_normalizer() (*dicee.models.octonion.ConvO static method*), 143
 octonion_normalizer() (*dicee.models.octonion.OMult static method*), 142
 octonion_normalizer() (*dicee.models.OMult static method*), 226
 OMult (class in *dicee.models*), 226
 OMult (class in *dicee.models.octonion*), 142
 on_epoch_end() (*dicee.callbacks.KGESaveCallback method*), 24
 on_epoch_end() (*dicee.callbacks.PseudoLabellingCallback method*), 25
 on_fit_end() (*dicee.abstracts.AbstractCallback method*), 19
 on_fit_end() (*dicee.abstracts.AbstractPPECallback method*), 19
 on_fit_end() (*dicee.abstracts.AbstractTrainer method*), 13
 on_fit_end() (*dicee.callbacks.AccumulateEpochLossCallback method*), 23
 on_fit_end() (*dicee.callbacks.Eval method*), 25
 on_fit_end() (*dicee.callbacks.KGESaveCallback method*), 24
 on_fit_end() (*dicee.callbacks.LRScheduler method*), 28
 on_fit_end() (*dicee.callbacks.PeriodicEvalCallback method*), 27
 on_fit_end() (*dicee.callbacks.PrintCallback method*), 23
 on_fit_end() (*dicee.weight_averaging.AMWA method*), 282
 on_fit_end() (*dicee.weight_averaging.ASWA method*), 283
 on_fit_end() (*dicee.weight_averaging.EMA method*), 286
 on_fit_end() (*dicee.weight_averaging.SWA method*), 284
 on_fit_end() (*dicee.weight_averaging.SWAG method*), 285
 on_fit_end() (*dicee.weight_averaging.TWA method*), 287
 on_fit_start() (*dicee.abstracts.AbstractCallback method*), 18
 on_fit_start() (*dicee.abstracts.AbstractPPECallback method*), 19
 on_fit_start() (*dicee.abstracts.AbstractTrainer method*), 13
 on_fit_start() (*dicee.callbacks.Eval method*), 25
 on_fit_start() (*dicee.callbacks.KGESaveCallback method*), 24
 on_fit_start() (*dicee.callbacks.KronE method*), 26
 on_fit_start() (*dicee.callbacks.PrintCallback method*), 23
 on_init_end() (*dicee.abstracts.AbstractCallback method*), 18
 on_init_start() (*dicee.abstracts.AbstractCallback method*), 18
 on_train_batch_end() (*dicee.abstracts.AbstractCallback method*), 19
 on_train_batch_end() (*dicee.abstracts.AbstractTrainer method*), 13
 on_train_batch_end() (*dicee.callbacks.Eval method*), 26

- `on_train_batch_end()` (*dicee.callbacks.KGESaveCallback* method), 24
- `on_train_batch_end()` (*dicee.callbacks.LRScheduler* method), 28
- `on_train_batch_end()` (*dicee.callbacks.PrintCallback* method), 23
- `on_train_batch_start()` (*dicee.callbacks.Perturb* method), 27
- `on_train_epoch_end()` (*dicee.abstracts.AbstractCallback* method), 19
- `on_train_epoch_end()` (*dicee.abstracts.AbstractTrainer* method), 13
- `on_train_epoch_end()` (*dicee.callbacks.Eval* method), 25
- `on_train_epoch_end()` (*dicee.callbacks.KGESaveCallback* method), 24
- `on_train_epoch_end()` (*dicee.callbacks.PeriodicEvalCallback* method), 27
- `on_train_epoch_end()` (*dicee.callbacks.PrintCallback* method), 23
- `on_train_epoch_end()` (*dicee.models.base_model.BaseKGELightning* method), 100
- `on_train_epoch_end()` (*dicee.models.BaseKGELightning* method), 184
- `on_train_epoch_end()` (*dicee.weight_averaging.AMWA* method), 282
- `on_train_epoch_end()` (*dicee.weight_averaging.ASWA* method), 283
- `on_train_epoch_end()` (*dicee.weight_averaging.EMA* method), 286
- `on_train_epoch_end()` (*dicee.weight_averaging.SWA* method), 284
- `on_train_epoch_end()` (*dicee.weight_averaging.SWAG* method), 285
- `on_train_epoch_end()` (*dicee.weight_averaging.TWA* method), 287
- `on_train_epoch_start()` (*dicee.abstracts.AbstractTrainer* method), 13
- `on_train_epoch_start()` (*dicee.weight_averaging.AMWA* method), 282
- `on_train_epoch_start()` (*dicee.weight_averaging.EMA* method), 286
- `on_train_epoch_start()` (*dicee.weight_averaging.SWA* method), 284
- `on_train_epoch_start()` (*dicee.weight_averaging.SWAG* method), 285
- `on_train_epoch_start()` (*dicee.weight_averaging.TWA* method), 287
- `on_train_start()` (*dicee.callbacks.LRScheduler* method), 28
- `OnevsAllDataset` (class in *dicee.dataset_classes*), 39
- `OnevsSample` (class in *dicee.dataset_classes*), 41
- `optim` (*dicee.config.Namespace* attribute), 30
- `OptimizedModule` (in module *dicee.trainer.torch_trainer_fsdp*), 277
- `optimizer` (*dicee.trainer.torch_trainer_ddp.NodeTrainer* attribute), 276
- `optimizer` (*dicee.trainer.torch_trainer_fsdp.TorchFSDPTrainer* attribute), 278
- `optimizer` (*dicee.trainer.torch_trainer.TorchTrainer* attribute), 275
- `optimizer_name` (*dicee.models.base_model.BaseKGE* attribute), 104
- `optimizer_name` (*dicee.models.BaseKGE* attribute), 188, 193, 206, 214, 222, 236, 241
- `ordered_bpe_entities` (*dicee.dataset_classes.BPE_NegativeSamplingDataset* attribute), 35
- `ordered_bpe_entities` (*dicee.knowledge_graph.KG* attribute), 75
- `ordered_shaped_bpe_tokens` (*dicee.knowledge_graph.KG* attribute), 74

P

- `p` (*dicee.config.Namespace* attribute), 32
- `p` (*dicee.models.clifford.DeCaL* attribute), 114
- `p` (*dicee.models.clifford.Keci* attribute), 109
- `p` (*dicee.models.DeCaL* attribute), 232
- `p` (*dicee.models.Keci* attribute), 228
- `P` (*dicee.weight_averaging.TWA* attribute), 287
- `pack_walks` (*dicee.models.flock.FlockBase* attribute), 128
- `padding` (*dicee.knowledge_graph.KG* attribute), 75
- `pandas_dataframe_indexer()` (in module *dicee.read_preprocess_save_load_kg.util*), 257
- `param_init` (*dicee.models.base_model.BaseKGE* attribute), 104
- `param_init` (*dicee.models.BaseKGE* attribute), 188, 194, 206, 215, 222, 236, 242
- `parameters()` (*dicee.abstracts.BaseInteractiveKGE* method), 17
- `parameters()` (*dicee.models.ensemble.EnsembleKGE* method), 123
- `parse_types()` (*dicee.models.flock_walks.WalkGraph* method), 129
- `path` (*dicee.abstracts.AbstractPPECallback* attribute), 19
- `path` (*dicee.callbacks.AccumulateEpochLossCallback* attribute), 23
- `path` (*dicee.callbacks.Eval* attribute), 25
- `path` (*dicee.callbacks.KGESaveCallback* attribute), 24
- `path` (*dicee.weight_averaging.AMWA* attribute), 282
- `path` (*dicee.weight_averaging.ASWA* attribute), 283
- `path_dataset_folder` (*dicee.analyse_experiments.Experiment* attribute), 21
- `path_for_deserialization` (*dicee.knowledge_graph.KG* attribute), 74
- `path_for_serialization` (*dicee.knowledge_graph.KG* attribute), 74
- `path_single_kg` (*dicee.config.Namespace* attribute), 28
- `path_single_kg` (*dicee.knowledge_graph.KG* attribute), 74
- `path_to_store_single_run` (*dicee.config.Namespace* attribute), 28
- `PeriodicEvalCallback` (class in *dicee.callbacks*), 27
- `Perturb` (class in *dicee.callbacks*), 26

`pl_trainer_kwargs` (*dicee.config.Namespace* attribute), 31
`polars_dataframe_indexer()` (in module *dicee.read_preprocess_save_load_kg.util*), 256
`poly_NN()` (*dicee.models.function_space.LFMult* method), 135
`poly_NN()` (*dicee.models.LFMult* method), 248
`polynomial()` (*dicee.models.function_space.LFMult* method), 136
`polynomial()` (*dicee.models.LFMult* method), 249
`pop()` (*dicee.models.function_space.LFMult* method), 136
`pop()` (*dicee.models.LFMult* method), 249
`pos_emb` (*dicee.models.CoKE* attribute), 204
`pos_emb` (*dicee.models.real.CoKE* attribute), 156
`pos_lambda` (*dicee.losses.custom_losses.ACLS* attribute), 90
`pos_lambda` (*dicee.models.ACLS* attribute), 173
`pq` (*dicee.analyse_experiments.Experiment* attribute), 21
`predict()` (*dicee.KGE* method), 291
`predict()` (*dicee.knowledge_graph_embeddings.KGE* method), 77
`predict_data_loader()` (*dicee.models.base_model.BaseKGELightning* method), 102
`predict_data_loader()` (*dicee.models.BaseKGELightning* method), 186
`predict_literals()` (*dicee.KGE* method), 295
`predict_literals()` (*dicee.knowledge_graph_embeddings.KGE* method), 81
`predict_missing_head_entity()` (*dicee.KGE* method), 290
`predict_missing_head_entity()` (*dicee.knowledge_graph_embeddings.KGE* method), 76
`predict_missing_relations()` (*dicee.KGE* method), 291
`predict_missing_relations()` (*dicee.knowledge_graph_embeddings.KGE* method), 77
`predict_missing_tail_entity()` (*dicee.KGE* method), 291
`predict_missing_tail_entity()` (*dicee.knowledge_graph_embeddings.KGE* method), 77
`predict_topk()` (*dicee.KGE* method), 292
`predict_topk()` (*dicee.knowledge_graph_embeddings.KGE* method), 78
`prefetch_factor` (*dicee.trainer.torch_trainer_fsdp.TorchFSDPTrainer* attribute), 278
`prefetch_walks` (*dicee.models.flock.FlockBase* attribute), 128
`prepare_graph_model()` (*dicee.DICE_Trainer* method), 297
`prepare_graph_model()` (*dicee.trainer.DICE_Trainer* method), 280
`prepare_graph_model()` (*dicee.trainer.dice_trainer.DICE_Trainer* method), 273
`prepare_ultra` (*dicee.DICE_Trainer* attribute), 297
`prepare_ultra` (*dicee.trainer.DICE_Trainer* attribute), 280
`prepare_ultra` (*dicee.trainer.dice_trainer.DICE_Trainer* attribute), 273
`preprocess_with_byte_pair_encoding()` (*dicee.read_preprocess_save_load_kg.PreprocessKG* method), 259
`preprocess_with_byte_pair_encoding()` (*dicee.read_preprocess_save_load_kg.preprocess.PreprocessKG* method), 254
`preprocess_with_byte_pair_encoding_with_padding()` (*dicee.read_preprocess_save_load_kg.preprocess.PreprocessKG* method), 254
`preprocess_with_pandas()` (*dicee.read_preprocess_save_load_kg.PreprocessKG* method), 259
`preprocess_with_pandas()` (*dicee.read_preprocess_save_load_kg.preprocess.PreprocessKG* method), 254
`preprocess_with_polars()` (*dicee.read_preprocess_save_load_kg.PreprocessKG* method), 259
`preprocess_with_polars()` (*dicee.read_preprocess_save_load_kg.preprocess.PreprocessKG* method), 254
`preprocesses_input_args()` (in module *dicee.static_preprocess_funcs*), 270
`PreprocessKG` (class in *dicee.read_preprocess_save_load_kg*), 259
`PreprocessKG` (class in *dicee.read_preprocess_save_load_kg.preprocess*), 253
`prev_loss` (*dicee.losses.custom_losses.AdaptiveLabelRelaxationLoss* attribute), 86
`prev_loss` (*dicee.losses.custom_losses.AdaptiveLabelSmoothingLoss* attribute), 85
`prev_loss` (*dicee.models.AdaptiveLabelRelaxationLoss* attribute), 174
`prev_loss` (*dicee.models.AdaptiveLabelSmoothingLoss* attribute), 175
`PrintCallback` (class in *dicee.callbacks*), 23
`process` (*dicee.trainer.torch_trainer.TorchTrainer* attribute), 275
`projection_cache_mb` (*dicee.models.ULTRA* attribute), 251
`projection_cache_mb` (*dicee.models.ultra.ULTRA* attribute), 170
`PseudoLabellingCallback` (class in *dicee.callbacks*), 24
`ptdtype` (*dicee.trainer.torch_trainer_fsdp.TorchFSDPTrainer* attribute), 278
`Pyke` (class in *dicee.models*), 203
`Pyke` (class in *dicee.models.real*), 155
`pykeen_model_kwargs` (*dicee.config.Namespace* attribute), 31
`PykeenKGE` (class in *dicee.models*), 240
`PykeenKGE` (class in *dicee.models.pykeen_models*), 144

Q

`q` (*dicee.config.Namespace* attribute), 32
`q` (*dicee.models.clifford.DeCaL* attribute), 114
`q` (*dicee.models.clifford.Keci* attribute), 109
`q` (*dicee.models.DeCaL* attribute), 232

- `q` (*dicee.models.Keci* attribute), 228
- `qdrant_client` (*dicee.scripts.index_serve.NeuralSearcher* attribute), 262
- `QMult` (class in *dicee.models*), 218
- `QMult` (class in *dicee.models.quaternion*), 146
- `quaternion_mul()` (in module *dicee.models*), 218
- `quaternion_mul()` (in module *dicee.models.static_funcs*), 157
- `quaternion_mul_with_unit_norm()` (in module *dicee.models*), 218
- `quaternion_mul_with_unit_norm()` (in module *dicee.models.quaternion*), 146
- `quaternion_multiplication_followed_by_inner_product()` (*dicee.models.QMult* method), 219
- `quaternion_multiplication_followed_by_inner_product()` (*dicee.models.quaternion.QMult* method), 146
- `quaternion_normalizer()` (*dicee.models.QMult* static method), 219
- `quaternion_normalizer()` (*dicee.models.quaternion.QMult* static method), 146
- `queries` (*dicee.scripts.index_serve.StringListRequest* attribute), 262
- `query_batch_size` (*dicee.models.graph_model.GraphKGE* attribute), 137
- `query_name_to_struct` (*dicee.query_generator.QueryGenerator* attribute), 252
- `query_name_to_struct` (*dicee.QueryGenerator* attribute), 295
- `QueryGenerator` (class in *dicee*), 295
- `QueryGenerator` (class in *dicee.query_generator*), 252

R

- `r` (*dicee.models.clifford.DeCaL* attribute), 114
- `r` (*dicee.models.clifford.Keci* attribute), 109
- `r` (*dicee.models.DeCaL* attribute), 232
- `r` (*dicee.models.Keci* attribute), 228
- `random_seed` (*dicee.config.Namespace* attribute), 31
- `range_constraints_per_rel` (*dicee.evaluation.Evaluator* attribute), 60
- `range_constraints_per_rel` (*dicee.evaluation.evaluator.Evaluator* attribute), 49
- `range_constraints_per_rel` (*dicee.Evaluator* attribute), 299
- `range_constraints_per_rel` (*dicee.evaluator.Evaluator* attribute), 68
- `rank` (*dicee.Execute* attribute), 288
- `rank` (*dicee.executer.Execute* attribute), 70, 71
- `rank` (*dicee.models.fsdp_models.RowWiseShardedEmbedding* attribute), 130
- `ratio` (*dicee.callbacks.Perturb* attribute), 26
- `raw_model` (*dicee.trainer.torch_trainer_fsdp.TorchFSDPTrainer* attribute), 278
- `raw_test_set` (*dicee.knowledge_graph.KG* attribute), 74
- `raw_train_set` (*dicee.knowledge_graph.KG* attribute), 74
- `raw_valid_set` (*dicee.knowledge_graph.KG* attribute), 74
- `re` (*dicee.models.clifford.DeCaL* attribute), 114
- `re` (*dicee.models.DeCaL* attribute), 232
- `re_vocab` (*dicee.evaluation.Evaluator* attribute), 60
- `re_vocab` (*dicee.evaluation.evaluator.Evaluator* attribute), 48, 49
- `re_vocab` (*dicee.Evaluator* attribute), 298
- `re_vocab` (*dicee.evaluator.Evaluator* attribute), 67
- `read_from_disk()` (in module *dicee.read_preprocess_save_load_kg.util*), 258
- `read_from_triple_store_with_pandas()` (in module *dicee.read_preprocess_save_load_kg.util*), 258
- `read_from_triple_store_with_polars()` (in module *dicee.read_preprocess_save_load_kg.util*), 258
- `read_only_few` (*dicee.config.Namespace* attribute), 31
- `read_only_few` (*dicee.knowledge_graph.KG* attribute), 74
- `read_or_load_kg()` (in module *dicee.static_funcs*), 266
- `read_with_pandas()` (in module *dicee.read_preprocess_save_load_kg.util*), 258
- `read_with_polars()` (in module *dicee.read_preprocess_save_load_kg.util*), 258
- `ReadFromDisk` (class in *dicee.read_preprocess_save_load_kg*), 260
- `ReadFromDisk` (class in *dicee.read_preprocess_save_load_kg.read_from_disk*), 254
- `reducer` (*dicee.scripts.index_serve.StringListRequest* attribute), 262
- `refinements` (*dicee.models.flock.FlockBase* attribute), 128
- `reg_lambda` (*dicee.weight_averaging.TWA* attribute), 287
- `rel2id` (*dicee.query_generator.QueryGenerator* attribute), 252
- `rel2id` (*dicee.QueryGenerator* attribute), 295
- `relation_cache_mb` (*dicee.models.TRIX* attribute), 251
- `relation_cache_mb` (*dicee.models.trix.TRIX* attribute), 166
- `relation_cache_mb` (*dicee.models.ULTRA* attribute), 250
- `relation_cache_mb` (*dicee.models.ultra.ULTRA* attribute), 170
- `relation_embeddings` (*dicee.models.AConvQ* attribute), 220
- `relation_embeddings` (*dicee.models.complex.RotatE* attribute), 121
- `relation_embeddings` (*dicee.models.ConvQ* attribute), 220
- `relation_embeddings` (*dicee.models.DualE* attribute), 249
- `relation_embeddings` (*dicee.models.dualE.DualE* attribute), 122

`relation_embeddings` (*dicee.models.FMult* attribute), 246
`relation_embeddings` (*dicee.models.FMult2* attribute), 247
`relation_embeddings` (*dicee.models.function_space.FMult* attribute), 132
`relation_embeddings` (*dicee.models.function_space.FMult2* attribute), 134
`relation_embeddings` (*dicee.models.function_space.GFMult* attribute), 133
`relation_embeddings` (*dicee.models.function_space.LFMult* attribute), 135
`relation_embeddings` (*dicee.models.function_space.LFMult1* attribute), 134
`relation_embeddings` (*dicee.models.GFMult* attribute), 246
`relation_embeddings` (*dicee.models.LFMult* attribute), 248
`relation_embeddings` (*dicee.models.LFMult1* attribute), 247
`relation_embeddings` (*dicee.models.pykeen_models.PykeenKGE* attribute), 145
`relation_embeddings` (*dicee.models.PykeenKGE* attribute), 240
`relation_embeddings` (*dicee.models.quaternion.AConvQ* attribute), 148
`relation_embeddings` (*dicee.models.quaternion.ConvQ* attribute), 147
`relation_embeddings` (*dicee.models.RotatE* attribute), 213
`relation_graph` (*dicee.models.trix.TRIXBase* property), 166
`relation_model` (*dicee.models.TRIX* attribute), 251
`relation_model` (*dicee.models.TRIXRelation* attribute), 251
`relation_model` (*dicee.models.trix.TRIX* attribute), 166
`relation_model` (*dicee.models.trix.TRIXRelation* attribute), 167
`relation_model` (*dicee.models.ULTRA* attribute), 250
`relation_model` (*dicee.models.ultra.ULTRA* attribute), 170
`relation_normal_translations` (*dicee.models.real.TransH* attribute), 152
`relation_normal_translations` (*dicee.models.TransH* attribute), 200
`relation_prediction` (*dicee.models.flock.FlockBase* attribute), 127
`relation_prediction` (*dicee.models.flock.FlockRelation* attribute), 129
`relation_prediction` (*dicee.models.FlockRelation* attribute), 251
`relation_to_idx` (*dicee.knowledge_graph.KG* attribute), 74
`relation_translations` (*dicee.models.MuRE* attribute), 201
`relation_translations` (*dicee.models.real.MuRE* attribute), 153
`RelationalConv` (class in *dicee.models.ultra*), 168
`RelationGraphKGE` (class in *dicee.models.graph_model*), 139
`RelationReasoner` (class in *dicee.models.trix*), 165
`relations_str` (*dicee.knowledge_graph.KG* property), 75
`RelNBFNet` (class in *dicee.models.ultra*), 168
`reload_dataset()` (in module *dicee.dataset_classes*), 36
`report` (*dicee.DICE_Trainer* attribute), 296
`report` (*dicee.evaluation.Evaluator* attribute), 60
`report` (*dicee.evaluation.evaluator.Evaluator* attribute), 49
`report` (*dicee.Evaluator* attribute), 298, 299
`report` (*dicee.evaluator.Evaluator* attribute), 67, 68
`report` (*dicee.Execute* attribute), 288, 289
`report` (*dicee.executer.Execute* attribute), 71
`report` (*dicee.trainer.DICE_Trainer* attribute), 279
`report` (*dicee.trainer.dice_trainer.DICE_Trainer* attribute), 272
`reports` (*dicee.callbacks.Eval* attribute), 25
`reports` (*dicee.callbacks.PeriodicEvalCallback* attribute), 27
`requires_grad_for_interactions` (*dicee.models.CKeci* attribute), 232
`requires_grad_for_interactions` (*dicee.models.clifford.CKeci* attribute), 114
`requires_grad_for_interactions` (*dicee.models.clifford.Keci* attribute), 109
`requires_grad_for_interactions` (*dicee.models.Keci* attribute), 228
`resid_dropout` (*dicee.models.clifford.TransformerSelfAttention* attribute), 113
`resid_dropout` (*dicee.models.transformers.SelfAttention* attribute), 160
`residual_convolution()` (*dicee.models.AConEx* method), 211
`residual_convolution()` (*dicee.models.AConvO* method), 228
`residual_convolution()` (*dicee.models.AConvQ* method), 221
`residual_convolution()` (*dicee.models.complex.AConEx* method), 118
`residual_convolution()` (*dicee.models.complex.ConEx* method), 117
`residual_convolution()` (*dicee.models.ConEx* method), 210
`residual_convolution()` (*dicee.models.ConvO* method), 227
`residual_convolution()` (*dicee.models.ConvQ* method), 220
`residual_convolution()` (*dicee.models.octonion.AConvO* method), 144
`residual_convolution()` (*dicee.models.octonion.ConvO* method), 143
`residual_convolution()` (*dicee.models.quaternion.AConvQ* method), 148
`residual_convolution()` (*dicee.models.quaternion.ConvQ* method), 148
`retrieve_embedding()` (*dicee.scripts.index_serve.NeuralSearcher* method), 262
`retrieve_embeddings()` (in module *dicee.scripts.index_serve*), 262
`return_multi_hop_query_results()` (*dicee.KGE* method), 293

`return_multi_hop_query_results()` (*dicее.knowledge_graph_embeddings.KGE method*), 79
`RMSNorm` (*class in dicее.models.flock*), 124
`root()` (*in module dicее.scripts.index_serve*), 262
`roots` (*dicее.models.FMult attribute*), 246
`roots` (*dicее.models.function_space.FMult attribute*), 133
`roots` (*dicее.models.function_space.GFMult attribute*), 133
`roots` (*dicее.models.GFMult attribute*), 246
`RotatE` (*class in dicее.models*), 212
`RotatE` (*class in dicее.models.complex*), 120
`RowWiseShardedEmbedding` (*class in dicее.models.fsdp_models*), 130
`runtime` (*dicее.analyse_experiments.Experiment attribute*), 22

S

`sample()` (*dicее.models.flock_walks.WalkGraph method*), 129
`sample()` (*dicее.weight_averaging.SWAG method*), 285
`sample_counter` (*dicее.abstracts.AbstractPPECallback attribute*), 19
`sample_entity()` (*dicее.abstracts.BaseInteractiveKGE method*), 15
`sample_relation()` (*dicее.abstracts.BaseInteractiveKGE method*), 15
`sample_triples_ratio` (*dicее.config.Namespace attribute*), 31
`sample_triples_ratio` (*dicее.knowledge_graph.KG attribute*), 74
`sample_walks()` (*dicее.models.flock.FlockBase method*), 128
`sample_weights()` (*dicее.weight_averaging.TWA method*), 287
`sampling_ratio` (*dicее.dataset_classes.LiteralDataset attribute*), 39
`sanity_check_callback_args()` (*in module dicее.sanity_checkers*), 261
`sanity_checking_with_arguments()` (*in module dicее.sanity_checkers*), 261
`save()` (*dicее.abstracts.BaseInteractiveKGE method*), 15
`save()` (*dicее.read_preprocess_save_load_kg.LoadSaveToDisk method*), 260
`save()` (*dicее.read_preprocess_save_load_kg.save_load_disk.LoadSaveToDisk method*), 255
`save_checkpoint()` (*dicее.abstracts.AbstractTrainer static method*), 13
`save_checkpoint_model()` (*in module dicее.static_funcs*), 265
`save_embeddings()` (*in module dicее.static_funcs*), 266
`save_embeddings_as_csv` (*dicее.config.Namespace attribute*), 28
`save_every_n_epochs` (*dicее.config.Namespace attribute*), 32
`save_experiment()` (*dicее.analyse_experiments.Experiment method*), 22
`save_graph()` (*dicее.models.graph_model.GraphKGE method*), 138
`save_local_shard_checkpoint()` (*dicее.models.fsdp_models.FSDPShardedEntityModel method*), 131
`save_model_at_every_epoch` (*dicее.config.Namespace attribute*), 31
`save_model_every_n_epoch` (*dicее.callbacks.PeriodicEvalCallback attribute*), 27
`save_numpy_ndarray()` (*in module dicее.read_preprocess_save_load_kg.util*), 259
`save_numpy_ndarray()` (*in module dicее.static_funcs*), 265
`save_pickle()` (*in module dicее.read_preprocess_save_load_kg.util*), 259
`save_pickle()` (*in module dicее.static_funcs*), 265
`save_queries()` (*dicее.query_generator.QueryGenerator method*), 253
`save_queries()` (*dicее.QueryGenerator method*), 296
`save_queries_and_answers()` (*dicее.query_generator.QueryGenerator static method*), 253
`save_queries_and_answers()` (*dicее.QueryGenerator static method*), 296
`save_trained_model()` (*dicее.Execute method*), 289
`save_trained_model()` (*dicее.executer.Execute method*), 72
`scalar_batch_NN()` (*dicее.models.function_space.LFMult method*), 135
`scalar_batch_NN()` (*dicее.models.LFMult method*), 248
`scaler` (*dicее.callbacks.Perturb attribute*), 27
`scaler` (*dicее.trainer.torch_trainer_ddp.NodeTrainer attribute*), 277
`scaler` (*dicее.trainer.torch_trainer_fsdp.TorchFSDPTrainer attribute*), 278
`scheduler` (*dicее.callbacks.LRScheduler attribute*), 28
`score()` (*dicее.models.clifford.Keci method*), 112
`score()` (*dicее.models.CoKE method*), 205
`score()` (*dicее.models.ComplEx static method*), 212
`score()` (*dicее.models.complex.ComplEx static method*), 120
`score()` (*dicее.models.complex.RotatE method*), 121
`score()` (*dicее.models.DistMult method*), 198
`score()` (*dicее.models.Keci method*), 231
`score()` (*dicее.models.octonion.OMult method*), 142
`score()` (*dicее.models.OMult method*), 226
`score()` (*dicее.models.QMult method*), 219
`score()` (*dicее.models.quaternion.QMult method*), 147
`score()` (*dicее.models.real.CoKE method*), 156
`score()` (*dicее.models.real.DistMult method*), 150

`score()` (*dicee.models.real.TransE method*), 151
`score()` (*dicee.models.RotatE method*), 213
`score()` (*dicee.models.TransE method*), 199
`score_func` (*dicee.models.FMult2 attribute*), 247
`score_func` (*dicee.models.function_space.FMult2 attribute*), 134
`score_walks()` (*dicee.models.flock.FlockBase method*), 128
`scoring_technique` (*dicee.analyse_experiments.Experiment attribute*), 22
`scoring_technique` (*dicee.config.Namespace attribute*), 30
`search()` (*dicee.scripts.index_serve.NeuralSearcher method*), 262
`search_embeddings()` (*in module dicee.scripts.index_serve*), 262
`search_embeddings_batch()` (*in module dicee.scripts.index_serve*), 262
`seed` (*dicee.dataset_classes.FixedNegSampleDataset attribute*), 41
`seed` (*dicee.dataset_classes.TriplePredictionDataset attribute*), 42
`seed` (*dicee.models.flock.FlockBase attribute*), 128
`seed` (*dicee.query_generator.QueryGenerator attribute*), 252
`seed` (*dicee.QueryGenerator attribute*), 295
`select_model()` (*in module dicee.static_funcs*), 265
`selected_optimizer` (*dicee.models.base_model.BaseKGE attribute*), 104
`selected_optimizer` (*dicee.models.BaseKGE attribute*), 188, 194, 206, 214, 222, 236, 242
`SelfAttention` (*class in dicee.models.transformers*), 159
`separator` (*dicee.config.Namespace attribute*), 30
`separator` (*dicee.knowledge_graph.KG attribute*), 74
`seq_len` (*dicee.models.clifford.KeciTransformer attribute*), 112
`seq_len` (*dicee.models.KeciTransformer attribute*), 235
`sequential_vocabulary_construction()` (*dicee.read_preprocess_save_load_kg.PreprocessKG method*), 260
`sequential_vocabulary_construction()` (*dicee.read_preprocess_save_load_kg.preprocess.PreprocessKG method*), 254
`serve()` (*in module dicee.scripts.index_serve*), 262
`set_global_seed()` (*dicee.query_generator.QueryGenerator method*), 252
`set_global_seed()` (*dicee.QueryGenerator method*), 296
`set_graph()` (*dicee.models.graph_model.GraphKGE method*), 138
`set_inference_backend()` (*dicee.models.graph_model.GraphKGE method*), 137
`set_model_eval_mode()` (*dicee.abstracts.BaseInteractiveKGE method*), 14
`set_model_train_mode()` (*dicee.abstracts.BaseInteractiveKGE method*), 14
`setup_distributed_training()` (*in module dicee.static_funcs*), 264
`setup_executor()` (*dicee.Execute method*), 289
`setup_executor()` (*dicee.executor.Execute method*), 71
`setup_fsdp_training()` (*dicee.models.fsdp_models.FSDPShardedEntityModel method*), 131
`Shallom` (*class in dicee.models*), 202
`Shallom` (*class in dicee.models.real*), 154
`shallom` (*dicee.models.real.Shallom attribute*), 154
`shallom` (*dicee.models.Shallom attribute*), 202
`shard_size` (*dicee.models.fsdp_models.RowWiseShardedEmbedding attribute*), 130
`sharding_strategy` (*dicee.trainer.torch_trainer_fsdp.TorchFSDPTrainer attribute*), 278
`single_hop_query_answering()` (*dicee.KGE method*), 293
`single_hop_query_answering()` (*dicee.knowledge_graph_embeddings.KGE method*), 79
`smoothing_factor` (*dicee.losses.custom_losses.AdaptiveLabelSmoothingLoss attribute*), 84
`smoothing_factor` (*dicee.models.AdaptiveLabelSmoothingLoss attribute*), 175
`smoothing_factor_step` (*dicee.losses.custom_losses.AdaptiveLabelSmoothingLoss attribute*), 84
`smoothing_factor_step` (*dicee.models.AdaptiveLabelSmoothingLoss attribute*), 175
`smoothness_ratio` (*dicee.losses.custom_losses.AggregatedLSandLR attribute*), 89
`smoothness_ratio` (*dicee.losses.custom_losses.CombinedLSandLR attribute*), 88
`smoothness_ratio` (*dicee.losses.custom_losses.LabelSmoothingLoss attribute*), 84
`smoothness_ratio` (*dicee.models.AggregatedLSandLR attribute*), 176
`smoothness_ratio` (*dicee.models.CombinedLSandLR attribute*), 177
`smoothness_ratio` (*dicee.models.LabelSmoothingLoss attribute*), 180
`snapshot_dir` (*dicee.callbacks.LRScheduler attribute*), 27
`snapshot_loss` (*dicee.callbacks.LRScheduler attribute*), 28
`sparql_endpoint` (*dicee.config.Namespace attribute*), 29
`sparql_endpoint` (*dicee.knowledge_graph.KG attribute*), 74
`sq_mean` (*dicee.weight_averaging.SWAG attribute*), 285
`stable_history` (*dicee.weight_averaging.AMWA attribute*), 282
`stable_state_dict` (*dicee.weight_averaging.AMWA attribute*), 282
`start()` (*dicee.DICE_Trainer method*), 297
`start()` (*dicee.Execute method*), 290
`start()` (*dicee.executor.Execute method*), 72
`start()` (*dicee.read_preprocess_save_load_kg.PreprocessKG method*), 259
`start()` (*dicee.read_preprocess_save_load_kg.preprocess.PreprocessKG method*), 254
`start()` (*dicee.read_preprocess_save_load_kg.read_from_disk.ReadFromDisk method*), 254

- `start()` (*dicee.read_preprocess_save_load_kg.ReadFromDisk method*), 260
- `start()` (*dicee.trainer.DICE_Trainer method*), 280
- `start()` (*dicee.trainer.dice_trainer.DICE_Trainer method*), 273
- `start_row` (*dicee.models.fsdg_models.RowWiseShardedEmbedding attribute*), 130
- `start_time` (*dicee.callbacks.PrintCallback attribute*), 23
- `start_time` (*dicee.Execute attribute*), 289
- `start_time` (*dicee.executer.Execute attribute*), 71
- `state_dict()` (*dicee.models.ensemble.EnsembleKGE method*), 123
- `step()` (*dicee.models.ADOPT method*), 182
- `step()` (*dicee.models.adopt.ADOPT method*), 94
- `step()` (*dicee.models.ensemble.EnsembleKGE method*), 123
- `step()` (*dicee.models.trix.RelationReasoner method*), 166
- `step_count` (*dicee.callbacks.LRScheduler attribute*), 28
- `storage_path` (*dicee.config.Namespace attribute*), 28
- `storage_path` (*dicee.DICE_Trainer attribute*), 297
- `storage_path` (*dicee.trainer.DICE_Trainer attribute*), 279
- `storage_path` (*dicee.trainer.dice_trainer.DICE_Trainer attribute*), 272
- `store()` (*in module dicee.static_funcs*), 265
- `store_ensemble()` (*dicee.abstracts.AbstractPPECallback method*), 20
- `strategy` (*dicee.abstracts.AbstractTrainer attribute*), 13
- `strict_negative_sampling` (*dicee.config.Namespace attribute*), 29
- `StringListRequest` (*class in dicee.scripts.index_serve*), 262
- `SWA` (*class in dicee.weight_averaging*), 283
- `swa` (*dicee.config.Namespace attribute*), 32
- `swa_c_epochs` (*dicee.config.Namespace attribute*), 33
- `swa_c_epochs` (*dicee.weight_averaging.SWA attribute*), 284
- `swa_c_epochs` (*dicee.weight_averaging.SWAG attribute*), 285
- `swa_lr` (*dicee.weight_averaging.SWA attribute*), 284
- `swa_lr` (*dicee.weight_averaging.SWAG attribute*), 285
- `swa_model` (*dicee.weight_averaging.SWA attribute*), 284
- `swa_n` (*dicee.weight_averaging.SWA attribute*), 284
- `swa_start_epoch` (*dicee.config.Namespace attribute*), 33
- `swa_start_epoch` (*dicee.weight_averaging.SWA attribute*), 284
- `swa_start_epoch` (*dicee.weight_averaging.SWAG attribute*), 285
- `SWAG` (*class in dicee.weight_averaging*), 284
- `swag` (*dicee.config.Namespace attribute*), 32

T

- `T()` (*dicee.models.DualE method*), 250
- `T()` (*dicee.models.dualE.DualE method*), 123
- `t_conorm()` (*dicee.abstracts.InteractiveQueryDecomposition method*), 17
- `t_norm()` (*dicee.abstracts.InteractiveQueryDecomposition method*), 17
- `target_dim` (*dicee.dataset_classes.AllvsAll attribute*), 37
- `target_dim` (*dicee.dataset_classes.MultiLabelDataset attribute*), 36
- `target_dim` (*dicee.dataset_classes.OnevsAllDataset attribute*), 39
- `target_dim` (*dicee.knowledge_graph.KG attribute*), 75
- `temperature` (*dicee.models.transformers.BytE attribute*), 158
- `tensor_t_norm()` (*dicee.abstracts.InteractiveQueryDecomposition method*), 17
- `TensorParallel` (*class in dicee.trainer.model_parallelism*), 274
- `test_data_loader()` (*dicee.models.base_model.BaseKGELightning method*), 101
- `test_data_loader()` (*dicee.models.BaseKGELightning method*), 185
- `test_epoch_end()` (*dicee.models.base_model.BaseKGELightning method*), 101
- `test_epoch_end()` (*dicee.models.BaseKGELightning method*), 185
- `test_h1` (*dicee.analyse_experiments.Experiment attribute*), 22
- `test_h3` (*dicee.analyse_experiments.Experiment attribute*), 22
- `test_h10` (*dicee.analyse_experiments.Experiment attribute*), 22
- `test_mrr` (*dicee.analyse_experiments.Experiment attribute*), 22
- `test_path` (*dicee.query_generator.QueryGenerator attribute*), 252
- `test_path` (*dicee.QueryGenerator attribute*), 295
- `test_samples` (*dicee.models.flock.FlockBase attribute*), 128
- `test_set` (*dicee.knowledge_graph.KG attribute*), 73, 74
- `timeit()` (*in module dicee.read_preprocess_save_load_kg.util*), 258
- `timeit()` (*in module dicee.static_funcs*), 264
- `timeit()` (*in module dicee.static_preprocess_funcs*), 270
- `to()` (*dicee.KGE method*), 290
- `to()` (*dicee.knowledge_graph_embeddings.KGE method*), 76
- `to()` (*dicee.models.ensemble.EnsembleKGE method*), 123

`to_df()` (*dicee.analyse_experiments.Experiment* method), 22
`to_node` (*dicee.models.flock.FlockBase* attribute), 128
`to_type` (*dicee.models.flock.FlockBase* attribute), 128
`topk` (*dicee.models.transformers.BytE* attribute), 158
`topk` (*dicee.scripts.index_serve.NeuralSearcher* attribute), 262
`torch_ordered_shaped_bpe_entities` (*dicee.dataset_classes.MultiLabelDataset* attribute), 36
`TorchDDPTrainer` (class in *dicee.trainer.torch_trainer_ddp*), 276
`TorchFSDPTrainer` (class in *dicee.trainer.torch_trainer_fsdp*), 277
`TorchTrainer` (class in *dicee.trainer.torch_trainer*), 275
`total_epochs` (*dicee.callbacks.LRScheduler* attribute), 27
`total_steps` (*dicee.callbacks.LRScheduler* attribute), 27
`train()` (*dicee.abstracts.BaseInteractiveTrainKGE* method), 20
`train()` (*dicee.models.graph_model.GraphKGE* method), 137
`train()` (*dicee.trainer.torch_trainer_ddp.NodeTrainer* method), 277
`train_data` (*dicee.dataset_classes.AllvsAll* attribute), 37
`train_data` (*dicee.dataset_classes.FSDP1vsSampleDataset* attribute), 37
`train_data` (*dicee.dataset_classes.KvsAll* attribute), 38
`train_data` (*dicee.dataset_classes.KvsSampleDataset* attribute), 38
`train_data` (*dicee.dataset_classes.MultiClassClassificationDataset* attribute), 35
`train_data` (*dicee.dataset_classes.OnevsAllDataset* attribute), 39
`train_data` (*dicee.dataset_classes.OnevsSample* attribute), 41
`train_data_loader()` (*dicee.models.base_model.BaseKGELightning* method), 102
`train_data_loader()` (*dicee.models.BaseKGELightning* method), 186
`train_data_loaders` (*dicee.trainer.torch_trainer.TorchTrainer* attribute), 275
`train_dataset_loader` (*dicee.trainer.torch_trainer_ddp.NodeTrainer* attribute), 277
`train_dataset_loader` (*dicee.trainer.torch_trainer_fsdp.TorchFSDPTrainer* attribute), 278
`train_file_path` (*dicee.dataset_classes.LiteralDataset* attribute), 39
`train_h1` (*dicee.analyse_experiments.Experiment* attribute), 21
`train_h3` (*dicee.analyse_experiments.Experiment* attribute), 21
`train_h10` (*dicee.analyse_experiments.Experiment* attribute), 21
`train_indices_target` (*dicee.dataset_classes.MultiLabelDataset* attribute), 36
`train_k_vs_all()` (*dicee.abstracts.BaseInteractiveTrainKGE* method), 20
`train_literals()` (*dicee.abstracts.BaseInteractiveTrainKGE* method), 20
`train_mode` (*dicee.models.ensemble.EnsembleKGE* attribute), 123
`train_mrr` (*dicee.analyse_experiments.Experiment* attribute), 21
`train_path` (*dicee.query_generator.QueryGenerator* attribute), 252
`train_path` (*dicee.QueryGenerator* attribute), 295
`train_set` (*dicee.dataset_classes.BPE_NegativeSamplingDataset* attribute), 35
`train_set` (*dicee.dataset_classes.MultiLabelDataset* attribute), 36
`train_set` (*dicee.knowledge_graph.KG* attribute), 73, 74
`train_set_target` (*dicee.knowledge_graph.KG* attribute), 75
`train_target` (*dicee.dataset_classes.AllvsAll* attribute), 37
`train_target` (*dicee.dataset_classes.KvsAll* attribute), 38
`train_target` (*dicee.dataset_classes.KvsSampleDataset* attribute), 38
`train_target_indices` (*dicee.knowledge_graph.KG* attribute), 75
`train_triples` (*dicee.dataset_classes.FixedNegSampleDataset* attribute), 41
`train_triples()` (*dicee.abstracts.BaseInteractiveTrainKGE* method), 20
`trained_model` (*dicee.Execute* attribute), 288, 289
`trained_model` (*dicee.executer.Execute* attribute), 71
`trainer` (*dicee.config.Namespace* attribute), 30
`trainer` (*dicee.DICE_Trainer* attribute), 297
`trainer` (*dicee.Execute* attribute), 288, 289
`trainer` (*dicee.executer.Execute* attribute), 71
`trainer` (*dicee.trainer.DICE_Trainer* attribute), 279
`trainer` (*dicee.trainer.dice_trainer.DICE_Trainer* attribute), 272
`trainer` (*dicee.trainer.torch_trainer_ddp.NodeTrainer* attribute), 276
`training_step` (*dicee.trainer.torch_trainer.TorchTrainer* attribute), 275
`training_step()` (*dicee.models.base_model.BaseKGELightning* method), 100
`training_step()` (*dicee.models.BaseKGELightning* method), 184
`training_step()` (*dicee.models.transformers.BytE* method), 159
`training_step_outputs` (*dicee.models.base_model.BaseKGELightning* attribute), 100
`training_step_outputs` (*dicee.models.BaseKGELightning* attribute), 184
`training_technique` (*dicee.knowledge_graph.KG* attribute), 74
`TransE` (class in *dicee.models*), 199
`TransE` (class in *dicee.models.real*), 150
`transformer` (*dicee.models.clifford.KeciTransformer* attribute), 112
`transformer` (*dicee.models.KeciTransformer* attribute), 235
`transformer` (*dicee.models.transformers.BytE* attribute), 158

transformer (*dicee.models.transformers.GPT attribute*), 163
 TransformerBlock (*class in dicee.models.clifford*), 113
 TransformerMLP (*class in dicee.models.clifford*), 113
 TransformerSelfAttention (*class in dicee.models.clifford*), 113
 TransH (*class in dicee.models*), 200
 TransH (*class in dicee.models.real*), 151
 trapezoid() (*dicee.models.FMult2 method*), 247
 trapezoid() (*dicee.models.function_space.FMult2 method*), 134
 tri_score() (*dicee.models.function_space.LFMult method*), 135
 tri_score() (*dicee.models.function_space.LFMult1 method*), 135
 tri_score() (*dicee.models.LFMult method*), 249
 tri_score() (*dicee.models.LFMult1 method*), 248
 triple_score() (*dicee.KGE method*), 292
 triple_score() (*dicee.knowledge_graph_embeddings.KGE method*), 79
 TriplePredictionDataset (*class in dicee.dataset_classes*), 41
 TRIX (*class in dicee.models*), 251
 TRIX (*class in dicee.models.trix*), 166
 trix_checkpoint (*dicee.config.Namespace attribute*), 29
 trix_dim (*dicee.config.Namespace attribute*), 29
 trix_query_batch_size (*dicee.config.Namespace attribute*), 29
 TRIXBase (*class in dicee.models.trix*), 166
 TRIXRelation (*class in dicee.models*), 251
 TRIXRelation (*class in dicee.models.trix*), 166
 tuple2list() (*dicee.query_generator.QueryGenerator method*), 252
 tuple2list() (*dicee.QueryGenerator method*), 296
 TWA (*class in dicee.weight_averaging*), 286
 twa (*dicee.config.Namespace attribute*), 32
 twa_c_epochs (*dicee.weight_averaging.TWA attribute*), 287
 twa_model (*dicee.weight_averaging.TWA attribute*), 287
 twa_start_epoch (*dicee.weight_averaging.TWA attribute*), 286
 type_init (*dicee.models.flock.FlockBase attribute*), 128
 type_logit (*dicee.models.flock.FlockBase attribute*), 128

U

ULTRA (*class in dicee.models*), 250
 ULTRA (*class in dicee.models.ultra*), 170
 ultra_checkpoint (*dicee.config.Namespace attribute*), 29
 ultra_dim (*dicee.config.Namespace attribute*), 29
 ultra_num_layers (*dicee.config.Namespace attribute*), 29
 ultra_query_batch_size (*dicee.config.Namespace attribute*), 29
 unlabelled_size (*dicee.callbacks.PseudoLabellingCallback attribute*), 25
 unmap() (*dicee.query_generator.QueryGenerator method*), 253
 unmap() (*dicee.QueryGenerator method*), 296
 unmap_query() (*dicee.query_generator.QueryGenerator method*), 253
 unmap_query() (*dicee.QueryGenerator method*), 296
 update_hits() (*in module dicee.evaluation.utils*), 58
 UPSTREAM_COMMIT (*in module dicee.models.flock*), 124
 UPSTREAM_COMMIT (*in module dicee.models.trix*), 164
 UPSTREAM_COMMIT (*in module dicee.models.ultra*), 168
 use_clifford_mul (*dicee.models.clifford.KeciTransformer attribute*), 112
 use_clifford_mul (*dicee.models.KeciTransformer attribute*), 235
 use_compile (*dicee.trainer.torch_trainer_fsdp.TorchFSDPTrainer attribute*), 278

V

val_aswa (*dicee.weight_averaging.ASWA attribute*), 283
 val_data_loader() (*dicee.models.base_model.BaseKGELighting method*), 102
 val_data_loader() (*dicee.models.BaseKGELighting method*), 186
 val_h1 (*dicee.analyse_experiments.Experiment attribute*), 21
 val_h3 (*dicee.analyse_experiments.Experiment attribute*), 21
 val_h10 (*dicee.analyse_experiments.Experiment attribute*), 22
 val_mrr (*dicee.analyse_experiments.Experiment attribute*), 21
 val_path (*dicee.query_generator.QueryGenerator attribute*), 252
 val_path (*dicee.QueryGenerator attribute*), 295
 VALID_SCORING_TECHNIQUES (*in module dicee.evaluation.evaluator*), 48
 valid_set (*dicee.knowledge_graph.KG attribute*), 73, 74
 validate_knowledge_graph() (*in module dicee.sanity_checkers*), 261
 var_clamp (*dicee.weight_averaging.SWAG attribute*), 285

vocab_preparation() (*dicee.evaluation.Evaluator method*), 60
 vocab_preparation() (*dicee.evaluation.evaluator.Evaluator method*), 49
 vocab_preparation() (*dicee.Evaluator method*), 299
 vocab_preparation() (*dicee.evaluator.Evaluator method*), 68
 vocab_size (*dicee.models.CoKEConfig attribute*), 203, 204
 vocab_size (*dicee.models.real.CoKEConfig attribute*), 155, 156
 vocab_size (*dicee.models.transformers.GPTConfig attribute*), 162
 vocab_to_parquet() (*in module dicee.static_funcs*), 266
 vtp_score() (*dicee.models.function_space.LFMult method*), 136
 vtp_score() (*dicee.models.function_space.LFMultI method*), 135
 vtp_score() (*dicee.models.LFMult method*), 249
 vtp_score() (*dicee.models.LFMultI method*), 248

W

w1 (*dicee.models.flock.FeedForward attribute*), 126
 w2 (*dicee.models.flock.FeedForward attribute*), 126
 w3 (*dicee.models.flock.FeedForward attribute*), 126
 walk() (*dicee.models.flock_walks.WalkGraph method*), 129
 walk_len (*dicee.models.flock.FlockBase attribute*), 128
 walk_num (*dicee.models.flock.FlockBase attribute*), 128
 WalkGraph (*class in dicee.models.flock_walks*), 129
 warmup_steps (*dicee.callbacks.LRScheduler attribute*), 28
 weight (*dicee.models.flock.RMSNorm attribute*), 125
 weight (*dicee.models.fsdp_models.RowWiseShardedEmbedding attribute*), 131
 weight (*dicee.models.transformers.LayerNorm attribute*), 159
 weight_decay (*dicee.config.Namespace attribute*), 30
 weight_decay (*dicee.models.base_model.BaseKGE attribute*), 104
 weight_decay (*dicee.models.BaseKGE attribute*), 188, 194, 206, 214, 222, 236, 242
 weight_samples (*dicee.weight_averaging.TWA attribute*), 287
 WeightedBCELoss (*class in dicee.losses.custom_losses*), 82
 weights (*dicee.models.FMult attribute*), 246
 weights (*dicee.models.function_space.FMult attribute*), 133
 weights (*dicee.models.function_space.GFMult attribute*), 133
 weights (*dicee.models.GFMult attribute*), 246
 world_size (*dicee.Execute attribute*), 288
 world_size (*dicee.executer.Execute attribute*), 70, 71
 world_size (*dicee.models.fsdp_models.RowWiseShardedEmbedding attribute*), 130
 write_csv_from_model_parallel() (*in module dicee.static_funcs*), 267
 write_links() (*dicee.query_generator.QueryGenerator method*), 253
 write_links() (*dicee.QueryGenerator method*), 296
 write_report() (*dicee.Execute method*), 290
 write_report() (*dicee.executer.Execute method*), 72

X

x_values (*dicee.models.function_space.LFMult attribute*), 135
 x_values (*dicee.models.LFMult attribute*), 248